

Sample Path Analysis and Simulation of Stochastic Systems

Nicky D. van Foreest

March 30, 2026



This work is licensed by the University of Groningen under a [Creative Commons Attribution-ShareAlike 4.0 International License](#).

CONTENTS

1	INTRODUCTION	1
1.1	Motivation and Overview	1
1.2	Exponential and Poisson Distribution	3
1.3	Fighting the Exponential Distribution is Futile	8
1.4	Probabilistic Arithmetic	15
2	CONSTRUCTION OF SIMPLE DISCRETE-TIME STOCHASTIC PROCESSES	21
2.1	Waiting for a Psychiatrist	21
2.2	Simulating the Psychiatrists Case	28
2.3	General Behavior of Queueing Systems	30
2.4	Queueing Processes as Regulated Random Walks	35
2.5	Control of Single-item Inventory Systems	36
2.6	Simulating Single-Item Inventory Systems	40
3	CONSTRUCTION OF SIMPLE CONTINUOUS-TIME STOCHASTIC PROCESSES	42
3.1	Queueing Process in Continuous Time	42
3.2	Kendall's Notation	46
3.3	Simulating Waiting Times	47
3.4	Simulation of Queue Length with Heap Queues	51
3.5	Discrete-event Simulation	53
3.6	Discrete-Event Simulations: Examples	55
4	FUNDAMENTAL TOOLS	59
4.1	Rate, Stability and Load	59
4.2	(Limits of) Empirical Performance Measures	62
4.3	Renewal Reward Theorem	63
4.4	Little's Law	65
4.5	Poisson Arrivals See Time Averages	67
4.6	Level Crossing and Balance Equations	70
4.7	Graphical Summaries	76
5	EXACT MODELS	78
5.1	The $M(n)/M(n)/1$ Queue	78
5.2	Applications of the $M(n)/M(n)/1$ queueing model	82
5.3	$M^B/M/1$ Queue and Rejection Policies	86
5.4	$M/G/1$ Queue: Expected Waiting Time	90
5.5	$M/G/1$ Queue Length Distribution	92
5.6	N -policies for the $M/G/1$ Queue	94
5.7	Analytic Results for the Base-Stock Policy	97
5.8	Analytic Results for the (Q, r) Policy	100
5.9	An algorithm for the (s, S) and (T, S) policies	102
5.10	Hitting problems with applications in single-item inventory control	106
6	APPROXIMATE MODELS	111
6.1	The $G/G/c$ Queue	111
6.2	Server Setups	114
6.3	Server Adjustments	117
6.4	Server Failures	119

7	QUEUEING NETWORKS	122
7.1	$G/G/c$ Queues in Tandem	122
7.2	Deterministic Queueing Networks	123
7.3	Open Single-Class Product-Form Networks	126
7.4	On $\lambda = \gamma + \lambda P$	129
	HINTS	131
	SOLUTIONS	136
	BIBLIOGRAPHY	173
	NOTATION	175

INTRODUCTION

1.1 MOTIVATION AND OVERVIEW

Queueing and inventory systems are ubiquitous, and their analysis, performance evaluation, control, and optimization are major topics in Operations Research. Let us illustrate this with two examples.

Fast food restaurants deal with many difficult queueing and inventory situations. For instance, they prepackage hamburgers and put them on a shelf⁰ so that customers need not wait for their orders. The lifetime of prepackaged hamburgers is on the order of a few minutes, so when customers do not buy a hamburger in time, some must be scrapped. The question is how many hamburgers to stock so that both the probability of scrapping and that of lost sales¹ are small. Second, fast food restaurants keep an inventory of baked patties not yet turned into hamburgers.² Baked patties have a longer lifetime than prepared hamburgers, but since preparing a hamburger from a baked patty takes time, customers may still have to wait. In this case too, the inventory level of these patties needs to be controlled. This does not conclude the matter: the rate of customer demand changes over time, so inventory levels for the different types of hamburgers and patties should vary with the time of day.³

As a second example, hospitals (call centers, courts, etc.) have a certain capacity to serve customers. The performance of hospitals is, in part, measured by the total number of jobs served per year and the sojourn-time distribution, that is, the distribution of the time a job spends in the system. Here the problem is to organize the system so that the average sojourn time is short without wasting capacity.

In all these examples, the performance of the stochastic system must be monitored and controlled. Typically, the following key performance measures (KPIs) are relevant for queueing systems.⁴

1. The fraction of time $p(n)$ that the system contains n customers. In particular, $p(0)$ is the fraction of time the system is empty, hence measures the time-average idle time of servers.
2. The fraction of customers $\pi(n)$ that 'see upon arrival' n customers in the system. This measure relates to customer perception of queue length, and hence to waiting time.
3. The average, variance, and/or distribution of the waiting time.
4. The average, variance, and/or distribution of the number of customers in the system.

With these examples in mind, let us give an overview of this book.

Chapter 2 shows how to construct rules that govern the dynamic behavior of many different queueing systems and inventory systems in discrete time. This has several purposes. First, these rules can often be easily implemented in computer code, which we can then use to simulate and observe

⁰ This is called *make-to-stock* production.

¹ Customers who do not buy because they have to wait.

² A hamburger is a bun plus a baked patty.

³ Inventory and queueing systems are tightly related by changing perspective. Hamburgers in stock can be interpreted as customers in queue waiting for service, where the 'service time' as perceived by a hamburger is the time between the arrival of two customers that buy a hamburger.

⁴ Inventory systems have similar performance measures, such as the average time items spend on the shelf, the fraction of customers served from stock, and the fraction of lost sales.

system behavior over time.⁵ The resulting *sample paths*⁶ can then be analyzed, visually or statistically. Second, we will use sample-path arguments to develop the theoretical results in later parts of the book.

In Chapter 3 we describe how to construct sample paths of continuous-time stochastic systems. In particular, we will be concerned with *discrete-event simulation*.⁷

Once the scope of queueing and inventory theory is clear, the stage is set for a more mathematical treatment of such systems. Despite the power of simulation, structural understanding of stochastic systems is difficult to obtain. Mathematical models, whether exact or approximate, help us reason about and improve such systems.

In Chapter 4 we develop some key results needed for the mathematical analysis of stochastic systems⁸. For this, we use sample paths⁹ of stochastic processes, and by assuming that such sample paths capture the ‘normal’ stochastic behavior, we can use these sample paths to estimate key performance measures.

In Chapter 5 we use these tools to build exact models for single-station queueing systems. We focus on developing an intuitive understanding of the analytical tools.¹⁰

In Chapter 6 we use approximations and results from probability theory to study how system parameters, such as service speed, batching rules, and outages, affect performance.

In Chapter 7 we study queueing networks, first in a deterministic and then in a stochastic setting.

IT REMAINS TO discuss the rest of this first chapter. Section 1.2 introduces the Exponential distribution and relates it to the Poisson distribution. These two are arguably the most important distributions for modeling dynamic stochastic systems. In Section 1.3 we use simulation and some mathematics to explain why these distributions are particularly useful for modeling arrival processes in queueing systems and demand processes for inventory systems. The amount of computational work involved is considerable; in Section 1.4 we discuss Python code that handles the numerical aspects.

WHILE THE MAIN text contains many examples and derivations, a considerable number of examples are left to the exercises. Also, some exercises check consistency between results derived for different models, thereby revealing important connections between different parts of the text.¹¹ The exercises are not intended to be easy; they require effort. Hints and solutions for the exercises are available at the end of the book.

TRUE–FALSE QUESTIONS ARE simple statements that can be either true or false; your task is to determine which.

TF 1.1.1. Consider the following code.

```

1  a = 8
2  a += 9
3  a = 10
4  a -= 3

```

⁵ Simulation is generally the best way to analyze practical stochastic systems, as realistic systems seldom yield to mathematics.

⁶ That is, realizations of these stochastic processes.

⁷ Nearly all professional simulation tools are built on the same principles.

⁸ Of which queueing systems are examples.

⁹ In fact, the same sample paths as those obtained through simulation.

¹⁰ For most proofs and more extensive results, we refer to the bibliography at the end of the book.

¹¹ Note that while such checks are conceptually simple, the algebra can be tedious.

Claim: $a = 10$.

TF 1.1.2. Consider the following code.

```
1 a = [0, 2, 4, 6]
2 a[1] += 1
```

Claim: $a = [0, 3, 4, 6]$.

TF 1.1.3. Consider the following code.

```
1 a = {-10: 1, 2: 8}
2 a[1] += 1
```

Claim: $a = \{-10: 1, 1: 1, 2: 8\}$.

TF 1.1.4. Consider the following code.

```
1 from collections import defaultdict
2
3 a = defaultdict(int)
4 a[-8] = 1
5 a[2] = 8
6 a[1] += 3
7 a[2] -= 3
```

Claim: $a = \{-8: 1, 1: 3, 2: 5\}$.

TF 1.1.5. Consider the following code.

```
1 from collections import defaultdict
2
3 a = defaultdict(int)
4 a[-8] = 1
5 a[2] = 8
6 a[1] += 3
7 a[2] -= 3
```

Claim: The keys of a are $-8, 1, 2$ and the values $1, 3, 5$.

Ex 1.1.6. For a discrete non-negative rv X , use indicator functions to prove:

1. $E[X] = \sum_{k=0}^{\infty} G(k),$
2. $\sum_{i=0}^{\infty} iG(i) = E[X^2]/2 - E[X]/2.$



1.2 EXPONENTIAL AND POISSON DISTRIBUTION

In this section, we make a model for the arrival process that is the most useful for queueing and inventory systems.⁰ In particular, we show how the exponential and gamma distributions relate to the Poisson distribution, and we use the memoryless property of job inter-arrival times as our point of departure.

⁰ In queueing systems, an arrival is often called customer or job, while in inventory systems it is called demand.

WE START WITH two examples to illustrate why it is reasonable to model the time between two events as memoryless. For instance, assume an hour ago a patient with a broken arm arrived at the first aid of a hospital. What can we say about the time the next patient with a broken arm will arrive? In other words, does the information that an hour ago a broken arm arrived help to make a better prediction of the next? Similarly, suppose that the average time between the sales of some shirt in a clothing store is 5 minutes. The shop owner informs us that no shirt has been sold in the past 7 minutes. Surely, this knowledge does not imply that the next shirt will necessarily be sold within the next minute.

Let us formalize this. If a person does not tell us anything about the past, we write $P\{X > t\}$ for the probability that no event occurs before time t . However, if the person tells us at time s there was no arrival during $[0, s]$, we write $P\{X > s + t \mid X > s\}$ for the probability that there will not be an arrival for an additional amount of time t . If we believe that the information $\{X > s\}$ is immaterial to predicting the lifetime of X after s , then necessarily $P\{X > s + t \mid X > s\} = P\{X > t\}$.

Definition 1.2.1. A rv¹ X whose cdf² satisfies

$$P\{X > s + t \mid X > s\} = P\{X > t\}, \quad s, t \geq 0, \quad (1.2.1)$$

is said to be *memoryless*.³

The next theorem is fundamental.

Theorem 1.2.2. $X \sim \text{Exp}(\lambda)$ iff X is memoryless.

Proof. Observing that $P\{X \leq t\} = 1 - e^{-\lambda t} \implies P\{X > t\} = e^{-\lambda t}$,

$$\begin{aligned} P\{X > s + t \mid X > s\} &\stackrel{1}{=} \frac{P\{X > s + t\}}{P\{X > s\}} = \frac{e^{-\lambda(s+t)}}{e^{-\lambda s}} \\ &= e^{-\lambda t} = P\{X > t\}, \end{aligned}$$

where Step 1 follows from the standard formula for conditional probability and that $\{X > t + s\} \subset \{X > s\}$. The other direction is somewhat more difficult; we skip it.⁴ $\square$

Note that for $X \sim \text{Exp}(\lambda)$, the following properties hold:⁵

$$E[X] = \lambda^{-1}, \quad V[X] = \lambda^{-2}.$$

IF WE HAVE a sequence of inter-arrival times $\{X_i\}_{i=1}^{\infty}$, then we construct arrival times according to the recursive rule

$$A_k = A_{k-1} + X_k, \quad A_0 = 0. \quad (1.2.2)$$

When inter-arrival times $\{X_k\}$ are iid⁶ rvs and distributed as the common rv⁷ $X \sim \text{Exp}(\lambda)$, it is clear that the density of the first arrival time is

$$f_{A_1}(t) = f_X(t) = \lambda e^{-\lambda t} = \lambda \frac{(\lambda t)^0}{0!} e^{-\lambda t},$$

¹ $rv(s) := \text{random variable}(s)$.

² $cdf := \text{cumulative distribution function}$.

³ Not all rvs related to lifetime are memoryless. As a counterexample, take X as the lifespan of an arbitrary person born in 1820, and $s = 100$ and $t = 99$ years. Then $P\{X > 99\}$ was small, but not zero, but $P\{X > 199 \mid X > 100\} = 0$.

⁴ For a proof, see?, Appendix 3.

⁵ See [1.2.6] for details.

⁶ iid := independent and identically distributed.

⁷ Henceforth, we write $X_k \text{ iid } \sim X$, or $X_k \text{ iid } \sim \text{Exp}(\lambda)$.

where we add the last term to see how a pattern emerges. Using convolution, for the second arrival time,

$$f_{A_2}(t) = \int_0^t f_{A_1}(s) f_X(t-s) ds = \int_0^t \lambda e^{-\lambda s} \lambda e^{-\lambda(t-s)} ds = \lambda \frac{(\lambda t)^1}{1!} e^{-\lambda t}.$$

With induction it is straightforward to extend the pattern and prove that $A_k \sim \text{Gamma}(\lambda, k)$, i.e.,

$$f_{A_k}(t) = \int_0^t f_{A_{k-1}}(s) f_X(t-s) ds = \lambda \frac{(\lambda t)^{k-1}}{(k-1)!} e^{-\lambda t}. \quad (1.2.3)$$

WHEN THE ARRIVAL times⁸ are gamma distributed, the cdf of the number $N(t)$ of jobs arriving during $[0, t]$ follows readily by observing that

$$P\{N(t) = k\} = P\{A_k \leq t < A_{k+1}\} = P\{A_k \leq t\} - P\{A_{k+1} \leq t\}, \quad (1.2.4)$$

that is, to have precisely k arrivals during $[0, t]$, the arrival time A_k of job k must lie in $[0, t]$, but job $k+1$ must arrive after t . Using the cdf of A_k and integration by parts,

$$\begin{aligned} P\{A_{k+1} \leq t\} &= \lambda \int_0^t \frac{(\lambda s)^k}{k!} e^{-\lambda s} ds = \lambda \frac{(\lambda s)^k}{k!} \frac{e^{-\lambda s}}{-\lambda} \Big|_0^t + \lambda \int_0^t \frac{(\lambda s)^{k-1}}{(k-1)!} e^{-\lambda s} ds \\ &= -\frac{(\lambda t)^k}{k!} e^{-\lambda t} + P\{A_k \leq t\}. \end{aligned}$$

Combining this with (1.2.4), we find

$$P\{N(t) = k\} = \frac{(\lambda t)^k}{k!} e^{-\lambda t}, \quad (1.2.5)$$

that is, $N(t) \sim \text{Pois}(\lambda t)$, i.e., *Poisson distributed* with parameter λt . It is simple to see from (1.2.5) that⁹

$$E[N(t)] = \lambda t \quad V[N(t)] = \lambda t.$$

Interestingly, we can use (1.2.4) and (1.2.5) to derive the pdf of A_k from the Poisson distribution. From (1.2.5), $P\{N(t) = 0\} = e^{-\lambda t}$ when $k = 0$, and as $A_0 = 0$ by definition, the RHS¹⁰ of (1.2.4) becomes $1 - P\{A_1 \leq t\}$. Consequently, $P\{A_1 \leq t\} = 1 - e^{-\lambda t}$, and by differentiating this, the pdf is seen to be $f_{A_1}(t) = \lambda e^{-\lambda t}$. For $k \geq 1$, take the derivative with respect to t of (1.2.4) to see that $dP\{N(t) = k\}/dt = f_{A_k}(t) - f_{A_{k+1}}(t)$. It then follows from (1.2.5) that

$$\lambda \frac{(\lambda t)^{k-1}}{(k-1)!} e^{-\lambda t} - \lambda \frac{(\lambda t)^k}{k!} e^{-\lambda t} = f_{A_k}(t) - f_{A_{k+1}}(t).$$

With recursion, we retrieve (1.2.3).

THE FAMILY OF rvs $N_\lambda = \{N(t), t \geq 0\}$ is a *Poisson process*¹¹ with *arrival rate* λ . Once we have N_λ , we can define $N(s, t] := N(t) - N(s)$ as the number of customers that arrive in a general time period $(s, t]$.¹²

It is important to know that N_λ has *stationary and independent increments*. Stationarity means that the distributions of the number of arrivals are the same for all intervals of equal length, that is, $N(s, t]$ has the same

⁸ Arrival times are not the same as inter-arrival times.

⁹ See [1.2.8] for details.

¹⁰ RHS := right hand side, LHS := left hand side.

¹¹ A random process is a much more complicated mathematical object than a rv: a process is a (possibly uncountable) set of rvs indexed by time, not just one rv.

¹² Note that $[0, t]$ is closed at both ends, but $(s, t]$ is open at the left.

probability distribution as $N(u, v]$ if $t - s = v - u$. Independence means, roughly speaking, that knowing that $N(s, t] = n$ does not help to make any predictions about the value of $N(u, v]$ if the intervals $(s, t]$ and $(u, v]$ do not overlap.

(1.2.5) implies a number of useful properties of the Poisson process. With small o notation¹³, if $t \ll 1$,

$$\begin{aligned} P\{N(t) = 0\} &= e^{-\lambda t} = 1 - \lambda t + o(t), \\ P\{N(t) = 1\} &= \lambda t e^{-\lambda t} = \lambda t(1 - \lambda t + o(t)) = \lambda t + o(t), \\ P\{N(t) \geq 2\} &= 1 - P\{N(t) = 0\} - P\{N(t) = 1\} \\ &= 1 - (1 - \lambda t) - \lambda t + o(t) = o(t). \end{aligned} \quad (1.2.6)$$

It is important to know that the reverse of the above holds, that is, a stochastic process with stationary and independent increments and satisfying (1.2.6) is necessarily a Poisson process.¹⁴

WHEN WE MERGE two independent Poisson processes N_λ and N_μ , e.g., adults and children entering a store¹⁵, we obtain a new Poisson process $N_{\lambda+\mu}$ with rate $\lambda + \mu$. To see this, we use the above. For $t \ll 1$,

$$\begin{aligned} P\{N_{\lambda+\mu}(t) = 0\} &= P\{N_\lambda(t) = 0\} P\{N_\mu(t) = 0\} \\ &= (1 - \lambda t)(1 - \mu t) + o(t) \\ &= 1 - (\lambda + \mu)t + o(t), \\ P\{N_{\lambda+\mu}(t) = 1\} &= P\{N_\lambda(t) = 1\} P\{N_\mu(t) = 0\} \\ &\quad + P\{N_\lambda(t) = 0\} P\{N_\mu(t) = 1\} \\ &= \lambda t(1 - \mu t) + (1 - \lambda t)\mu t + o(t) \\ &= (\lambda + \mu)t + o(t), \end{aligned} \quad (1.2.7)$$

and the rest of (1.2.6) follows similarly. So we see that we obtain the same type of equations as in (1.2.6) but with λ replaced by $\lambda + \mu$. Realizing that N_λ and N_μ have stationary and independent increments, it follows that $N_{\lambda+\mu}$ is a Poisson process with rate $\lambda + \mu$.

We can also *split*, or *thin*, a Poisson process into several substreams. For instance, consider a Poisson process N_λ of people passing by a shop, and suppose that some arbitrary customer decides to enter the shop as a Bernoulli distributed rv B , independent of N_λ , with success probability p and failure probability $q = 1 - p$. Then the process of customers entering the shop is a Poisson process $N_{\lambda p}$ with rate λp , because, for $t \ll 1$, from (1.2.6),¹⁶

$$\begin{aligned} P\{N_{\lambda p}(t) = 0\} &= P\{N_\lambda(t) = 0\} + P\{N_\lambda(t) = 1\} P\{B = 0\} + o(t) \\ &= 1 - \lambda t + \lambda t q + o(t) = 1 - \lambda p t + o(t), \\ P\{N_{\lambda p}(t) = 1\} &= P\{N_\lambda(t) = 1\} P\{B = 1\} + o(t) = \lambda t p + o(t), \\ P\{N_{\lambda p}(t) \geq 2\} &\leq P\{N_\lambda(t) \geq 2\} = o(t) \end{aligned}$$

The resulting process $N_{\lambda p}$ has an arrival rate of λp and has stationary and independent increments because N_λ is a Poisson process.

The concepts of *merging* and *thinning* are useful to analyze queueing networks. Suppose the departure stream of a machine splits into two

¹³ A function $f(h) = o(h)$ when $f(h)/h \rightarrow 0$ as $h \rightarrow 0$, e.g., $x^2 = o(x)$.

¹⁴ We refer to the literature on (mathematical) probability theory for further background.

¹⁵ Or some other feature by which you want to distinguish between customers.

¹⁶ Why are there $o(t)$ symbols already after the first equality signs? (Hint, it can happen that $N(t) = 2$, but both persons do not enter.)

substreams, e.g., a fraction p of the jobs moves on to another machine and the rest $(1 - p)$ of the jobs leaves the system. Then we can model the arrival stream at the second machine as a thinned stream (with probability p) of the departures of the first machine. Merging occurs where the output streams of various stations are sent to a common downstream station.

A FINAL INTERESTING property of the Poisson process is the distribution of arrivals over an interval $[0, t]$ given that $N(t) = n$.

Theorem 1.2.3. Conditional on the event $N(t) = n$, $t > 0$, the distribution of the arrival times $\{A_i\}_{i=1}^n$ is the same as the distribution of the order statistic of n iid rvs $\sim \text{Unif}([0, t])$.¹⁷

Proof. We write for convenience $dt_i = [t_i, t_i + dt]$, and similarly $d(t_i - t_{i-1}) = [t_i - t_{i-1}, t_i - t_{i-1} + dt]$.¹⁸ We provide the proof for $N(t) = 2$; the proof for the general case is nearly identical, but needs more notation.

Consider the Poisson process at $0 \leq t_1 < t_2 \leq t$. Then,

$$P\{A_1 \in dt_1, A_2 \in dt_2 \mid N(t) = 2\} = \frac{P\{A_1 \in dt_1, A_2 \in dt_2, A_3 > t\}}{P\{N(t) = 2\}}, \quad (\star)$$

because $N(t) = 2 \iff A_2 \leq t < A_3$. For the numerator,

$$\begin{aligned} P\{A_1 \in dt_1, A_2 \in dt_2, A_3 > t\} &\stackrel{1}{=} P\{X_1 \in dt_1, X_2 \in d(t_2 - t_1), X_3 > t - t_2\} \\ &\stackrel{2}{=} \lambda e^{-\lambda t_1} dt \lambda e^{-\lambda(t_2 - t_1)} dt e^{-\lambda(t - t_2)} \\ &\stackrel{3}{=} \lambda^2 e^{-\lambda t} dt^2, \end{aligned}$$

where 1 follows from the recursive relation $A_i = A_{i-1} + X_i$; 2 because the X_i are iid $\sim \text{Exp}(\lambda)$; 3 is algebra. Because $P\{N(t) = 2\} = (\lambda t)^2 e^{-\lambda t} / 2!$, we find that $P\{A_1 \in dt_1, A_2 \in dt_2 \mid N(t) = 2\} = 2! dt^2 / t^2$.

As for the iid uniform rvs, $P\{U_1 \in dt_1, U_2 \in dt_2\} = dt^2 / t^2$. However, there are $2!$ ways that the uniform rvs can hit the set $dt_1 dt_2$, so the probability that the *order statistic* of U_1, U_2 lies in this set is $2! dt^2 / t^2$. This equals the conditional probability in $(\star)$. $\square$

SUMMARIZING, THE POISSON process N_λ and the exponential distribution are very closely related: a counting process $\{N_\lambda(t)\}$ is a *Poisson process* with rate λ if and only if the inter-arrival times $\{X_k\}$ are iid and $X_k \sim \text{Exp}(\lambda)$. Thus, if you find it reasonable to model inter-arrival times as memoryless, then the number of arrivals in an interval is necessarily Poisson distributed. Moreover, if you find it reasonable that the occurrence of an event in a small time interval is equally likely over time and independent from one interval to another, then the arrival process is Poisson, and the inter-arrival times are exponential.

TF 1.2.1. Let N be a Poisson process with rate λ and $0 < s < t$. Claim:

$$\{N(0, s] + N(s, t] = 1\} \cap \{N(0, t] = 1\} = \{N(0, s] = 1\}.$$

TF 1.2.2. Assume that $N_a(t) \sim \text{Pois}(\lambda t)$, $N_s(t) \sim \text{Pois}(\mu t)$ and independent. Claim:

$$P\{N_a(t) + N_s(t) = n\} = e^{-(\mu+\lambda)t} \sum_{i=0}^n \frac{(\mu t)^{n-i}}{(n-i)!} \frac{(\lambda t)^i}{i!}.$$

¹⁷ $\text{Unif}(A)$ is the uniform distribution on the set A .

¹⁸ This notation allows us to write $P\{X_i \in dt_i\} = \lambda e^{-\lambda t_i} dt$ when $X_i \sim \text{Exp}(\lambda)$, and $P\{U_i \in dt_i\} = dt/t$ when $U_i \sim \text{Unif}([0, t])$.

TF 1.2.3. Claim: this program prints 6.

```

1 X = [0, 1, 2, 3, 4, 7]
2 A = [1] * len(X)
3
4 for i in range(2, 5):
5     A[i] += A[i - 1] + X[i]
6
7 print(A[3])

```

TF 1.2.4. The inter-arrival times $\{X_k\}$ are iid and exponentially distributed with mean $1/\lambda$, and $A_k = \sum_{i=1}^k X_i$, $A_0 = 0$. Claim:

$$P\{A_{k+1} \leq t\} = -\frac{(\lambda t)^k}{k!} e^{-\lambda t} + P\{A_k \leq t\}.$$

TF 1.2.5. Claim: this program prints 31.

```

1 tot, thres = 8, 30
2 while tot < thres:
3     tot += 5
4
5 print(tot)

```

THE EXERCISES REQUIRE pen and paper. All problems have solutions, but not all have hints.

Ex 1.2.6. Show that when $X \sim \text{Exp}(\lambda)$ its moment generating function (MGF) is $M_X(t) := E[e^{tX}] = \lambda/(\lambda - t)$, and use this to calculate $E[X]$ and $V[X]$.



Ex 1.2.7. Use MGFs to prove that A_i has density $f_{A_i}(t) = \lambda e^{-\lambda t} \frac{(\lambda t)^{i-1}}{(i-1)!}$.



Ex 1.2.8. When $N \sim \text{Pois}(\lambda)$ and $\alpha > 0$, show that $E[\alpha^N] = e^{\lambda(\alpha-1)}$. Use this to see that $M_{N(t)}(s) = \exp(\lambda t(e^s - 1))$. Then find $E[N(t)]$ and $V[N(t)]$.



Ex 1.2.9. If $X \sim \text{Exp}(\lambda)$, $S \sim \text{Exp}(\mu)$ and are independent, show that $Z = \min\{X, S\} \sim \text{Exp}(\lambda + \mu)$, hence $E[Z] = (\lambda + \mu)^{-1}$.



Ex 1.2.10. If the Poisson arrival processes N_λ and N_μ are independent, show that

$$P\{N_\lambda(t) = 1 \mid N_\lambda(t) + N_\mu(t) = 1\} = \frac{\lambda}{\lambda + \mu}.$$

In other words, given that a customer arrived in $[0, t]$, the probability that the customer is of the first type is $\lambda/(\lambda + \mu)$.

Ex 1.2.11. If $X \sim \text{Exp}(\lambda)$, $S \sim \text{Exp}(\mu)$ and are independent, show that

$$P\{X \leq S\} = \frac{\lambda}{\lambda + \mu}.$$



IF THE SOMEWHAT heuristic derivations above do not satisfy your sense for rigor, then you should solve the next set of exercises; otherwise you can skip them.

Opt 1.2.12. If the Poisson arrival processes N_λ and N_μ are independent, use moment generating functions to show that $N_\lambda + N_\mu$ is a Poisson process with rate $\lambda + \mu$.

Opt 1.2.13. Show with moment-generating functions that thinning the Poisson process N_λ by means of Bernoulli rvs with success probability p results in a Poisson process $N_{\lambda p}$.



1.3 FIGHTING THE EXPONENTIAL DISTRIBUTION IS FUTILE

Perhaps the mathematical arguments presented in the preceding section have not fully convinced you of the universality of the exponential distribution and its counterpart, the Poisson distribution. To enhance your understanding, let us use simulation to see how the exponential distribution emerges as a sort of law of nature.

The simulation is based on an internship of one of my master's students with the task of evaluating the scheduling system for patients with inflammatory bowel disease (IBD) at a hospital. Patients with IBD can be categorized as regular and urgent. Regular patients required check-ups approximately every six months, whereas urgent patients needed to be seen within one week, that is, five working days.⁰ Each consultation takes 10 minutes. Immediately after a consultation, regular patients scheduled their next visit for approximately six months later. There are about 1000 patients in the system and a working year consists of 200 working days. Given that each patient is seen twice per year, a physician needed to consult with an average of 10 patients daily.¹ The problem was to design a planning procedure that allowed enough flexibility to meet the time constraints for the urgent patients without leading to idle times for the physicians. Of course, new patient intakes occurred, but for simplicity we assume that the patient population remains constant.

⁰ This procedure has been changed recently; however, the ideas of this case apply much more generally.

¹ Two visits per year per patient times 1000 patients distributed over 200 days per year is 10 patients per day.

Planning regular patients six months in advance seems to lead to stable arrival times, but this is not true, as there are many practical disturbances. A significant number of patients rescheduled their appointment date, for example, due to altered vacation plans. There were no-shows and yet other patients relocated to other cities; some died. However, even without such practical problems and even when each patient individually shows relatively predictable behavior, the simulation results below show that *at the population level* the inter-arrival times are well modeled as exponentially distributed rvs.

In conclusion, whether the planner plans six months in advance or does not plan at all, there will not be much difference in the variability of the number of patients arriving per day. In other words, the idea that making appointments half a year in advance will reduce variability by a large amount is simply not true. The verdict must then be clear: When rules do not have an effect, it is best to drop them altogether. It is simply better to send patients a reminder for a new appointment three weeks before a new visit is due, rather than planning six months in advance. Far fewer replanning actions are needed, and the probability for no-shows decreases.

WE NOW TURN to developing the simulation to show how the exponential distribution arises 'out of thin air'. We focus only on regular patients and neglect urgent patients as this is a relatively small group. Assume that the hospital has just 50 regular patients. For ease, scale the time so that each patient plans the next appointment uniformly in the interval $[0.9, 1.1]$, independent of previous appointments and other patients.²

The question of interest is to estimate the distribution of inter-arrival times between two patients *as seen by the planner* of the hospital. Of course, with only one patient this is straightforward, the planner sees $\sim \text{Unif}([0.9, 1.1])$. But what happens for many patients?

Fig. 1.3.1 provides the answer to this question, but we first discuss the code that produces this figure, as the code clarifies how to interpret the figure.³

IN PYTHON, WE nearly always start with loading a set of modules.⁴ Modules provide specific functionality not required by every Python program; hence, these should only be loaded when needed. For example, the `numpy` module offers support for numerical work, while `matplotlib` is a plotting library.

```
IBD_simulations.py
1 import numpy as np
2 import matplotlib.pyplot as plt
```

To create a pdf file with a plot that looks good in $\text{\LaTeX}$, we use the `seaborn` library and provide it with some extra options⁵, such as good font sizes.⁶

```
IBD_simulations.py
1 def initialize_plots():
2     import seaborn as sns
3
4     rc = {
5         "text.usetex": True,
6         "font.family": "fourier",
7         "axes.labelsize": 10,
8         "font.size": 10,
9         "legend.fontsize": 8,
10        "xtick.labelsize": 8,
11        "ytick.labelsize": 8,
12    }
13    sns.set(style="whitegrid", rc=rc)
14
15
16 initialize_plots()
```

The simulation environment contains `num_patients` patients and runs for a number of `num_periods` half years. With `default_rng` we can set up a random number generator and by setting a seed⁷ we ensure to always obtain the same random deviates.⁸

```
IBD_simulations.py
1 num_periods = 10
2 num_patients = 50
3 rng = np.random.default_rng(2)
```

² Like this, the inter-arrival times between two consecutive appointments correspond to half a year (100 working days) plus or minus 10 days (observe that $[0.9, 1.1] \times 100 = [90, 110]$).

³ If you don't understand certain commands of python, just consult the web or an LLM.

⁴ Here and elsewhere, we place comments above the code to which they relate. We will not explain commands that are trivial to understand.

⁵ Such configurations are not algorithmic, so you don't have to memorize this.

⁶ While designing a figure, I comment out this code because running $\text{\LaTeX}$ slows down the script.

⁷ Here the seed is 2, but any number would do.

⁸ A random deviate is a sample taken from a distribution; it is not the same as a rv.

We next generate a number of inter-arrival times in the following way. The matrix X has `num_patients` rows and `num_periods` columns. Row k of X corresponds to the inter-arrival times of patient k . Each inter-arrival time is $\sim \text{Unif}([0.9, 1.1])$. With these inter-arrival times, we use (1.2.2) to compute the arrival times for patient k as $A_i^k = A_{i-1}^k + X_i^k$.

The `cumsum` of a matrix along `axis=1` adds elements one by one along a row.⁹ As such, the elements of the first column of A would be equal to the first column of x . To ensure that the first row of A is zero, so that all patients start at time 0, we set the first column of x to 0. However, setting the first column of inter-arrivals X to zero, we effectively remove one arrival. Thus, we should generate `num_periods + 1` inter-arrival times.

```
IBD_simulations.py
1 X = rng.uniform(low=0.9, high=1.1, size=(num_patients, num_periods + 1))
2 X[:, 0] = 0 # set first column of X to 0
3 A = X.cumsum(axis=1)
```

We need an environment to control the plots. The `fig` object can be thought of as a canvas on which we draw the graphs, and `axes` is a list of `ax` objects, where each `ax` represents a single graph within the figure. As is apparent from Fig. 1.3.1, we have a matrix of four `ax` objects. For easy reference in the code below, we flatten¹⁰ the `axes` object and unpack it into the four separate objects `ax1, ..., ax4`.

```
IBD_simulations.py
1 fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(6, 3))
2 ax1, ax2, ax3, ax4 = axes.flatten()
```

Next, we build the upper left panel of Fig. 1.3.1. Each row contains a number of dots, and each dot corresponds to an arrival of a patient.¹¹ As we have 50 patients, we have 50 such rows of dots. The for loop writes the dots of each patient to the upper left panel (which is referenced by `ax1`). To beautify the plot, we give a few options to the plot command. We do not want the dots to be connected by lines, so we set the `linestyle` `ls='None'`.

```
IBD_simulations.py
1 ax1.set_xlim(0, 11)
2 for i in range(num_patients):
3     x = A[i, 1:]
4     y = i * np.ones(num_periods)
5     ax1.plot(x, y, ls='None', marker="o", ms=0.2, c="k")
```

We plot the arrivals as seen by the planner as a jitter plot, that is, we add small random noise to the data points along the y -axis. By spreading out the points slightly, the jitter plot allows for a clearer visualization of the distribution of individual points. We plot the dots at $y = 0$ plus a jittering distributed as $\sim \text{Unif}([-1/2, 1/2])$. Without the jitter, we would just see small line segments instead of somewhat rectangular shapes.

```
IBD_simulations.py
1 for i in range(num_patients):
2     x = A[i, 1:]
3     y = rng.uniform(-0.5, 0.5, size=num_periods) # add jitter
4     ax1.plot(x, y, ls='None', marker="o", ms=0.2, c="k")
```

⁹ The `cumsum` of the sequence of numbers 1, 3, 5 is 1, 4, 9. Hence, if we want to implement $A_0 = 0$, we set just set $X_0 = 0$ and keep $X_1 = 3, X_2 = 5$, so that $[0, 3, 5].\text{cumsum}()$ expands to $[0, 3, 8]$.

¹⁰ Ask an LLM for further explanation.

¹¹ The x coordinate of the i th dot of patient k is A_i^k and its y coordinate is k .

OUR NEXT TASK is to estimate the density of inter-arrival times as observed by the planner and plot this in the right panel of Fig. 1.3.1. Recall that each row of the matrix A contains the arrival times of one specific patient. By flattening A we obtain all arrival times in a long list. By sorting, we obtain the arrival times in order of time. Finally, the inter-arrival times for the planner are the times between the elements of this list. In other words, if the arrival times $\{A_k\}$ are known, the k th inter-arrival time is given by

$$X_k = A_k - A_{k-1}. \quad (1.3.1)$$

```
IBD_simulations.py
1 A = np.sort(A.flatten())
2 X = A[1:] - A[:-1]
```

We assemble all inter-arrival times of X in 50 bins. On average, we have $\lambda = 50$ arrivals per unit time; thus, the mean inter-arrival time is $1/\lambda = 1/50$. Next, recall that the pdf of the exponential distribution is $\lambda e^{-\lambda x}$. To plot the pdf, we take as support for the plot 5 times the mean time between arrivals, that is, $5/\lambda = 0.1$. We then plot the histogram of the inter-arrival times and the pdf on `ax2`, which corresponds to the upper right panel. The argument `density = True` normalizes the area of the bars in the histogram to one, enabling us to compare the histogram with the plot in the pdf.

```
IBD_simulations.py
1 bins = np.linspace(0, 5 / num_patients, 50)
2 exp_density = num_patients * np.exp(-num_patients * bins)
3 ax2.hist(X, bins, density=True, color="k")
4 ax2.plot(bins, exp_density, ":", c="k", lw=0.75, label="pdf")
5 ax2.legend()
```

WHAT CAN WE see in the upper panels of Fig. 1.3.1? In the upper left panel, the arrival times of each patient are at first tightly concentrated around 1, but as time progresses, the arrival times become more spread out. The jitter plot around $y = 0$ also demonstrates this. The rectangles become wider until they almost overlap. In the right upper panel, we see that in comparison to the exponential density, there are relatively many short inter-arrival times. This is expected because all patients start at the same time, so arrival times will be concentrated for the first couple of visits.¹² Therefore, letting all patients start at the same time is not a good model for a hospital that has been serving patients for years.

To compensate for these unnatural start times, we set the initial arrival time of each patient as uniformly distributed on $[0, 1]$. With this change, we run the above code again and plot the results in `ax3` and `ax4` to obtain the lower left and right panels of Fig. 1.3.1. Now we see that the jitter rectangles, that is, the arrivals as seen by the hospital, overlap right from the start, and the exponential distribution is an excellent fit.

The next code saves the figure. The `tight_layout()` function ensures that the axes labels are nicely formatted.

```
IBD_simulations.py
1 fig.tight_layout()
2 fig.savefig("/home/nicky/Dropbox/vakken/qts/stochastic_or/figures/IBD_exponential.pdf")
3 fig.savefig("/home/nicky/Dropbox/vakken/qts/stochastic_or/figures/IBD_exponential.png", dpi=300)
```

¹² But even with these unrealistic starting times, the exponential distribution is not a bad fit.

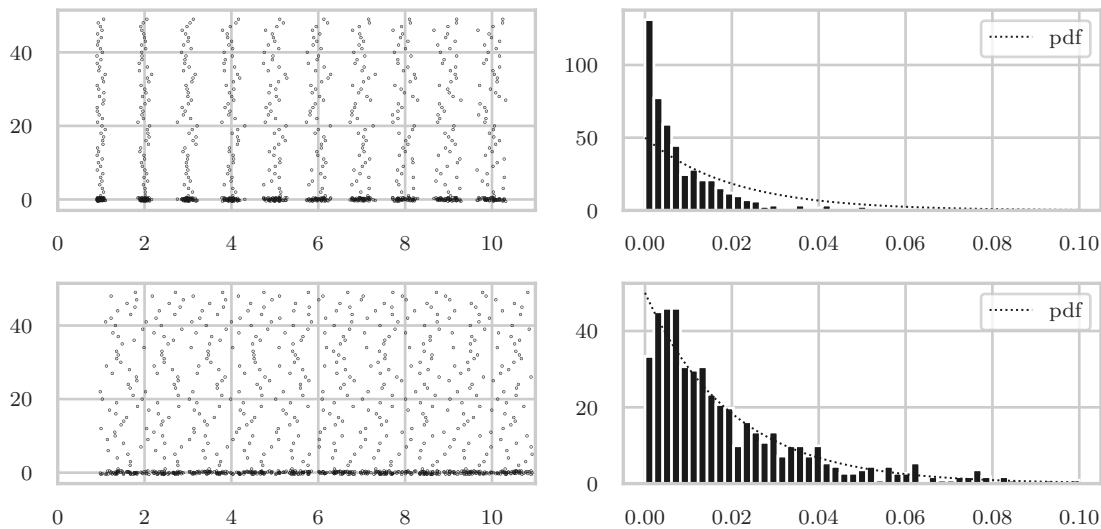


FIG. 1.3.1: How the uniform distribution leads to the exponential distribution.

TO CONCLUDE, IF many patients arrive at the hospital and each contributes some small variation to the inter-arrival times, the hospital sees exponential inter-arrival times for the population. Clearly, this is not specific to hospitals: It applies to any system that serves a large pool of customers, each arriving with some variation. To avoid any confusion, here we model the inter-arrival times for one patient as uniformly distributed, but the same phenomenon occurs when the inter-arrival times for one patient have other types of distribution.

TF 1.3.1. Consider the following chunk of code, where the i th row of X corresponds to the inter-arrival times of the i th patient to a hospital.

```

1 import numpy as np
2
3 rng = np.random.default_rng(3)
4 N, T = 5, 8
5 a, b = 0, 3
6 X = rng.uniform(low=a, high=b, size=(N, T))
7 X[:, 0] = 0
8 A = X.cumsum(axis=1)

```

Claim: the i -th row of A contains the T arrival times of the i -th patient.

TF 1.3.2. Claim: this program prints 22.

```

1 x = [3 * i + 1 for i in range(10)]
2 print(x[8])

```

TF 1.3.3. Claim: the keys and the values of the dict a are 8, 13 and 18.

```

1 a = {}
2 start, thres = 3, 19
3 while True:

```

```

4     start += 5
5     if start >= thres:
6         break
7     a[start] = start
8
9     print(a.keys())

```

TF 1.3.4. Claim: this program prints 64.

```

1  def doit(a):
2      match a:
3          case int():
4              return 8 * a
5          case float():
6              return 9 * a
7          case _:
8              raise ValueError("Unknown type passed")
9
10 print(doit("8"))

```

TF 1.3.5. Given that the function `uniform()` returns a uniform random number on the interval $(0, 1)$ then the following function generates a random variate with a Poisson distribution with parameter $\lambda = \text{src_python}\{\text{labda}\}$.

```

1  def Pois(labda):
2      R = 0
3      T = - np.log(uniform())
4      while T < labda:
5          R += 1
6          T += -np.log(uniform())
7      return R

```

IT REMAINS TO formalize the observations from the above plots. In fact, we should distinguish between two limiting regimes. Patients recurrently plan appointments, separated around some mean period, that is, roughly every half a year. The upper left panel in Fig. 1.3.1 suggests that if the number of appointments increases, the appointments for each patient eventually become uniformly distributed on the period length (here half a year). The lower left panel suggests that if the appointments are uniformly distributed to begin with, they remain uniformly distributed. So, our first step is to prove the claim that in the limit $t \rightarrow \infty$, stochastically recurring inter-arrival times become uniformly distributed over an interval. The second claim is that, in the limit of many patients, each arriving at a uniformly distributed time in an interval, the inter-arrival times of all patients together become exponentially distributed, in a scaled sense.

The theorems and proofs are elegant and combine order statistics, the beta distribution, first-step analysis, and some infinitesimal calculus to give the reasoning a heuristic flavor. Finally, we discuss an important smoothing property of taking expectations. This idea is also relevant in data science when moving averages are used. However, if you are not interested in probability theory, skip the rest of the section.

LET US FIRST show that periodically planned arrivals, with some stochasticity, tend to become uniformly distributed over an interval. To formalize this claim, we consider a discrete-time model. There are on average C days between two successive visits of a patient (for the moment, the time on a day is of no importance). For instance, let the inter-arrival times X_i be such that $P\{X_i = C - 1\} = P\{X_i = C + 1\} = 1/2$. In this case, if the process starts on day 0 and $X_1 = C - 1$, the next visit will be on day $C - 1$, but if $X_1 = C + 1$, the next visit will be on day 1 of the next cycle. Looking at the arrival process like this, the patient *moves on a circle* with states $\{0, 1, \dots, C - 1\}$, and with every visit the patient moves with probability $1/2$ one state to the left or to the right. Thus, when $X = C - 1$, we can just as well model this as $X = -1$.¹³

Theorem 1.3.1. Assume that the arrival times $\{A_i\}$ live on the circle space $\{0, 1, \dots, C - 1\}$, i.e., $A_i = (A_{i-1} + X_i) \bmod C$. If $\{X_k\}_{k=1}^\infty$ is a set of iid rvs such that $E[X_k] < \infty$ and $\sup_j P\{X_k = j\} < 1$, then $\alpha_k(i) = P\{A_k = i\}$, i.e., the probability that the k th arrival is on the i th day, converges to $1/C$ as $k \rightarrow \infty$.

Proof. We impose the condition $E[X_k] < \infty$ to prevent having to deal with $A_k = \infty$ for some finite k , and we need $\sup_j P\{X_k = j\} < 1$ to exclude trivial rvs.¹⁴

For the proof, assume for the moment that $P\{X_k = 1\} = p \in (0, 1)$ and $P\{X_k = -1\} = q = 1 - p$, and let C be odd.^{??}¹³ By first-step analysis,

$$\alpha_{k+1}(i) = p\alpha_k(i - 1) + q\alpha_k(i + 1).$$

Clearly, if $\alpha_k(i) = 1/C$ for all i , then $\alpha_{k+1}(i) = 1/C$ also. However, if not all $\alpha_k(\cdot)$ are equal, there must be at least one state i such that $\alpha_k(i)$ is strictly larger than at least one of its neighbors, that is, $\alpha_k(i) \geq \max\{\alpha_k(i - 1), \alpha_k(i + 1)\}$ and $\alpha_k(i) > \min\{\alpha_k(i - 1), \alpha_k(i + 1)\}$.¹⁵ But then, on the next visit,

$$\alpha_{k+1}(i) = p\alpha_k(i - 1) + q\alpha_k(i + 1) < \alpha_k(i).$$

In words, as long as not all $\alpha_k(i)$ are equal for all i , after each visit the maximal α_k becomes lower, and by symmetry the minimal α_k becomes higher. As the minimum and maximum cannot cross after any step, all $\alpha_k(i)$ have to converge to the same value as $k \rightarrow \infty$. Since there are C states, $\lim_{k \rightarrow \infty} \alpha_k(i) \rightarrow 1/C$ for all $i \in \{0, 1, \dots, C - 1\}$.

The argument is easy to generalize. Let $P\{X = j\} = p_j$. By assumption $p_j \in (0, 1)$. Then, $E[\alpha_k(i + X)] = \sum_j p_j \alpha_k(i + j) < \alpha_k(i)$ if $\alpha_k(i) \geq \alpha_k(\cdot)$ when there is at least one j such that $\alpha_k(i + j) < \alpha_k(i)$ and $p_j > 0$. And similarly for the minimum. In total, this implies that $\alpha_k(i) \rightarrow 1/C$ for all i as $k \rightarrow \infty$.

Clearly, the proof applies to any finite $C = 1, 2, \dots$. To see how to generalize the argument to the circle $[0, 1)$ on the reals, consider a continuous function f on the unit circle C . As the circle is compact and f continuous, f achieves its maximum $f(\bar{x})$ at $\bar{x}$, say. Suppose for some $\epsilon > 0$ there is set $A_\epsilon(\bar{x}) = \{x \in C : f(x) \leq f(\bar{x}) - \epsilon\}$ such that $P\{A_\epsilon(\bar{x})\} > 0$. (If there is no such set for any ϵ , then f is a constant.) But then $E[f(x + X)] \leq (f(\bar{x}) - \epsilon)P\{A_\epsilon(\bar{x})\} + f(\bar{x})(1 - P\{A_\epsilon(\bar{x})\}) < f(\bar{x})$. Again, taking an average over f makes the maximum smaller and the minimum larger. $\square$

Applied to the hospital, after many visits of a patient—each roughly half a year apart—the visits will appear to be uniformly distributed on a ‘circle’ of half-year length.

¹³ There is a small subtlety when the cycle length is even and $P\{X = C - 1\} = P\{X = C + 1\} = 1/2$. As an even number is a multiple of 2, the odd (even) days can only be visited after an odd (even) number of visits. The problem is easily resolved by assuming, in addition, that C is odd or $P\{X = 0\} > 0$.

¹⁴ A rv X is trivial when $P\{X = a\} = 1$ for some a .

¹⁵ Observe that, for this argument, it is necessary that C is finite.

There is one point that remains about the lower left panel of Fig. 1.3.1; recall that for this panel we started the simulation with uniformly distributed arrival times. What is the distribution of the arrival times *on the circle* if we start like this? Let us analyze this for the case in which the inter-arrival times are uniform on $[0, 1]$ and we take the circle as $C = [0, 1)$. Observe that when $C = [0, 1)$, $U + V \bmod 1$, that is, the sum of two iid uniform rvs on the circle, is just the fractional part of $U + V$.

Theorem 1.3.2. Let $Z = \sum_{i=1}^n U_i$ for n iid rvs $U_i \sim \text{Unif}([0, 1))$. The fractional part of Z , i.e., $Z - \lfloor Z \rfloor$, is uniformly distributed on $[0, 1)$.¹⁶

¹⁶ $\lfloor x \rfloor$ is the integer part of $x \in \mathbb{R}$.

Proof. Realizing that the density of the sum of two uniform rvs on $[0, 1]$ is a triangle with base $[0, 2]$ and height 1,

$$\begin{aligned} P\{U + V - \lfloor U + V \rfloor \leq x\} &= P\{U + V \leq x\} + P\{1 \leq U + V \leq 1 + x\} \\ &= x^2/2 + (1/2 - (1 - x)^2/2) = x. \end{aligned}$$

Therefore, the fractional part of the sum of two uniform rvs is uniform on $[0, 1]$. We can apply this result successively through the summation in Z . $\square$

We now deal with the case with many patients, that is, when the number n of patients goes to infinity. By the above results, we model the arrival times as iid uniform rvs $\{U_k\}_{k=1}^n$ such that $U_k \sim \text{Unif}([0, 1))$. Recall that in the simulation we sorted the arrival times of all patients together to obtain the ordered sequence of arrivals as seen by the hospital. In probabilistic terms, the hospital sees *the order statistic* of $\{U_k\}_{i=1}^n$. Write this as $0 = A_0^n < A_1^n < A_2^n < \dots < A_n^n < A_{n+1}^n = 1$.¹⁷ Define the size of the inter-arrivals as seen by the hospital as $X_k^n = A_k^n - A_{k-1}^n$, $k = 1, \dots, n + 1$.

¹⁷ We neglect the probability of simultaneous arrivals; these have probability zero.

As the population size n increases, the size of the first gap, i.e., the properly scaled arrival time of the first patient, becomes increasingly like an exponentially distributed random variable.¹⁸

¹⁸ We need to scale with a factor n because the number of patients increases.

Lemma 1.3.3.

$$\lim_{n \rightarrow \infty} P\{A_1^n \leq x/n\} = \lim_{n \rightarrow \infty} P\{X_1^n \leq x/n\} = 1 - e^{-x}.$$

Proof. The probability that the smallest of n rvs is less than some x is the same as 1 minus the probability that all n rvs are larger than x . Therefore,

$$P\{A_1^n \leq x/n\} = 1 - P\{\min\{U_1, \dots, U_n\} > x/n\} = 1 - (1 - x/n)^n,$$

because $P\{U_i > x/n\} = 1 - x/n$, and the U_i are iid. The RHS converges to $1 - e^{-x}$ as $n \rightarrow \infty$. Finally, from the definition, $X_1^n = A_1^n - A_0^n = A_1^n$. $\square$

Next, we show that all the hospital inter-arrival times have the same density.

Lemma 1.3.4. For $r \in (0, 1)$, the density of the k th inter-arrival time is $f_{X_k^n}(r) = n(1 - r)^{n-1}$, for $k = 1, \dots, n$.

Proof. From the proof of the previous lemma we have $P\{X_1^n \leq r/n\} = 1 - (1-r)^n$, therefore $f_{X_1^n}(r) dr = P\{X_1^n \in [r, r+dx]\} = n(1-r)^{n-1} dr$. By symmetry, $X_1^n \sim X_{n+1}^n$. It remains to deal with the intermediate inter-arrivals.

The probability of the event $A_1^n < A_2^n < \dots < A_{k-2}^n < x$, $A_{k-1}^n \in [x, x+dx]$, $A_k^n \in [y, y+dy]$, $y < A_{k+1}^n < \dots < A_n^n$ is given by

$$f_{A_{k-1}^n, A_k^n}(x, y) dx dy = \frac{n!}{(k-2)!(n-k)!} x^{k-2} (1-y)^{n-k} dx dy,$$

because $k-2$ arrivals must occur before k , $n-k$ after y and one arrival in $[x, x+dx]$ and one in $[y, y+dy]$. With this, the probability that $X_k^n \in [r, r+dr]$ becomes

$$f_{X_k^n}(r) dr = dr \int_0^{1-r} f_{A_{k-1}^n, A_k^n}(x, x+r) dx,$$

because $X_k^n = A_k^n - A_{k-1}^n = r \implies y = A_k^n = A_{k-1}^n + r = x + r$, and x can lie anywhere in $[0, 1-r]$.

Substituting in this integral the density $f_{A_{k-1}^n, A_k^n}$, but dropping the factorials for the moment for notational ease, we find

$$\begin{aligned} \int_0^{1-r} x^{k-2} (1-r-x)^{n-k} dx &= (1-r)^{n-1} \int_0^{1-r} \left(\frac{x}{1-r}\right)^{k-2} \left(\frac{1-r-x}{1-r}\right)^{n-k} dx \\ &= (1-r)^{n-1} \int_0^1 x^{k-2} (1-x)^{n-k} dx \\ &= (1-r)^{n-1} \frac{(k-2)!(n-k)!}{(n-1)!}, \end{aligned}$$

where we use the normalization constant of the Beta distribution to compute the last integral. Canceling the factorials proves the claim. $\square$

Combining the above two lemmas leads straightaway to the next theorem.

Theorem 1.3.5. Let n jobs arrive at a station such that the arrival times are iid rvs $U_k \sim \text{Unif}([0, 1])$. Then all inter-arrival times $\{X_k^n\}_{k=1}^{n+1}$ as seen by the server (the hospital) are iid and $\lim_{n \rightarrow \infty} P\{nX_k^n \leq x\} = 1 - e^{-x}$, i.e., exponentially distributed.

Proof. By Th. 1.3.4, all rvs $\{X_k^n\}_{k=1}^n$ have the same density $n(1-r)^{n-1}$. By Th. 1.3.3, $\lim_{n \rightarrow \infty} P\{nX_k^n \leq x\} = 1 - e^{-x}$ for each k . $\square$

1.4 PROBABILISTIC ARITHMETIC

In Section 1.2 we were interested in how the visiting times of a patient would become distributed over periods of half a year after many visits. We can formalize this problem as follows for one patient. Let X_k be the k th inter-arrival time, and write C for the length of the interval in which the patient has to see a doctor; here C is half a year. Then the times at which the arrivals are distributed over the periods behave like

$$Z_k = Z_{k-1} + X_k \bmod C, \quad Z_0 = 0.$$

To compute the distribution of Z_k we need to be able to compute the distribution of $Y_k = Z_{k-1} + X_k$, and then find the distribution of $f(Y_k)$ where $f(x) = x \bmod C$. Clearly, this is a very unattractive task to carry out by hand. In this section we develop Python code so that the computer can do all this tedious work.

More generally, in later sections, we will use a considerable amount of numerical work to obtain insight into the behavior of stochastic processes. For this we will use a Python class that allows us to do *probabilistic arithmetic*. By focusing on coding *patterns* we can achieve that and, in the meantime, turn coding into an interesting intellectual challenge. At the end of this section, we use this code to provide numerical insight into the distribution of $\{Z_k\}$ discussed above.

LET US START by making explicit what we mean by a pattern. As a first pattern, consider the $+$ operator. Suppose that we only know how to do addition on the positive integers.⁰ Observe that the definition of addition on integers cannot be directly applied to rationals. To add fractions, we need to extend the symbol $+$ to the following procedure: make the denominators of the fractions identical, and then add the numerators.¹ However, this procedure does not work to compute $\sqrt{2} + \sqrt{3}$.² To add general real numbers, we need to extend the definition of $+$ yet further.³ Next, we need to define addition for complex numbers, vectors, matrices ... In summary, when learning mathematics, we have been silently *overloading* the $+$ operator to many different situations.

In our present case, we *write* $X + Y$ for two independent rvs to mean the convolution:

$$P\{X + Y = z\} = \sum_i \sum_j p_i q_j \mathbb{1}_{i+j=z}, \quad (1.4.1)$$

where $p_i = P\{X = i\}$, $q_j = P\{Y = j\}$ and the indicator $\mathbb{1}_A = 1$ if the event A occurs, and 0 otherwise. Below we implement this in code.⁴

Thus, the first pattern that helps us to reason in more abstract ways about problems is to use *operator overloading*.

This brings us to a second pattern: observe that the expression $X + Y$ combines two rvs X and Y and then applies some function. Similarly, the operations $X - Y$, XY , $\max\{X, Y\}$ are all *functions applied to two independent rvs*. What we therefore need is a *method* to compute the pmf of a new rv $f(X, Y)$, where f is some general *binary* function.⁵

We identify a third pattern when we realize that to compute $V[X]$, we need $E[X^2]$. But $E[X^2]$ is the expectation of the rv $g(X)$ where $g(x) = x^2$. So, we also need a method to compute the pmf of the rv $g(X)$, where now g is some general *unary* function.⁶

In summary, we want to build code that supports for rvs:

1. operator overloading for addition, subtraction, and potentially other arithmetical operations,
2. the application of a unary function to obtain a new rv
3. the application of a binary function.

⁰ For example: $3 + 5 = 8$.

¹ $1/3 + 1/2 = 2/6 + 3/6 = (2 + 3)/6 = 5/6$.

² $\sqrt{2}$ and $\sqrt{3}$ are not rationals so we cannot make the denominators equal.

³ One way is like this: $\sqrt{2} + \sqrt{3} = \sup\{x + y : x, y \in \mathbb{Q}, x^2 \leq 2, y^2 \leq 3\}$.

⁴ For addition there exists a much faster method based on Fast Fourier transforms.

⁵ A binary function depends on two arguments.

⁶ A unary function depends on just one argument.

We call these procedures probabilistic arithmetic, and we will build, step by step, elegant and generic code to support this. However, we first need to introduce some computer science concepts.

A USEFUL DATA structure to store the pmf of discrete rvs is a dictionary.

```
1 X = {1: 1/2, 2: 1/2}
2 Y = {1: 1/2, 2: 1/2}
```

It is obvious that $Z = X + Y$ should be $Z = \{2: 1/4, 3: 1/2, 4: 1/4\}$. Let us use (1.4.1) to compute the pmf of $Z = X + Y$. Observe that the result of this code is wrong: the pmf of $Z = 3$ is not $1/2$.

```
1 Z = {}
2 for i, p in X.items():
3     for j, q in Y.items():
4         Z[i + j] = p * q
5 print(f"{Z.keys()}")
6 print(f"{Z.values()}")
```

```
1 Z.keys()=dict_keys([2, 3, 4])
2 Z.values()=dict_values([0.25, 0.25, 0.25])
```

The problem is that in line 4 $Z[3]$ gets overwritten by the $=$ operator.⁷ Actually, we need something that adds probabilities, rather than just assigning them, as in the code above. The correct data structure is a `defaultdict(float)` which provides a default value of 0 when a key in the dict is missing.

⁷ The $=$ is an assignment operator!

```
1 from collections import defaultdict
2
3 Z = defaultdict(float)
4 for i, p in X.items():
5     for j, q in Y.items():
6         Z[i + j] += p * q # note the +=
7 print(f"{Z.keys()}")
8 print(f"{Z.values()}")
```

```
1 Z.keys()=dict_keys([2, 3, 4])
2 Z.values()=dict_values([0.25, 0.5, 0.25])
```

Now the result is correct. Note that we use the operator $+=$, a convenient way to perform addition in place. To see why it is useful, consider the following code. In particular, the third line is a much cleaner (shorter) way to add 3 to the variable's value.

```
1 very_long_variable = 3
2 very_long_variable = very_long_variable + 3
3 very_long_variable += 3 # compare this to the line above
```

THE ABOVE CODE works in principle, but handling the pmf as a dictionary requires some care. Here is an example to understand the problem. We all agree that $1/3 = 1 - 2/3$, so let us see what happens if we run the following code.

```

1 Z = defaultdict(float)
2 Z[1/3] = 1
3 Z[1 - 2 / 3] += 1
4 print(Z.keys())

```

```

1 dict_keys([0.3333333333333333, 0.3333333333333337])

```

The dictionary `z` suddenly contains two elements instead of just one. Inspecting the last digit of the keys, we see that in one number the last digit is 7 and in the other it is 3. So, for a *computer* $1 - 2/3 \neq 1/3$.⁸

To avoid this, we round the elements of the support to a fixed precision and use fractions, instead of floats, as keys.

```

1 from fractions import Fraction
2
3 a, b = 1 / 3, 1 - 2 / 3
4 af = Fraction(a).limit_denominator(1000)
5 bf = Fraction(b).limit_denominator(1000)
6 print(a == b) # this should be False
7 print(af == bf) # this should be True

```

```

1 False
2 True

```

In the code below, we will use the so-called λ function. Sometimes functions are so short and simple that we simply do not want to bother to give them a name. For such cases, the λ function serves as an *anonymous function*. Note that to apply the function `lambda x: 3 * x` to the number 8, we must place the expression in parentheses.

```

1 print((lambda x: 3 * x)(8))

```

```

1 24

```

Sometimes we need to add many rvs, in particular we would like to be able to code the equivalent of $\sum_{i=1}^k X_i$. Let us first see how this works in Python for normal numbers.

```

1 print(sum(x for x in [1, 2, 3]))

```

```

1 6

```

⁸ This is not something specific to Python; the problem lies in the fact that most real numbers cannot be represented as floating point numbers.

However, there is a small catch. The `sum` needs a starting element. To see why, consider the following code and output; we will discuss both next.

```
1 print(sum(x for x in "dog"))
```

```
1 TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

So, we pass `x` as a string, and then `sum` fails, indicating that it cannot add an integer and a string. But where does the `int` in the error text come from? As it turns out, `sum` starts with a default value 0 and then sums from left to right. In other words, the above summation over the list `[1, 2, 3]` expands to `0 + 1 + 2 + 3`. This clarifies the error: the sum `0 + "dog"` is not well defined. Below we point out when this problem becomes relevant.

A final general concept we will use is *caching*. By caching, it is possible to store the result of a function in a dictionary rather than evaluating the function body again on a second call to the function. For simple functions, this is not necessary, but when the evaluation of the function requires substantial numerical work, caching decreases computation times beyond imagination (milliseconds instead of centuries for the same result).

```
1 from functools import cache
2
3 @cache
4 def f(x):
5     print("The body of the function is called.")
6     return 8 * x
7
8
9 print(f(3)) # first call: run function body
10 print(f(3)) # second call, retrieve from memory, don't run body
11 print(f(4)) # new argument: run function body
12 print(f(4)) # second call with argument, don't run body
```

```
1 The body of the function is called.
2 24
3 24
4 The body of the function is called.
5 32
6 32
```

Clearly, the print statement in the function body is only run when the function receives a new argument. Thus, `@cache` stores for a given value⁹ the output of `f` and does not evaluate the body of `f` for a second time when the function receives the same value for the argument.

⁹ Here 3 and 4.

WE NOW TURN to building a Python class to work conveniently with rvs.

We start with loading a number of modules that provide some basic functionality.

```

1 import operator
2 from collections import defaultdict
3 from fractions import Fraction
4
5 import numpy as np
6 from numpy.random import default_rng
7
8 from functions import normalize

```

The meaning of the next variables will become clear later. The function `toFrac` is a convenience function to hide how to convert a float to a fraction of the desired precision. If we want to change the precision at a later stage, or want to change the conversion, we only have to change the code here.

```

1 max_denominator = 1_000_000
2 thres = 1e-16 # Reject probabilities smaller than this.
3 seed = 3 # For the random number generator.
4
5
6 def toFrac(x):
7     "Convert x to fraction of specified precision."
8     return Fraction(x).limit_denominator(max_denominator)

```

The `normalize` function works like this:

```

1 def normalize(pmf):
2     norm = sum(pmf.values())
3     # no small prob events.
4     return {k: nv for k, v in pmf.items() if (nv := v / norm) > thres}

```

The `RV` class, which we build in the following code blocks, stores the pmf and the support of a rv as a dictionary; for instance, `{0: 1 / 3, 1: 2 / 3}` represents a biased coin with support `{0, 1}`, pmf $p_0 = 1/3$, and $p_1 = 2/3$. `RV` uses `_pmf`, `_support` and `_cdf` as private members.

```

1 class RV:
2     "A random variable whose keys form the support and values the pmfs."
3
4     def __init__(self, pmf):
5         self._pmf = self.make_pmf(pmf)
6         self._support = np.array(sorted(self._pmf.keys()))
7         self._cdf = np.cumsum([self._pmf[k] for k in self._support])

```

To make the pmf we use `Fraction` and `defaultdict` for the reasons mentioned earlier. As we do not include outcomes whose probabilities are too small, we must normalize at the end.

```

1 def make_pmf(self, pmf):
2     res = defaultdict(float)
3     for k, pk in pmf.items():
4         res[toFrac(k)] += pk
5     return normalize(res)

```

The next methods provide access to the pmf and the support, and the number of elements in the pmf. When x is not in the support, the pmf is 0.

```

1  def pmf(self, x):
2      return self._pmf.get(toFrac(x), 0)
3
4  def support(self):
5      return self._support
6
7  def __len__(self):
8      return len(self._support)
9
10 def __repr__(self):
11     return ", ".join(f"{k}: {self._pmf[k]}" for k in self._support)

```

Computing the cdf with binary search via `np.searchsorted` works really fast. With the cdf, the survivor function follows immediately. We do not need to cache the `sf`, because `cdf` already uses caching.

```

1  @cache
2  def cdf(self, x):
3      if x < self._support[0]:
4          return 0
5      if x >= self._support[-1]:
6          return 1
7      return self._cdf[np.searchsorted(self._support, x)]
8
9  def sf(self, x):
10     "Survivor function"
11     return 1 - self.cdf(x)

```

The computation of the mean, variance, and standard deviation becomes easy once we realize that all these functions can be expressed in terms of the *pattern* $E[f(X)]$ for suitably chosen functions f . In passing, we remark that by taking $f(x) = \mathbb{1}_{x \leq y}$, the cdf $F(x) = E[f(x)] = \sum_k \mathbb{1}_{k \leq x} p_k$. However, this is not efficient as compared to the binary search used above. The typing `f: Callable[numeric, numeric]` means that the function `f` is supposed to map a number to a number.

```

1  def E(self, f):
2      "Compute E(f(X))"
3      return sum(f(i) * self.pmf(i) for i in self.support())

```

With the expectation defined, the mean, variance and standard deviation follow from simple λ functions.

```

1  def mean(self):
2      return self.E(lambda x: x)
3
4  def var(self):
5      return self.E(lambda x: x**2) - self.mean() ** 2
6
7  def std(self):
8      return np.sqrt(self.var())

```

The method `sortedsupport` of `RV` sorts the elements in the support in decreasing size of probability mass. We explain below why we need this.

```

1  @cache
2  def sortedsupport(self):
3      "Return the support sorted in decreasing order of the pmf."
4      return np.array(sorted(self._pmf, key=self._pmf.get, reverse=True))

```

Besides this general functionality, we want to use our `RV` class to generate random deviates that are distributed according to the cdf F . Recall that if $U \sim \text{Unif}([0, 1])$, then $F^{-1}(U)$ has the required distribution, because

$$P\{F^{-1}(U) \leq x\} = P\{U \leq F(x)\} = F(x),$$

where the last equality follows from the fact that U is uniform on $[0, 1]$.

```

1  def rvs(self, size = 1, random_state=default_rng(seed)):
2      "Generate an array with 'size' number of random deviates."
3      U = random_state.uniform(size=size)
4      pos = np.searchsorted(self._cdf, U)
5      return self.support()[pos].astype(float)

```

We are almost finished with the class definition of `RV`, but we need two additional elements. To add or subtract two instances of `RV`, the class needs to know what to do when writing code like $X + Y$ or $X - Y$. The private method `__add__` achieves this by calling `map2`, which is defined below to compute $f(X, Y)$ for general f . Subtraction uses `__sub__`, and implements it as $X + (-Y)$. We explain the convert function below.

```

1  def __add__(self, other):
2      other = convert(other)
3      return map2(self, other, operator.add)
4
5  def __neg__(self):
6      return RV({-k: self.pmf(k) for k in self.support()})
7
8  def __sub__(self, other):
9      return self + (-other)
10
11 def __truediv__(self, other):
12     other = convert(other)
13     return map2(self, other, operator.truediv)
14
15 def __mul__(self, other):
16     other = convert(other)
17     return map2(self, other, operator.mul)

```

Next, we want to support the `+=` and `-=` operators, so that we can write $X -= Y$. At a later stage, we might also want to add many rvs with code such as `sum(X for _ in range(5))`. To support this, we must provide `sum` with an initial element to start the summations; recall the earlier `TypeError`. The next two methods enable this functionality for both cases.

```

1  def __radd__(self, other):
2      # support sum([rv for i in ...])
3      return self.__add__(convert(other))
4
5  def __rsub__(self, other):
6      return convert(other).__sub__(self) # mind the sequence of b - a

```

As a last feature, how to check that two rvs are equal? This is easy: when the dictionaries that store the pmfs are equal. Note that when overloading the method `__eq__`, it is necessary to implement the `__hash__` method too. (Ask an LLM why.)

```

1  def __eq__(self, other):
2      return self._pmf == other._pmf
3
4  def __hash__(self):
5      return id(self)

```

This nearly finishes the RV class; the `map` method follows below.

THE TWO REMAINING patterns are binary and unary functions. Supporting binary functions works the same as addition in (1.4.1). Observe that we break the inner `for` loop when the probability becomes too small. This rule assumes that the order in which the elements in the support pass by is in decreasing order of pmf. The method `sortedsupport` of RV ensures that this is the case.

```

1  def map2(X, Y, f):
2      "Make the rv f(X, Y) for the independent rvs X and Y."
3      c = defaultdict(float)
4      for i in X.sortedsupport():
5          for j in Y.sortedsupport():
6              p = X.pmf(i) * Y.pmf(j)
7              c[f(i, j)] += p
8              if p <= thres:
9                  break
10     return RV(c)

```

Here is how to turn the application of a unary function `f` to a RV into a RV.

```

1  def map(self, f):
2      "Make the rv f(X)."
3      c: defaultdict[B, float] = defaultdict(float)
4      for k in self.support():
5          c[f(k)] += self.pmf(k)
6      return RV(c)

```

Finally, what would happen if we write `2 + x` to mean that the support of `x` needs to be shifted two steps to the right? Observe that we are once again overloading the `+` operator: it adds an integer to a RV. Actually, we can handle this in our framework by interpreting the number 2 as a trivial

rv Y , i.e., with all probability mass concentrated on the number 2. So, if we convert ints and floats, by means of a function `convert`, into a RV, we are done. Observe that we already used `convert` in `__add__` and `__sub__`.

```

1 def convert(rv):
2     "Check and convert to rv if necessary."
3     match rv:
4         case RV():
5             return rv
6         case int():
7             return RV({rv: 1}) # An int is a shift.
8         case float():
9             return RV({rv: 1}) # A float is a shift too.
10        case _:
11            raise ValueError("Unknown type passed as a RV")

```

WITH THIS CLASS we can study the content of Th. 1.3.1 in numerical terms. More specifically, how fast do the arrival times on the circle converge to the uniform distribution? Recall that the arrival times on the circle are defined as $A_k = A_{k-1} + X_k \bmod C$. Thus, for our numerical study, we want to add rvs and apply the function $x \rightarrow x \bmod C$ where C is the length of the cycle.

The RV class already supports `+` for rvs. For the modulo function, the following function applies to integers, floats, and rvs.¹⁰

```

1 def mod(a, b):
2     "Compute a modulo b where a can also be a RV."
3     match a:
4         case int():
5             return a % b
6         case float():
7             return a % b
8         case RV():
9             return a.map(lambda x: x % b)
10        case _:
11            raise ValueError("Unknown type passed to mod")

```

¹⁰ I like to write (and read) code that resembles the mathematical operations I write on paper.

We iteratively compute the arrival time position $\sum_{i=1}^n X_i \bmod C$ where the inter-arrival time is $X_i \sim \text{Unif}(\{-1, 0, 1\})$, and $A_0 = 0$. We track the arrival times that occur with the smallest and the largest probability.

```

1 def minmax(cycle, num):
2     position = RV({0: 1})
3     mins, maxs = [], []
4     for i in range(num):
5         position = mod(position + step, cycle)
6         pmf = [position.pmf(k) for k in range(cycle)]
7         mins.append(min(pmf))
8         maxs.append(max(pmf))
9     return mins, maxs

```

This code creates the left panel in Fig. 1.4.1; the right panel is created in the same way. The caption discusses our findings.

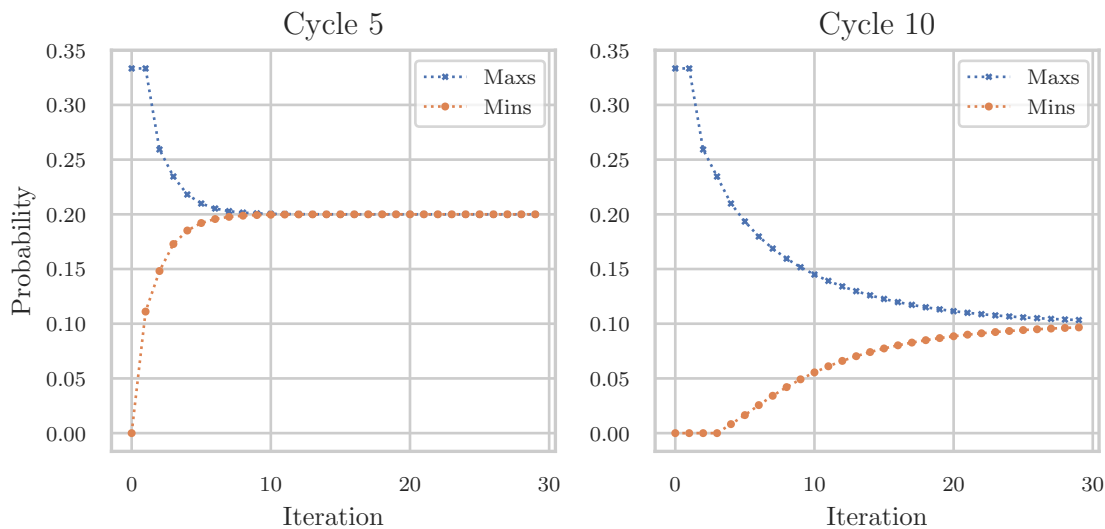


FIG. 1.4.1: Convergence speed of the time between two visits to the hospital. In the left panel the cycle length $C = 5$, in the right $C = 10$. The largest and the smallest probability on the circle converge to $1/C$ (recall Th. 1.3.1). The larger C , the longer it takes for the sequence of visits to converge to uniform inter-arrival times, from the perspective of the hospital. In actual practice, this is not a problem, because patients in the hospital start at arbitrary times, not all at time 0.

```

1 cycle = 5
2 num = 30
3 xx = range(num)
4 mins, maxs = minmax(cycle, num)
5 ax1.plot(xx, maxs, "x", ms=2, ls=":", lw=1, label="Maxs")
6 ax1.plot(xx, mins, "o", ms=2, ls=":", lw=1, label="Mins")
7 ax1.set_title("Cycle 5")
8 ax1.set_xlabel("Iteration")
9 ax1.set_ylabel("Probability")
10 ax1.legend()

```

TF 1.4.1. Claim: this program prints 40.

```

1 norm = 5
2 x = [100, 20, 30, 40]
3 y = {i: x[i] // norm for i in range(len((x)))}
4 print(y[3])

```

TF 1.4.2. Claim: if we apply this function to two independent rvs X and Y and take $f(i, j) = i/j$, we get a RV that represents $Z = X/Y$.

```

1 def map2(X, Y, f):
2     "Make the rv f(X, Y) for the independent rvs X and Y."
3     c = defaultdict(float)
4     for i in X.sortedsupport():
5         for j in Y.sortedsupport():
6             p = X.pmf(i) * Y.pmf(j)
7             c[i, j] += p
8             if p <= thres:

```

```

9         break
10    return RV(c)

```

TF 1.4.3. Let the rv X have cumulative distribution function F_X and let $U \sim \text{Unif}(0, 1)$. Claim: $F_X(F_X^{-1}(U)) \sim \text{Unif}(0, 1)$.

TF 1.4.4. Let X and Y be two independent rvs with support $\{0, 1, 2, \dots\}$ and probability mass functions f_X and f_Y respectively. Claim:

$$\mathbb{P}(X - Y \leq k) = \sum_{i=0}^{\infty} \sum_{j=0}^{\infty} 1\{i - j \leq k\} f_X(i) f_Y(j).$$

TF 1.4.5. Consider the following code.

```

1  a, b = 1000 / 3001, 1001 / 3000
2  af = Fraction(a).limit_denominator(1000)
3  bf = Fraction(b).limit_denominator(1000)
4  print(af, bf)

```

Claim: this program can print $1/3$ 333/998.

Ex 1.4.6. Consult the RealPython website to improve your understanding of `defaultdict`, the `lambda` function, caching, and classes. If you are interested in somewhat more abstract code, read also about decorators to see how caching is implemented. Of course, you can also ask an LLM to explain these concepts: I use it a lot myself and I still learn from the solutions it offers. However, the explanations on RealPython are better.

CONSTRUCTION OF SIMPLE DISCRETE-TIME STOCHASTIC PROCESSES

The first step to analyze a stochastic process⁰ is to model and simulate it. In this chapter we discuss how to do this by means of discrete-time models of queueing systems. Here, discrete time means that we chop up time in intervals of equal duration, and study how the system behaves from one period to the next.¹

Section 2.1 and Section 2.2 show how to make a simulation model of a case of mental health department. Section 2.3 uses simulation to obtain general insight into how queueing systems behave. Section 2.4 provides motivation as to why we concentrate on long-run behavior for queueing systems if we want to use mathematics to analyze such systems. Section 2.5 presents suitable recursions for single-item inventories in discrete time, and demonstrates in particular how to design good rules to control stochastic inventory systems. In Section 2.6 we show how to set up a simulation for such systems.

2.1 WAITING FOR A PSYCHIATRIST

We start with a case to provide motivation for studying queueing systems. Then we develop a set of fundamental recursions to simulate the case and evaluate the efficacy of several policies to improve the system.

AT A MENTAL-HEALTH department, five psychiatrists perform intakes⁰ to determine the best treatment process once patients enter the system. There are complaints about the time patients have to wait for the intake; the desired waiting time is around two weeks, but the realized waiting time is sometimes more than three months. This is unacceptably long, but what to do about it?

The five psychiatrists put forward various suggestions to reduce waiting times.

1. Give all psychiatrists the same weekly capacity to perform intakes. Motivation: Currently, one psychiatrist performs 9 intakes per week, one performs 3, and the other three psychiatrists only perform 1. This is not a problem during weeks when all psychiatrists are present; however, psychiatrists take vacations, visit conferences, etc. Hence, if the psychiatrist with the highest weekly number of intakes goes on leave, the total intake capacity is significantly reduced.
2. Synchronize holidays.¹ Motivation: to reduce the variation in the service capacity, the psychiatrists plan their holidays consecutively. However, perhaps it is better to work at full capacity or not at all.

After consultation, another suggestion was proposed:

3. Control the intake capacity as a function of the waiting time.

⁰ In our case, queueing or inventory processes.

¹ Such a period can be an hour, or a day; in fact, any amount of time that makes sense in the context of the model. For hospitals, it may be days, for supermarkets it may be 5 minutes.

⁰ Interviews with new patients.

¹ Later, with the models of Chapter 6, we will see that this will not work, in fact, synchronization might be detrimental.

Motivation: in analogy with supermarkets, increase (decrease) capacity when the queue is long (short). Observe that people often object to such policies because they believe that they have to do more work. However, this reasoning is incorrect when the arrival rate does not depend on the queue length. To see this, suppose that 1000 patients per year need treatment. This number does not depend on whether patients spend time in the queue or not.

We next develop a method to simulate the behavior of the system over time so that we can evaluate the effect of the above suggestions. We use simulation because mathematical analysis is intractable.²

LET US START with analyzing the two suggestions of the psychiatrists by making a queueing model. Often, the easiest way to construct queueing processes is to divide time into periods of equal length, and develop recursions that capture the behavior of the queue from one period to the next.³

Let us define:

- a_k = the number of jobs that arrive *in* period k ,
- c_k = the capacity, i.e., the maximal number of jobs that can be served, *during* period k ,
- d_k = the number of jobs that depart just before the *end* of period k ,
- L_k = the *system size*, i.e., the number of jobs in the system at the *end* of period k .

In what follows, we also call a_k the *size of the batch* arriving in period k . Note that the definition of a_k is subtle: we may assume that the arriving jobs arrive either at the start or at the end of the period. In the first case, the jobs can be served in period k ; in the latter case, they *cannot* be served in period k .

Since L_{k-1} is the length of the system at the *end* of period $k-1$, it must also be the number of customers at the *start* of period k . Assuming that jobs arriving in period k cannot be served in period k , the number of customers d_k that depart in period k and the number that remain in the system L_k at the end of period k are given by the recursions

$$d_k = \min\{L_{k-1}, c_k\}, \quad L_k = L_{k-1} - d_k + a_k. \quad (2.1.1)$$

Thus, if we are given L_0 and arrivals $\{a_k\}_{k=1}^n$ and capacities $\{c_k\}_{k=1}^n$, we can obtain $\{d_k\}_{k=1}^n$ and $\{L_k\}_{k=0}^n$.

Typically, we can use company data to model the arrival process $\{a_k\}$ and the capacity $\{c_k\}$, and feed these data into a computer to perform recursions (2.1.1). If insufficient data is available, we build a probability model for $\{a_k\}$ and $\{c_k\}$ and generate random deviates with these distributions.

Another interesting recursion is this:⁴

$$L_k = [L_{k-1} + a_k - c_k]^+. \quad (2.1.2)$$

[2.1.9] asks you to prove that this generates a system in which jobs can be served⁵ in the period they arrive.

² In case you doubt this, try to analyze transient multi-server queueing systems with vacations, of which this is an example.

³ Using fixed-length periods has the advantage that we do not have to specify inter-arrival times or service times of individual customers. In this case, only the number of arrivals in a period and the number of potential services are relevant.

⁴ $x^+ := \max\{x, 0\}$, $x^- := [-x]^+$.

⁵ If there is sufficient capacity.

CONTINUING WITH THE case of the five psychiatrists, we can now analyze how the first and second suggestion would affect the waiting times.

As a first step in the analysis, we model the arrival process of patients as a Poisson process. The duration of a period is taken to be a week. The average number of arrivals per period, based on company data, is slightly less than 12 per week; in the simulation we set $\lambda = 11.8$ per week.

WE MODEL THE capacity in the form of a matrix such that row i corresponds to the weekly capacity of psychiatrist i :

$$\begin{pmatrix} 1 & 1 & 1 & \dots \\ 1 & 1 & 1 & \dots \\ 1 & 1 & 1 & \dots \\ 3 & 3 & 3 & \dots \\ 9 & 9 & 9 & \dots \end{pmatrix}. \quad (2.1.3)$$

Thus, psychiatrists 1, 2, and 3 perform one intake per week, the fourth performs 3, and the fifth performs 9. The sum over column k is the total service capacity for week k of all psychiatrists together.

With such a matrix, it is easy to construct other capacity schemes. A more balanced scheme would be like this:

$$\begin{pmatrix} 2 & 2 & 2 & \dots \\ 2 & 2 & 2 & \dots \\ 3 & 3 & 3 & \dots \\ 4 & 4 & 4 & \dots \\ 4 & 4 & 4 & \dots \end{pmatrix}. \quad (2.1.4)$$

We include the effects of holidays by setting the capacity of one or some psychiatrists to 0 in a certain week. Let us assume that every week just one psychiatrist is on leave such that their holiday schemes rotate. To achieve this, we set the entries of the matrix like this:

$$\begin{pmatrix} 0 & 2 & 2 & 2 & 2 & 0 & \dots \\ 2 & 0 & 2 & 2 & 2 & 2 & \dots \\ 3 & 3 & 0 & 3 & 3 & 3 & \dots \\ 4 & 4 & 4 & 0 & 4 & 4 & \dots \\ 4 & 4 & 4 & 4 & 0 & 4 & \dots \end{pmatrix}. \quad (2.1.5)$$

Clearly, under this scheme, the average capacity is 12 patients per week. There is another simple holiday scheme: when all psychiatrists take holiday in the same week:

$$\begin{pmatrix} 2 & 2 & 2 & 2 & 0 & 2 & 2 & \dots \\ 2 & 2 & 2 & 2 & 0 & 2 & 2 & \dots \\ 3 & 3 & 3 & 3 & 0 & 3 & 3 & \dots \\ 4 & 4 & 4 & 4 & 0 & 4 & 4 & \dots \\ 4 & 4 & 4 & 4 & 0 & 0 & 4 & \dots \end{pmatrix}. \quad (2.1.6)$$

Of course, random holiday schemes are more likely, but these two are perhaps the simplest that result in the same average capacity.

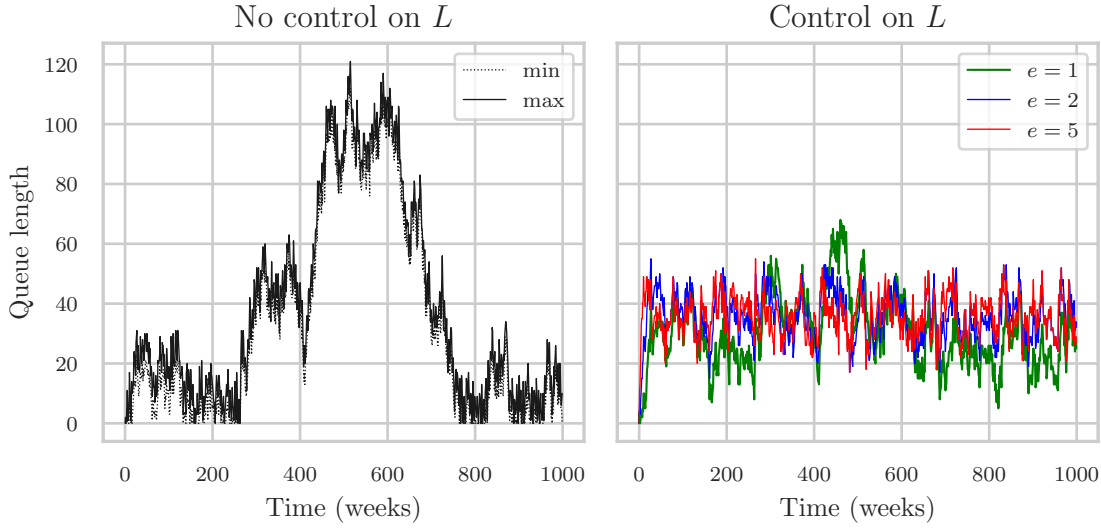


FIG. 2.1.1: The left panel shows the maximum and the minimum queue length under the different holiday and balancing policies on the queue length process $\{L_k\}$. The right panel shows the queue length under the control rule (2.1.7). We see that the fluctuations in queue length are largest when modulating capacity with $e = 1$. Further, the queue is never empty, so intake capacity is not wasted, but the queue also does not make large excursions.

By also applying the same holiday plans to the unbalanced case of (2.1.3), we obtain four different capacity schemes: unbalanced versus balanced capacity, and spread out holidays versus simultaneous holidays. We can now use the recursions (2.1.1) to simulate the queue length process.

The results are shown in Fig. 2.1.1. We plot, for each period, the largest and smallest queue that occurred under all four capacity plans that result from following the first and second suggestions of the psychiatrists. The graphs show that these suggestions hardly affect the behavior of the queue-length process.

This desk study brings us to an important observation: Suggestions 1 and 2 are unlikely to have a positive impact on waiting times. So, rather than setting up an experiment in real life, we should look for other rules.

NOW WE CONSIDER Suggestion 3, which consists of doing more intakes when it is busy and fewer when it is quiet. A simple rule is to let the capacity for week k depend on the queue length at the end of the previous week, for instance,

$$c_k = \begin{cases} 12 + e, & \text{if } L_{k-1} \geq 48 \\ 12 - e, & \text{if } L_{k-1} \leq 24. \end{cases} \quad (2.1.7)$$

We can take $e = 1$ or 2, or perhaps a larger number; the larger e , the larger the control we exercise.⁶ We can, of course, also adapt the thresholds 48 and 24.

Consider three different control levels, $e = 1$, $e = 2$, and $e = 5$; when $e = 1$, one psychiatrist does one additional intake, when $e = 5$, all do an extra intake in that week. The right panel of Fig. 2.1.1 shows a striking result. The queue does not explode anymore; even taking $e = 1$ has a large effect.

In conclusion, this simulation experiment shows that changing holiday plans or spreading the work more evenly over multiple servers, i.e., psychia-

⁶ Why would it not make much sense to set the lower threshold, here 24, smaller than 12? Hint, do we use the capacity of 12 patients efficiently?

trists, may not significantly affect queueing behavior. However, controlling the service rate as a function of the queue length can dramatically improve the situation.

WE NEXT DISCUSS some general points that need further attention. In (2.1.1) we assume that the jobs that arrive in period k cannot be served in period k . If the situation is such that jobs *can* be served in the same period as they arrive, then (2.1.1) should be changed to⁷

$$d_k = \min\{L_{k-1} + a_k, c_k\}. \quad (2.1.8)$$

Which of (2.1.1) or (2.1.8) to choose depends on what we need to model; in general, no rule is ‘best’, and what is ‘good’ depends essentially on the context. For instance, if we want to be ‘on the safe side’, then it is perhaps best to use (2.1.1) because with this rule, we overestimate the queue lengths, while with (2.1.8) we underestimate the queue lengths.

Note next that in the computation of d_k we make a fundamental modeling choice: If there are jobs in the system and the server capacity $c_k > 0$, the server will serve jobs. However, there is no formal need to serve jobs, even when there is service capacity available. A simple reason not to serve jobs is when it is too expensive; for example, if there is very little demand for a flight on some day, then the flight can be canceled. In the models we consider, we will *not* allow jobs to wait when there is capacity available; in other words, our service processes are assumed to be *work-conserving*.

The above recursions only construct $\{d_k\}$ and $\{L_k\}$. If we also need information about the *sojourn times*, that is, the time jobs spend in the system, it is necessary to specify the *service discipline*, i.e., a scheduling rule that decides on the order in which jobs in queue are taken into service. In this book, we henceforth assume that jobs move to the server in the order in which they arrive. This is known as *First-In-First-Out (FIFO)*.⁸ There are many other scheduling rules, such as Last-In-First-Out; we do not discuss these here.

It is quite remarkable that with (2.1.1) the computation of the system length process $\{L_k\}$ is very simple, but it is much harder to compute the sojourn time J_k for the jobs arriving in period k . In fact, the best we can obtain are bounds on the sojourn time J_k such that $J_k^- \leq J_k \leq J_k^+$, where

$$J_k^- = \min \left\{ m : \sum_{i=k}^{k+m} c_i > L_{k-1} \right\}, \quad J_k^+ = \min \left\{ m : \sum_{i=k}^{k+m} c_i \geq L_{k-1} + a_k \right\}.$$

To see how this works, suppose that $L_{k-1} = 20$, $a_k = 6$, and $c_k = c = 2$ for all k . Any job that arrives in the k th period, must wait at least $L_{k-1}/c = 20/2 = 10$ periods just to get access to the server under the FIFO discipline, and we can be sure that the last of these jobs is served after $(20 + 6)/2 = 13$ periods.

We note that there is a difference between *waiting time* W and sojourn time J . The former is the time spent by jobs in the queue, the latter is the time in the system, which is the waiting time plus the time at the server. However, in (2.1.1) and (2.1.8) jobs are not really in service; we only count the jobs in the system at the end of a period. Thus, in this model, the number of jobs in the system coincides with the number in queue.

⁷ [2.1.9].

⁸ First-Come, First-Served (FCFS) is also often used.

Finally, once the simulation is finished, we compute some performance measures to analyze how the (simulation of the) queueing system behaved. Besides making graphs, we can be interested in the arrival rate, the capacity⁹ and mean number in the system,

$$a = \frac{1}{n} \sum_{k=1}^n a_k \quad c = \frac{1}{n} \sum_{k=1}^n c_k \quad L = \frac{1}{n} \sum_{k=1}^n L_k.$$

Similarly, we can take the averages over d_k , J_k^- and J_k^+ . The variances of these (simulated) rvs can also be of interest. Perhaps the most important measure to check is the *load*, which is the rate at which work arrives per server. For instance, if $\lambda = 5$ jobs arrive per hour, and each job requires 20 minutes of work (on average), then we say that the load is $5 \times 20/60 = 5/3$. Since one server can do only 1 hour of work per hour, we need at least two servers to deal with this number of jobs. For a simulation in discrete time, we define the load as

$$\rho \approx \frac{\sum_{k=1}^n a_k}{\sum_{k=1}^n c_k}. \quad (2.1.9)$$

We expect that when $\rho > 1$, long queues build up.¹⁰

IN GENERAL, WITH recursions like (2.1.1) we can carry out numerous what-if analyses.

For example, a hospital considers buying a second MRI scanner because the current one is saturated: it is used from 8 am to 6 pm.¹¹ If the hospital can identify some additional capacity, it might postpone the purchase of an additional MRI scanner for a few years. Suppose that a percentage of the staff is willing to work from 8 pm to 11 pm, say, and that 30% are ready to come to the hospital for a scan in the evening, then the capacity would increase by about $3/10 = 30\%$. To see whether this idea is interesting, we can use (2.1.1) to let the computer make a graph of the influence on L .

Recursions are not just useful for modeling queueing systems. For instance, a common model in population dynamics has this form

$$N_{t+1} = N_t + B_t + I_t - D_t - E_t,$$

where N_t is the population size at year t , B_t the number of births, I_t the immigration, D_t the deaths, and E_t the emigration.

Yet another use of recursion is to understand how the reflection of infrared light by carbon dioxide affects the temperature of the earth. The atmosphere is modeled as a sequence of layers so that, per layer, the temperature, air density, and chemical composition are considered constant. These properties determine how much infrared light, which is initially reflected by the surface of the earth, reaches outer space. In each of the layers, we need to specify what fraction of infrared photons are transmitted to a higher layer, absorbed, or reflected again to a lower level. Thus, this physical process can be modeled as a queueing system in which photons (customers) go from one layer (station) to another layer (station) or are absorbed (leave the network).¹²

⁹ Also known as the service rate.

¹⁰ In Section 2.3 we discuss the concept of load more formally.

¹¹ That is, 10 h per working day.

¹² <https://github.com/scienceetonnante/RadiativeForcing/blob/main/RadiativeForcing.py>.

FORMULATING RECURSIONS IS the key step in making useful models; you should practice this a lot. In some exercises the solution is stated in code so that you practice reading (and thinking in) code.

TF 2.1.1. In a discrete-time queueing system, when job arrivals in period k cannot be served in period k , then $d_k = \min\{L_{k-1}, c_k\}$, $L_k = L_{k-1} - d_k + a_k$.

TF 2.1.2. We have a queueing system in discrete time. Take $c_k = c \mathbb{1}_{L_{k-1} > \alpha}$ as the service capacity in period k , with $\alpha > 0$. Claim: if $c < \alpha$ and $L_0 > 0$, then $L_k > 0$ for all k .

TF 2.1.3. A single machine serves two queues so that jobs in the first queue get priority over jobs in the other queue. Jobs cannot be served in the period they arrive. Claim: these recursions model this situation correctly:

```

1 d1[k] = min(L1[k - 1], c[k])
2 L1[k] = L1[k - 1] - d1[k] + a1[k]
3 c2[k] = min(L2[k - 1], c[k] - d1[k])
4 d2[k] = min(L2[k - 1], c2[k])
5 L2[k] = L2[k - 1] - d2[k] + a2[k]

```



TF 2.1.4. Consider a single server that serves two parallel queues. Queue i has minimal guaranteed service capacity r^i each period, such that $c_k \geq r^1 + r^2$. Extra capacity beyond the reserved capacity is given to queue 1 with priority. Arriving jobs cannot be served in the period they arrive. (An example is a psychiatrist who reserves capacity for different patient groups.) Claim: these recursions model this situation correctly:

```

1 c2[k] = min(L2[k - 1], r2)
2 d1[k] = min(L1[k - 1], c[k] - c2[k])
3 L1[k] = L1[k - 1] - d1[k] + a1[k]
4 d2[k] = min(L2[k - 1], c[k] - d1[k])
5 L2[k] = L2[k - 1] - d2[k] + a2[k]

```

The next true-false question relates to the control rule (2.1.7).

TF 2.1.5. Consider the following control rule for the psychiatrists case:

```

1 if L[k - 1] >= 48:
2     c[k] = 12 + e
3 elif L[k - 1] <= 24:
4     c[k] = 12 - e

```

Claim: When $24 < L_{k-1} < 48$, the capacity c_k is set to 12.

Ex 2.1.6. *Fast lanes at De Oosterpoort.* The theater *De Oosterpoort* in Groningen wants to increase customer satisfaction during event breaks. In particular, management would like to ensure that visitors receive their drinks before the break ends. At the same time, they aim to increase total sales.

One proposed intervention is the introduction of *fast lanes*. A fraction α of customers would use a separate lane with service times that are 30 seconds shorter than the average of 2 minutes, and a fraction β of staff would be assigned exclusively to this fast lane. Before implementing this change, management would like to simulate the system to understand under which parameter regimes fast lanes should (or should not) be installed.

Consider a discrete-time queueing model observed in time steps of one minute. Let a_k denote the number of arriving customers in minute k , and c_k the service capacity if all staff would be assigned to the regular queue. Develop the recursions for the regular queue L_k^r and fast lane L_k^f .

For real-life situations the recursions can become quite intricate.¹³

Ex 2.1.7. A nurse takes blood samples in two departments of a hospital. It takes time to walk from one location to another. The nurse serves all patients in one location, then walks to the other location, serves all patients there, walks back to the first location, and so on.

Give the recursions to see how this switching behavior affects the queue lengths at both locations.

Ex 2.1.8. Each week a civil court receives a random number of new cases. After a case is filed, it must first be processed by a clerk, then reviewed by a judge, and then handled once more by a clerk for the final administrative steps. All workloads are measured in hours. Every newly filed case creates 5 hours of initial clerk work, 3 hours of judge work, and 2 hours of final clerk work. If there is work at both queues of the clerks, they give priority to new cases. The clerks together have 80 hours available per week, and the judge has 34 hours. During week k , a server can only work on the backlog that is actually present.

Formulate the weekly recursion relations for all three queues of cases.

Ex 2.1.9. Prove that (2.1.2) produces a system in which jobs can be served in the period they arrive.



¹³ This example is a somewhat simplified case from a bachelor thesis.

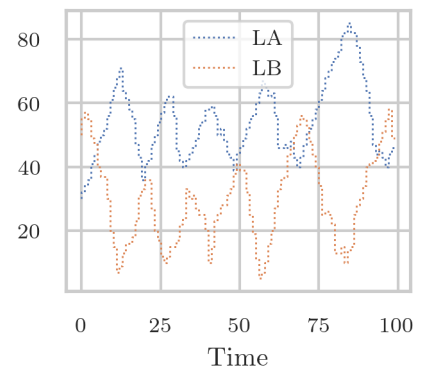


Fig. 2.1.2: A nurse walking between two locations.



2.2 SIMULATING THE PSYCHIATRISTS CASE

In the previous section, we showed how to analyze a case with five psychiatrists acting as five parallel servers processing a queue of patients waiting for an intake. Moreover, we proposed a good rule to control the queue length. In this section, we provide the Python code we used for the simulation leading to Fig. 2.1.1.

WE NEED THE following modules.

```

_____ psychiatrists.py _____
1 import numpy as np
2 from numpy.random import default_rng
3 import matplotlib.pyplot as plt
4
5 from latex_figures import apply_figure_settings
6
7 apply_figure_settings(use=True)
8

```

```

9
10 thres_low = 24
11 thres_high = 48

```

Recall that we compute the queue length recursively by first computing the number of departures $d = \min\{L_{k-1} + a_k, c_k\}$ and then $L_k = L_{k-1} + a_k - d$. The following function follows this logic. It first forms an empty array `L` to store all the queue lengths. Then it applies the recursion via a for loop. The loop has to start at $k = 1$ because it 'looks back' at L_{k-1} . For generality, we can provide an optional queue level `L0` if we want the queue to start at a level other than 0.

```

_____ psychiatrists.py _____
1 def compute_L(a, c, L0=0):
2     # L0 is the initial queue length
3     L = np.empty(len(a))
4     L[0] = L0
5     for k in range(1, len(a)):
6         d = min(L[k - 1] + a[k], c[k])
7         L[k] = L[k - 1] + a[k] - d
8     return L

```

Now we make the capacity schemes. The first step is to form an array with 5 rows such that each row corresponds to the weekly capacity of each psychiatrist. Since Python counts from 0 instead of 1, the row with index 0 corresponds to the first psychiatrist. We can set all values of the entire row with index 0 with notation like `p[0, :]` where the colon refers to all indices of the columns.⁰ Next, `np.ones(n)` makes a row array of just ones. For each psychiatrist, we fill the related row in `p` with the appropriate capacity of that specific psychiatrist. The result is a matrix that looks like the one in (2.1.3).

⁰ It evokes the dots in mathematical notation like the dots here: $i = 1, 2, \dots$

```

_____ psychiatrists.py _____
1 def unbalanced_load(n):
2     p = np.empty([5, n])
3     p[0, :] = 1 * np.ones(n)
4     p[1, :] = 1 * np.ones(n)
5     p[2, :] = 1 * np.ones(n)
6     p[3, :] = 3 * np.ones(n)
7     p[4, :] = 9 * np.ones(n)
8     return p

```

The following function builds the matrix (2.1.4).

```

_____ psychiatrists.py _____
1 def balanced_load(n):
2     p = np.empty([5, n])
3     p[0, :] = 2 * np.ones(n)
4     p[1, :] = 2 * np.ones(n)
5     p[2, :] = 3 * np.ones(n)
6     p[3, :] = 4 * np.ones(n)
7     p[4, :] = 4 * np.ones(n)
8     return p

```

We modify the capacity matrix so that it includes a holiday pattern. This function spreads the holidays over the week so that we obtain (2.1.5).

The operator % computes the remainder after division.¹ In the for loop, the variable j moves one column per iteration, and it cycles over the rows from 0 to 4. The `shape[1]` returns the number of columns in a matrix.²

¹ For example, $17\%5 = 2$.

² The function does not really need to return p because the change is in-place. (Ask an LLM what this means.) Here we return it so that all functions work in the same way.

```

1 def spread_holidays(p):
2     n_cols = p.shape[1]
3     for j in range(n_cols):
4         p[j % 5, j] = 0
5     return p

```

In the case of synchronized holidays, we set all capacities in one column to zero; see (2.1.6). The `range` function moves from 0 to the end in steps of 5.³

³ So it returns 0, 5, 10, ...

```

1 def synchronized_holidays(p):
2     n_cols = p.shape[1]
3     for j in range(0, n_cols, 5):
4         p[:, j] = 0
5     return p

```

ALL IS READY to start the simulation. We set the seed of the random generator and simulate `num_weeks` of Poisson distributed weekly arrivals. The matrix `L` will contain the lengths of the queues, one row for each different scenario. As we have four capacity scenarios, we need four rows.

```

1 rng = default_rng(3)
2 num_weeks = 1000
3 a = rng.poisson(11.8, num_weeks)
4 L = np.zeros((4, len(a)))

```

In the first scenario, the capacity is balanced and holiday plans are spread over the weeks. The weekly capacity c_i is equal to the sum of all rows in `p` that correspond to week i . This is established by summing over `axis=0` in `p`. For example, in (2.1.5), the capacity for the first week is $0 + 2 + 3 + 4 + 4$. Once we have the weekly capacities stored in `c`, we can use the recursion for L to compute the queue lengths at the end of each week. The other scenarios work likewise.

```

1 p = balanced_load(len(a))
2 p = spread_holidays(p)
3 c = np.sum(p, axis=0)
4 L[0, :] = compute_L(a, c)
5
6 p = balanced_load(len(a))
7 p = synchronized_holidays(p)
8 c = np.sum(p, axis=0)
9 L[1, :] = compute_L(a, c)
10
11 p = unbalanced_load(len(a))
12 p = synchronized_holidays(p)
13 c = np.sum(p, axis=0)

```

```

14 L[2, :] = compute_L(a, c)
15
16 p = unbalanced_load(len(a))
17 p = spread_holidays(p)
18 c = np.sum(p, axis=0)
19 L[3, :] = compute_L(a, c)

```

The code above suffices to make the left panel of Fig. 2.1.1. For the right panel, we need the control rule (2.1.7). This works almost the same as the regular recursion for L , but now we add or subtract an additional capacity e to the weekly capacity given in the array c . We use the result of [2.1.9] to update L .

```

_____ psychiatrists.py _____
1 def compute_L_with_control(a, c, e, L0=0):
2     L = np.empty(len(a))
3     L[0] = L0
4     for k in range(1, len(a)):
5         if L[k - 1] <= thres_low:
6             c[k] -= e
7         elif L[k - 1] >= thres_high:
8             c[k] += e
9         L[k] = max(L[k - 1] + a[k] - c[k], 0)
10    return L

```

We simulate the effect on L for the control rules with $e = 1$, $e = 2$, and $e = 5$. Recall from Section 2.1 that the total weekly capacity is 12 patients.

```

_____ psychiatrists.py _____
1 c = 12 * np.ones_like(a)
2 LL = np.zeros((4, len(a)))
3 LL[0, :] = compute_L(a, c)
4 LL[1, :] = compute_L_with_control(a, c, 1)
5 LL[2, :] = compute_L_with_control(a, c, 2)
6 LL[3, :] = compute_L_with_control(a, c, 5)

```

FINALLY, WE CAN make a figure. It is easy to find online what `ax1` and the other commands mean. Note, `L.min(axis=0)` computes the minimum queue length per week, and `L.max(axis=0)` the maximum.

```

_____ psychiatrists.py _____
1 fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(6, 3), sharey=True)
2 ax1.set_title("No control on $L$")
3 ax1.set_ylabel("Queue length")
4 ax1.set_xlabel("Time (weeks)")
5
6 ax1.plot(L.min(axis=0), ":", label="min", color='k', lw=0.5)
7 ax1.plot(L.max(axis=0), "-.", label="max", color="k", lw=0.5)
8 ax1.legend()
9
10 ax2.set_title("Control on $L$")
11 ax2.set_xlabel("Time (weeks)")
12 ax2.plot(LL[1], label="$e = 1$", color='green', lw=0.8)
13 ax2.plot(LL[2], label="$e = 2$", color='blue', lw=0.5)
14 ax2.plot(LL[3], label="$e = 5$", color='red', lw=0.5)
15 ax2.legend()
16

```

```

17 fig.tight_layout()
18 fig.savefig("/home/nicky/Dropbox/vakken/qts/stochastic_or/figures/psychiatrists.pdf")
19 fig.savefig("/home/nicky/Dropbox/vakken/qts/stochastic_or/figures/psychiatrists.png", dpi=300)

```

TF 2.2.1. Consider a single server serving two parallel queues. Queue i receives a minimal service capacity r^i every period. The reserved capacity unused for one queue cannot be used to serve the other queue. Any extra capacity beyond the reserved capacity, that is, $c_k - r^2$, is given to queue 1 with priority. Claim: The next recursions are correct:

```

1 d1 = min(L1[k - 1], c[k] - r2)
2 c2 = c[k] - d1
3 d2 = min(L2[k - 1], c2)
4 L1[k] = L1[k - 1] + a1[k] - d1
5 L2[k] = L2[k - 1] + a2[k] - d2

```

An example is the operation room of a hospital. There is a weekly capacity, part of which is reserved for emergencies. It might not be possible to assign this reserved capacity to other patient groups, because it should be available at all times for emergency patients. As a result, unused capacity is lost. In practice, it may not be as extreme as in the model, but part of the unused capacity is still lost. 'Use it, or lose it' often applies to service capacity.

The next true-false questions and exercises relate to the N -policy.

TF 2.2.2. Under an N -policy, the utilization ρ depends on N .

TF 2.2.3. Claim: Under an N -policy the average waiting time is smaller than without the policy.

TF 2.2.4. Consider the following code for the N -policy.

```

1 I[k] = (L[k - 1] >= N) + I[k - 1] * (0 < L[k - 1] < N)

```

Claim: When $L_{k-1} = 0$ and $I_{k-1} = 1$, the server switches off, i.e., $I_k = 0$.

TF 2.2.5. Claim: For the N -policy, the following code correctly counts the number of switch-on events.

```

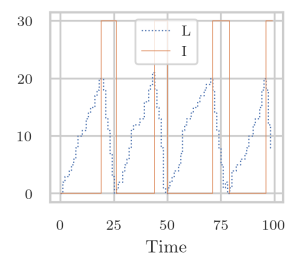
1 num_switchons = sum((I[1:] > I[:-1]))

```

In the queueing systems we have analyzed up to now, the server is always present to start serving jobs at the moment they arrive.⁴ However, in cases in which there is a significant cost associated with changing the server from idle to busy this condition is typically not satisfied. To reduce the average cost, we can use an N -policy, which works as follows.⁵

Ex 2.2.6. As soon as the system becomes empty (and the server idle), the server switches off. Then it waits until at least N jobs have arrived, and then it switches on. The server processes jobs until the system is empty

⁴ A queue under an N -policy.



⁵ N -policies are used to decide when to heat up a large oven.

again, then it switches off, and remains idle until N new jobs have arrived, and so on, cycle after cycle. Note that under an N policy, even though the load remains the same, the server has longer busy and idle times.

Derive recursions to model the situation in which jobs that arrive in period k can be served in that period. If the system length hits N (becomes empty) in period k , the machine switches on (off) in period $k + 1$. Assume the machine is off at $k = 0$.

Ex 2.2.7. In the setting of the previous exercise, suppose it costs K to switch on the machine. There is also a cost β per period when the machine is on, and it costs h per period per customer in the system. Can you make a model to compute the total cost for the first T periods?

2.3 GENERAL BEHAVIOR OF QUEUEING SYSTEMS

In this section, we develop a simulation environment for queueing systems to explore their behavior and visualize queue-length dynamics.

THE FIRST ASPECT to address is system stability. Intuitively, when work arrives faster than it can be served, on average, the queue grows; if work arrives slower, the queue remains close to the origin. To illustrate this, Fig. 2.3.1 shows queue-length sample paths for several arrival rates and service capacities.

After importing the standard modules; see Section 2.2, the function below simulates the queue-length process for arrivals $a = (a_0, a_1, \dots, a_n)$ and capacities $c = (c_0, c_1, \dots, c_n)$. We set the initial queue length to and start simulating from time 1, so a_0 and c_0 remain unused. Jobs arriving in period i can be served in that same period.

```

discrete_simulations.py
1 def compute_queue_length(a, c, L0=0):
2     "Departures and queue lengths for arrivals, capacities, and init level L0."
3     L = np.zeros_like(a)
4     d = np.zeros_like(a)
5     L[0] = L0
6     for i in range(1, len(a)):
7         d[i] = min(c[i], L[i - 1] + a[i])
8         L[i] = L[i - 1] + a[i] - d[i]
9     return d, L

```

Next, we set the parameters for the simulation.

```

discrete_simulations.py
1 labda, mu = 6, 7 # arrival and service rates
2 L0 = 40 # starting level of queue
3 num = 50 # run length
4 x = np.arange(0, num) # x values for the plot

```

To illustrate stochastic variability, we plot several sample paths of the queue-length process. We use ten random seeds, simulate each case, and plot all paths. For the simulation, we let a_k be uniformly distributed on the integers $\lambda - 1, \lambda, \lambda + 1$ with $\lambda = 6$, so that 6 jobs arrive on average in a period. Because draws iid uniformly distributed integers from to , we set . The constant service capacity $\mu = \lambda + 1 = 7$ makes the system stable.

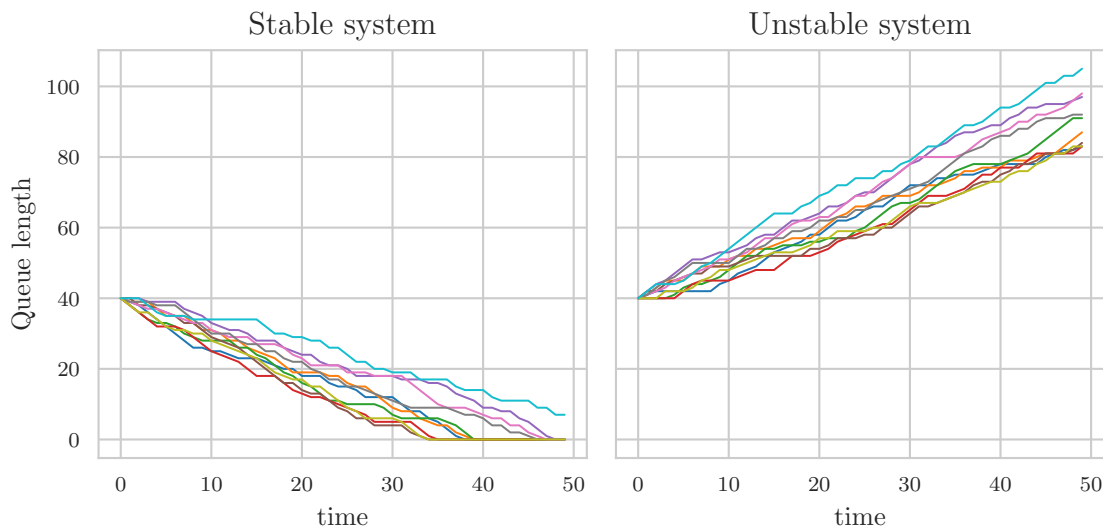


FIG. 2.3.1: Drift of queue-length process. In the left panel, $L_0 = 40$, $\mu = 7$ and $\lambda = 6$; the queue drains at about one job per period. In the right panel, $\mu = 5$, $L_0 = 40$; the queue grows by roughly one job per period. Clearly, when $\mu > \lambda$ the queue-length process decreases; in the other case, it increases.

```

discrete_simulations.py
1 fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(6, 3), sharey=True)
2 ax1.set_xlabel("time")
3 ax1.set_ylabel("Queue length")
4 ax1.set_title("Stable system")
5
6 for seed in range(10):
7     rng = np.random.default_rng(seed)
8     a = rng.integers(labda - 1, labda + 2, size=num)
9     c = mu * np.ones_like(a)
10    d, L = compute_queue_length(a, c, L0)
11    ax1.plot(x, L, linewidth=0.75)

```

Repeating with $\mu = 5$ yields an unstable system. We finish by writing the figure to a file. Fig. 2.3.1 shows the result of our work.

```

discrete_simulations.py
1 mu = 5
2 ax2.set_xlabel("time")
3 ax2.set_title("Unstable system")
4 for seed in range(10):
5     rng = np.random.default_rng(seed)
6     a = rng.integers(labda - 1, labda + 2, size=num)
7     c = mu * np.ones_like(a)
8     d, L = compute_queue_length(a, c, L0)
9     ax2.plot(x, L, linewidth=0.75)
10
11
12 fig.tight_layout()
13 fig.savefig('/home/nicky/Dropbox/vakken/qts/stochastic_or/figures/queue-discrete-time-stability.pdf')

```

HOW ABOUT REDUCING the difference between λ and μ ? When μ barely exceeds λ , large fluctuations take longer to die out. To see this in quantitative terms, we run two simulations with smaller values for μ while keeping

$\lambda = 6$, and we compute the mean and variance of the number of jobs in the system.

We need a sequence of random integers c_k with mean $\mu = 6.5$. One way to make such a sequence is to set $c_k = 6 + b_k$, where b_k is a Bernoulli rv with success probability $p = \mu - 6 = 0.5$.

```

discrete_simulations.py
1  labda, mu, L0 = 6, 6.5, 0
2  num = 5000
3  rng = np.random.default_rng(4)
4
5  a = rng.integers(labda - 1, labda + 2, size=num)
6  c = int(mu) * np.ones_like(a)
7  p = mu - int(mu) # fractional part of mu
8  c += rng.binomial(1, p, size=len(c))
9  print(f"{c.mean():.2f}") # just a check
10 d, L = compute_queue_length(a, c, L0)
11 print(f"{L.mean():.2f}, {L.var():.2f}")

```

If we rerun with $\mu = 6.1$, we obtain this.

```

1  c.mean()=6.10
2  L.mean()=4.36, L.var()=25.30

```

Here $E[L]$ stands for the sample average $n^{-1} \sum_{k=1}^n L_k$, and likewise for $V[L]$.⁰ As expected, the mean queue length rises when capacity tightens.

WOULD VARIABILITY AFFECT the average queue length? We may suspect this because when $a_k = 1$ and $c_k = 1$ for all k and $L_0 = 0$, $L_k = 0$ for all k . If instead $c_i = 5$ when i is a multiple of 5 and 0 otherwise, queues do appear and disappear.¹ Let us use simulation to see how variability can influence the queue-length process. The goal is now to make Fig. 2.3.2.

We first need a way to quantify *relative* variability. Suppose the standard deviation of the service time is 1 minute. If the mean service time is 1 hour, we are inclined to call the service times very regular, while if the mean is just 1 minute, we would call it irregular. To capture this relative effect, we define the *square coefficient of variation* (scv) of the rv X as²

$$C_X^2 := \frac{V[X]}{(E[X])^2}.$$

Continuing with the simulation, we seek a probability distribution for $\{c_k\}$ with mean μ and scv c^2 . A simple way to achieve this is to use a rv X such that $P\{X = b\} = p$ and $P\{X = 0\} = 1 - p$, and require that $E[X] = pb = \mu$ and $C_X^2 = c^2$. Using $V[X] = E[X^2] - (E[X])^2$, we see that $C_X^2 = E[X^2] / (E[X])^2 - 1$. Rewriting, we get $E[X^2] = \mu^2(c^2 + 1)$, but we also know that $E[X^2] = pb^2$ for a Bernoulli-distributed rv. Solving for b and p in terms of μ and c^2 gives $p = 1/(1 + c^2)$ and $b = \mu/p = \mu(1 + c^2)$.

The following function implements the construction of a list of rvs with the required properties.

```

discrete_simulations.py
1  def make_vector_mean_scv(mu, scv, num):
2      "Vector of iid rvs with a given mean mu and scv."

```

⁰ We are in a state of sin here: the mean of a rv is not the same as its sample average. The strong law of large numbers makes a statement only about the limit; for small sample sizes, the difference can be quite large.

¹ Observe that the mean service time is still 1; only the periods in which service is offered changed.

² The scv is unit-less.

```

3     p = 1 / (1 + scv)
4     b = mu / p
5     vec = b * rng.binomial(1, p, size=num)
6     return vec

```

We simulate for $c^2 = 0.5, 1, 2$ and with constant arrivals.

```

_____ discrete_simulations.py _____
1 num = 5000
2 x = np.arange(0, num)
3 fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(6, 3), sharey=True)
4
5 ax1.set_title("Constant arrivals")
6 ax1.set_xlabel("time")
7 ax1.set_ylabel("Queue length")
8 a = make_vector_mean_scv(mu=6, scv=0, num=num) # constant arrivals
9 for scv in (2, 1, 0.5):
10     c = make_vector_mean_scv(mu=6.5, scv=scv, num=num)
11     d, L = compute_queue_length(a, c, L0)
12     ax1.plot(x, L, linewidth=0.5, label=f"$c^2 = {scv}$")
13 ax1.legend()

```

We repeat this to see the effect of variability in both the arrivals and services. Fig. 2.3.2 shows that higher variability yields larger queue-length fluctuations.

```

_____ discrete_simulations.py _____
1 ax2.set_xlabel("time")
2 ax2.set_title("Both variable")
3 for scv in (2, 1, 0.5):
4     a = make_vector_mean_scv(mu=6, scv=scv, num=num)
5     c = make_vector_mean_scv(mu=6.5, scv=scv, num=num)
6     d, L = compute_queue_length(a, c, L0)
7     ax2.plot(x, L, linewidth=0.5, label=f"$c^2 = {scv}$")
8
9 fig.tight_layout()
10 fig.savefig('/home/nicky/Dropbox/vakken/qts/stochastic_or/figures/queue-discrete-time-scv.pdf')

```

In conclusion, tight capacity or high relative variability leads to large queues. To improve performance, we have basically four options: increase capacity, reduce load (or demand), reduce the scv of the arrivals or of the services.³

FINALLY, WE CONSIDER queues in sequence. For example, obtaining grades for a course involves several sequential steps. First, I grade the exams (and handle all changes that might occur during the review). I send a csv file with the grades to the secretary who uploads it. The secretary prints the file for me to sign, after which it is logged by the administration. In abstract terms, there are three stations (me, the secretary, and the back office), and I am involved at least twice, once for the grading and a second time for the signing. More generally, most service systems involve multiple servers visited⁴ to complete a service.

To gain insight into such systems, we simulate a chain of 4 stations in tandem, which means that jobs arrive at station 1, are then sent to station

³ Of course, these options are not exclusive; for example, we can try to reduce the variability of the service process and add some capacity at the same time.

⁴ Perhaps even multiple times.

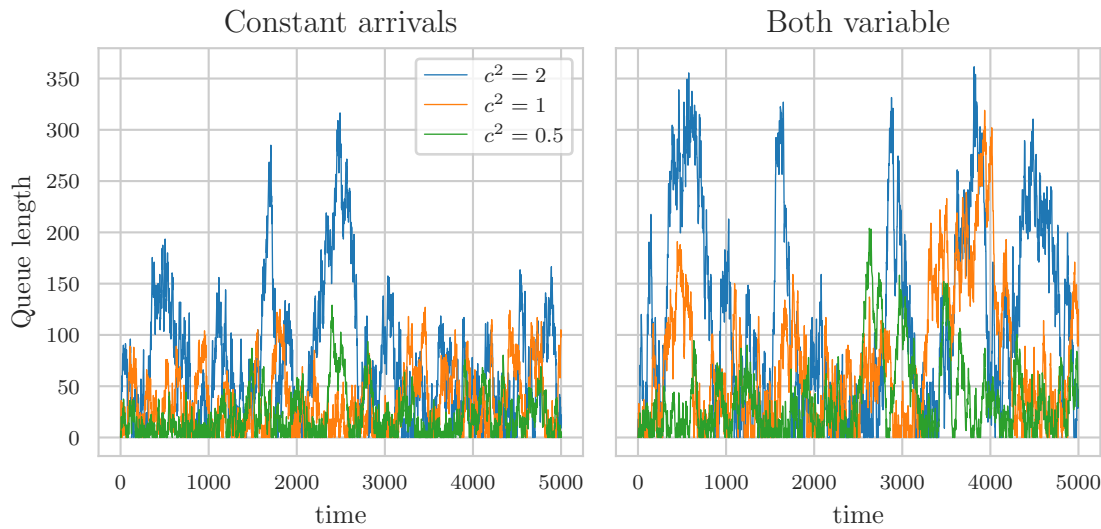


FIG. 2.3.2: The influence of variability on queue length. In the left panel the arrivals are constant, but the number of services per period is variable with scv c^2 as indicated in the legend. In the right panel, we have variability in both arrivals and services. In the second case, the queue length is more variable. (Note that the scales on the y-axes are the same in the left and right panels.)

2, etc., until they leave the network after station 4. Jobs neither visit the same server nor leave the network prematurely.

We generate arrivals at Station 1 and service capacities at each of the stations. Note that station 2 has the slower server.⁵

⁵ Even though a Poisson distributed rv can have a fractional rate, the values of the rv are integer.

```
discrete_simulations.py
1 labda, L0 = 6, 0
2 num = 5000
3 x = np.arange(num)
4
5 rng = np.random.default_rng(3)
6 a = rng.integers(labda - 1, labda + 2, size=num)
7 c1 = rng.poisson(labda + 1, num)
8 c2 = rng.poisson(labda + 0.1, num)
9 c3 = rng.poisson(labda + 1.5, num)
10 c4 = rng.poisson(labda + 1.0, num)
```

It remains to make the plots.

```
discrete_simulations.py
1 fig, ax = plt.subplots(figsize=(5, 3))
2 ax.set_xlabel("time")
3 ax.set_ylabel("Queue length")
4
5 d1, L1 = compute_queue_length(a, c1)
6 ax.plot(x, L1, linewidth=0.5, label="Q1")
7
8 d2, L2 = compute_queue_length(d1, c2)
9 ax.plot(x, L2, linewidth=0.5, label="Q2")
10
11 d3, L3 = compute_queue_length(d2, c3)
12 ax.plot(x, L3, linewidth=0.5, label="Q3")
13
14 d4, L4 = compute_queue_length(d3, c4)
```

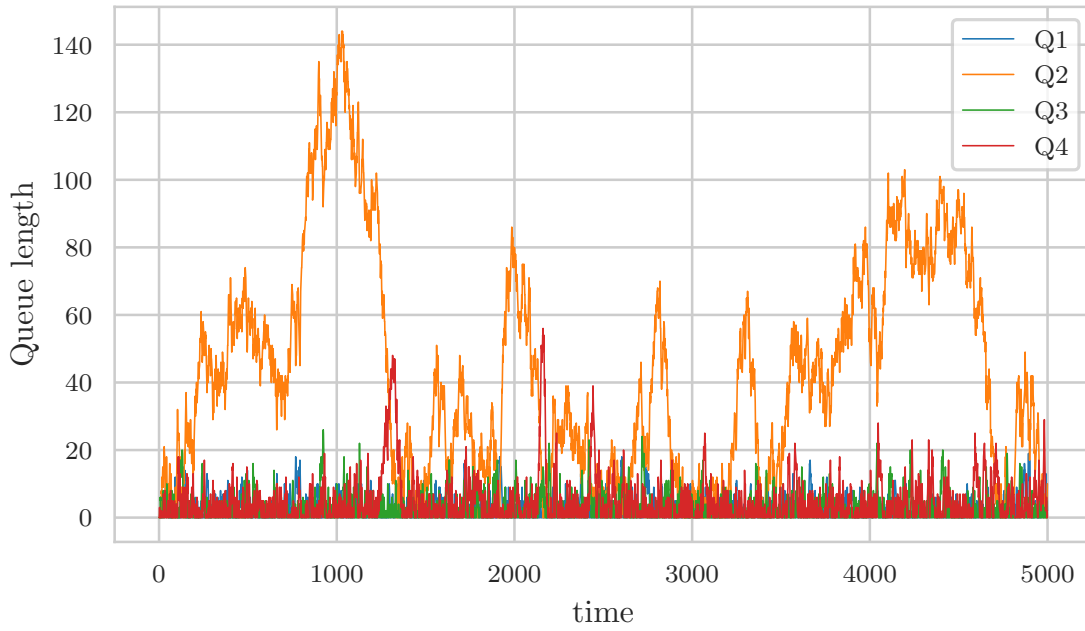


FIG. 2.3.3: A queueing network of 4 stations in tandem.

```

15 ax.plot(x, L4, linewidth=0.5, label="Q4")
16
17 plt.legend()
18 fig.tight_layout()
19 fig.savefig('/home/nicky/Dropbox/vakken/qts/stochastic_or/figures/queue-discrete-network.pdf')

```

Fig. 2.3.3 shows that the station with the slowest server typically has the longest queue. This brings us to an interesting idea: to improve performance, invest in the bottleneck. In practice, identifying the bottleneck can be difficult because service capacities are hard to estimate.⁶ However, often the station with the largest queue is the bottleneck.

TF 2.3.1. A general queueing system is stable when the arrival rate is not higher than the service rate.

TF 2.3.2. If we find the average queue length too large, then we must increase the average service rate to decrease the average queue length.

TF 2.3.3. We consider a discrete-time queueing system with a_k the number of arrivals in period k , and c_k the service capacity. Let $d_k = \min\{L_{k-1}, c_k\}$, $L_k = L_{k-1} - d_k + a_k$. Take $a_k \sim \text{Pois}(2)$, $c_k \sim \text{Pois}(1)$, and $L_0 = 0$. Claim: $P\{L_{1000} \geq 100\} \leq 1/2$.

TF 2.3.4. Claim: The scv of a rv X is the same as the scv of αX if α is a positive scalar.

TF 2.3.5. The parking meters at the universities are redesigned. A study has shown that the average amount of time individuals need to spend to pay for parking meters has decreased. Claim: As long as the inter-arrival distribution stays the same, the average queue length at the parking meter will be smaller.

⁶ The capacity may depend on the person at the station on a certain day. There can also be disturbances due to missing raw materials or machine breakdowns. In short, in practical situations, there are many messy details that disturb the assembly of high-quality data.

TF 2.3.6. Claim: When $X \sim \text{Exp}(\lambda)$ its scv exceeds 1.

TF 2.3.7. Consider a queue with an exponential inter-arrival distribution and a uniform service time distribution on $[0, A]$. Claim: $C_s^2 = \frac{1}{3}$.

Ex 2.3.8. A queueing system is under periodic review, that is, at the end of each period, the queue length is measured. Jobs arriving at period k cannot be served in period k and the system cannot contain more than K jobs; in other words, jobs are blocked when the system is too full. Develop code to simulate $\{L_k\}$ and compute the amount of jobs lost per period and the fraction of jobs lost after simulating for T periods.

Ex 2.3.9. All items produced by a machine are tested. With probability p an item turns out to be defective and put at the head of the queue. The production time of new and faulty items is equal. Suppose that items cannot be served in the period in which they arrive, develop a set of recursions to simulate $\{L_k\}$.

Ex 2.3.10. Sometimes items need re-work. Suppose a fraction p of items does not meet the quality requirements after the first pass at a machine, but requires a second pass to repair the problems. Assume that repair jobs need half the service time of new jobs and are served with priority over new jobs. Develop the recursions.

Ex 2.3.11. Consider a network with two production stations in tandem, i.e., all jobs from station 1 move on to station 2, and blocking, i.e., the server at station 1 is not allowed to produce more than the amount M station 2 can maximally contain. Assume that jobs moving from station 1 to station 2 arrive at the end of a period (hence cannot be served in that period), and must fit at the second station before any jobs from station 2 leave. Find recursions.

Ex 2.3.12. Consider the discrete-time model specified by (2.1.1). We assume that a_k is a *batch* of jobs arriving in period k after the departures d_k have left. Provide a simulation to estimate $P\{L \leq m\}$ for a situation in which the first job of the batch sees $L_{k-1} - d_k$ jobs in the system, the second sees $L_{k-1} - d_k + 1$ jobs, and so on.

2.4 QUEUEING PROCESSES AS REGULATED RANDOM WALKS

In this section, we provide an elegant construction of a queueing process based on a *random walk*.⁰ This serves two aims. The first is to show that queueing theory is essentially based on fundamental probabilistic concepts—the random walk—and its close relation to applications in finance, inventory, and insurance. The second goal is to show that it is very hard to characterize the transient behavior of a queueing process analytically. Thus, we will henceforth study queueing systems only in steady state, i.e., their long-run average limits; see Section 4.1.

IN THE CONSTRUCTION of discrete-time queueing processes as set out in Section 2.1, we consider two sequences of iid rvs: arrivals $\{a_k\}$ and capacities $\{c_k\}$ per period. The system size then satisfies $L_k = [L_{k-1} + a_k -$



⁰ If you don't like math, then you can skip this section.

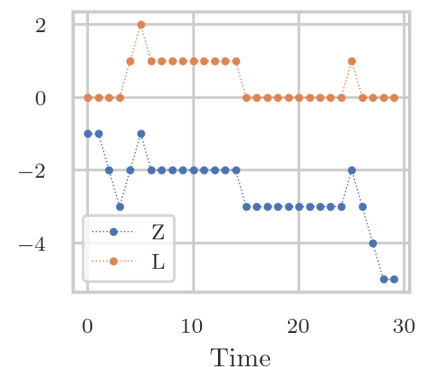


Fig. 2.4.1: An instance of (2.4.1).

$c_k]^+$. The $[\cdot]^+$ operator complicates computations. Removing it yields a related stochastic process $\{Z_k, k = 0, 1, \dots\}$ defined by

$$Z_k = Z_{k-1} + a_k - c_k. \quad (2.4.1)$$

In fact, $\{Z_k\}$ is a random walk, since Z_k makes a jump of size $a_k - c_k$, where $\{a_k - c_k\}$ forms a sequence of iid rvs. Observe that $\{Z_k\}$ is ‘free’—it can take positive and negative values—while $\{L_k\}$ is restricted to non-negative integers.

It is useful to express $\{L_k\}$ in terms of $\{Z_k\}$. From (2.4.1), we have $a_k - c_k = Z_k - Z_{k-1}$, hence

$$L_k = [L_{k-1} + a_k - c_k]^+ = [L_{k-1} + Z_k - Z_{k-1}]^+.$$

It follows that $L_k - Z_k = \max\{L_{k-1} - Z_{k-1}, -Z_k\}$, and by iterating the recursion,¹

$$L_k = Z_k - \left(\min_{1 \leq i \leq k} Z_i \right) \wedge 0, \quad (2.4.2)$$

where $a \wedge b := \min\{a, b\}$.

This recursion for L_k yields instructive plots. In Fig. 2.4.1, let $a_k \sim \text{Bern}(0.3)$ and $c_k \sim \text{Bern}(0.4)$. We clearly see that $L_k = Z_k - \min_{i \leq k} \{Z_i\}$. In Fig. 2.4.2, $a_k \sim \text{Bern}(0.49)$ so that the random walk satisfies

$$Z_k = Z_{k-1} + 2a_k - 1. \quad (2.4.3)$$

Thus, if $a_k = 1$, the random walk increases by one, while if $a_k = 0$, the random walk decreases by one, so $Z_k \neq Z_{k-1}$ always. This differs slightly from (2.4.1), where $Z_k = Z_{k-1}$, if $a_k = c_k$.

WHAT CAN WE say about the transient (time-dependent) behavior of $\{L_k\}$? To gain insight, suppose $a_k \sim \text{Pois}(\lambda)$ and $c_k \sim \text{Pois}(\mu)$.

$$Z_k = Z_0 + N_{\lambda,k} - N_{\mu,k}.$$

Writing $Z_0 = m$, one can show² that³

$$\mathbb{P}_m\{Z_k = n\} = e^{-(\lambda+\mu)k} (\lambda k)^{n-m} \sum_{j=0}^{\infty} \frac{(\lambda \mu k^2)^j}{j!(n-m+j)!}. \quad (2.4.4)$$

Formally, one could derive the distribution of L_k from this and (2.4.2), but no simple expression results. Consequently, no tractable expressions exist for $\mathbb{P}\{L_k = n\}$ as a function of n .

Interestingly, $\{L_k\}$ is among the simplest transient queueing system to analyze; most others are far more complex. For this reason, we henceforth focus on the steady-state regime, i.e., the limit as $t \rightarrow \infty$.

Opt 2.4.1. Show that L_k satisfies (2.4.2).

Opt 2.4.2. Show that, when $Z_0 = m$, $\mathbb{P}\{Z_k = n\}$ satisfies (2.4.4).

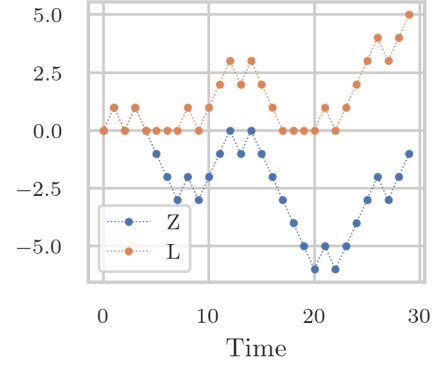


Fig. 2.4.2: An instance of (2.4.3).

¹ [2.4.1].

² [2.4.2].

³ Unlike their sum, the difference of two Poisson rvs is not Poisson.

2.5 CONTROL OF SINGLE-ITEM INVENTORY SYSTEMS

In this section, we analyze so-called single-item inventory systems under *periodic review*, which means that decisions to order a replenishment are made at the end of discrete intervals, e.g., daily. As a simple example, think of macaroni boxes on a supermarket shelf. At the end of each period (say, a day), the supermarket uses the remaining stock to decide whether a replenishment is necessary. If so, it orders in the evening a batch of, say, ten boxes to fill the stock. The batch will be delivered by truck at the start of the next day after which a stocker unpacks the batch and places all ten boxes on the shelf.

In general, there are many types of items in inventory, ranging from cheap to expensive; hence different rules are needed⁰ to determine *when and how much to order*. We assume that demand is discrete, stochastic, and items are not too perishable¹ so unsold items carry over to the next period as on-hand inventory. With these assumptions, we can develop elegant formulas that allow us to simulate the discrete-time dynamics of common inventory-control systems. We then define a number of performance measures to assess their performance. We use the following notation, the meaning of which we explain below:

Q_k = size of order placed at the *start* of period k ,

D_k = demand arriving *during* period k ,

I_k = inventory level at the *start* of period k ,

I_k^+ = inventory on hand at the *start* of period k ,

I_k^- = backlogged demand at the *start* of period k .

P_k = inventory position at the *start* of period k ,

l = fixed lead time.

We are explicit about the *timing* of each of the variables and distinguish between the start and the end of a period. Quantities measured at the *end* of a period are marked with a prime, for instance I'_k .

THE INVENTORY DYNAMICS can be constructed simply if we are given period demands $\{D_k, k = 1, 2, \dots\}$ and order sizes $\{Q_k, k = 0, 1, \dots\}$, but first we need to address a subtle point. In many inventory systems, there is a delay between *placing* and *receiving* a replenishment order: only after the receipt can the order be used to replenish on-hand stock and serve demand. The time between placing the order and its arrival is called the *lead time* l ; the number of orders under way are the *outstanding* orders. We assume l to be constant and independent of the number of outstanding orders.² Then, when placing an order of size Q_{k-l} at the start of period $k-l$ so that it arrives at the start of period k , the *inventory level* follows the rules

$$I'_{k-1} = I_{k-1} - D_{k-1}, \quad I_k = I'_{k-1} + Q_{k-l}. \quad (2.5.1)$$

In words, the inventory level at the end of the period is the level at the start of the period minus the period's demand. The level at the start of the next period includes any replenishments.

⁰ As such, inventory systems are examples of stochastic processes subject to control.

¹ Yogurt is acceptable, but fresh fish not.

² If $l = 0$, the order arrives immediately.

We call $I_k^+ = \max\{I_k, 0\}$ the *inventory on hand*, as these items can directly meet demand from stock. Next, $I_k^- = \max\{-I_k, 0\}$ is the *backlogged demand*, as it represents demand waiting for future replenishments.

Clearly, if we used the inventory level to determine when and how much to order, we might receive replenishments too late because of the lead time between placing and receiving them. The *inventory position* is a more suitable control variable to compensate for this delay. It updates according to the rules

$$P'_{k-1} = P_{k-1} - D_{k-1}, \quad P_k = P'_{k-1} + Q_k. \quad (2.5.2)$$

Thus, the inventory position includes outstanding orders directly on the books, so it accounts for orders under way but not yet arrived. Next, we study a number of inventory control policies. Note that these policies use the *inventory position*, not the *inventory level*, to decide when and how much to order.

THE /BASE-STOCK POLICY/ places an order for each unit demanded. Specifically, the size of the order is given by

$$Q_k = S - P'_{k-1},$$

where S is the *order-up-to level*.³ From (2.5.2) we see that $P_k = S$ for all k .

A base-stock policy is often used to control the inventories of expensive items when the ordering cost K is small relative to the item's price. For example, when someone buys a washing machine, the customer typically pays for delivery and installation. Moreover, customers rarely buy multiple washing machines at once; hence, any ordering or setup cost must be covered per sale.

AN / (s, S) POLICY/ is appropriate in situations where delivery sizes can vary and order costs are significant. When the inventory position is at or below the *reorder level* s , we place an order up to S , specifically,

$$Q_k = (S - P'_{k-1}) \mathbb{1}_{P'_{k-1} \leq s}. \quad (2.5.3)$$

Note that the base-stock policy is a special case of the (s, S) policy by setting $S = s + 1$.

Supermarkets could use (s, S) policies to control the inventory of yogurt, for instance, as yogurt has a fixed amount of shelf space, thereby setting the order-up-to level S .

THE / (Q, r) POLICY/ is useful when the ordering cost is relatively large and replenishments arrive in fixed-size batches. The order quantity is a multiple of a minimal order quantity Q such that P_k always lies between the reorder level r and $r + Q + 1$.⁴ In a formula,

$$Q_k = Q \left\lceil \frac{r + 1 - P'_{k-1}}{Q} \right\rceil \mathbb{1}_{P'_{k-1} \leq r}.$$

So, a base-stock policy is a (Q, r) policy with $Q = 1$ and $r = s$.

The (Q, r) policy is used mainly in supermarket supply chains. Factories pack items in batches and ship them on pallets to distribution centers, which then send smaller batches to supermarkets, where staff unpacks and shelves single items. Thus, to reduce ordering costs, the entire supply chain handles batches instead of single units.

³ The amount ordered depends on the end-of-period inventory position.

⁴ It may seem a bit odd that the (Q, r) policy is not called the (Q, s) policy, but this is the conventional terminology.

FINALLY, THE (T, S) POLICY specifies ordering up to S once every T periods. In a formula:

$$Q_k = (S - P'_{k-1}) \mathbb{1}_{\text{mod}\{k, T\}=0},$$

where $\text{mod}\{k, T\}$ is the remainder of k/T .

This policy can be used by, for instance, gas stations at which every three days a truck refills all tanks.

CLEARLY, WHEN GIVEN a sequence of period demands $D_1, D_2, \dots$ and a chosen inventory control rule, we simulate how the inventory level and position behave over time. It remains to develop a number of KPIs to assess how the inventory control rule performs. We can then search for a good policy by running simulations for various policy parameters and compare their performance using these KPIs.

For cost computations, we have to account for the cost of inventory and backlog. Here we choose to charge at the start of the period: each backlogged unit costs b euros, and each unit in stock costs h euros. If there is a cost K per order placed, the average cost up to period n is

$$Kq + hI^+ + bI^-,$$

where the order frequency, the average on-hand inventory, and the average backlog are given by, respectively,

$$q = \frac{1}{n} \sum_{k=1}^n \mathbb{1}_{Q_k > 0}, \quad I^+ = \frac{1}{n} \sum_{k=1}^n I_k^+, \quad I^- = \frac{1}{n} \sum_{k=1}^n I_k^-.$$

Service levels form another set of KPIs to quantify how well demand is met. We discuss three such measures. The *ready rate* is the fraction of periods in which the end-of-period inventory is non-negative, i.e.,

$$\alpha := \frac{1}{n} \sum_{k=1}^n \mathbb{1}_{I'_k \geq 0}.$$

The *fill rate* β measures the fraction of demand satisfied from on-hand stock. In each period, we can serve the demand D_k as long as it does not exceed the on-hand stock I_k^+ :

$$\beta := \frac{\sum_{k=1}^n \min\{D_k, I_k^+\}}{\sum_{k=1}^n D_k}. \quad (2.5.4)$$

The third measure is *cycle service level*, which is defined as the fraction of cycles⁵ in which the inventory was not negative; a cycle is the time between the arrival of two consecutive replenishments. To capture this we use (2.5.1). A cycle ends just in period $k-1$ if in period $k-l$ a replenishment is placed.⁶ Thus, the cycle ends in period $k-1$ when $Q_{k-l} > 0$. When in this case $I'_{k-1} \geq 0$, there is still on-hand inventory. Thus, the number of cycles in which the inventory was non-negative is $\sum_{k=l+1}^n \mathbb{1}_{Q_{k-l} > 0} \mathbb{1}_{I'_{k-1} \geq 0}$. The cycle service level then follows from averaging over the total number of cycles $\sum_{k=l+1}^n \mathbb{1}_{Q_{k-l} > 0}$:

$$\alpha_c = \frac{\sum_{k=l+1}^n \mathbb{1}_{Q_{k-l} > 0} \mathbb{1}_{I'_{k-1} \geq 0}}{\sum_{k=l+1}^n \mathbb{1}_{Q_{k-l} > 0}}.$$

⁵ A cycle consists typically of many periods; it is not a single period.

⁶ A replenishment placed in period $k-l$ arrives just before period k starts.

The cycle service level is much used in practice because it is simple to measure: when a replenishment arrives, check whether there is still on-hand stock. However, in plain terms, it is a poor performance measure because it completely ignores the amount of unmet demand.⁷

TF 2.5.1. Claim: In a single-item inventory system, the inventory level triggers orders.

TF 2.5.2. Claim: In a single-item inventory system, the inventory level depends on the lead time l when $l > 0$.

TF 2.5.3. Claim: In a single-item inventory system, the inventory position and inventory level coincide when the lead time $l = 0$.

TF 2.5.4. The base-stock and (s, S) models assume that the order size is unlimited. Suppose now that it can be maximally Q . Claim: These recursions cover this case.

$$Q_k = \min\{Q, S - P'_{k-1}\}.$$

TF 2.5.5. Claim: In an (s, S) inventory policy, an order is placed at the end of every period.

Ex 2.5.6. In production environments it is important to decide which products should be make-to-order (MTO) or make-to-stock (MTS). Why can a queueing system model MTO production? What inventory control model would best model a queueing system and what are the parameters?

Ex 2.5.7. Suppose $h \gg b$, i.e., inventory on hand is much more expensive than demand in backlog. For any of the above inventory control models, how would you prefer to perform on the three service levels?

Ex 2.5.8. Which variables (period, order quantity) are fixed, which can vary, in each of the inventory policies we discussed above? Which policy should be able to achieve the lowest cost?

Ex 2.5.9. So far, we assumed that demand in excess of the stock level is backlogged. Modify the recursions of the base-stock model such that it can cope with lost demand. For example, if at the start of period k the inventory level $I_k = 5$ and $D_k = 9$, then 4 items are lost and 5 are fulfilled. How are the (s, S) and (Q, r) policies affected by loss? How is the average cost affected?

Ex 2.5.10. Can-order policies are often used in joint inventory systems in which items can share a delivery (Replenishing supermarkets with trucks forms an example.) Placing an order includes a common setup cost K plus a (much smaller) cost K' for each item included. The can-order policy is characterized by three parameters. If the inventory position $P_{i,k}$ of item i , $i = 1, \dots, N$, falls at or below its reorder level s_i in period k , this item triggers a joint order. Make the recursions to simulate this situation. Find an expression for the cost up to period n . Explain why searching for good parameters with just simulation is not quite possible for any joint inventory system of reasonable size.

⁷ It is an ideal performance measure for managers: easy to understand, but not very informative. Knowing only that stock was empty provides little information about how much demand was lost.

2.6 SIMULATING SINGLE-ITEM INVENTORY SYSTEMS

In this section, we develop a simulation environment to analyze a single-item inventory system under periodic review controlled by an (s, S) policy.

THE LIGHTHOUSE COMPANY sells ceiling lamps and, since the replenishment cost $K = 50$ euro is relatively high, it considers using an (s, S) policy to control the inventory. The company wants to use simulation to find sound values for s and S .

The details of the case are as follows. Ceiling lamps sell for 100 euro, while the purchase price is 40 euro. The monthly holding cost is estimated at 50% of the purchase price. To prevent customers from buying from a competitor (e.g., online), the Lighthouse Company offers a discount of 20% per day on the regular selling price when demand cannot be met from on-hand stock. The pmf $f_i = P\{D = i\}$ of daily demand is given by

$$f_0 = 1/6, \quad f_1 = 1/5, \quad f_2 = 1/4, \quad f_3 = 1/8, \quad f_4 = 11/120, \quad f_5 = 1/6.$$

The lead time $l = 2$ days.

Here are the model parameters. The daily demand is a rv. The cost parameters must have consistent units; here we convert costs to a per-day basis. We write this data to the file `lighthouse_case.py` so that we can use the same case on other occasions.

```

_____ lighthouse_case.py _____
1  """Lighthouse case: model parameters."""
2  from random_variable import NumericRV as RV
3
4  D = RV({0: 1 / 6, 1: 1 / 5, 2: 1 / 4, 3: 1 / 8, 4: 11 / 120, 5: 1 / 6})
5
6  leadtime = 2 # lead time
7  h = 40 * 0.5 / 30 # daily holding cost
8  b = 100 * 0.2 # daily backlog cost
9  K = 50 # Order cost, used for sS and Qr
10 N = 100 # simulation duration
11
12 if N <= leadtime:
13     print(f"The simulation duration {N=} is shorter than the lead time {leadtime=}.")
14     quit()

```

WITH THE CODE below, we set up a simulation to estimate the performance measures for one specific (s, S) policy. For the KPIs we introduce two general helper functions, and write these to the file `functions.py`. By importing this file, we can use these functions at other places, just like the `RV` class.

```

_____ functions.py _____
1  import numpy as np
2
3
4  def normalize(pmf):
5      norm = sum(pmf.values())
6      return {k: pk / norm for k, pk in pmf.items()}
7
8
9  def Plus(x):

```

```

10     if isinstance(x, (int, float)):
11         return max(x, 0)
12     elif isinstance(x, (list, tuple)):
13         return [Plus(v) for v in x]
14     else:
15         return np.maximum(x, 0)
16
17
18 def Min(x):
19     if isinstance(x, (int, float)):
20         return Plus(-x)
21     elif isinstance(x, (list, tuple)):
22         return [Min(v) for v in x]
23     else:
24         return Plus(-x)

```

We now proceed with the inventory model. Most of the following modules should be familiar. The last module loads the RV class, which was discussed in Section 1.4.

```

_____ ss_inventory_simulation.py _____
1 import numpy as np
2 from numpy.random import default_rng
3
4 import random_variable as rv
5 from functions import Plus, Min
6
7 from lighthouse_case import D, K, N, leadtime, h, b

```

This function computes, for the demand D , the inventory positions and inventory levels under specified s and S . For the inventory levels, we combine (2.5.1) into one recursion. The first **for** loop for the inventory levels computes the levels for $t < l$, since no replenishments occur in those periods.

```

_____ ss_inventory_simulation.py _____
1 def simulate_sS(demands, s, S):
2     P = np.zeros(N) # inventory position
3     Q = np.zeros(N) # order quantities
4     IL = np.zeros(N) # inventory level
5     P[0] = IL[0] = S # initial values
6
7     for t in range(1, N):
8         Pprime = P[t - 1] - demands[t - 1]
9         Q[t] = (S - Pprime) * (Pprime <= s)
10        P[t] = Pprime + Q[t]
11
12    for t in range(1, leadtime):
13        IL[t] = IL[t - 1] - demands[t - 1]
14    for t in range(min(leadtime, N), N):
15        IL[t] = IL[t - 1] - demands[t - 1] + Q[t - leadtime]
16
17    return P, IL, Q

```

For the service levels, the cycle service level α_c is of particular interest; we compute it using slicing, a common Python technique.⁰

⁰ Ask an LLM for explanation if you find this line hard. I found it hard to get it right.

```

ss_inventory_simulation.py
1 def cost(IL, Q):
2     return K * (Q > 0).mean() + h * Plus(IL).mean() + b * Min(IL).mean()
3
4
5 def alpha(IL):
6     return (IL >= 0).mean()
7
8
9 def beta(demands, IL):
10    return np.minimum(demands, Plus(IL)).sum() / demands.sum()
11
12
13 def alpha_c(IL, Q):
14    return ((Q[:-leadtime] > 0) * (IL[leadtime - 1 : -1] >= 0)).sum() / (Q[:-leadtime] > 0).sum()

```

We can now run a simulation to compute the KPIs.

```

ss_inventory_simulation.py
1 rng = default_rng(3) # set the seed
2 demands = D.rvs(size=N, random_state=rng)
3
4 P, IL, Q = simulate_sS(demands, s=3, S=20)
5
6 print(f"{demands.mean():.2f}, {P.mean():.2f}, {IL.mean():.2f}, {Q.mean():.2f}")
7 print(f"{Plus(IL).mean():.2f}, {Min(IL).mean():.2f}")
8 print(f"{cost(IL, Q):.2f}")
9 print(f"{alpha(IL):.2f}, {beta(demands, IL):.2f}, {alpha_c(IL, Q):.2f}")

```

```

1 demands.mean()=2.32, P.mean()=13.11, IL.mean()=8.64, Q.mean()=2.32
2 Plus(IL).mean()=8.71, Min(IL).mean()=0.07
3 cost(IL, Q)=13.71
4 alpha(IL)=0.95, beta(demands, IL)=0.88, alpha_c(IL, Q)=0.67

```

IT REMAINS TO find good policy parameters. In principle, this is simple: vary s and S over a range of values to find the cost-minimizing combination.¹ For this we can use the next general grid search algorithm, which we store in `functions.py`; below we apply it to our case.

¹ Finding the optimal s and S efficiently is not an easy problem. If you are interested, consult me for details.

```

functions.py
1 def grid_search(grid, f):
2     "Minimize a f on the grid."
3     best_x, best_value = None, float('inf')
4     for x in grid:
5         if not isinstance(x, (tuple, list)):
6             # allow unpacking *x for non-iterable x
7             x = (x,)
8             value = f(*x)
9             if value < best_value:
10                 best_x, best_value = x, value
11    return best_x, best_value

```

The results show that the best s and S lie in the search area. This suggests they are optimal.

```

ss_inventory_simulation.py
1 def cost_of(demands, s, S):

```

```

2     P, IL, Q = simulate_sS(demands, s, S)
3     return cost(IL, Q)
4
5 from functions import grid_search
6
7 pairs = ((s, S) for s in range(-5, 10) for S in range(s + 1, 30))
8 (s_opt, S_opt), V = grid_search(pairs, lambda s, S: cost_of(demands, s, S))
9
10 print(f"s_opt={s_opt}, {S_opt=}, {V=:.2f}")

```

```

1 s_opt=5, S_opt=25, V=13.00

```

Ex 2.6.1. What inventory policy does the following code try to implement; what is wrong; and what are its policy parameters?

```

1 import numpy as np
2
3 gen = np.random.default_rng(seed=42)
4
5 leadtime = 2
6 n = 100
7 Q_size = 10
8 s = 20
9 labda = 3
10 D = gen.poisson(labda, size=n)
11
12 P = np.zeros_like(D)
13 Q = np.zeros_like(D)
14 for t in range(1, len(D)):
15     Q[t] = Q_size * (P[t - 1] <= s)
16     P[t] = P[t - 1] + Q[t] - D[t]
17
18 IL = np.zeros_like(D)
19 for t in range(leadtime, len(D)):
20     IL[t] = IL[t - 1] + Q[t - leadtime] - D[t]

```

In Section 3.1 we develop recursions to construct queueing processes in continuous time. This shows that a queueing system can be characterized by a few elements; Section 3.2 introduces useful notation for this purpose. Section 3.3 shows how to implement these recursions in Python and visualize the results. Some processes of interest, such as the number of jobs $L(t)$ in the system, cannot be expressed directly by recursion. Section 3.3 and Section 3.4 introduce new ideas to simulate $L(t)$ and other, more complex, queueing processes. The models of these two sections are simple, but cannot capture complex systems such as multi-server queues with heterogeneous service rates. Therefore, in Section 3.5 we extend the ideas of Section 3.3 and Section 3.4 to discrete-event simulation. Section 3.6 then applies this framework to several examples, including queues with heterogeneous server speeds and priority scheduling.

3.1 QUEUEING PROCESS IN CONTINUOUS TIME

In Section 2.1 we modeled time as progressing in discrete chunks. However, we can also model queueing systems in continuous time, so that jobs can arrive at any moment in time and have arbitrary service times. In this section, we develop a set of recursions to construct the waiting times of jobs served in their order of arrival.⁰

LET US IMAGINE that a machine¹ starts working on a one-hour job at 9 am, and there is no job in the queue. When the next job arrives before 10 am, this second job has to wait in the queue until the first job finishes at 10 am. If, however, this second job arrives after 10 am, it finds the server free, so its service can start immediately upon arrival.

More generally, suppose that we are given an ordered sequence of *arrival times*² $\{A_k\}$ and a set of iid *service times* $\{S_k\}$, so that job k arrives at time A_k and needs an amount S_k of service. The departure times $\{D_k\}$ follow from the recursion³

$$\tilde{A}_k = \max\{A_k, D_{k-1}\}, \quad D_k = \tilde{A}_k + S_k, \quad D_0 = 0, \quad (3.1.1)$$

where $\tilde{A}_k$ is the time the service of job k starts⁴, and $D_0 = 0$ is meant to start the recursions. Why is this true? The service of job k cannot start before it arrives at A_k or before job $k-1$ leaves the server at D_{k-1} . Thus, the first moment the service of job k can start is $\max\{A_k, D_{k-1}\}$. When started, it takes S_k to get served.

Once A_k and D_k are known, we compute the *sojourn time*, i.e., the total time in the system, and the *waiting time*, i.e., the time in queue⁵, as

$$J_k = D_k - A_k, \quad W_k = \tilde{A}_k - A_k = J_k - S_k.$$

⁰ That is, according to the FIFO scheduling rule.

¹ A single server.

² Moments at which the system state is inspected or changes are often called epochs.

³ Note that we assume that job k ‘reveals’ its required service at the moment it arrives.

⁴ That is, when this job moves from the queue to the server.

⁵ But not at the server.

WHILE IT IS often simple to measure arrival times in practical situations, it is harder to generate $\{A_k\}$ directly with a simulator. For this we let a random number generator generate iid rvs $\{X_k\}$ that represent the *inter-arrival times* between jobs. The arrival times can then be computed recursively from (1.2.2) as $A_k = A_{k-1} + X_k$, $A_0 = 0$. And conversely, from (1.3.1), if $\{A_k\}$ is known, $X_k = A_k - A_{k-1}$. Interestingly, if $\{X_k\}$ and $\{S_k\}$ are given, we need not first compute $\{A_k\}$ and $\{D_k\}$ using (3.1.1) to find the sojourn times and waiting times. In fact, suppose that job $k-1$ has to wait a time W_{k-1} in queue and spends S_{k-1} in service, then its sojourn time $J_{k-1} = W_{k-1} + S_{k-1}$. If X_k time units elapse between the arrivals of job $k-1$ and k , then job k must wait in queue⁶

$$W_k = [W_{k-1} + S_{k-1} - X_k]^+, \quad W_0 = 0, \quad J_k = W_k + S_k. \quad (3.1.2)$$

However, if only J_k is needed, use $J_k = [J_{k-1} - X_k]^+ + S_k$, $J_0 = 0$.

LATER WE WILL need the system state at arbitrary times, not only at arrivals. The number of arrivals $A(t)$ during $[0, t]$ can be computed from $\{A_k\}$ as⁷

$$A(t) := \sum_{k=1}^{\infty} \mathbb{1}_{A_k \leq t} = \max\{k : A_k \leq t\}. \quad (3.1.3)$$

The function $t \rightarrow A(t)$ is right-continuous.⁸

Conversely, we can retrieve the arrival times $\{A_k\}$ from the process $\{A(t)\}$. For example, if $A(s) = k-1$ and $A(t) = k$, then the arrival time A_k of the k th job must lie somewhere in $(s, t]$. Specifically,

$$A_k = \min\{t : A(t) \geq k\}, \quad A(0) = 0.$$

And if $\{A_k\}$ is known, then $X_k = A_k - A_{k-1}$. Thus, the arrival times, the inter-arrival times, and the number of arrivals as a function of time can be derived from one another.

Similarly, for the number of jobs that started service, $\tilde{A}(t)$, and the departures $D(t)$ we have

$$\begin{aligned} \tilde{A}(t) &= \max\{k : \tilde{A}_k \leq t\}, & \tilde{A}_k &= \min\{t : \tilde{A}(t) \geq k\}, \\ D(t) &= \max\{k : D_k \leq t\}, & D_k &= \min\{t : D(t) \geq k\}. \end{aligned}$$

The *work in the system* $W(t)$ is the time needed to clear the remaining service of all jobs present at time t . To construct $\{W(t)\}$ graphically, we draw lines that start at points (A_k, J_k) and go down with slope -1 until the lines hit the x -axis.⁹ If the system is empty, the work in the system remains at zero until an arrival occurs. To express $W(t)$ as a function of time, note that $A(t)$ is the last arrival at t , thus, $D_{A(t)}$ is the time this job leaves. Therefore,

$$W(t) = [D_{A(t)} - t]^+. \quad (3.1.4)$$

The work in the system is equal to the so-called *virtual waiting time*. For example, if $W(t) = 5$ hours¹⁰, then a job arriving at t would have a waiting time of 5 hours. Fig. 3.1.1 shows how the above concepts relate to each other.

⁶ As we start counting jobs for $k = 1, 2, \dots$, W_0 has no physical meaning.

⁷ Read the RHS as the maximum of all k such that $A_k \leq t$.

⁸ Define $f(x-) = \lim_{y \uparrow x} f(y)$ and $f(x+) = \lim_{y \downarrow x} f(y)$. A function f is right-continuous if $f(x) = f(x+)$ for all x , and left-continuous if $f(x) = f(x-)$ for all x .

⁹ The slope is -1 because a single server processes one hour of work per hour.

¹⁰ It takes 5 hours to complete all jobs in the system at time t .

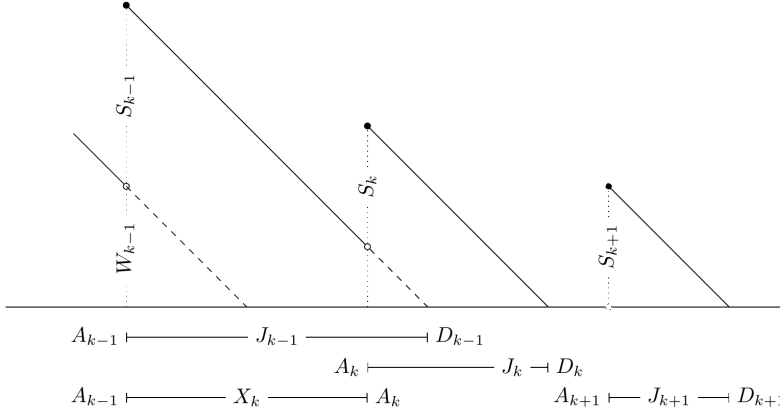


FIG. 3.1.1: The work in the system $W(t)$ as a function of time. Right after job k arrives at A_k , the work in the system satisfies $W(A_k) = J_k$, where the sojourn time J_k is the sum of the waiting time W_k and service time S_k . The server processes work at a rate of one hour per hour so W decreases as the solid lines with slope -1 . Once the system is empty, there is nothing to process, and W remains at 0 as long as the server is idle.

ONCE WE HAVE the arrival and departure processes, it is easy to compute the *number of jobs in the system* at time t , as illustrated in Fig. 3.1.2,

$$L(t) = A(t) - D(t) + L(0), \quad (3.1.5)$$

where $L(0)$ is the number of jobs in the system at time $t = 0$; typically $L(0) = 0$. Recalling that a job can be either waiting or in service, we distinguish $L(t)$ (in the system), $Q(t)$ (in queue), and $L_s(t)$ (in service):

$$L(t) = Q(t) + L_s(t), \quad L_s(t) = \tilde{A}(t) - D(t) = L(t) - Q(t).$$

Note that statistics obtained from a sample path $t \rightarrow Q(t)$ can be quite different from the samples $k \rightarrow Q(A_k-)$ as observed by arriving jobs, and likewise for $L(t)$ and $L(A_k-)$.¹¹

THE FOLLOWING INDICATOR function

$$I_k(s) := \mathbb{1}_{A_k \leq s < D_k}, \quad (3.1.6)$$

represents the presence of job k at time s .¹² Clearly, $\int_0^t I_k(s) ds$ corresponds to the total time job k spends in the system up to time t while summing over all jobs present at time s gives $L(s)$. Therefore,

$$J_k = \int_0^\infty I_k(s) ds, \quad L(t) = \sum_{k=1}^\infty I_k(t). \quad (3.1.7)$$

With this we can relate sojourn times and the number of jobs in the system. When n is the index of the last departure of a simulation, we have on the one hand,¹³

$$\frac{1}{D_n} \sum_{k=1}^n \int_0^{D_n} I_k(s) ds = \frac{1}{D_n} \sum_{k=1}^n J_k,$$

while on the other hand,

$$\frac{1}{D_n} \int_0^{D_n} \sum_{k=1}^n I_k(s) ds = \frac{1}{D_n} \int_0^{D_n} L(s) ds = \bar{L}.$$

¹¹ We need to be careful about the left and right limits at jump epochs.

¹² $I_k(s) = 1$ if job k is present in the system at time s and zero otherwise.

¹³ At time D_n all n jobs have departed.

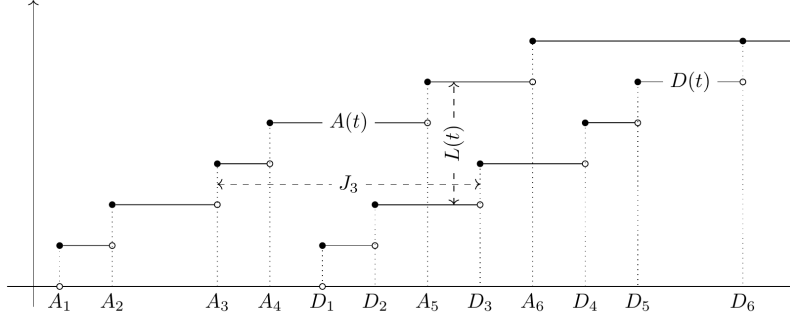


FIG. 3.1.2: Relation between the arrival process $\{A(t)\}$, the departure process $\{D(t)\}$, the number in the system $\{L(t)\}$ and the sojourn times $\{J_k\}$. Observe that the arrival and departure times of different jobs are intertwined.

As the expressions at the LHSs are equal, the sum over all sojourn times divided by the time of the last departure equals the time-average number of jobs in the system:

$$\bar{L} \approx \frac{1}{D_n} \sum_{k=1}^n J_k. \quad (3.1.8)$$

WE FINISH THIS section with constructing the recursions for the waiting times in a multi-server queue. Jobs line up in queue i , $i = 1, \dots, c$ that is served by a server working with rate $\mu(i)$. When job k arrives, it sees a vector $w_k = (w_k(1), \dots, w_k(c))$ of waiting times at the queues. Suppose the job selects the line with the shortest waiting time.¹⁴ Let $e(i)$ denote the i th unit vector¹⁵, and $\mathbf{1} = \sum_{i=1}^c e_i$. Then, in analogy with (3.1.2), the waiting time w_{k+1} updates as¹⁶

$$s_k = \operatorname{argmin}_{i \in \{1, \dots, c\}} \{w_k(i)\}, \quad w_{k+1} = [w_k + S_k e(s_k) / \mu(s_k) - X_k \mathbf{1}]^+, \quad (3.1.9)$$

where s_k is the queue selected by job k .

WHILE THE ABOVE recursions are charming in their simplicity, it is hard to generalize these recursions to more realistic queueing situations involving control. In later sections we discuss how to use discrete-event simulation to analyze such harder cases.

TF 3.1.1. Claim: with our notation, the following mapping is correct:

$$A_k : \mathbb{N} \rightarrow \mathbb{R}, \quad \text{job id (integer) to arrival time (real number).}$$

TF 3.1.2. Claim: the number of arrivals $A(t)$ during $[0, t]$ can be defined as $\min\{k : A_k \geq t\}$.

TF 3.1.3. Claim: The waiting time of the third job is correctly represented in Fig. 3.1.3.

TF 3.1.4. Claim: $A_{A(t)} = t$ for all t and $A(A_n) = n$ for all n .

TF 3.1.5. For the $G/G/1$ queue, the virtual waiting time process $\{W(t), t \geq 0\}$ satisfies

$$W(t) = [J(A_{A(t)}) + (A_{A(t)} - t)]^+.$$

¹⁴ This is not necessarily the same as the shortest queue.

¹⁵ A vector with a 1 at place i and zeros elsewhere.

¹⁶ $[\cdot]^+$ applies element-wise to the vector.

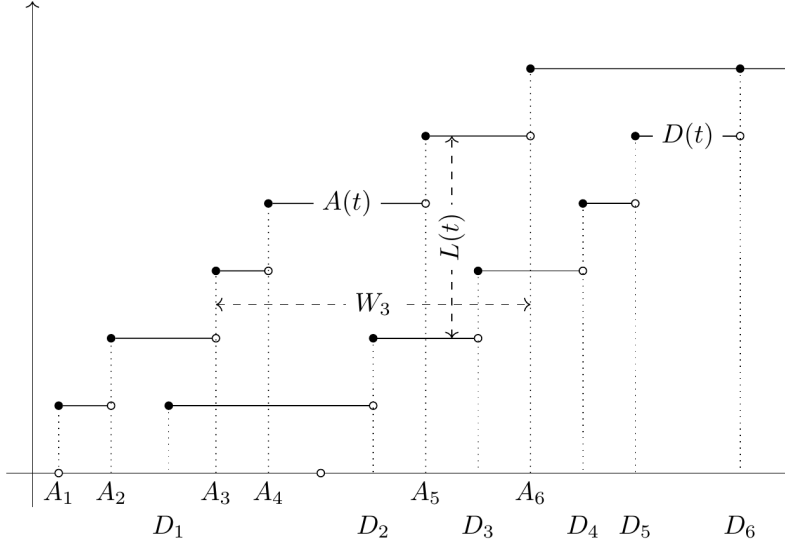


FIG. 3.1.3: The figure for [3.1.3].

Ex 3.1.6. Show that $L(A_k-) > 0 \iff A_k \leq D_{k-1}$.

Ex 3.1.7. Why is $W(A_k) = J_k$?

Ex 3.1.8. Modify (3.1.9) such that job k selects the queue with the shortest sojourn time.

3.2 KENDALL'S NOTATION

As is apparent from Sections 2.1 and 3.1, the construction of a queueing process for a single station involves a few main elements: the distribution of inter-arrival times for jobs, the distribution of service times, the number of servers present to process jobs, and the number of jobs that arrive and can be served simultaneously as a batch.

When $\{X_k\}$ and $\{S_k\}$ are iid and mutually independent, we can use *Kendall's abbreviation* $A^B/Y^C/c/K$ to characterize the type of queueing process. Here, A is the distribution of the iid inter-arrival times, and Y the distribution of the iid service times. When in an arrival epoch multiple jobs can arrive simultaneously⁰, we say that a *batch of jobs* arrives. The letter B denotes the (distribution of the) random batch size. Similarly, when the server can process multiple jobs as a batch¹, the letter C refers to the (distribution of the) service batch size. The letter c specifies the number of servers, and K the system size, i.e., the total number of customers that can be simultaneously present in the system, either waiting or in service.² Finally, the service discipline is assumed to be FIFO. When $B \equiv 1$ or $C \equiv 1$, that is, jobs arrive and are served individually, B or C are omitted. Likewise, if the system is unbounded, K is omitted. Moreover, service is assumed to be *work-conserving*, which means that a server cannot be idle when there is at least one job in the system.³ The most important inter-arrival and service distributions are the exponential distribution, denoted with the shorthand⁴ M , and the general distribution, denoted G . We write D for a



⁰ Like a group of people entering a restaurant.

¹ For instance, an oven can contain multiple jobs simultaneously.

² The meaning of K differs among authors. Sometimes, it represents the size of the queue, not the entire system. In this book, K denotes the system size.

³ Thus, when service is work-conserving, capacity is not wasted.

⁴ $M :=$ Markov or Memoryless.

deterministic rv.

IMPORTANT EXAMPLES INCLUDE the $M/M/1$ queue, with exponentially distributed inter-arrival times and service times; the $M/G/1$ queue, with exponential inter-arrival times and general service times; and the $G/G/c$ queue, where both the inter-arrival and service times are unspecified.

A common model for determining the number of beds required in a hospital ward is the $M/M/c/c$ queue. By the arguments of Section 1.3 patient inter-arrival times are exponentially distributed, and as there are many patients with various reasons for occupying a bed, the recovery times are (roughly) exponentially distributed. There are c beds available, and since each bed serves one patient,⁵ the hospital is ‘full’ when all c beds are occupied. Therefore $K = c$. The hospital’s problem is to determine the number c of beds needed such that the blocking probability is below some threshold, say 1%.

⁵ In a hospital patients typically don’t share beds.

The discrete-time model that corresponds to the recursion (2.1.1) can be represented as a $D^B/D^C/1$ queue in which $a_k \sim B$ and $c_k \sim C$.

An inventory under base-stock control with $S = 0$ is a $D^B/D/\infty$ queue: the total demand in a period is B , and each demand is delivered (served) after the lead time, hence it can be seen as if each demand receives its own server.

TF 3.2.1. Consider an $M^2/G/c/K$ queue, with $K \geq c$. Claim: Exactly one of the following statements is false.

- Jobs arrive in pairs of two.
- Service times can be deterministic.
- The queue shared among c parallel servers cannot contain more than $K - c$ jobs.

TF 3.2.2. In the $D/M/1$ queue, jobs have deterministic service times.

TF 3.2.3. Consider a check-in desk at an airport. One desk is dedicated to business customers. However, when this desk is idle (i.e., no business customer in service or in queue), it also serves economy class customers. The other c desks are reserved for economy class customers. The queueing process perceived by the economy class customers can be modeled as an $M/M/(c + 1)$ queue.

TF 3.2.4. Claim: The $M/G/1$ queue has Poisson arrivals, general service times, and a single server.

TF 3.2.5. Claim: In an $M/M/c/K$ queue, the K denotes the number of servers.

3.3 SIMULATING WAITING TIMES

Using the recursions of Section 3.1, we can simulate the waiting time of the $G/G/1$ queue. We use this to analyze how various blocking strategies can prevent long waiting times. In passing we will encounter subtle problems with stochastic dependence and with (plotting) pmfs.

THE DYNAMICS OF the single-server queue in continuous time is given by the recursions in (3.1.1). The corresponding code implements the formulas of Section 3.1 to compute the departure times D , the sojourn times J , the waiting times W , and the time-average number of jobs in the system from (3.1.8). We set $X[0] = 0$ so that $A[0] = 0$. Since job 0 is not served, we set $S[0] = 0$.

```

1  import numpy as np
2
3  from functions import Plus
4
5  rng = np.random.default_rng(3)
6  labda, mu = 1, 1.1
7  num = 10000
8  X = rng.exponential(scale=1 / labda, size=num)
9  S = rng.exponential(scale=1 / mu, size=num)
10 X[0] = S[0] = 0
11 A = X.cumsum()
12
13 D = np.zeros_like(X)
14 for i in range(1, num):
15     D[i] = max(A[i], D[i - 1]) + S[i]
16
17 J = D - A # sojourn times
18 W = J - S # waiting times
19 L_tilde = J.sum() / D[-1] # time-average number in system
20
21 print(f"{J.mean():.2f}, {J.std():.2f}")
22 print(f"{W.mean():.2f}, {W.std():.2f}")
23 print(f"{L_tilde:=0.3f}")

```

```

1  J.mean()=10.11, J.std()=9.03
2  W.mean()=9.20, W.std()=8.99
3  L_tilde=10.167

```

There is a small point of concern. Calling `J.mean()` computes the mean sojourn time, but this includes job 0. Since the simulation excludes this job, the correct mean is `J.mean() * len(J) / (len(J) - 1)`. Here we neglect this.⁰

We can also compute the maximum waiting time and the fraction of jobs that have to wait longer than $T = 5$. The expression `(W >= T)` returns a Boolean array whose i th element equals $\mathbb{1}_{W_i \geq T}$, so `(W >= T)` gives the total number of jobs with long waits, and `(W >= T).mean()` the fraction. We see that the maximum waiting time is indeed much larger than T , and that more than half of the jobs have to wait a long time.

```

1  T = 5 # threshold for long waiting
2
3  print(f"{W.max():.2f}, {(W >= T).mean():.2f}")

```

```

1  W.max()=52.12, (W >= T).mean()=0.59

```

⁰ Be aware, in critical software you should not use such tricks.

BLOCKING JOBS CAN be a remarkably effective strategy to prevent long and variable waiting times. One rule, known as *complete acceptance*, only accepts a job when $W_k \leq T$, and otherwise blocks the job.

Here is a hack to implement this rule. When $W_k > T$, job k should not enter. That means that its service time should not be added to the waiting time, which can be achieved by setting $S_k = 0$.

```

1 W = np.zeros_like(S)
2 for k in range(1, len(S)):
3     W[k] = Plus(W[k - 1] + S[k - 1] - X[k])
4     S[k] = S[k] if W[k] <= T else 0

```

Such code is undesirable; it modifies the primary data.¹ Consequently, these data cannot be reused later.²

A better approach uses an indicator I_k . When $W_k \geq T$, set $I_k = 0$ (reject); otherwise $I_k = 1$ (accept). Note that job k depends on $I[k-1]$.

```

1 W = np.zeros_like(S) # Reset waiting times
2 I = np.zeros_like(S, dtype=int) # Accept or not
3 for k in range(1, len(S)):
4     W[k] = Plus(W[k - 1] + S[k - 1] * I[k - 1] - X[k])
5     I[k] = 1 if W[k] <= T else 0

```

Under this rule, about 90% of the jobs are accepted ($I.mean()$ is 0.9), yet $E[W]$ drops by a factor 4, and the maximum by more than a factor 10.

```

1 print(f"W.mean()=:.2f", {I.mean()=:.2f})
2 print(f"W[I == 1].mean()=:.2f", {W[I==1].max()=:.2f})
3 print(f"len(W)=, {len(W[I == 1])=}")

```

```

1 W.mean()=2.23, I.mean()=0.90
2 W[I == 1].mean()=1.83, W[I==1].max()=5.00
3 len(W)=10000, len(W[I == 1])=8994

```

There are other interesting rules to block jobs. A second rule, *complete rejection*, accepts only if $J_k \leq T$.

```

1 W = np.zeros_like(S)
2 I = np.zeros_like(S, dtype=int) # Accept or not
3 for k in range(1, len(S)):
4     W[k] = Plus(W[k - 1] + S[k - 1] * I[k - 1] - X[k])
5     I[k] = 1 if W[k] + S[k] <= T else 0

```

This is an interesting result. Compared with complete acceptance, the acceptance probability is *higher* because jobs never occupy the server longer than $T = 5$.³

```

1 print(f"W.mean()=:.2f", {I.mean()=:.2f})
2 print(f"S.mean()=:.2f", {S[I == 1].mean()=:.2f})
3 print(f"S.max()=:.2f", {S[I == 1].max()=:.2f})

```

¹ Here the service times.

² Golden rule: never modify primary data.

³ In other words, only small jobs are accepted.

```

1 W.mean()=1.21, I.mean()=0.97
2 S.mean()=0.82, S[I == 1].mean()=0.75
3 S.max()=9.25, S[I == 1].max()=4.92

```

A third rule, *partial acceptance*, keeps a middle ground: it accepts only the portion of a job's service time that fits, that is, $\min\{T - W_k, S_k\}$.

```

1 W = np.zeros_like(S)
2 J = np.zeros_like(S) # Sojourn times
3 accepted_service = np.ones_like(S)
4 for k in range(1, len(S)):
5     W[k] = Plus(W[k - 1] + accepted_service[k - 1] - X[k])
6     accepted_service[k] = min(T - W[k], S[k])
7     J[k] = W[k] + accepted_service[k]
8
9 print(f"W.mean()=:.2f, {J.mean()=:.2f}")
10 print(f"accepted_service.mean()=:.2f")
11 print(f"accepted_service.max()=:.2f")

```

```

1 W.mean()=1.58, J.mean()=2.36
2 accepted_service.mean()=0.78
3 accepted_service.max()=5.00

```

WE NEXT SHOW how to obtain the *distribution* of the waiting time and departure time of job $k = 30$.⁴ We use both the probabilistic arithmetic of Section 1.4 and simulation.

⁴ Job $k = 30$ is chosen arbitrarily.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 from functions import Plus
5 from random_variable import NumericRV as RV

```

The inter-arrival and service distributions are as follows. We choose $E[S] > E[X]$ to get more widely spread values in the figures.

```

1 X = RV({3: 1 / 3, 4: 1 / 3, 5: 1 / 3})
2 S = RV({4: 1 / 3, 5: 1 / 3, 7: 1 / 3})
3 print(f"{X.mean()=:.2f}, {S.mean()=:.2f}")

```

```

1 X.mean()=4.00, S.mean()=5.33

```

We set the initial arrival such that $P\{A_0 = 0\} = 1$. The first call `A += X` computes the pmf of the rv A_1 . Since the system starts empty, $P\{W_1 = 0\} = 1$. In the for loop, we update A_k according to the rule $A_k = A_{k-1} + X_k$ and the waiting time with $W_k = [W_{k-1} + S_{k-1} - X_k]^+$. We start the for loop at 2, because the lines before the loop compute the pmf of A_1 and W_1 . We later explain why the line computing D is *wrong*.

```

1 horizon = 30
2 A = RV({0: 1}) # Arrival time A_0
3 A += X # Arrival time A_1
4 W = RV({0: 1}) # Waiting time W_1
5 # Mind that we start at 2 instead of 1
6 for i in range(2, horizon):
7     A += X
8     W = (W + S - X).map(Plus)
9
10 D = A + W + S # These are not the departure times!
11
12 print(f"W.mean()=:0.2f}, {W.std()=:0.2f}")
13 print(f"D.mean()=:0.2f}, {D.std()=:0.2f}")

```

```

1 W.mean()=37.51, W.std()=7.82
2 D.mean()=158.84, D.std()=9.06

```

Next, we use simulation to estimate the pmf and cdf of W_{30} and D_{30} . The printed values show that the mean and the standard deviations for the waiting time match well between theory and simulation. However, the standard deviation of d_{30} differs noticeably, indicating an issue.

```

1 num_runs = 1000
2 w30 = np.zeros(num_runs)
3 d30 = np.zeros(num_runs)
4 rng = np.random.default_rng(3)
5 for run_no in range(num_runs):
6     x = X.rvs(horizon, rng)
7     s = S.rvs(horizon, rng)
8     x[0] = s[0] = w = 0
9     for i in range(1, horizon):
10         w = Plus(w + s[i - 1] - x[i])
11
12     w30[run_no] = w
13     a = x.cumsum()
14     d30[run_no] = a[i] + w + s[i]
15
16 print(f"w30.mean()=:0.2f}, {w30.std()=:0.2f}")
17 print(f"d30.mean()=:0.2f}, {d30.std()=:0.2f}")

```

```

1 w30.mean()=37.36, w30.std()=7.61
2 d30.mean()=158.74, d30.std()=6.45

```

We now make a plot of the cdf and pmf of W_{30} , see Fig. 3.3.1. The keyword `cumulative=True` converts a histogram into a cdf.⁵

⁵ Recall that a histogram approximates a pmf.

```

1 fig, (ax1, ax2) = plt.subplots(nrows=1, ncols=2, figsize=(6, 3))
2 ax1.set_title("Waiting times cdf")
3 ax1.set_xlabel("Time")
4 ax1.plot(
5     W.support(), [W.cdf(k) for k in W.support()], c='k', lw=0.75, label="exact"
6 )

```

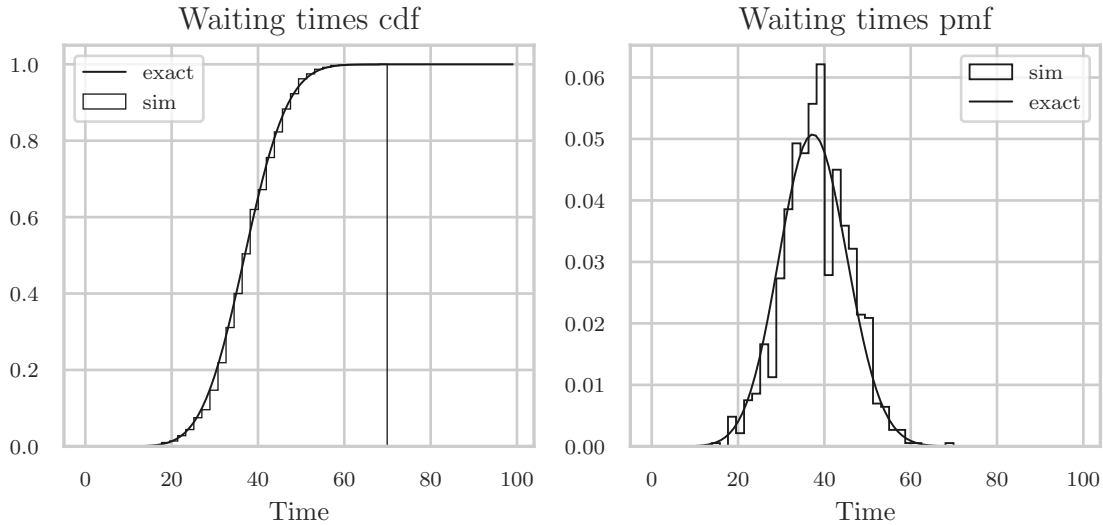


FIG. 3.3.1: The cumulative distribution and pmf of the waiting time of job 30 with the inter-arrival and service times as mentioned in the text. Notice that the distribution shifts to the right because the queue is unstable. The results of theory and simulation are very similar.

```

7  ax1.hist(
8      w30,
9      bins=30,
10     density=True,
11     cumulative=True,
12     histtype='step',
13     align="right",
14     color='k',
15     lw=0.5,
16     label="sim",
17 )
18 ax1.legend()
19
20 ax2.set_title("Waiting times pmf")
21 ax2.set_xlabel("Time")
22 ax2.hist(
23     w30,
24     bins=30,
25     density=True,
26     histtype='step',
27     align="right",
28     color='k',
29     lw=0.7,
30     label="sim",
31 )
32 x = W.support()
33 y = [W.pmf(k) for k in W.support()]
34 ax2.plot(x, y, color='k', lw=0.75, label="exact")
35 ax2.legend()
36 fig.tight_layout()
37 fig.savefig("/home/nicky/Dropbox/vakken/qts/stochastic_or/figures/waiting_time_distribution.pdf")

```

LET US NOW discuss why the theoretical D_{30} and simulated d_{30} have different standard deviations. The reason is that A_k and W_k are negatively correlated. Recall that $A_k = A_{k-1} + X_k$ and $W_k = [W_{k-1} + S_{k-1} - X_k]^+$. If X_k

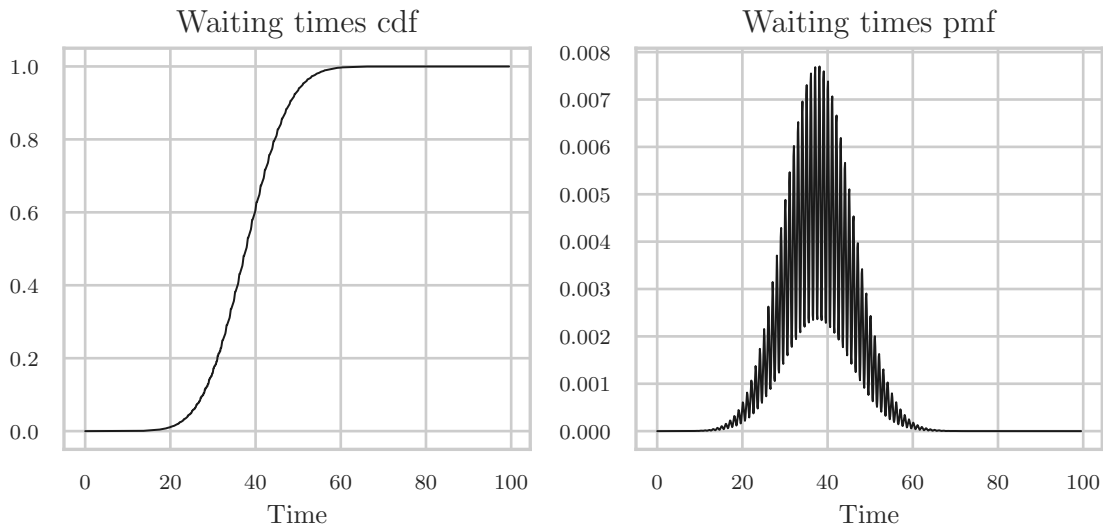


FIG. 3.3.2: The cdf and pmf of the waiting time of job 30 with X with support $\{3, 4.1, 5\}$ instead of $\{3, 4, 5\}$; the rest of the parameters are as in Fig. 3.3.1. Note how pathological the pmf of the theoretical distribution behaves.

is very large, A_k is large, but W_k is zero (or small), and then $D_k \approx A_k + S_k$. In probabilistic arithmetic, addition of two rvs is only allowed when they are independent.⁶ The expression $D = A + W + S$ violates this assumption. But why then is the recursion $W_k = [W_{k-1} + S_{k-1} - X_k]^+$ correct? This is because W_{k-1} is independent of S_{k-1} and X_k .

Stochastic dependence is easily overlooked. To avoid making mistakes, check analytical results with simulation. If the results disagree, there is probably an error⁷ somewhere.

IN GENERAL, THE cdf behaves more regularly than the pmf, even though the pmf seems easier to interpret. To show this, modify the support of X to $\{3, 4.1, 5\}$, keep all else the same, and simulate. The resulting pmf in Fig. 3.3.2 differs markedly from that in Fig. 3.3.1!

WE FINISH THIS section with coding the multi-server queue of (3.1.9). We take $\mu(i) = 1$ for all servers.

⁶ Handling such dependencies requires the modeling of the joint distribution of rvs. The Python package *lea* supports this.

⁷ Often not just one error!

```

1  import numpy as np
2
3  c = 3
4  N = 10
5
6  one = np.ones(c, dtype=int) # vector with ones
7  mu = np.array([1, 1, 1])
8  X = np.ones(N + 1, dtype=int)
9  S = 5 * np.ones(N, dtype=int)
10 w = np.zeros(c, dtype=int)
11 W = J = A = D = 0
12
13 for k in range(1, N):
14     A += X[k] # arrival time
15     s = w.argmin() # server chosen
16     W = w[s] # waiting time
17     S = S[k] / mu[s]
```

```

18     J = W + S # sojourn time
19     D = A + J # departure time
20     print(f"{k=}, {S=:.2f}, {W=:.2f}, {w=}")
21     w[s] += S # update w
22     w = np.maximum(0, w - X[k + 1] * one)

```

```

1 k=1, S=5.00, W=0.00, w=array([0, 0, 0])

```

TF 3.3.1. Claim: This code prints the mean departure time.

```

1 import numpy as np
2
3 rng = np.random.default_rng(3)
4 labda, mu = 3, 4
5 num = 10
6 X = rng.exponential(scale=1 / labda, size=num)
7 Z = rng.exponential(scale=1 / mu, size=num)
8 X[0] = Z[0] = 0
9 A = X.cumsum()
10
11 D = np.zeros_like(X)
12 for i in range(1, num):
13     D[i] = max(A[i], D[i - 1]) + Z[i]
14
15 print(f"D.mean()=:.2f")

```

```

1 1.9085250664726694

```

TF 3.3.2. Consider a continuous-time queue with inter-arrival times $\{X_k\}$, service times $\{S_k\}$, waiting times $\{W_k\}$, and sojourn times $\{J_k\}$. Claim: we can compute the waiting times and sojourn times using the recursion:

$$W_k = [J_{k-1} - X_k]^+, \quad J_k = W_k + S_k.$$

TF 3.3.3. Claim: This code uses the recursion $D_k = \max\{D_{k-1}, A_k\} + S_k$ in the correct way to compute the probability distribution of $\{D_k\}$.

```

1 A = .RV({0: 1})
2 D = RV({0: 1})
3 for i in range(1, horizon):
4     A += X
5     D = RV.map2(D, A, max) + S

```

TF 3.3.4. Claim: In the $G/G/c$ queue, the next rule selects the queue with the smallest sojourn time.

```

1 for k in range(1, N):
2     A += X[k] # arrival time
3     s = w.argmin() # server chosen
4     W = w[s] # waiting time
5     S = S[k] / mu[s]

```

```

6     J = W + S # sojourn time
7     D = A + J # departure time
8     print(f"{k=}, {S=:.2f}, {W=:.2f}, {w=}")
9     w[s] += S # update w
10    w = np.maximum(0, w - X[k + 1] * one)

```

TF 3.3.5. Claim: In a $G/G/c$ queue simulation, the waiting time of job k can be computed as $W_k = [W_{k-1} + S_{k-1} - X_k]^+$.

3.4 SIMULATION OF QUEUE LENGTH WITH HEAP QUEUES

We have seen in Section 3.3 that the recursions to compute waiting and sojourn times for a continuous-time single-server queue can be implemented in a straightforward way. Here we handle the work process $\{W(t)\}$ and the system-size process $\{L(t)\}$. This is somewhat complicated because job arrival and departure events are intertwined⁰, while to plot properly, these events have to be chronologically sorted.

⁰ See Fig. 3.1.2.

WE WORK TOWARDS Fig. 3.4.1. The top panel ax1 shows the graph of a sample path of $\{W(t)\}$, the lower panel ax2 the graph of $\{L(t)\}$.

```

1 fig, (ax1, ax2) = plt.subplots(nrows=2, figsize=(6, 3), sharex=True)
2 ax1.set_ylabel("Work $W$")
3 ax2.set_ylabel("Length $L$")
4 ax2.set_xlabel("time")
5 ax2.set_yticks([0, 1, 2, 3, 4, 5])

```

With A_k, D_k, J_k and W_k , we can plot triangles for each job with height and base equal to the sojourn or waiting time.¹

¹ Setting the opacity `alpha` to 0.2 gives nice results for overlapping triangles.

```

1 for i in range(1, num):
2     ax1.fill([A[i], A[i], D[i]], [0, J[i], 0], c='green', alpha=0.2)
3     ax1.fill([A[i], A[i], A[i] + W[i]], [0, W[i], 0], c='blue', alpha=0.2)

```

For clarity, we also want to plot vertical dotted lines to indicate that at an arrival at t , the work moves from $W(t-)$ to $W(t)$, and downward sloping solid lines between the arrivals. Consequently, we need to be able to compute $W(t-)$ and $W(t)$ for arbitrary t , which, in view of the definition $W(t) = [D_{A(t)} - t]^+$, requires computing $A(t-)$ and $A(t)$. The following function uses `np.searchsorted` to efficiently² find the index k in an array a such that $a_{k-1} \leq t < a_k$ for a given t .³ The option `right` specifies the right-continuous (by default) or left-continuous version of $A(t-)$.

² For the same reason, we used binary search in the `~RV` class in Section 1.4.

³ [3.4.7] explains why the following function returns $A(t)$, in particular why to subtract 1.

```

1 def At(t, side='right'):
2     "The right or left continuous version of A(t)."
3     return max(np.searchsorted(A, t, side=side) - 1, 0)

```

From this, the work $W(t)$ can now be implemented directly.

```

1 def W(t, side='right'):

```

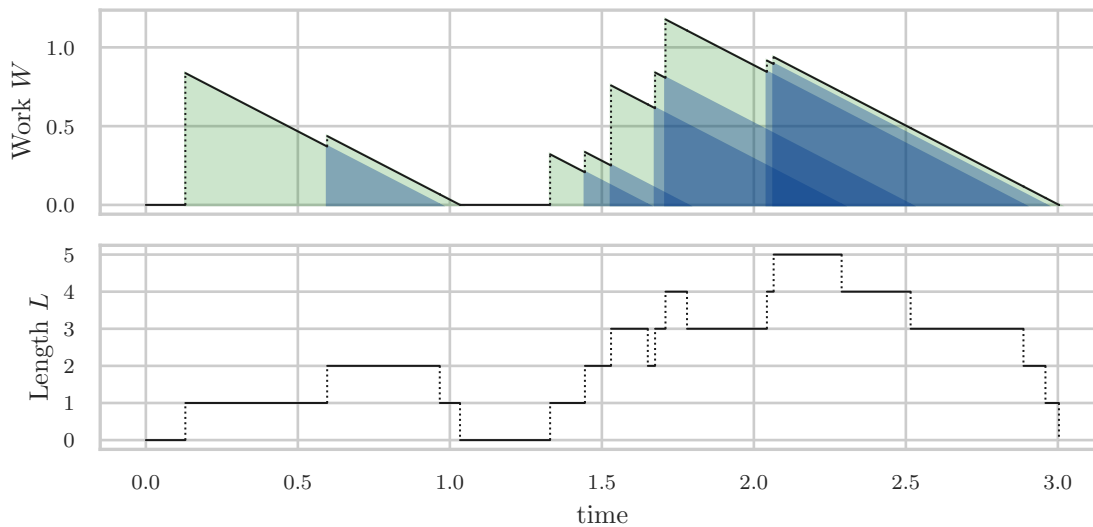


FIG. 3.4.1: The top panel shows the work in the system $W(t)$; the green triangles correspond to the sojourn times J_k and the blue triangles to the waiting times W_k . The lower panel shows the number of jobs in the system.

```

2      "The right or left continuous work in the system."
3      k = At(t, side)
4      return max(D[k] - t, 0)

```

To plot W we still need an efficient algorithm to sort chronologically arrival and departure times. For this, we use a heap queue.

A `HEAP QUEUE` stores tuples (or lists) in a list such that the tuples (or lists) can be retrieved according to some ordering rule. Here is an example. Tuples with ages and animal names can be arbitrarily *pushed* onto a *heap*.

```

1  from heapq import heappop, heappush
2
3  heap = []
4
5  heappush(heap, (25, "Turtle"))
6  heappush(heap, (14, "Dog"))
7  heappush(heap, (18, "Cat"))
8  heappush(heap, (21, "Horse"))
9  heappush(heap, (20, "Lion"))

```

Popping from the heap yields the animals in lexicographical order.⁴

```

1  print(heappop(heap)) # Why is it the dog?
2  print(heappop(heap)) # Why is it the cat?

```

```

1  (14, 'Dog')
2  (18, 'Cat')

```

We can also push a new animal after popping.

⁴ As here the age is the first element of the tuple, the sorting is here in age.

```
1 heappush(heap, (23, "Elephant")) # Add Elephant
```

Let's pop until the heap is empty. Observe that the dog and cat are removed from the heap, due to the earlier popping.

```
1 while heap:
2     e = heappop(heap)
3     print(e)
```

```
1 (20, 'Lion')
2 (21, 'Horse')
3 (23, 'Elephant')
4 (25, 'Turtle')
```

A heap queue is algorithmically efficient: it supports pushing at our convenience while guaranteeing that elements will be popped in the correct order. Thus, it is the ideal data structure for ordering arrival and departure events that are intertwined.

WITH HEAP QUEUES we can plot the work W and the system size L for Fig. 3.4.1. For $\{L(t)\}$, we start with setting $L=0$ and add 1 when an arrival occurs, and subtract 1 for a departure. We therefore form a list of arrival events of type $(A_k, 1)$ and a second list $(D_k, -1)$ for the departures. As these lists are already sorted, merging them suffices.⁵

```
1 arrivals = [(t, 1) for t in A[1:]]
2 departures = [(t, -1) for t in D[1:]]
3 events = list(heapq.merge(arrivals, departures))
4 heapq.heapify(events)
```

Finally, we can plot the dotted and solid lines for W , and L . Note that $W(\text{now}, \text{'left'})$ corresponds to $W(t-)$ when the time t is now.

```
1 L, past = 0, 0
2 while events:
3     now, delta = heapq.heappop(events)
4     ax1.plot([past, now], [W(past), W(now, 'left')], c='k', lw=0.75)
5     ax1.plot([now, now], [W(now, 'left'), W(now)], ":", c='k', lw=0.75)
6     ax2.plot([past, now], [L, L], c='k', lw=0.75)
7     ax2.plot([now, now], [L, L + delta], ":", c='k', lw=0.75)
8     L += delta
9     past = now
```

⁵ The function `merge` returns a generator, not a list. However, `heapify` expects a list instead of a generator. We call `list()` to convert the generator.

TF 3.4.1. We have a single-server queue where the inter-arrival and service times have a general distribution. Claim: when $L(T) = 0$ at time T , then

$$\begin{aligned} \int_0^T L(s) \, ds &= \int_0^T \sum_{k=1}^{A(T)} 1\{A_k \leq s < D_k\} \, ds \\ &= \sum_{k=1}^{A(T)} \int_0^T 1\{A_k \leq s < D_k\} \, ds = \sum_{k=1}^{A(T)} J_k. \end{aligned}$$

TF 3.4.2. Claim: the following code pops the element: (1.9, 'Long').

```

1  from heapq import heappop, heappush
2  heap = []
3  heappush(heap, (1.9, "Long"))
4  heappush(heap, (1.7, "Egg"))
5  heappush(heap, (1.3, "Zeno"))
6  print(heappop(heap))

```

TF 3.4.3. Claim: In a heap queue (min-heap), the element with the largest key is always popped first.

TF 3.4.4. Claim: In the simulation of a single-server queue, the time-average number of jobs $E[L] = \lim_{T \rightarrow \infty} \frac{1}{T} \int_0^T L(s) ds$ can be computed from the sojourn times $\{J_k\}$ alone, without tracking $L(s)$ at every moment.

TF 3.4.5. Claim: In a discrete-event simulation, events must be processed in the order in which they were created.

Ex 3.4.6. Suppose that we want to compute the time-average of $\bar{L}$ for some time $t < D_n$, i.e., before the departure of the last job. How would you compute this?

Ex 3.4.7. Recall that `np.searchsorted(A, t)` efficiently finds the index k in an array A such that $A_{k-1} \leq t < A_k$ for some given value t . When using this function, do we obtain $A(t) - 1$, $A(t)$ or $A(t) + 1$?



3.5 DISCRETE-EVENT SIMULATION

The queueing and inventory systems in the previous sections were based on simple recursions, and therefore straightforward to implement. When we need to analyze more difficult situations, we need discrete-event simulation. In this section we build a simulator for a $G/G/c$ queue with different server speeds and test it. In the next section, we apply this to some non-trivial queueing situations. We will see the power of Python classes.

THE FIRST CLASS is the `Server` which has an `id` and a service rate. The method `__lt__` enables us to express a preference for a server with a faster service rate.

```

server.py
1  from dataclasses import dataclass
2
3
4  @dataclass
5  class Server:
6      id: int
7      rate: float
8
9      def __lt__(self, other):
10         return self.rate > other.rate

```

The `Servers` class represents the pool of free servers. We subclass `Servers` from `list` so that it can be used by the `heapq` algorithms to push and pop

servers. When a job leaves, Servers assigns the next job in queue to a server based on the priority rule stated in `Server.__lt__`. A server that becomes idle after finishing a job is pushed to the server pool. The convenience methods `num_free` and `is_server_available` improve readability compared to using `len` directly.

```

1  from heapq import heappop, heappush
2
3
4  class Servers(list):
5      def push(self, server):
6          heappush(self, server)
7
8      def pop(self):
9          return heappop(self)
10
11     def num_free(self):
12         return len(self)
13
14     def is_server_available(self):
15         return self.num_free() > 0

```

The Job class speaks mostly for itself. The service time equals the job's load divided by the rate of the server that serves the job.⁰ The job keeps a reference to its assigned server. The other attributes are used for the statistics computations at the end of the simulation. We implement the sojourn time and waiting time as a `property`, as this allows us to access them the same way as, for example, the departure time. The `__lt__` method determines which job to select from the queue when a service can start. As it is now, we select jobs in FIFO sequence. The `__hash__` method assigns each job a unique id, enabling its use in a set or as a key in a dict.

⁰ Since servers may have different rates, we can only compute the service time after the job is assigned to a server.

```

1  from dataclasses import dataclass
2
3  from server import Server
4
5
6  @dataclass
7  class Job:
8      id: int
9      arrival_time: float
10     load: float
11     service_time: float = 0
12     departure_time: float = 0
13     q_length_at_arrival: int = 0
14     q_length_at_departure: int = 0
15     free_servers: int = 0
16     server: Server = None
17
18     @property
19     def sojourn_time(self):
20         return self.departure_time - self.arrival_time
21
22     @property
23     def waiting_time(self):
24         return self.sojourn_time - self.service_time

```

```

25
26     def __str__(self):
27         return (
28             f"{self.id}, load={self.load:.2f}, service={self.service_time:.2f},"
29             f" arr={self.arrival_time:.2f}, dep={self.departure_time:.2f}"
30         )
31
32     def __hash__(self):
33         return self.id
34
35     def __lt__(self, other):
36         return self.arrival_time < other.arrival_time

```

The Queue class also subclasses `list` so that it can act as a heap and use `Job.__lt__` to select the next job from the queue.

```

_____ queues.py _____
1  from heapq import heappop, heappush
2
3
4  class Queue(list):
5      def pop(self):
6          return heappop(self)
7
8      def push(self, job):
9          heappush(self, job)
10
11     def length(self):
12         return len(self)
13
14     def is_empty(self):
15         return self.length() == 0

```

An Event contains a time and a job; the `__lt__` method specifies how to order events.¹ The `ArrivalEvent` and `DepartureEvent` allow us to distinguish the relevant type of event.

¹ Clearly, this must be by time.

```

_____ event.py _____
1  from dataclasses import dataclass
2
3  from job import Job
4
5
6  @dataclass
7  class Event:
8      time: float
9      job: Job
10
11     def __lt__(self, other):
12         return self.time < other.time
13
14
15     class ArrivalEvent(Event):
16         pass
17
18     class DepartureEvent(Event):
19         pass
20

```

The Events class is a heap queue and keeps the events *chronologically* sorted.²

```

1  from heapq import heappop, heappush
2
3
4  class Events(list):
5      def push(self, event):
6          heappush(self, event)
7
8      def pop(self):
9          return heappop(self)
10
11     def is_empty(self):
12         return len(self) == 0

```

² Once you understand that the Events class offers the functionality to remove and insert events arbitrarily but still pop them sorted in time, you understand the core of discrete-event simulation.

The Statistics class is a subclass of `list` allowing us to append the jobs in the order in which they depart from the server.

```

1  from job import Job
2
3
4  class Statistics(list[Job]):
5      def push(self, job: Job):
6          self.append(job)
7
8      def num_jobs(self):
9          return len(self)
10
11     def mean_waiting_time(self):
12         tot = sum(job.waiting_time for job in self)
13         return tot / self.num_jobs()
14
15     def mean_sojourn_time(self):
16         tot = sum(job.sojourn_time for job in self)
17         return tot / self.num_jobs()
18
19     def mean_queue_length(self):
20         tot = sum(job.q_length_at_arrival for job in self)
21         return tot / self.num_jobs()
22
23     def mean_servers_free(self):
24         tot = sum(job.free_servers for job in self)
25         return tot / self.num_jobs()

```

The Simulator is the final class³ and it stores the events, queue, statistics, and servers. The attribute `now` points to the current simulation time. When starting the simulation, we provide a list of jobs to the simulator, which pushes an `ArrivalEvent` for each new job onto the event stack.

When a job service starts, the `serve_job` method asks for a free server from the server pool⁴ and assigns it to the job. Then it computes the job service time and departure time, and pushes a `DepartureEvent` to the event stack to inform the simulator that a job will leave at that time.

The `run` method checks whether events remain; if so, it pops the next event from the event stack. It then updates `now` to the event time and retrieves the associated job. If the event is an `ArrivalEvent`, the job is served

³ Study it carefully.

⁴ At a later stage, we might have different server pools. We therefore add this as an argument.

if a server is free; otherwise, it is queued. In case of a `DepartureEvent`, the server assigned to the job becomes free and is returned to the server pool. The departing job is pushed to the statistics tracker. If there are still jobs in the queue, a new job is taken into service.

```

1  import events
2  import queues
3  import server
4  import servers
5  import stats
6  from event import ArrivalEvent, DepartureEvent
7  from job import Job
8
9
10 class Simulation:
11     def __init__(self):
12         self.events = events.Events()
13         self.queue = queues.Queue()
14         self.stats = stats.Statistics()
15         self.servers = servers.Servers()
16         self.now = 0
17
18     def initialize_jobs(self, jobs):
19         for job in jobs:
20             self.events.push(ArrivalEvent(job.arrival_time, job))
21
22     def initialize_servers(self, server_rates):
23         for i, rate in enumerate(server_rates):
24             self.servers.push(server.Server(id=i, rate=rate))
25
26     def serve_job(self, job: Job, server_pool=None):
27         "Serve job from a free server of the server pool."
28         if server_pool is None:
29             server_pool = self.servers
30         server = server_pool.pop()
31         job.free_servers = server_pool.num_free()
32         job.server = server
33         job.service_time = job.load / server.rate
34         job.departure_time = self.now + job.service_time
35         self.events.push(DepartureEvent(job.departure_time, job))
36
37     def handle_arrival(self, job):
38         job.q_length_at_arrival = self.queue.length()
39         if self.servers.is_server_available():
40             self.serve_job(job)
41         else:
42             self.queue.push(job)
43
44     def handle_departure(self, job):
45         self.servers.push(job.server)
46         self.stats.push(job)
47         if not self.queue.is_empty():
48             self.serve_job(self.queue.pop())
49         job.q_length_at_departure = self.queue.length()
50
51     def run(self):
52         while not self.events.is_empty():
53             event = self.events.pop()
54             self.now, job = event.time, event.job
55             if isinstance(event, ArrivalEvent):

```

```

56         self.handle_arrival(job)
57     elif isinstance(event, DepartureEvent):
58         self.handle_departure(job)
59     else:
60         raise ValueError("Unknown event")

```

TESTING THE CODE comes next. In Section 5.1, we derive exact results for the expected waiting time of the $M/M/c$ queue, so this model provides a good test. The library `mmc.py`⁵ contains the code for the $M/M/c$ queue.

⁵ See *GitHub*.

For the generation of the data for the simulation, we follow the same procedure as in Section 3.3.

```

test_mmc.py
1  import numpy as np
2  import mmc
3  from job import Job
4  from simulator import Simulation
5
6
7  def run_mmc(num, labda=3, mu=4, server_rates=[1, 1]):
8      rng = np.random.default_rng(3)
9      X = rng.exponential(scale=1 / labda, size=num)
10     X[0] = 0
11     A = X.cumsum()
12     loads = rng.exponential(scale=1 / mu, size=num)
13     jobs = [Job(id=i, arrival_time=A[i], load=loads[i]) for i in range(1, num)]
14
15     sim = Simulation()
16     sim.initialize_jobs(jobs)
17     sim.initialize_servers(server_rates=server_rates)
18     sim.run()
19
20     c = len(server_rates)
21     mm = mmc.MMC(labda, mu, c=c)
22
23     print(f"W: {sim.stats.mean_waiting_time():0.4f}, {mm.EW=:0.4f}")
24     print(f"J: {sim.stats.mean_sojourn_time():0.4f}, {mm.EJ=:0.4f}")
25     print(f"Q: {sim.stats.mean_queue_length():0.4f}, {mm.EQ=:0.4f}")

```

Here is the output for 10^4 jobs. The results are reasonable, but the differences in $E[W]$ and $E[Q]$ are still quite large.

```

1  W: 0.0441, mm.EW=0.0409
2  J: 0.2952, mm.EJ=0.2909
3  Q: 0.1467, mm.EQ=0.1227

```

Let us repeat the experiment, but now with 10^6 jobs. The results show a remarkable improvement.⁶

```

1  W: 0.0410, mm.EW=0.0409
2  J: 0.2915, mm.EJ=0.2909
3  Q: 0.1225, mm.EQ=0.1227

```

In the next section we demonstrate with a few examples how flexible this simulation environment actually is.

⁶ This brings up a philosophical problem. If we need to simulate with one million jobs to see the time-average statistics appear, what is the significance of such time-averages? In how many practical situations will we see such numbers of jobs?

TF 3.5.1. Claim: This part of the `Simulator` class handles arrivals correctly.

```

1 class Simulation:
2     def handle_arrival(self, job):
3         job.q_length_at_arrival = self.queue.length()
4         if self.servers.is_server_available():
5             self.queue.push(job)
6         else:
7             self.serve_job(job)

```

TF 3.5.2. Claim: This part of the `simulator` class handles the serving of jobs correctly.

```

1 class Simulation:
2     def serve_job(self, job):
3         server = self.servers.pop()
4         job.server = server
5         job.service_time = job.load / server.rate
6         job.departure_time = self.now + job.service_time
7         job.queue_length = self.queue.length()
8         job.free_servers = self.servers.num_free()
9         self.events.push(DepartureEvent(job.departure_time, job))

```

TF 3.5.3. Claim: This part of the `Simulator` class handles departures correctly.

```

1 class Simulation:
2     def handle_departure(self, job):
3         self.servers.push(job.server)
4         self.stats.push(job)
5         if not self.queue.is_empty():
6             self.serve_job(self.queue.pop())
7         job.q_length_at_departure = self.queue.length()

```

TF 3.5.4. Claim: In discrete-event simulation, time advances by a fixed increment each step.

TF 3.5.5. Claim: In a discrete-event simulation framework based on object-oriented programming, subclassing allows us to reuse the event loop and only override specific behavior such as arrival handling.

Ex 3.5.6. Consult the website of RealPython to gain a better understanding of classes, dataclasses, `heapq`, and `deque`.

Ex 3.5.7. There are some advantages to subclassing `Queue` from `deque`. What are these advantages?

Ex 3.5.8. Consider a multi-server queue with m servers. Suppose that at some time t it happens that $\tilde{A}(t) - D(t) < m$ but $A(t) - D(t) > m$. How can this occur?

3.6 DISCRETE-EVENT SIMULATIONS: EXAMPLES

The discrete-event simulator for the $G/G/c$ queue developed in the previous section can be easily adapted to other queueing systems. We first analyze the effects of several server speeds for the $G/G/c$ queue. Then we implement a queue in which jobs can be served with priority.⁰ Next, we focus on a policy that controls the server based on the number of jobs in the system. Our final example is more extensive: we analyze whether servers dedicated to business customers in an airport check-in process should be shared with economy class customers. This would correspond to a part of a business consultancy project.

CONSIDER A STATION with three servers with rates 1, 0.1, and 0.01, respectively, and such that whenever multiple servers are free, the fastest available one is chosen. If queue sizes are very long, then it could help to use the extra capacity offered by the two slower servers. However, when the load is small and a job ends up at the slowest server, its sojourn time is large¹. Should we actually use this extra capacity? To obtain insight into the value of the extra capacity, we compare this system with an $M/M/1$ queue with service rate 1²

We take $E[S] = 1/\mu = 1/4$ at first.

⁰ A hospital serves as an example.

¹ Because its service time is large.

² In other words, a queue with no extra capacity.

```

multiple_speeds.py
1  """Simulate a multi-server queue with different speeds and compare
2  to an M/M/C queue."""
3
4  from collections import Counter
5
6  import numpy as np
7
8  import mmc
9  from job import Job
10 from simulator import Simulation
11
12
13 def run_multiple_speeds(labda=3, mu=4, server_rates=[1, 0.1, 0.01], num=10000):
14     rng = np.random.default_rng(3)
15     X = rng.exponential(scale=1 / labda, size=num)
16     X[0] = 0
17     A = X.cumsum()
18     loads = rng.exponential(scale=1 / mu, size=num)
19     jobs = [Job(id=i, arrival_time=A[i], load=loads[i]) for i in range(1, num)]
20
21     sim = Simulation()
22     sim.initialize_jobs(jobs)
23     sim.initialize_servers(server_rates=server_rates)
24     sim.run()
25
26     mm1 = mmc.MMC(labda, mu, c=1)
27     print(f"W: sim: {sim.stats.mean_waiting_time():.2f}, exact: {mm1.EW:.2f}")
28     print(f"J: sim: {sim.stats.mean_sojourn_time():.2f}, exact: {mm1.EJ:.2f}")
29     print(f"Q: sim: {sim.stats.mean_queue_length():.2f}, exact: {mm1.EQ:.2f}")
30
31     count = Counter([j.free_servers for j in sim.stats])
32     print(count)

```

Here is the output. We see that the average queue length is smaller in the multi-server station, but the sojourn time is longer. The counter shows that about 91 of the 10000 jobs are served by the slowest server, thereby providing the explanation.

```

1 W: sim: 0.42, exact: 0.75
2 J: sim: 1.24, exact: 1.00
3 Q: sim: 1.27, exact: 2.25
4 Counter({0: 8772, 1: 1136, 2: 91})

```

Since the slow servers increase sojourn times, we may question whether to use them at all. Let us examine the output if the job load becomes quite a bit larger. We increase $E[S]$ from $1/4$ to $1/3.1$, so ρ increases from $3/4$ to $3/3.1$.

The new results make it clear that the extra servers do help; the expected waiting and sojourn go down as compared to the $M/M/1$ queue, i.e., the queue without extra capacity. Interestingly, the fraction of jobs served by the fastest server is *larger* than when $E[S] = 1/4$.

```

1 W: sim: 2.04, exact: 9.68
2 J: sim: 2.97, exact: 10.00
3 Q: sim: 6.17, exact: 29.03
4 Counter({0: 9615, 1: 357, 2: 27})

```

In conclusion, if the load is low, it is best to use only the fastest server. But when the load is high, queueing times form the larger part of the sojourn time, and it then becomes better to use the slower servers as well.

DIFFERENT SCHEDULING AND priority rules are easy to implement by subclassing `Job` and overriding its `__lt__` method. For instance, here is the code to implement LIFO³, and SPTF⁴.

```

1 from dataclasses import dataclass
2
3 from job import Job
4
5
6 @dataclass(eq=False)
7 class LiFOJob(Job):
8     def __lt__(self, other):
9         return self.arrival_time > other.arrival_time
10
11
12 @dataclass(eq=False)
13 class SPTFJob(Job):
14     def __lt__(self, other):
15         return self.load < other.load

```

³ LIFO = Last In First Out.

⁴ SPTF = Shortest Processing Time First; a rule giving preference to smaller jobs.

In priority queues, higher priority is better, and only when jobs have the same priority, they should be sorted according to FIFO.⁵

⁵ A variation would be to provide jobs with due dates: the job with the smaller due date should be served first.

```

1  from dataclasses import dataclass
2
3  from job import Job
4
5
6  @dataclass(eq=False)
7  class PriorityJob(Job):
8      priority: int = 0
9
10     def __lt__(self, other):
11         if self.priority == other.priority:
12             return self.arrival_time < other.arrival_time
13         else:
14             return self.priority > other.priority

```

Here is how to use it in a simulation. In the simulation we give ten percent of the jobs a higher priority, and then we compare the results of the simulation to a regular $M/M/1$ queue with the same service capacity but without job priorities.

```

1  """The effect of priority service in multi-server queues."""
2
3  from collections import defaultdict
4
5  import numpy as np
6
7  import mmc
8  from priority_job import PriorityJob
9  from simulator import Simulation
10
11  num = 10000
12  labda, mu = 3, 3.2
13
14  rng = np.random.default_rng(3)
15  X = rng.exponential(scale=1 / labda, size=num)
16  X[0] = 0
17  A = X.cumsum()
18  loads = rng.exponential(scale=1 / mu, size=num)
19  priorities = rng.binomial(1, 0.1, size=num)
20  jobs = []
21  for i in range(1, num):
22      job = PriorityJob(id=i, arrival_time=A[i], load=loads[i])
23      job.priority = priorities[i]
24      jobs.append(job)
25
26
27  sim = Simulation()
28  sim.initialize_jobs(jobs)
29  sim.initialize_servers(server_rates=[1])
30  sim.run()
31
32  mm1 = mmc.MMC(labda, mu, c=1)
33  print(f"W: sim: {sim.stats.mean_waiting_time():0.2f}, exact: {mm1.EW:0.2f}")
34  print(f"J: sim: {sim.stats.mean_sojourn_time():0.2f}, exact: {mm1.EJ:0.2f}")
35  print(f"Q: sim: {sim.stats.mean_queue_length():0.2f}, exact: {mm1.EQ:0.2f}")
36
37  wait = defaultdict(float)
38  nums = defaultdict(float)

```

```

39 for j in sim.stats:
40     nums[j.priority] += 1
41     wait[j.priority] += j.waiting_time
42 for k in sorted(nums.keys()):
43     print(f"Priority: {k}, EW: {wait[k] / nums[k]:.2f}")

```

```

1 W: sim: 4.82, exact: 4.69
2 J: sim: 5.14, exact: 5.00
3 Q: sim: 14.46, exact: 14.06
4 Priority: 0, EW: 5.34
5 Priority: 1, EW: 0.32

```

From the output we see that for the population as a whole, there is no difference in average waiting time⁶: there is hardly any difference in waiting time between the simulation and the exact model (without priorities). However, for the jobs with the higher priority class $E[W] = 0.32$, while for the lower priority $E[W] = 5.34$. That is, the lower priority jobs do wait somewhat (but not much) longer than in the case without priorities. We explain this by the fact that the fraction of jobs with higher priority is just 10%.

⁶ Which is in accordance with our intuition.

IMPLEMENTING THE N POLICY is now very simple⁷: we *only* have to subclass the `Simulation` class and override the `handle_arrival` method, and nothing else! To see why, realize that in `Simulation.handle_departure` when there are jobs in queue and a job departs, a new job is assigned to the server; this does not depend on the state of the server. Fig. 3.6.1 shows a sample path of the queue length process.

⁷ See [2.2.6].

```

_____ n_policy.py _____
1  """Queueing behavior under an N policy."""
2
3  import numpy as np
4
5  from job import Job
6  from simulator import Simulation
7
8
9  class N_policy(Simulation):
10     def __init__(self, thres_N):
11         self.thres_N = thres_N
12         super().__init__()
13
14     def handle_arrival(self, job: Job):
15         job.q_length_at_arrival = self.queue.length()
16         self.queue.push(job)
17         if (
18             self.queue.length() >= self.thres_N
19             and self.servers.is_server_available()
20         ):
21             self.serve_job(self.queue.pop(), self.servers)

```

THE CHECK-IN PROCESS at an airport is a somewhat more involved queueing system, because at a check-in desk business and economy class customers typically have their own set of servers. Here, we are going to use

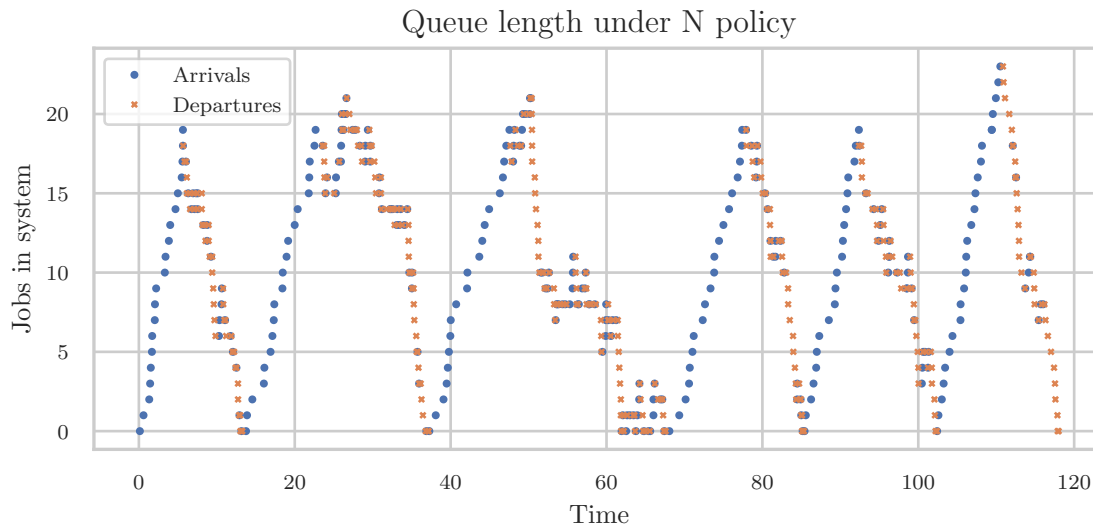


FIG. 3.6.1: A queue under an N policy for $N = 20$. Clearly, the queue builds up to 20, then the server switches on, and continues serving jobs until the system is empty. Then the server switches off until the system again contains $N = 20$ jobs, switches on, and so on.

simulation to compare different ways to organize the server allocation. One policy is like this: some of the servers are strictly allocated to business class customers, and the rest of the servers are strictly used for the economy class customers. Another policy is that if a business server is free, this server accepts just one economy customer for service⁸. When, after this service, there is still no business customer, the business server serves a second economy customer. Once a business customer arrives, the business server finishes the economy customer⁹, and then serves the business customer.¹⁰ Which of these two policies is best?

Assume that a flight departing at 11 am consists of 300 economy class and 100 business class people. For ease, assume that all N customers arrive between 8 and 10, and that the desks are open from 8 to 10h30 to serve the demand.

Clearly, this is a transient queueing process that ends when all jobs are served. As this system is mathematically intractable, we only have simulation at our disposal.

We first load the relevant modules.

```

airport_check_in.py
1  """Queueing at an airport check-in desk with business and economy customers."""
2
3  from dataclasses import dataclass
4
5  import numpy as np
6
7  import events
8  import job
9  import queues
10 import server
11 import servers
12 import simulator
13 import stats

```

⁸ Assuming there is such a customer in queue.

⁹ If one is in service.

¹⁰ Note that this system is not identical to a priority queue. When the business servers are shared, business customers are served with priority; however, the economy servers are not shared.

Since jobs and servers now are of the business or economy class, we subclass Job and add the relevant attribute.

```

1  ECONOMY = 0
2  BUSINESS = 1
3
4
5  @dataclass
6  class Job(job.Job):
7      type: int = ECONOMY
8
9
10 @dataclass
11 class Server(server.Server):
12     type: int = ECONOMY

```

In the simulator we configure separate queues and servers for the economy and the business customers. The boolean argument `share` indicates whether the business servers have to be shared or not. When handling arrivals and departures, we need to take the type of job and server into account. For the rest, we can subclass from the `Simulation` class.

```

1  class Simulation(simulator.Simulation):
2      def __init__(self, num_e_servers, num_b_servers, share=False):
3          self.share = share
4          self.events = events.Events()
5          self.stats = stats.Statistics()
6          self.now = 0
7          # ECONOMY
8          self.e_queue = queues.Queue()
9          self.e_servers = servers.Servers()
10         for i in range(num_e_servers):
11             self.e_servers.push(Server(id=i, rate=1, type=ECONOMY))
12         # BUSINESS
13         self.b_queue = queues.Queue()
14         self.b_servers = servers.Servers()
15         for i in range(num_b_servers):
16             self.b_servers.push(Server(id=i, rate=1, type=BUSINESS))
17
18     def handle_arrival(self, job):
19         if job.type == ECONOMY:
20             if self.e_servers.is_server_available():
21                 self.serve_job(job, self.e_servers)
22             elif self.share and self.b_servers.is_server_available():
23                 self.serve_job(job, self.b_servers)
24             else:
25                 self.e_queue.push(job)
26         elif job.type == BUSINESS:
27             if self.b_servers.is_server_available():
28                 self.serve_job(job, self.b_servers)
29             else:
30                 self.b_queue.push(job)
31         else:
32             raise ValueError("Unknown job type")
33
34     def handle_departure(self, job):
35         self.stats.push(job)
36         if job.server.type == ECONOMY:

```

```

37         self.e_servers.push(job.server)
38         if not self.e_queue.is_empty():
39             self.serve_job(self.e_queue.pop(), self.e_servers)
40     elif job.server.type == BUSINESS:
41         self.b_servers.push(job.server)
42         if not self.b_queue.is_empty():
43             self.serve_job(self.b_queue.pop(), self.b_servers)
44         elif self.share and not self.e_queue.is_empty():
45             self.serve_job(self.e_queue.pop(), self.b_servers)
46     else:
47         raise ValueError("Unknown job type")

```

The next step is to configure the jobs and define the inter-arrival times and the time the desks have to be open to serve all customers.

```

_____ airport_check_in.py _____
1  rng = np.random.default_rng(3)
2  check_in_window = 120 # minutes
3  desks_open = check_in_window + 30
4
5
6  # Make ECONOMY jobs
7  num = 300
8  A = np.sort(rng.uniform(low=0, high=check_in_window, size=num))
9  loads = rng.exponential(scale=2, size=num)
10 e_jobs = [
11     Job(id=i, arrival_time=A[i], load=loads[i], type=ECONOMY)
12     for i in range(1, num)
13 ]
14
15
16 # BUSINESS jobs
17 num = 100
18 A = np.sort(rng.uniform(low=0, high=check_in_window, size=num))
19 loads = rng.exponential(scale=2, size=num)
20 # loads = rng.uniform(low=1, high=3, size=num)
21 b_jobs = [
22     Job(id=i, arrival_time=A[i], load=loads[i], type=BUSINESS)
23     for i in range(1, num)
24 ]

```

It is evident that we need to determine the required number of servers c such that the performance is acceptable. The following code prints the total capacity and the demand so that we can check whether the available server capacity suffices. It turns out that the capacities suffice to get the work done.

```

_____ airport_check_in.py _____
1  num_e_servers, num_b_servers = 4, 2
2
3  e_work = sum(j.load for j in e_jobs)
4  e_service_available = num_e_servers * desks_open
5  print(f"Check e capacity: {e_work:.0f}, {e_service_available:.0f}")
6  b_work = sum(j.load for j in b_jobs)
7  b_service_available = num_b_servers * desks_open
8  print(f"Check b capacity: {b_work:.0f}, {b_service_available:.0f}")
9  tot_work = e_work + b_work
10 tot_service_available = (num_e_servers + num_b_servers) * desks_open
11 print(f"Check b capacity: {tot_work:.0f}, {tot_service_available:.0f}")

```

```

1 Check e capacity: e_work=553, e_service_available=600
2 Check b capacity: b_work=193, b_service_available=300
3 Check b capacity: tot_work=747, tot_service_available=900

```

We can run the simulation.

```

_____ airport_check_in.py _____
1 sim = Simulation(num_e_servers, num_b_servers, share=False)
2 sim.initialize_jobs(e_jobs)
3 sim.initialize_jobs(b_jobs)
4 sim.run()

```

And we can print the KPIs. It is reasonable¹¹ that besides the mean waiting time the largest waiting time is important. Apparently, all customers are served in time, but the waiting times of the economy class customers are quite long.

¹¹ Within this context, that is.

```

_____ airport_check_in.py _____
1 def mean(iterable):
2     "Compute mean of values in an iterable."
3     total = count = 0
4     for value in iterable:
5         total += value
6         count += 1
7     if count == 0:
8         raise ValueError("iterable is empty.")
9     return total / count
10
11
12 print(f"Last departure e job: {e_jobs[-1].departure_time:.2f}")
13 print(f"Last departure b job: {b_jobs[-1].departure_time:.2f}")
14 print(f"Largest waiting time e job: {max(j.waiting_time for j in e_jobs):.2f}")
15 print(f"Largest waiting time b job: {max(j.waiting_time for j in b_jobs):.2f}")
16 print(f"Overall mean waiting time: {sim.stats.mean_waiting_time():.2f}")
17 print(f"mean waiting e jobs: {mean(job.waiting_time for job in e_jobs):.2f}")
18 print(f"mean waiting b jobs: {mean(job.waiting_time for job in b_jobs):.2f}")

```

```

1 Last departure e job: 146.96
2 Last departure b job: 135.60
3 Largest waiting time e job: 25.17
4 Largest waiting time b job: 7.47
5 Overall mean waiting time: 8.17
6 mean waiting e jobs: 10.51
7 mean waiting b jobs: 1.09

```

One way to reduce the waiting times for the economy class is by opening an extra economy desk, that is, use 5 + 2 desks instead of 4 + 2.¹² The results show that this decision has a large impact on the waiting time.

¹² This comes at the expense of hiring an extra server: 7 instead of 6.

```

1 Last departure e job: 126.90
2 Last departure b job: 135.60
3 Largest waiting time e job: 8.32
4 Largest waiting time b job: 7.47
5 Overall mean waiting time: 2.36
6 mean waiting e jobs: 2.78
7 mean waiting b jobs: 1.09

```

However, rather than paying for this extra desk, we can share the business class servers and keep using $4 + 2 = 6$ desks. Clearly, the business customers are hardly affected, but the mean waiting time of the economy class is reduced by a factor two, as compared to the situation without server sharing and 6 servers.

```

1 Last departure e job: 134.48
2 Last departure b job: 137.10
3 Largest waiting time e job: 13.61
4 Largest waiting time b job: 7.58
5 Overall mean waiting time: 4.59
6 mean waiting e jobs: 5.64
7 mean waiting b jobs: 1.43

```

To conclude, adding extra service capacity has the largest effect on waiting time, but server sharing can also be an effective strategy.

TF 3.6.1. Claim: this class implements a sorting rule in which jobs with the same priority are served in LIFO order.

```

1 class PriorityJob(Job):
2     priority: int = 0
3
4     def __lt__(self, other):
5         if self.priority == other.priority:
6             return self.arrival_time < other.arrival_time
7         else:
8             return self.priority > other.priority

```

TF 3.6.2. Claim: This method of the simulation class switches on the server when the system contains `thres_N` jobs or more.

```

1 def handle_arrival(self, job: Job):
2     job.q_length_at_arrival = self.queue.length()
3     if (
4         self.queue.length() >= self.thres_N
5         and self.servers.is_server_available()
6     ):
7         self.serve_job(self.queue.pop(), self.servers)
8     else:
9         self.queue.push(job)

```

TF 3.6.3. Consider the check-in process at an airport when the business servers are shared with the economy customers. Claim: This method of the simulation class handles the arrivals correctly.

```

1 def handle_arrival(self, job):
2     if job.type == ECONOMY:
3         if self.e_servers.is_server_available():
4             self.serve_job(job, self.e_servers)
5         elif self.share:
6             self.serve_job(job, self.b_servers)
7         else:
8             self.e_queue.push(job)

```

TF 3.6.4. Claim: In a discrete-event simulation of a multi-server queue, the server with the shortest remaining service time should always be chosen for new arrivals.

TF 3.6.5. Claim: In a priority queue, jobs with higher priority always have a shorter expected sojourn time than jobs with lower priority.

With the tools developed in the previous chapters, we can *simulate* very general stochastic processes. While useful, we still need analytical models to make reasoning about such processes simpler; in this chapter we make a start with that.

As we have seen in Section 2.4, characterizing the *transient behavior* of stochastic systems is complicated, so we focus on the *long-run time-average* behavior. Arrival and service rates play a crucial role. A queueing system is stable only if the service rate exceeds the rate at which work arrives. Once stability is established, we can define performance measures such as the long-run average waiting time.

We derive some results fundamental to analyzing any stochastic process. These results are based on sample paths which thereby provide a unifying link between, on the one hand, the construction of a stochastic process in Chapter 2 and Chapter 3, and, on the other hand, the theoretical results that follow here. To illustrate the relations among these concepts, we provide two mind maps at the end of the chapter, see Figs. 4.7.1 and 4.7.2.

We keep the discussion intuitive and refer to ? for proofs and further background.

4.1 RATE, STABILITY AND LOAD

In this section, we define the load and stability of a queueing system, which is, arguably, the most important performance measure of such a system. We first develop the concepts for the $G/G/1$ queue, then we consider the $G^B/G^C/c$ queue.

WE START WITH defining the *arrival rate* and *departure rate* as the long-run average number of jobs that arrive and depart along a sample path:

$$\lambda = \lim_{t \rightarrow \infty} \frac{A(t)}{t}, \quad \delta = \lim_{t \rightarrow \infty} \frac{D(t)}{t}, \quad (4.1.1)$$

where $A(t)$ and $D(t)$ are discussed in Section 3.1. We assume that both limits exist⁰, are finite, and $0 < \delta \leq \lambda < \infty$.

FOR THE ARRIVAL rate, suppose we are given a sequence of iid inter-arrival times $X_k \sim X$. By the strong law of large numbers, $A_n/n = n^{-1} \sum_{k=1}^n X_k \xrightarrow{a.s.} E[X]$ as $n \rightarrow \infty$ when $E[X] < \infty$. The next theorem relates $E[X]$ to the limit of $A(t)/t$ as $t \rightarrow \infty$.

Theorem 4.1.1. Assume that $\lambda \in (0, \infty)$. Then, the existence of one of the next two limits implies the existence of the other, in which case,

$$\frac{A_k}{k} \rightarrow \lambda^{-1} > 0 \iff \frac{A(t)}{t} \rightarrow \lambda > 0.$$

⁰ The first limit does not necessarily exist if $A(t)$ is some pathological function.

Proof. $\Rightarrow$: $A_{A(t)}$ is the arrival time of the last job *before* time t and $A_{A(t)+1}$ is the arrival time of the first job *after* time t . Therefore $A_{A(t)} \leq t < A_{A(t)+1}$ and, for $A(t) > 0$,

$$\frac{A_{A(t)}}{A(t)} \leq \frac{t}{A(t)} < \frac{A_{A(t)+1}}{A(t)} = \frac{A_{A(t)+1}}{A(t)+1} \frac{A(t)+1}{A(t)}.$$

For the LHS, observe that the assumption $A_k/k \rightarrow \lambda^{-1} > 0$ as $k \rightarrow \infty$ implies that $A_k \rightarrow \infty$. This in turn implies that $A(t) := \max\{k : A_k \leq t\} \rightarrow \infty$ as $t \rightarrow \infty$. Next, if k and t are such that $A(t) = k$ then¹ $A_{A(t)} = A_k$, hence, $A_{A(t)}/A(t) = A_k/k$. Consequently, $\lim_{t \rightarrow \infty} A_{A(t)+1}/(A(t)+1) = \lim_{t \rightarrow \infty} A_{A(t)}/A(t) = \lim_{k \rightarrow \infty} A_k/k = \lambda^{-1}$. Finally, since $A(t) \rightarrow \infty$ implies that $(A(t)+1)/A(t) \rightarrow 1$, we conclude that the LHS and RHS both converge to $1/\lambda$.

$\Leftarrow$: If $A(t)/t \rightarrow \lambda$, then at the epochs A_k we also have $A(A_k)/A_k \rightarrow \lambda$. (Observe that the epochs $\{A_k\}$ are just a subset of $\mathbb{R}^+$ while t takes any value in $\mathbb{R}^+$.)² Noting that $A(A_k) = k$, it follows that $k/A_k \rightarrow \lambda$. $\square$

¹ [3.1.4].

² Recall: $A(t)/t \rightarrow \lambda$ as $t \rightarrow \infty \iff \forall \epsilon > 0 : \exists s : \forall t > s : |A(t)/t - \lambda| < \epsilon$.

Thus, from this theorem it follows that

$$E[X] = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n X_k = \lim_{n \rightarrow \infty} \frac{A_n}{n} = \lim_{t \rightarrow \infty} \frac{t}{A(t)} = \frac{1}{\lambda}.$$

For the service rate of a *single* server, let S_k denote the required service time of the k th job served, so that $U_n = \sum_{k=1}^n S_k$ is the total service time of the first n jobs. Similarly to the definition of $A(t)$, we let $U(t) = \max\{k : U_k \leq t\}$ and define the *service*, or *processing*, rate of a single server as

$$\mu := \lim_{t \rightarrow \infty} \frac{U(t)}{t}, \quad 0 < \mu < \infty,$$

assuming that this limit exists. This provides us with the relation

$$E[S] = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n S_k = \lim_{n \rightarrow \infty} \frac{U_n}{n} = \lim_{t \rightarrow \infty} \frac{t}{U(t)} = \frac{1}{\mu}.$$

Hence, the service rate μ of a single server equals $1/E[S]$.

GIVEN THE ARRIVAL and service rate, we define for the single-server queue the

$$\text{Server load} = \rho := \lambda E[S] = \frac{\lambda}{\mu} = \frac{E[S]}{E[X]},$$

and the

$$\text{Server utilization} = \delta E[S] = \frac{\delta}{\mu}.$$

We need to distinguish between these two concepts either when $\lambda > \mu$ or when arriving customers are blocked. In the first situation, the queue in front of the server increases without bound so that $\lambda > \mu = \delta$.³ In the second situation, the blocked jobs do not enter the system and therefore are not served, so $\lambda > \delta$. Bear in mind that the load can exceed 1 (when $\lambda > \mu$), while the utilization is always ≤ 1 .

³ Of course, the departure rate can never exceed the service rate.

In general we say that a system is *rate-stable* if

$$\lambda = \delta,$$

in words, the system is rate-stable whenever in the long run jobs leave the system just as fast as they enter the system. In this case, load and utilization are equal; we use these terms interchangeably for rate-stable systems. Rate-stability requires that $\lambda \leq \mu$.

It is easy to generalize the definition of the load to the $G^B/G^C/c$ queue. When each batch contains $E[B]$ jobs each of which requires $E[S]$ time to serve, the expected work per job is $E[B] E[S]$. Similarly, when a server serves, on average, a batch of $E[C]$ jobs per service time, its service rate is $E[C] / E[S]$. Finally, when the station contains c identical servers, the station's capacity is c times as large as that of a single server. Hence, in this more general case,

$$\rho := \frac{\lambda E[S] E[B]}{c E[C]}. \quad (4.1.2)$$

The definition of utilization changes accordingly.

WE CAN USE the memoryless property of the inter-arrival times in the $M/G/1$ queue to show that the expected idle and busy times⁴ of the $M/G/1$ satisfy

$$E[I] = \frac{1}{\lambda} \quad E[U] = \frac{E[S]}{1 - \rho}. \quad (4.1.3)$$

For this, observe that since inter-arrival times are memoryless in the $M/G/1$ queue, the expected time to the next arrival is $1/\lambda$. For the busy time, observe that during the service time of the customer starting a busy time⁵, an expected number $\lambda E[S]$ of new jobs arrives. As each of these jobs generates a busy period of its own, each of which is independent and has the same expected duration $E[U]$, the expected busy time started by the first job must be equal to the expected service time of this job plus the expected busy periods generated by jobs arriving during this service time; hence $E[U] = E[S] + \lambda E[S] E[U]$. But from this, $E[U] = E[S] / (1 - \rho)$, because $\rho = \lambda E[S]$.

TF 4.1.1. Claim: If for a queueing system $\lim_{t \rightarrow \infty} A(t) = \infty$, then the system is not rate stable.

TF 4.1.2. Claim: $\lim_{t \rightarrow \infty} \frac{A(t)}{t} = \lim_{n \rightarrow \infty} \frac{A_n}{n}$.

TF 4.1.3. Claim: $\lambda = \delta \implies \rho = 1$, so the queue is not stable.

TF 4.1.4. Claim: A queueing system with $\lambda E[S] > 1$ and a single server can still be rate-stable if jobs are sometimes rejected.

TF 4.1.5. Claim: The load $\rho = \lambda E[S] / c$ of a queueing system with c servers is always at most 1.

Opt 4.1.6. Can you make an arrival process such that $A(t)/t$ does not have a limit?

⁴ The busy time of a server is the time between an arrival to an empty system until the server becomes idle again.

⁵ This customer necessarily arrives at an empty system.



4.2 (LIMITS OF) EMPIRICAL PERFORMANCE MEASURES

In Section 3.1 we use the arrival times $\{A_k\}$ and the service times $\{S_k\}$ to construct the waiting times $\{W_k\}$, the sojourn times $\{J_k\}$, the arrival process $\{A(t)\}$, the departure process $\{D(t)\}$, and the number in the system $\{L(t)\}$. If the queueing system is rate stable, several long-run average performance measures⁰ along sample paths exist almost surely.¹

As the next example demonstrates, we need to distinguish between sampling the system at arrival times and sampling it over time. Suppose that jobs arrive exactly at the start of an hour and require 59 minutes of service time. If we sample the server occupancy at arrival epochs, the server is always idle, whereas over time it is almost always busy. Thus, what jobs see upon arrival *generally differs* from what the server perceives.

ONE SET OF measures focuses on what customers perceive at the moment of arrival.² The expectation and distribution function³ of the waiting time are obtained from, respectively,

$$E[W] = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n W_k, \quad P\{W \leq x\} = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n \mathbb{1}_{W_k \leq x}. \quad (4.2.1)$$

For the sojourn time J , we use similar definitions.

Since $L(A_k-)$ denotes the number of jobs in the system just before the k th arrival⁴, the average and distribution function of the number of jobs in the system as seen by arrivals are given by

$$E[L] = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n L(A_k-), \quad P\{L \leq m\} = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n \mathbb{1}_{L(A_k-) \leq m}. \quad (4.2.2)$$

A SECOND SET of performance measures follows by tracking the system's behavior over time and taking the *time-average*.⁵ Since at time s the number in the system is $L(s) = A(s) - D(s)$, we define the time-average number of jobs⁶ and the time-average fraction of time the system contains at most m jobs, respectively, as

$$E[L] = \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t L(s) ds, \quad P\{L \leq m\} = \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t \mathbb{1}_{L(s) \leq m} ds. \quad (4.2.3)$$

TF 4.2.1. Claim: If the limit exists, then

$$\frac{1}{t} \sum_{k=1}^{A(t)} \mathbb{1}_{W_k \leq x} \rightarrow P\{W \leq x\}, \quad t \rightarrow \infty.$$

TF 4.2.2. The departure process $\{D(t)\}$ can be computed from the set $\{D_k\}$ of departure times as

$$D(t) = \sum_{k=1}^{\infty} \mathbb{1}_{D_k \leq t}.$$

TF 4.2.3. Claim: In general, the time-average number of jobs in the system equals the number of jobs seen by arrivals.

TF 4.2.4. Claim: The fraction of time the system contains n or more jobs can be computed as $P\{L \geq n\} = \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t \mathbb{1}_{L(s) \geq n} ds$.

⁰ Also known as 'steady-state' or 'stationary' limits.

¹ Proving the existence of these limits below requires substantial mathematics; see § for details. Here we assume these limits exist.

² Such statistics are often described as 'as seen by arrivals'.

³ This is simply counting.

⁴ Note the A_k- ; since $L(t)$ is right-continuous, we focus on what an arrival sees just before its arrival.

⁵ These performance measures are 'as seen by the server'.

⁶ Although the symbol is the same, this expectation need not be the same as (4.2.2).

TF 4.2.5. Claim: The empirical distribution of the waiting time $P\{W \leq x\} = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n \mathbb{1}_{W_k \leq x}$ samples at arrival epochs, hence it captures what customers experience.

Ex 4.2.6. Design a queueing system where the time-average number of jobs in the system differs markedly from what arrivals observe. 

Ex 4.2.7. If $L(t)/t \rightarrow 0$ as $t \rightarrow \infty$, can $E[L]$ still be positive?

4.3 RENEWAL REWARD THEOREM

In a one-dollar store, each item costs \$1. If every minute a customer arrives and each customer buys exactly 10 items, this shop earns money at a rate of \$10 per minute. In another one-dollar store, customers spend variable amounts, but on average \$10 per customer, and customers come in irregularly, but the average inter-arrival time between two customers is 1 minute. The *renewal reward* theorem says that both shops have the same earning rate, even though the second shop perceives variability while the first does not. In this section, we prove this extremely useful theorem and show how to apply it to a famous inventory system.

WE ARE GIVEN a sequence of strictly increasing epochs $0 = T_0 < T_1 < T_2 < \dots$ such that $T_k \rightarrow \infty$ as $k \rightarrow \infty$, and a non-decreasing right-continuous (deterministic) function $t \rightarrow R(t)$ with $R(0) = 0$. Let $N(t) = \sum_{k=1}^{\infty} \mathbb{1}_{T_k \leq t}$ count the number of epochs that occurred during $(0, t]$, and $Z_k = R(T_k) - R(T_{k-1})$ be the increment of $R(t)$ during $(T_{k-1}, T_k]$.⁰ Fig. 4.3.1 shows graphically how all concepts relate.

Theorem 4.3.1 (Renewal Reward Theorem, $R = \alpha Z$). Assume that the limit $\alpha = \lim_{t \rightarrow \infty} N(t)/t$ exists with $0 < \alpha < \infty$. Then, the limit $R = \lim_{t \rightarrow \infty} R(t)/t$ exists iff $Z = \lim_{n \rightarrow \infty} n^{-1} \sum_{k=1}^n Z_k$ exists, in which case $R = \alpha Z$.

Proof. As $T_{N(t)} \leq t < T_{N(t)+1}$ and $R(\cdot)$ is non-decreasing,

$$R(T_{N(t)}) \leq R(t) \leq R(T_{N(t)+1}).$$

Hence

$$\frac{N(t)}{t} \frac{R(T_{N(t)})}{N(t)} \leq \frac{R(t)}{t} \leq \frac{R(T_{N(t)+1})}{N(t)+1} \frac{N(t)+1}{t}.$$

Since $R(T_{N(t)}) = \sum_{k=1}^{N(t)} Z_k$ and $N(t)/t \rightarrow \alpha$, we conclude that $R(t)/t \rightarrow R$ iff $(N(t))^{-1} \sum_{k=1}^{N(t)} Z_k \rightarrow Z$. By the same arguments as in Th. 4.1.1, if one of these limits exists, so does the other, completing the proof. $\square$

THE RENEWAL REWARD theorem finds an easy application in the analysis of the *Economic Production Quantity (EPQ)* model. This is an inventory system in which demand arrives at a fixed rate d and a machine produces items, when on, at a constant rate $r \geq d$.¹ The inventory is controlled by an (s, S) policy, the lead time $l = 0$, the holding and backlogging cost are h and b per unit per time, and it costs K to switch on the machine. Fig. 4.3.1 shows a sample path of the inventory level. We use the renewal reward

⁰ Here Z_k is not necessarily an inter-arrival time between two jobs.

¹ We need $r \geq d$ for stability.

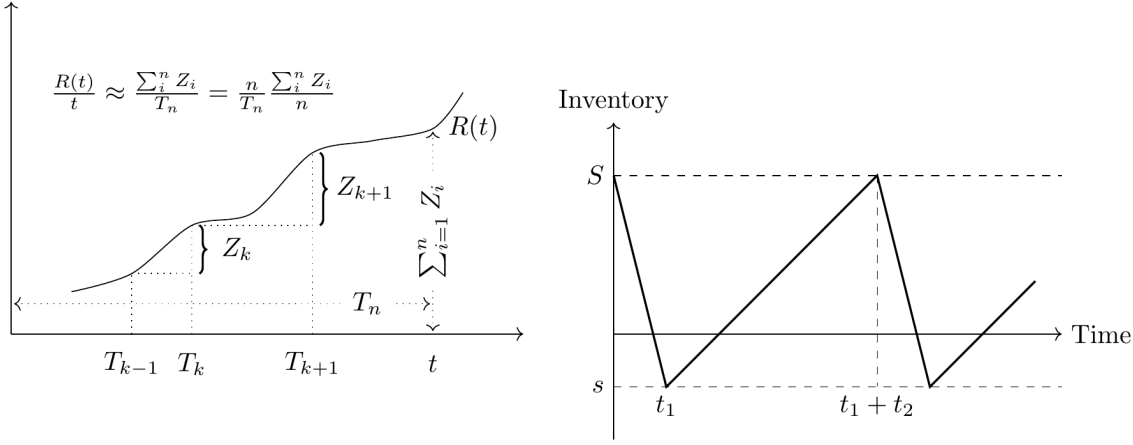


Fig. 4.3.1: The left panel provides a graphical ‘proof’ of the renewal reward equation $R = \alpha Z$. The right panel shows a sample path of an inventory system with constant production capacity and a fixed demand rate. Since the inventory operates under an (s, S) policy, it moves from cycle to cycle.

theorem (noting that the cumulative cost is non-decreasing) to find the policy parameters² that minimize the long-run time-average cost. Clearly, the inventory level repeats over time, so it suffices to study just one cycle that starts and stops when the inventory level hits S . From Fig. 4.3.1 it is simple to see that, respectively,

$$t_1 := \frac{S-s}{d}, \quad t_2 := \frac{S-s}{r-d}, \quad T := t_1 + t_2 = \frac{S-s}{d} \frac{r}{r-d},$$

are the time to empty the inventory from level S to s (after which production switches on), the time needed to replenish the inventory from level s to S (after which production switches off), and the cycle time.

Assume next that $s \leq 0 < S$.³ Writing T_+ (T_-) for the amount of time that the inventory is positive (negative) during a cycle, it follows that

$$T_+ = \frac{S}{d} + \frac{S}{r-d}, \quad T_- = \frac{-s}{d} + \frac{-s}{r-d}, \quad T = T_+ + T_-,$$

as $s \leq 0$. Therefore,⁴

$$\frac{T_+}{T} = \frac{S}{S-s}, \quad \frac{T_-}{T} = \frac{-s}{S-s}.$$

Since the holding and backlogging cost are proportional to the areas of the corresponding triangles, the average cost of one cycle becomes

$$g = \frac{K}{T} + b \frac{-s}{2} \frac{T_-}{T} + h \frac{S}{2} \frac{T_+}{T}.$$

Substituting the expressions for T_- , T_+ , and T in terms of s and S , taking the partial derivatives with respect to s and S and solving for the optimal values by setting $\partial g / \partial s = \partial g / \partial S = 0$ gives⁵

$$g^* = \sqrt{2Kd} \sqrt{\frac{hb}{h+b}} \sqrt{1 - \frac{d}{r}}, \quad s^* = -\frac{g^*}{b}, \quad S^* = \frac{g^*}{h}. \quad (4.3.1)$$

Take the limits $r \rightarrow \infty$ and $b \rightarrow \infty$ to obtain the *Economic Order Quantity (EOQ)* formula.⁶

² That is, the optimal s and S .

³ [4.3.8] asks you to generalize this assumption.

⁴ This also follows from the geometry of the triangles in the right panel of Fig. 4.3.1.

⁵ Interestingly, when $r = d$, the cost g^* equals 0. Why is that so?

⁶ See also [5.8.7].

$$g_{EOQ}^* = \sqrt{2Kdh} \quad s^* = 0, \quad Q^* = \sqrt{\frac{2Kd}{h}},$$

where we write Q^* instead of S^* to indicate that we order in fixed quantities. Observe that the EOQ policy is simultaneously an (s, S) policy, (Q, r) policy, and (T, S) policy.⁷

An interesting extension is a system with constraints on cycle length and order size.⁸

TF 4.3.1. Consider a machine that fails regularly and needs to be repaired. Assuming that repairs take a negligible amount of time, the time between machine failures has density f_X , and each repair costs c . Claim: The renewal reward theorem tells us that the cost rate equals $\frac{c}{\int_0^\infty xf(x)dx}$.

TF 4.3.2. For a $G/G/1$ queue, write $E[U]$ for the expected busy time and $E[I]$ for the expected idle time. Claim:

$$\rho = E[U] / (E[U] + E[I]).$$

TF 4.3.3. An (s, S) policy is a restocking policy where s denotes the size of replenishment orders and S represents the speed at which items are produced.

TF 4.3.4. Claim: In the renewal reward theorem $R = \alpha Z$, the quantity α must be the arrival rate λ .

Ex 4.3.5. Use the renewal reward theorem to relate the load $\rho = \lambda E[S]$ to the limiting fraction of time the server of a stable $G/G/1$ queue is busy. 

Ex 4.3.6. We can derive the relation between utilization and the fraction of busy time of the server of a $G/G/1$ queue by considering the inequalities

$$\sum_{k=1}^{A(t)} S_k \geq \int_0^t \mathbb{1}_{L(s) > 0} ds \geq \sum_{k=1}^{D(t)} S_k. \quad (4.3.2)$$

Explain the inequalities and show that, by taking appropriate limits, we obtain $\lambda E[S] \geq \rho \geq \delta E[S]$. 

Ex 4.3.7. Show for the $G/G/1$ queue that $\rho = E[U] / (E[U] + E[I])$ where $E[U]$ is the expected busy time and $E[I]$ the expected idle time. 

Ex 4.3.8. For the EPQ model, take T to be the cycle time. Use elementary geometry to show that the average cost $g = K/T + h(s+S)/2$ when $0 \leq s < S$, and $g = K/T - b(s+S)/2$ when $s < S \leq 0$.

4.4 LITTLE'S LAW

There exists a fascinating connection, known as *Little's law*, between the average sojourn time of a job in an input-output system and the long-run time-average number of jobs within that system. To heuristically understand this relationship, imagine a 100 km highway segment between points A and B. Every minute, a car enters the highway and each car maintains a constant speed of 50 km/h. Now, since each car spends 2 hours⁰ on the

⁷ Recall the policies of Section 2.5.

⁸ Can you find an optimal policy?

⁰ The sojourn time equals 100 km / 50 km/h = 2 hours.

highway, there must be 120 cars¹ on the highway simultaneously. The objective in this section is to demonstrate that, under a few simple conditions, this relation generalizes to stochastic systems, in which case it takes the form $E[L] = \lambda E[J]$, i.e., the average number of 'things' in a system is the rate at which these 'things' enter times the average time they spend in the system.²

Theorem 4.4.1 (Little's law). If $\delta = \lambda \in (0, \infty)$ and $\frac{1}{n} \sum_{k=1}^n J_k \rightarrow E[J] < \infty$ as $n \rightarrow \infty$, then the limit $E[L] := \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t L(s) ds$ exists, and $E[L] = \lambda E[J]$.

Proof. Suppose we charge a job for the time it spends in the system. This can be done at arrival, while present, or upon departure. A comparison of the costs under each of these three charging schemes shows that for some job k

$$J_k \mathbb{1}_{D_k \leq t} \leq \int_0^t I_k(s) ds \leq J_k \mathbb{1}_{A_k \leq t},$$

where $I_k(s)$ is defined in (3.1.6). Taking the sum over this inequality and using the reasoning that led to (3.1.8) gives³

$$\frac{1}{t} \sum_{k=1}^{D(t)} J_k \leq \frac{1}{t} \int_0^t \sum_{k=1}^{\infty} I_k(s) ds = \frac{1}{t} \int_0^t L(s) ds \leq \frac{1}{t} \sum_{k=1}^{A(t)} J_k.$$

The assumptions imply that $D(t) \rightarrow \infty$ as $t \rightarrow \infty$. Thus, for the LHS,

$$\lim_{t \rightarrow \infty} \frac{1}{t} \sum_{k=1}^{D(t)} J_k = \lim_{t \rightarrow \infty} \frac{D(t)}{t} \frac{1}{D(t)} \sum_{k=1}^{D(t)} J_k = \delta E[J].$$

Likewise, the RHS converges to $\lambda E[J]$. As $\delta = \lambda < \infty$, the limits of the LHS and RHS are the same. Consequently, the limit $\frac{1}{t} \int_0^t L(s) ds \rightarrow E[L]$ exists and $\lambda E[J] = E[L]$. $\square$

The conditions are subtle: assuming only that $\delta = \lambda < \infty$ does not imply that $\lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n J_k < \infty$. For example, consider a single-server queue, take $A_k = k$, and $S_k = 1$ if k is not a square but $S_k = 2$ if k is a square. Then for very large k , $D_k \sim k - \sqrt{k}$. Hence, $A_k/k \rightarrow 1$ and $D_k/k \rightarrow 1$. However, for large s , $L(s) = A(s) - D(s) \approx \sqrt{s}$, so that $t^{-1} \int_0^t \sqrt{s} ds \rightarrow \infty$ as $t \rightarrow \infty$.

TF 4.4.1. Claim: Little's law states for a rate-stable $G/G/1$ queue that $E[J] = \lambda E[L]$.

TF 4.4.2. Claim: For Little's law to be true for any input-output system the equality

$$\int_0^T L(s) ds = \sum_{k=1}^{A(T)} W_k$$

must hold for all $T \geq 0$.

TF 4.4.3. Consider the server of the $G/G/1$ queue as a system in itself. The time jobs stay in this system is $E[S]$, and jobs arrive at a rate λ . Claim: It follows from Little's law that the fraction of time the server is busy is $\lambda E[S]$.

TF 4.4.4. Suppose that there are 10 jobs present in the $M/M/1$ queue with arrival rate $\lambda = 3$ and service rate $\mu = 4$ per hour. Claim: The time to clear the system follows from Little's law and is $3 \times 10 = 30$ hours.

¹ $2 \text{ h} \times 60 \text{ min/h} \times 1 \text{ car/min} = 120$ cars.

² Use the physical dimensions of the components of Little's law to check that $E[J] \neq \lambda E[L]$.

³ Observe that $\sum_{k=1}^{\infty} J_k \mathbb{1}_{D_k \leq t} = \sum_{k=1}^{D(t)} J_k$, and similarly for $A(t)$.

TF 4.4.5. Claim: Little's law $E[L] = \lambda E[J]$ implies that reducing the arrival rate λ always reduces the average number of jobs $E[L]$ in the system.

Ex 4.4.6. Think of all servers in the (stable) $G/G/c$ queue as being placed in a box. Jobs enter the box at a rate λ and stay for a duration $E[S]$ in this box. Use Little's law to conclude that the time-average number of busy servers is $E[L_s] = \lambda E[S]$.

Let's take 'life' itself as an input-output system⁴, and ask how many people are born and die in the Netherlands per week.

Ex 4.4.7. For simplicity, assume that in 2023 the Netherlands has 16 M inhabitants, and people have an expected lifetime (i.e., the time they remain in the system called 'life') of 80 years. By Little's law, the rate λ of people entering the system (births) is $16 \text{ M}/80 = 0.2 \text{ M}$ per year. For simplicity, if a year has 50 weeks, there are 4000 births per week. If the size of the population remains about constant, also around 4000 people die per week.

As it turns out, the death rate in 2023 is about 2900 per week. This is much lower than the estimate, and a more precise computation does not close the gap. Can you explain this paradoxical difference?



⁴ An application to actuarial sciences.

4.5 POISSON ARRIVALS SEE TIME AVERAGES

Recall from the example of Section 4.2 that sampling at arrival times can give very different results from sampling over time. Interestingly, as we show here, when jobs arrive as a Poisson process, both sampling methods yield the same value. This result is known as *Poisson arrivals see time-averages* (PASTA).

SAY THE SYSTEM is in *state* n at time t when $L(t) = n$. Then define

$$A(n, t) := \sum_{k=1}^{A(t)} \mathbb{1}_{L(A_k-) = n}$$

as the number of arrivals up to time t that saw the system in *state* n upon arrival.⁰ Next, let

$$Y(n, t) := \int_0^t \mathbb{1}_{L(s) = n} ds$$

be the total time the system spends in *state* n during $[0, t]$.

Assume the following limits exist:

$$\lambda := \lim_{t \rightarrow \infty} \frac{A(t)}{t}, \quad \lambda(n) := \lim_{t \rightarrow \infty} \frac{A(n, t)}{Y(n, t)}, \quad (4.5.1a)$$

$$p(n) := \lim_{t \rightarrow \infty} \frac{Y(n, t)}{t}, \quad \pi(n) := \lim_{t \rightarrow \infty} \frac{A(n, t)}{A(t)}. \quad (4.5.1b)$$

Here, $\lambda(n)$ denotes the arrival rate *when* the system is in *state* n , $p(n)$ the long-run fraction of time the system spends in *state* n , and $\pi(n)$ the long-run fraction of *arrivals* that find n customers present.

Theorem 4.5.1. If all limits in (4.5.1) exist,

$$\lambda \pi(n) = \lambda(n) p(n).$$

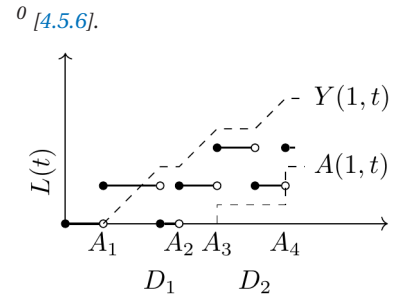


Fig. 4.5.1: We subtracted $1/2$ from the graph of $A(1, t)$, otherwise it would overlap with the graph of Y .

Proof. On the one hand,

$$\lim_{t \rightarrow \infty} \frac{A(n, t)}{t} = \lim_{t \rightarrow \infty} \frac{A(n, t)}{Y(n, t)} \frac{Y(n, t)}{t} = \lambda(n)p(n).$$

On the other,

$$\lim_{t \rightarrow \infty} \frac{A(n, t)}{t} = \lim_{t \rightarrow \infty} \frac{A(n, t)}{A(t)} \frac{A(t)}{t} = \pi(n)\lambda.$$

□

This result has a simple consequence. Write $\mathcal{S}$ for all states the system can attain.¹ Clearly, any arrival must see the system in some state, hence $\sum_{n \in \mathcal{S}} \pi(n) = 1$. Thus, summing over n in $\lambda\pi(n) = \lambda(n)p(n)$, the overall arrival rate λ satisfies

$$\lambda = \sum_{n \in \mathcal{S}} \lambda(n)p(n). \quad (4.5.2)$$

With the above theorem, we can obtain the second result of utmost importance.

Theorem 4.5.2. If the arrival rate does not depend on the state of the system, i.e., $\lambda(n) = \lambda$, the sample probabilities $\{\pi(n)\}$ equal the time-average probabilities $\{p(n)\}$, i.e.,

$$\lambda(n) = \lambda \iff \pi(n) = p(n). \quad (4.5.3)$$

Proof. From Th. 4.5.1, $\pi(n) = \lambda(n)p(n)/\lambda$. By assumption, $\lambda = \lambda(n)$, so the λ 's cancel. □

As the example in the introduction of this section shows, this property is not satisfied in general. However,²

Corollary 4.5.3. If the arrival process is Poisson, then $\lambda(n) = \lambda$, in which case $\pi(n) = p(n)$.

BY DISCRETIZING TIME we can give an intuitive explanation of when PASTA holds or fails. If in some period k a job arrives ($a_k = 1$) and the system contains m jobs at the end of period $k-1$, i.e., $L_{k-1} = m$, then we say that the arrival in period k sees m jobs in the system upon arrival. Thus, for k large, $\pi(m) \approx \mathbb{P}\{L_{k-1} = m | a_k = 1\}$. With Bayes' law:

$$\pi(m) \approx \mathbb{P}\{L_{k-1} = m | a_k = 1\} = \frac{\mathbb{P}\{a_k = 1, L_{k-1} = m\}}{\mathbb{P}\{a_k = 1\}}.$$

Observe that L_{k-1} is a function of $\{a_i\}_{i=1}^{k-1}$. If a_k and $\{a_i\}_{i=1}^{k-1}$ are independent, then a_k and L_{k-1} are independent too, and then $\mathbb{P}\{a_k = 1, L_{k-1} = m\} = \mathbb{P}\{a_k = 1\} \mathbb{P}\{L_{k-1} = m\}$. By canceling $\mathbb{P}\{a_k = 1\}$ in the fraction, we see that

$$\pi(m) \approx \mathbb{P}\{L_{k-1} = m\} \approx p(m),$$

where the last approximation follows from the fact that $\mathbb{P}\{L_{k-1} = m\}$ is the fraction of periods that $L = m$.³

However, PASTA does not hold when a_k depends on L_{k-1} . For example, if $a_k = 0$ when $L_{k-1} > 0$ and $c_k \sim \text{Bern}(d)$ for some small probability d , then $\pi(1) = 0$ while in general $\mathbb{P}\{L_{k-1} = 0\} > 0$.

¹ When there is no upper bound on L , then $\mathcal{S} = \mathbb{N}$, but when $L(t)$ is bounded by some number K , then $\mathcal{S} = \{0, 1, \dots, K\}$.

² A rigorous proof of this is difficult, see e.g., ?.

³ For mathematically interested students: The assumed strong convergence $N^{-1} \sum_{k=1}^N \mathbb{1}_{L_k=m} \rightarrow p(m)$ as $N \rightarrow \infty$ implies the weak convergence $\mathbb{P}\{L_k = m\} \rightarrow p(m)$ as $k \rightarrow \infty$.

WITH PASTA, LITTLE'S law and memorylessness we can derive some initial results for the $M/M/1$ queue.⁴ By PASTA, an arrival sees on average $E[L]$ jobs in the system, and the expected time to serve these jobs is $E[L] E[S]$. This is subtler than it seems: the expected time to serve the jobs in queue is $E[Q] E[S]$, and by *the memoryless property of S* , the expected remaining time of the job in service (if any) is also $E[S]$.⁵ Next, as the arriving job has to be served itself before leaving, its expected sojourn time $E[J] = E[L] E[S] + E[S]$. With Little's law, $E[L] = \lambda E[J]$, and therefore,

$$E[J] = E[L] E[S] + E[S] = \lambda E[J] E[S] + E[S] = \rho E[J] + E[S].$$

Consequently,

$$E[J] = \frac{E[S]}{1 - \rho}, \quad (4.5.4a)$$

from which,

$$E[L] = \lambda E[J] = \frac{\lambda E[S]}{1 - \rho} = \frac{\rho}{1 - \rho}, \quad (4.5.4b)$$

$$E[W] = E[L] E[S] = \frac{\rho}{1 - \rho} E[S] \quad (4.5.4c)$$

$$E[Q] = \lambda E[W] = \frac{\rho^2}{1 - \rho}, \quad (4.5.4d)$$

$$E[L_s] = E[L] - E[Q] = \frac{\rho}{1 - \rho} - \frac{\rho^2}{1 - \rho} = \rho, \quad (4.5.4e)$$

TO LET YOU think a bit deeper about the PASTA property, and obtaining statistics by sampling in general, consider the next cute riddle. The problem is known as the 'inspection paradox'.⁶

In a small midwestern town⁷ there lived a retired railroad engineer named William Johnson. The main line on which he had worked for so many years passed through the town. Mr. Johnson suffered from insomnia and would often wake up at any odd hour of the night and be unable to fall asleep again. He found it helpful, in such cases, to take a walk along the deserted streets of the town, and his way always led him to the railroad crossing. He would stand there thoughtfully watching the track until a train thundered by through the dead of the night. The sight always cheered the old railroad man, and he would walk back home with a good chance of falling asleep.

After a while he made a curious observation; it seemed to him that most of the trains he saw at the crossing were traveling eastward, and only a few were going west. Knowing very well that this line was carrying equal numbers of eastbound and westbound trains, and that they alternated regularly, he decided at first that he must have been mistaken in this reckoning. To make sure, he got his little notebook, and began putting down "E" or "W," depending on which way the first train to pass was traveling. At the end of a week, there were five "E's" and only two "W's" and the observations of the next week gave essentially the same proportion. Could it be that he always woke up at the same hour of night, mostly before the passage of eastbound trains?

⁴ For the $M/G/1$ queue, things are a bit subtler; see [4.5.8] and Section 5.6.

⁵ This is not true for the $M/G/1$ queue.

⁶ Look this paradox up on the web. You should be aware of such biases in data science.

⁷ This cute story comes from the book 'Puzzle Math' by G. Gamow and M. Stern, <https://www.arvindguptatoys.com/arvindgupta/puzzlemath.pdf>. The interesting solution can also be found there.

Being puzzled by this situation, he decided to undertake a rigorous statistical study of the problem, extending it also to the daytime. He asked a friend to make a long list of arbitrary times such as 9:35 a.m., 12:00 noon, 3:07 p.m., and so on, and he went to the railroad crossing punctually at these times to see which train would come first. However, the result was the same as before. Out of one hundred trains he met, about seventy-five were going east and only twenty-five west. In despair, he called the depot in the nearest big city to find whether some of the westbound trains had been rerouted through another line, but this was not the case. He was, in fact, assured that the trains were running exactly on schedule, and that equal numbers of trains daily were going each way. This mystery brought him to such despair that he became completely unable to sleep and was a very sick man.

Can you reveal the mystery, thereby solving the sleeping problems of Mr. Johnson without prescribing pills?

TF 4.5.1. Claim: The PASTA property implies that $\sum_{n=0}^{\infty} np(n) = \sum_{n=0}^{\infty} n\pi(n)$ for any $G/M/1$ queue.

TF 4.5.2. Consider a stable $M/G/1$ queue. Claim: By the PASTA property, a fraction ρ of the arrivals sees the server occupied, while a fraction $1 - \rho$ sees a free server.

TF 4.5.3. Claim: For the $M/G/1$ queue it follows from the PASTA property that

$$E[J] = \sum_{n=0}^{\infty} E[W | L = n] \pi(n) + E[S].$$

TF 4.5.4. Let $L_S(s)$ be the number of jobs in service at time s in an $M/G/1$ queue. Define

$$\alpha = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n L_S(A_k).$$

Claim: PASTA implies that $1 - \rho = \alpha$.

TF 4.5.5. Claim: PASTA implies that for any $G/G/1$ queue, $\pi(n) = p(n)$.

Ex 4.5.6. If $\lambda > \delta$, can it happen that $\lim_{t \rightarrow \infty} A(n, t)/t > 0$ for some (finite) n ?

Ex 4.5.7. When $\lambda \neq \delta$, is $\pi(n) \geq \delta(n)$?



Ex 4.5.8. While (4.5.4c) is true for the $M/M/1$, it is *not true for the $M/G/1$ queue*. Why?

4.6 LEVEL CROSSING AND BALANCE EQUATIONS

Consider a sample path of $\{L(t) : t \geq 0\}$ of a queueing or inventory system in which jobs arrive and depart one by one. We say that the sample path *up-crosses level n* at time t when the number of jobs in the system changes from n to $n + 1$ due to an arrival, that is, when $L(t-) = n$ and $L(t) = n + 1$. The sample path *down-crosses level n* at time t when, due to a departure, $L(t-) = n + 1$ and $L(t) = n$. Clearly, the number of up-crossings

and down-crossings cannot differ by more than 1 at any time, because a down-crossing of level n can only occur after an up-crossing (or the other way around). This simple idea is a key tool in the analysis of stationary stochastic systems.

SIMILAR TO THE definitions for $A(n, t)$ and $\lambda(n)$, let⁰

$$D(n, t) := \sum_{k=1}^{D(t)} \mathbb{1}_{L(D_k)=n}, \quad \mu(n+1) := \lim_{t \rightarrow \infty} \frac{D(n, t)}{Y(n+1, t)}, \quad (4.6.1)$$

denote the number of departures up to time t that *leave n jobs behind*, and the departure rate from state $n+1$.

Theorem 4.6.1. When jobs arrive and depart as single units and the limits (4.5.1) and (4.6.1) hold,

$$\lambda(n)p(n) = \mu(n+1)p(n+1). \quad (4.6.2)$$

Proof. Since jobs arrive one by one, $|L(t) - L(t-)| \leq 1$ for all $t \geq 0$. This implies that

$$|A(n, t) - D(n, t)| \leq 1, \quad (4.6.3)$$

hence,

$$\lim_{t \rightarrow \infty} \frac{A(n, t)}{t} = \lim_{t \rightarrow \infty} \frac{D(n, t)}{t}. \quad (4.6.4)$$

The claim follows by noting that

$$\begin{aligned} \lim_{t \rightarrow \infty} \frac{A(n, t)}{t} &= \lim_{t \rightarrow \infty} \frac{A(n, t)}{Y(n, t)} \frac{Y(n, t)}{t} = \lambda(n)p(n), \\ \lim_{t \rightarrow \infty} \frac{D(n, t)}{t} &= \lim_{t \rightarrow \infty} \frac{D(n, t)}{Y(n+1, t)} \frac{Y(n+1, t)}{t} = \mu(n+1)p(n+1). \end{aligned}$$

□

SUPPOSE WE CAN specify¹ the arrival and service rates $\lambda(n)$ and $\mu(n)$, then we can easily compute the long-run fraction of time $p(n)$ that the system contains n jobs. To see this, rewrite (4.6.2) as

$$p(n+1) = \frac{\lambda(n)}{\mu(n+1)} p(n). \quad (4.6.5)$$

A simple recursion yields:

$$p(n+1) = \frac{\lambda(n)\lambda(n-1)\cdots\lambda(0)}{\mu(n+1)\mu(n)\cdots\mu(1)} p(0).$$

Since $p(n)$, $n \geq 0$, represent probabilities,

$$1 = \sum_{n=0}^{\infty} p(n) = p(0) \left(1 + \sum_{n=0}^{\infty} \frac{\lambda(n)\lambda(n-1)\cdots\lambda(0)}{\mu(n+1)\mu(n)\cdots\mu(1)} \right).$$

Thus, we can *normalize* $p(n)$ by setting

$$p(0)^{-1} = 1 + \sum_{n=0}^{\infty} \frac{\lambda(n)\lambda(n-1)\cdots\lambda(0)}{\mu(n+1)\mu(n)\cdots\mu(1)} \quad (4.6.6)$$

⁰ Note that we write $L(D_k)$ and not $L(D_k-)$: to leave n jobs behind, the system must contain $n+1$ jobs just prior to the departure.

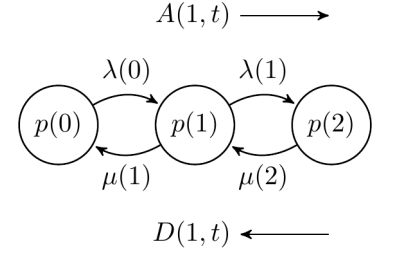


Fig. 4.6.1: Level crossing transitions between states showing arrival and departure rates.

¹ In Section 5.1 and onward we will model many queueing situations by making suitable choices for $\lambda(n)$ and $\mu(n)$.

Finally, once we have $p(n)$, it is easy to compute the time-average number of jobs in the system and the long-run fraction of time it contains at least n jobs:

$$E[L] = \sum_{n=0}^{\infty} np(n), \quad P\{L \geq n\} = \sum_{i=n}^{\infty} p(i).$$

WITH THE DEFINITIONS of $A(n, t)$ and $D(n, t)$, we establish a relation between $\pi(n)$ and $\delta(n)$, i.e., statistics as obtained by arrivals and departures. Define, analogous to $\pi(n)$,

$$\delta(n) := \lim_{t \rightarrow \infty} \frac{D(n, t)}{D(t)}$$

as the long-run fraction of jobs that leave n jobs *behind*. From (4.6.4) and using that $A(n, t) \approx D(n, t)$,

$$\frac{A(t)}{t} \frac{A(n, t)}{A(t)} = \frac{A(n, t)}{t} \approx \frac{D(n, t)}{t} = \frac{D(t)}{t} \frac{D(n, t)}{D(t)}.$$

Taking limits at the left and right, and using (4.1.1), we obtain for the $G/G/c$ queue²

$$\lambda \pi(n) = \delta \delta(n). \quad (4.6.7)$$

Hence,

$$\lambda = \delta \iff \pi(n) = \delta(n). \quad (4.6.8)$$

WHEN JOBS CAN be blocked, we must distinguish between jobs that arrive and those that actually enter.³ To handle this, define

$$A^+(n, t) := \sum_{k=1}^{A(t)} \mathbb{1}_{L(A_k-) = n} \mathbb{1}_{L(A_k) \geq n+1}, \quad \lambda^+(n) := \lim_{t \rightarrow \infty} \frac{A^+(n, t)}{Y(n, t)}, \quad (4.6.9)$$

i.e., $A^+(n, t)$ counts arrivals that up-cross level n , and $\lambda^+(n)$ corresponds to the rate at which jobs *enter* when the system contains n jobs.⁴ We should replace $\lambda(n)$ by $\lambda^+(n)$ at all other relevant places, for instance, (4.6.2) becomes

$$\lambda^+(n)p(n) = \mu(n+1)p(n+1). \quad (4.6.10)$$

IT IS IMPORTANT to realize that level-crossing arguments in the simple form as presented above cannot always be applied, since it is not always possible to split the state space into two disjoint subsets by ‘drawing a line’ between two states. For a more general approach, we focus on a single state and count how often this state is entered and left.

Specifically, define $I(n, t) = A(n-1, t) + D(n, t)$ as the number of times the queueing process enters state n either by an arrival from state $n-1$ or a departure leaving n jobs behind. Similarly, $O(n, t) = A(n, t) + D(n-1, t)$ counts how often state n is left either by an arrival (to state $n+1$) or a departure (to state $n-1$). Just like (4.6.3), it is evident that $|I(n, t) - O(n, t)| \leq 1$, and this implies⁵

$$\lambda(n-1)p(n-1) + \mu(n+1)p(n+1) = (\lambda(n) + \mu(n))p(n). \quad (4.6.11)$$

These equations hold for any $n \geq 1$ and are known as the *balance equations*. These equations are useful for analyzing queueing networks.

² Because customers arrive and leave as single units in a (rate-stable) $G/G/c$ queue.

³ Blocked jobs do not enter, hence do not produce up-crossings.

⁴ $\lambda(n) > \lambda^+(n)$ when a fraction of jobs is blocked, otherwise $\lambda(n) = \lambda^+(n)$.

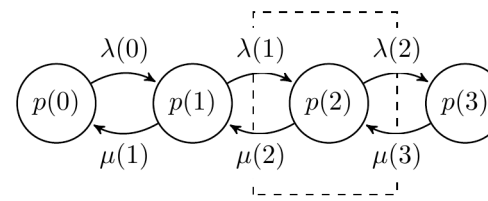


Fig. 4.6.2: Balance equations: counting transitions in and out of a state.

⁵ [4.6.10].

LEVEL-CROSSING ARGUMENTS CAN also be applied to continuous systems. For instance, we can find a closed-form expression for the waiting time of the $M/G/1$ queue. However, this is a bit more complicated as the argument hinges on an integral equation. If you do not like developing mathematical skills, you may skip this derivation.

Suppose that the waiting time W of the $M/G/1$ queue has a density f , that is, the cdf $F(x) = P\{W \leq x\}$ has a derivative $f(x)$ for all $x > 0$.^{6,7} Consider some level $x > 0$. By PASTA, the rate of arriving jobs that see a waiting time $y \in [0, x]$ is equal to $\lambda f(y)$.⁸ To up-cross level x from state $y < x$, the service time must be at least $x - y$, the probability of which is $G(x - y) = P\{S > x - y\}$. Adding up the rates for all possible waiting times below x gives that $\lambda \int_0^x f(y)G(x - y) dy$ is the up-crossing rate of level x . The down-crossing rate equals $f(x)$ because the server drains work at unit rate, so level x is crossed downward whenever the workload is at x . Equating these rates gives the integral equation $\lambda \int_0^x f(y)G(x - y) dy = f(x)$. For $x = 0$, we need to consider the atom $P\{W = 0\}$. In that case, $\lambda P\{W = 0\} = f(0+)$. We can combine these observations into the following level-crossing equation:

$$\lambda \int_0^x G(x - y) dF(y) = f(x). \quad (4.6.12)$$

Using this integral equation

$$\begin{aligned} E[W] &= \int_0^\infty x f(x) dx = \lambda \int_0^\infty x \int_0^x G(x - y) dF(y) dx \\ &= \lambda \int_0^\infty \int_0^\infty x \mathbb{1}_{y \leq x} G(x - y) dx dF(y) \\ &= \lambda \int_0^\infty \int_0^\infty (u + y) G(u) du dF(y) \\ &\stackrel{1}{=} \lambda \int_0^\infty (E[S^2]/2 + y E[S]) dF(y) \\ &\stackrel{2}{=} \lambda E[S^2]/2 + \lambda E[S] E[W]. \end{aligned}$$

Step 1 follows from substituting $u = x - y$, and 2 from the fact that $E[W] = \int_0^\infty y dF(y)$. Bringing the term $\lambda E[S] E[W] = \rho E[W]$ to the left, and dividing by $1 - \rho$ gives

$$E[W] = \lambda \frac{E[S^2]}{2(1 - \rho)}.$$

As a last point of interest, let us derive the density of the waiting time for the $M/M/1$ queue. As in this case $S \sim \text{Exp}(\mu)$, f must satisfy

$$f(x) = \lambda \int_0^x e^{-(x-y)\mu} dF(y) = \lambda \int_0^x e^{-(x-y)\mu} f(y) dy + \lambda e^{-x\mu} F(0).$$

Multiplying both sides by $e^{\mu x}$ and defining $g(x) = e^{\mu x} f(x)$, this can be rewritten to $g(x) = \lambda \int_0^x g(y) dy + \lambda F(0)$. Differentiating the LHS and RHS with respect to x gives the differential equation $g'(x) = \lambda g(x)$. Hence, $g(x) = A e^{\lambda x}$, and therefore, $f(x) = A e^{-(\mu - \lambda)x}$. At $x = 0$, $f(0) = \lambda P\{W = 0\} = \lambda p(0)$. Since $p(0) = 1 - \rho$, it follows that

$$f(x) = \lambda(1 - \rho) e^{-\mu(1 - \rho)x}, \quad x > 0. \quad (4.6.13)$$

⁶ A formal proof would first establish that f exists.

⁷ At $x = 0$, F is only differentiable from the right because $P\{W = 0\} > 0$, while $P\{W \leq x\} = 0$ for $x < 0$.

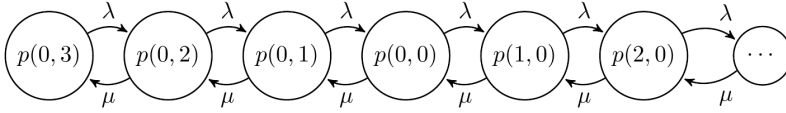
⁸ Observe that we use the splitting property of Poisson processes.

Another way to obtain this result is to condition on the number L of jobs found upon arrival, and then using that the total service of these L customers has a gamma density:⁹

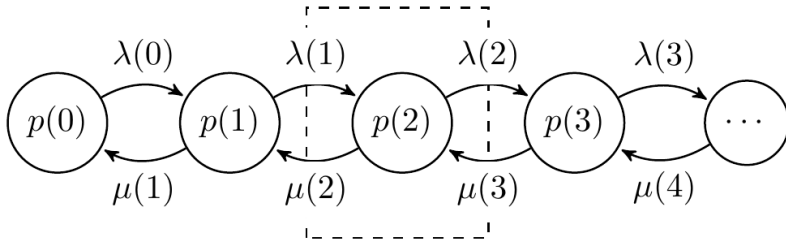
$$\begin{aligned} f(x) &= \sum_{k=1}^{\infty} P\{W = x | L = k\} P\{L = k\} \\ &= \sum_{k=1}^{\infty} P\{S_1 + S_2 + \cdots + S_k = x\} P\{L = k\} = (1 - \rho) \sum_{k=1}^{\infty} \rho^k \mu \frac{(\mu x)^{k-1}}{(k-1)!} e^{-\mu x} \\ &= (1 - \rho) \lambda e^{-\mu x} \sum_{k=0}^{\infty} \rho^k \frac{(\mu x)^k}{k!} = (1 - \rho) \lambda e^{-\mu(1-\rho)x}. \end{aligned}$$

⁹ If $L(t) = k > 0$, the new arrival finds one job at the server and $k - 1$ in queue. Each of these jobs in the system has to be served before it is the turn of the new job.

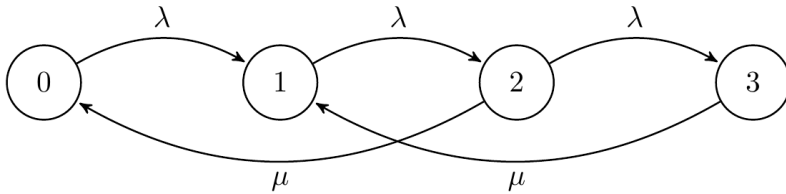
TF 4.6.1. In a large hotel, taxis arrive at a rate of $\lambda = 15$ per hour and riders' parties arrive at a rate of $\mu = 12$ per hour. Whenever taxis are waiting, riders are served immediately upon arrival. Whenever riders are waiting, taxis are loaded immediately upon arrival. A maximum of three taxis can wait at a time (other cabs must go elsewhere). Let $p(i, j)$ be the steady-state probability that there are i parties of riders and j taxis waiting at the hotel. Claim: the transitions are modeled by the graph below.



TF 4.6.2. Claim: To obtain *balance equations*, we do not count the number of up- and down-crossings of a level. Instead, we count how often a box around a state, such as state 2 in the figure below, is crossed from inside and outside.



TF 4.6.3. Consider the $M^2/M^2/1/3$ queue. The graph below shows all relevant transitions.



TF 4.6.4. For the $G/G/1$, the difference between the number of ‘out-transitions’ and the number of ‘in-transitions’ is at most 1 for all t . As a consequence,

$$\begin{aligned} \text{transitions out} &\approx \text{transitions in} \iff \\ A(n, t) + D(n-1, t) &\approx A(n-1, t) + D(n, t) \iff \\ \frac{A(n, t) + D(n-1, t)}{t} &\approx \frac{A(n-1, t) + D(n, t)}{t} \iff \\ \frac{A(n, t)}{t} + \frac{D(n-1, t)}{t} &\approx \frac{A(n-1, t)}{t} + \frac{D(n, t)}{t}. \end{aligned}$$

Thus, under proper technical assumptions (which you can assume to be satisfied) this becomes for $t \rightarrow \infty$,

$$(\lambda(n) + \mu(n))p(n) = \lambda(n-1)p(n-1) + \mu(n+1)p(n+1).$$

We claim that if we specialize this result for the $M/D/1$ queue we get $\lambda(n) = \lambda$ and $\mu(n) = \mu$, hence using PASTA,

$$(\lambda + \mu)\pi(n) = \lambda\pi(n-1) + \mu\pi(n+1).$$

TF 4.6.5. Claim: The level-crossing equation $\lambda(n)p(n) = \mu(n+1)p(n+1)$ holds for any queueing system in which jobs arrive and depart one at a time.

Ex 4.6.6. Consider the following (silly) queueing process. At times $0, 2, 4, \dots$ customers arrive, each customer requires 1 unit of service, and there is one server.

1. Find an expression for $A(n, t)$, when $L(0) = 0$.
2. Show that $\pi(0) = 1$ and $\pi(n) = 0$, for $n > 0$.
3. Check that $\lambda\pi(n) = \lambda(n)p(n)$.
4. Find an expression for $Y(n, t)$.
5. Compute $p(n)$ and $\lambda(n)$.
6. Compute $D(n, t)$ and $\mu(n+1)$ for $n \geq 0$.
7. Compute $\lambda(n)p(n)$ for $n \geq 0$, and check $\lambda(n)p(n) = \mu(n+1)p(n+1)$.

Ex 4.6.7. Derive $E[L] = \sum_{n=0}^{\infty} np(n)$ from (4.2.3).

Ex 4.6.8. Show for the $M/G/1$ that with probability ρ a job leaves behind a busy server. 

Ex 4.6.9. Consider a single server that serves one queue and serves only in batches of 2 jobs at a time (so never 1 job or more than 2 jobs). At most 3 jobs fit in the system. Single jobs arrive as a Poisson process with λ . Due to blocking, we set $\lambda(n) = \lambda$ for $n < 3$ and $\lambda(n) = 0$ for $n \geq 3$. The batch service times are exponentially distributed with mean $1/\mu$, so that by the memoryless property, $\mu(n) = \mu$.

Make a graph of the state-space and show, with arrows, the transitions that can occur and use level-crossing arguments to express the steady-state probabilities $p(n)$, $n = 0, \dots, 3$ in terms of λ and μ .

Ex 4.6.10. Show (4.6.11) from $|I(n, t) - O(n, t)| \leq 1$.

4.7 GRAPHICAL SUMMARIES

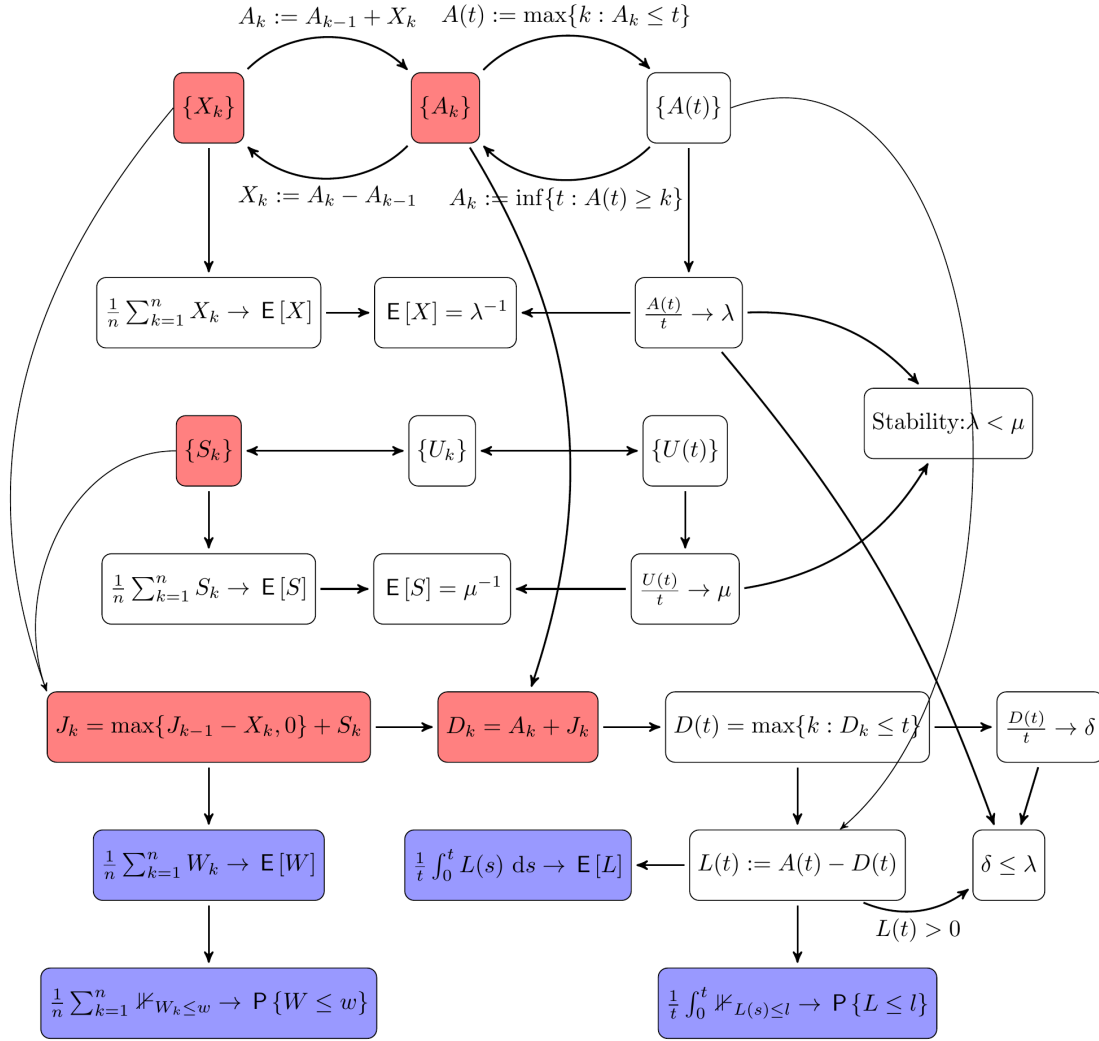


Fig. 4.7.1: An overview of the relations between the different types of times (inter-arrivals, arrivals, departures and so on) we use to construct a queueing process in continuous time.

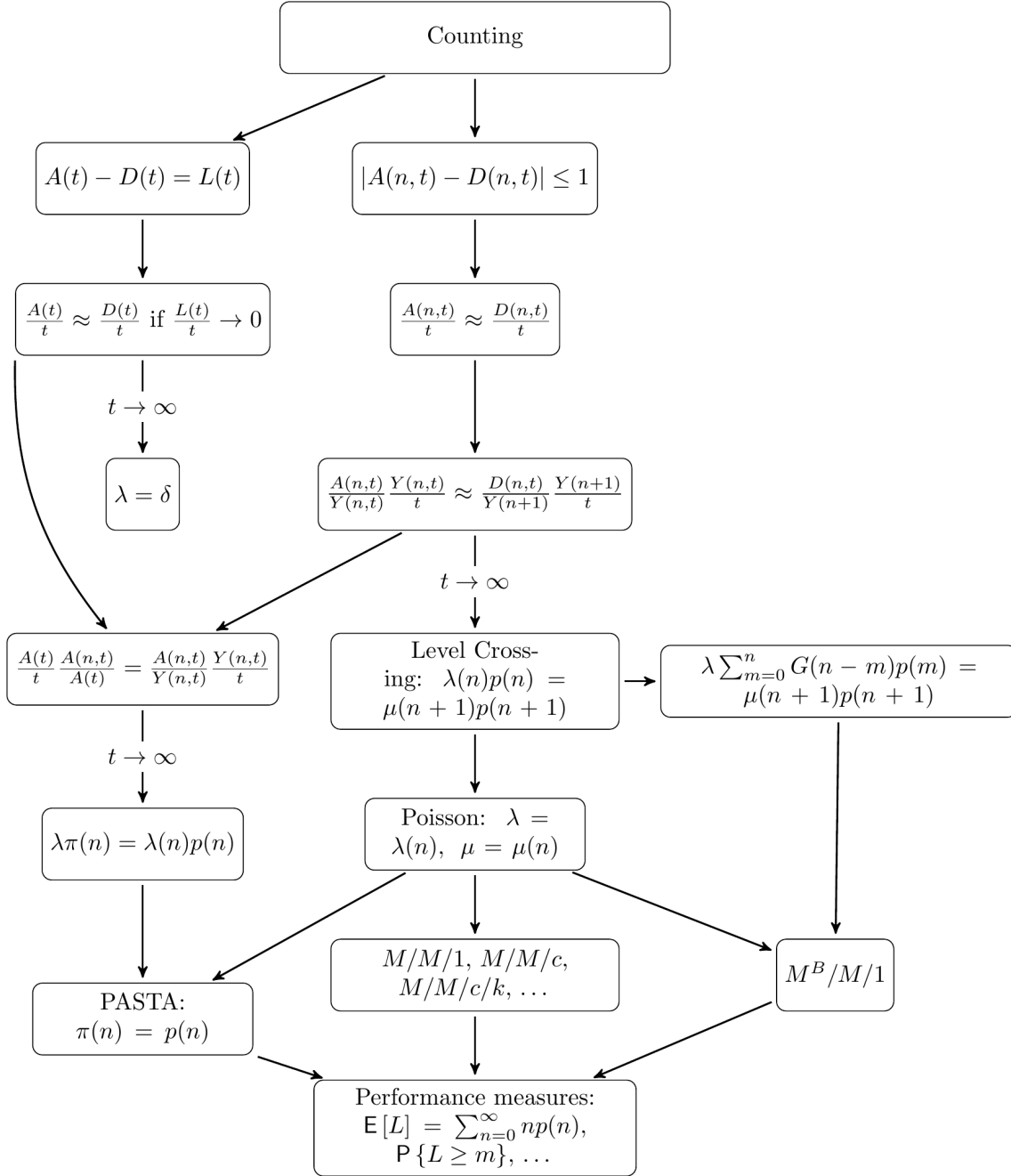


Fig. 4.7.2: This figure presents an overview of the results that derive from level-crossing arguments.

In this chapter we use the concepts of sample-path analysis and level-crossing, developed in Chapter 4, to model and analyze many single-station queueing systems in steady state. Section 5.1 considers several variants of the $M(n)/M(n)/1$ queue. In Section 5.2 we develop Python code to handle the numerical aspects, and we apply it to a simple business case. Section 5.3 discusses the $M^X/M/1$ queue and blocking policies. This analysis paves the way for deriving the formula for the expected waiting time of the $M/G/1$ queue; Section 5.4 provides the details. Section 5.5 then derives the queue length distribution of the $M/G/1$ queue. Section 5.6 discusses an example of the control of a queueing system. Section 5.7 and Section 5.8 provide models for inventory systems controlled by the base-stock policy or the (Q, r) policy. Section 5.9 develops an algorithm to compute the long-run average cost for the (s, S) and (T, S) inventory policies. Finally, Section 5.10 formulates this cost computation as a hitting problem. The theory of hitting and stopping problems extends far beyond inventory control; it is, for instance, one of the cornerstones of mathematical finance, ruin theory in actuarial science, and connects to the theory of partial differential equations.

5.1 THE $M(n)/M(n)/1$ QUEUE

In this section, we specify appropriate functions for $\lambda(n)$ and $\mu(n)$ in (4.6.2), or $\lambda^+(n)$ in (4.6.10) in case of blocking, for several of the queueing models discussed in Section 3.2. We emphasize that understanding how to specify the level-crossing equations is more important than deriving closed-form expressions for the performance measures.⁰ Further, it is assumed that a system without blocking is stable.¹

IN THE $M(n)/M(n)/1$ queue the inter-arrival and service times are exponentially distributed conditional on $L = n$, that is,

$$X|L = n \sim \text{Exp}(\lambda(n)), \quad S|L = n \sim \text{Exp}(\mu(n)).$$

Somewhat more specifically, similar to (1.2.6), we require that²

$$P\{L(t + \Delta t) = n + 1 | L(t) = n\} = \lambda^+(n)\Delta t + o(\Delta t), \quad n \geq 0,$$

$$P\{L(t + \Delta t) = n - 1 | L(t) = n\} = \mu(n)\Delta t + o(\Delta t), \quad n \geq 1.$$

Note that, when all arrivals are accepted³, we write $\lambda(n)$, but otherwise, we use $\lambda^+(n)$. The next models are all examples of the $M(n)/M(n)/1$ queue with different choices for $\lambda(n)$ (or $\lambda^+(n)$) and $\mu(n)$.

THE $M/M/1$ QUEUE has a constant arrival and service rate, so

$$\lambda(n) = \lambda^+(n) = \lambda, \quad \mu(n) = \mu.$$

⁰ The code of Section 5.2 deals with the numerical aspects.

¹ A system that blocks demand is stable anyway.

² In the general case, arrivals need not enter. We therefore consider λ^+ rather than λ .

³ In which case $\lambda^+(n) = \lambda(n)$.

The level-crossing equations (4.6.5) result in

$$p(n+1) = \frac{\lambda(n)}{\mu(n+1)} p(n) = \frac{\lambda}{\mu} p(n) = \rho p(n) = \rho^{n+1} p(0), \quad \rho = \frac{\lambda}{\mu},$$

and from (4.6.6),

$$p(0)^{-1} = \sum_{n=0}^{\infty} \rho^n = \frac{1}{1-\rho} \implies p(n) = (1-\rho)\rho^n, \quad n \geq 0. \quad (5.1.1)$$

With this expression for $p(n)$, it is easy to compute the key performance measures. The load equals the utilization since all jobs are accepted and served. With a bit of algebra,⁴

⁴ [5.1.8].

$$E[L] = \frac{\rho}{1-\rho}, \quad E[L_s] = \rho, \quad (5.1.2)$$

$$E[Q] = E[L] - E[L_s] = \frac{\rho^2}{1-\rho}, \quad P\{L > n\} = \rho^{n+1}. \quad (5.1.3)$$

THE $M/M/c$ QUEUE has c identical servers to process jobs, thus

$$\lambda(n) = \lambda^+(n) = \lambda, \quad \mu(n) = \mu \min\{n, c\}.$$

Closed-form expressions for $E[Q]$ and $p(n)$ exist⁵, but are not very informative. However, two measures are particularly simple. The load is $\rho = \lambda/c\mu$, and

⁵ [5.1.9].

$$E[L_s] \stackrel{1}{=} \sum_{n=0}^{\infty} \min\{n, c\} p(n) \stackrel{2}{=} \frac{\lambda}{\mu},$$

where 1 shows how to compute $E[L_s]$ as an expectation, and 2 follows from Little's law because jobs spend an average time $E[S] = 1/\mu$ in service, and jobs arrive at a rate λ at the servers.⁶

⁶ Note that $E[L_s] \neq \rho$.

THE $M/M/1/K$ QUEUE blocks jobs when $L \geq K$, hence⁷

⁷ Thus, $\lambda(n) \neq \lambda^+(n)$.

$$\lambda(n) = \lambda, \quad \lambda^+(n) = \lambda \mathbb{1}_{n < K}, \quad \mu(n) = \mu.$$

This gives $\lambda p(n) = \mu p(n+1)$ for $n < K$, and $p(n) = 0$ for $n > K$, because $\lambda^+(n) = 0$ for $n \geq K$. Thus, in general, $\lambda^+(n) p(n) = \mu(n+1) p(n+1)$, from which follows that

$$\rho = \frac{\lambda}{\mu}, \quad p(n) = \frac{1-\rho}{1-\rho^{K+1}} \rho^n.$$

There is a subtle point to make here. When the system contains K jobs, the rate at which jobs arrive at the system is $\lambda(K) = \lambda$ because jobs still *arrive* as a Poisson process. However, the rate at which jobs *enter* is $\lambda^+(K) = 0$, consequently,

$$\lambda \pi(K) = \lambda(K) p(K) \neq \lambda^+(K) p(K) = 0.$$

Since $\lambda(K) = \lambda$, we still have⁸ $\pi(K) = p(K)$, and therefore the *fraction* of jobs rejected upon arrival is equal to $p(K)$. Note that because jobs are

⁸ PASTA.

blocked, the utilization is no longer equal to the load $\rho = \lambda/\mu$. Specifically, if we define the mean acceptance rate as

$$\lambda^+ = \sum_{n=0}^{\infty} \lambda^+(n) p(n), \quad (5.1.4)$$

we see for the $M/M/1/K$ queue that $\lambda^+ = \lambda(1 - p(K))$. With this, the utilization becomes

$$\tilde{\rho} = \lambda^+/\mu = \lambda(1 - p(K))/\mu = E[L_s], \quad (5.1.5)$$

where the last equation follows from Little's law.

THE $M/M/c/K$ QUEUE is a multi-server queue with blocking⁹,

$$\lambda(n) = \lambda, \quad \lambda^+(n) = \lambda \mathbb{1}_{n < K}, \quad \mu(n) = \mu \min\{n, c\}.$$

THE $M/M/c/c$ QUEUE¹⁰ blocks jobs when all of its c servers are occupied,¹¹

$$\lambda(n) = \lambda, \quad \lambda^+(n) = \lambda \mathbb{1}_{n < c}, \quad \mu(n) = n\mu.$$

When $n < c$, $\lambda p(n) = (n+1)\mu p(n+1)$, hence

$$p(n+1) = \frac{\lambda}{(n+1)\mu} p(n) = \cdots = \frac{\lambda^{n+1}}{(n+1)!\mu^{n+1}} p(0).$$

The normalizing constant $p(0)$ follows from the constraint $1 = \sum_{n=0}^c p(n)$. As there are as many servers as jobs, $E[Q] = 0$. Moreover, jobs get accepted at a rate $1 - p(c)$, so that by PASTA,

$$E[L] = E[L_s] = \frac{\lambda}{\mu} (1 - p(c)).$$

THE $M/M/\infty$ QUEUE assigns a free server to each arrival,¹²

$$\lambda(n) = \lambda^+(n) = \lambda, \quad \mu(n) = n\mu.$$

By taking the limit $c \rightarrow \infty$ in the expressions of the $M/M/c/c$ queue¹³ we obtain

$$p(n) = e^{-\lambda/\mu} \frac{(\lambda/\mu)^n}{n!}, \quad E[Q] = 0, \quad E[L] = E[L_s] = \frac{\lambda}{\mu}.$$

We see that the number of busy servers in the $M/M/\infty$ queue is Poisson distributed with parameter $\lambda E[S]$.

A FINITE CALLING population has a finite number N of jobs. This model is useful for analyzing repair systems and inventory systems of spare parts. To see this, consider a factory with N machines and c mechanics.¹⁴ A machine can be in one of two states: working or failed. When a machine breaks down, it waits at the repair department until repaired. When n machines are in repair, there are still $N - n$ machines running. Thus, if λ is the rate at which an individual machine fails,

$$\lambda(n) = \lambda(N - n), \quad \mu(n) = \mu \min\{n, c\},$$

A CUSTOMER BALKS when finding the queue too long and deciding not to join. Balking differs from blocking. In the latter case, $\lambda^+(n) = \lambda \mathbb{1}_{n < K}$; in the former case, a fraction of the customers may already choose not to join the system at a lower level than K . A simple model for balking is

$$\lambda^+(n) = \lambda[K - n]^+, \quad \mu(n) = \mu.$$

⁹ Hence, it combines the $M/M/c$ and the $M/M/1/K$ queue.

¹⁰ Also known as the Erlang-B model.

¹¹ Why is it not necessary to write $\mu(n) = \mu \min\{c, n\}$?

¹² This system is said to have ample servers.

¹³ Or the $M/M/c$ queue.

¹⁴ When $c = N$, we obtain the Ehrenfest model of diffusion, which is used to provide some insight into the second law of thermodynamics.

TF 5.1.1. Consider the level-crossing analysis of the $M(n)/M(n)/1$ queue. Claim: it is necessary that the inter-arrival times of jobs are iid.

TF 5.1.2. For the $M/M/1$ queue, the following reasoning leads to the expected number of jobs in the system.

$$\begin{aligned} M_L(s) &= E[e^{sL}] = \sum_{n=0}^{\infty} e^{sn} p(n) = (1-\rho) \sum_n e^{sn} \rho^n \\ &= \frac{1-\rho}{1-e^s \rho}, \end{aligned}$$

where we assume that s is such that $e^s \rho < 1$. Then,

$$M'_L(s) = (1-\rho) \frac{1}{(1-e^s \rho)^2} e^s \rho.$$

Claim: This implies $E[L] = M'_L(0) = \rho/(1-\rho)$.

TF 5.1.3. Claim: To model the $M/M/c/c$ queue as an $M(n)/M(n)/1$ queue, we must set $\lambda(n) = \lambda$ for all n .

TF 5.1.4. Claim: For the $M/M/c$ queue, $E[Q] = \sum_{n=0}^{\infty} \max\{n-c, 0\} p(n)$, wherein $p(n) = P\{L = n\}$.

TF 5.1.5. Claim: For the $M/M/c$ queue, $p(n) = p(0) \frac{(\lambda/\mu)^n}{n!}$ for all $n \geq 0$.

Ex 5.1.6. Show that for the $M/M/1$ queue, $E[Q] = \sum_{n=1}^{\infty} (n-1)\pi(n)$ and use this to find $E[Q] = \rho^2/(1-\rho)$.

Ex 5.1.7. Derive the steady-state probabilities $p(n)$ for a queue with a finite calling population with N jobs and N servers. What happens if $N \rightarrow \infty$?

Ex 5.1.8. Use moment-generating functions to derive $E[L]$ and $V[L]$ for the $M/M/1$ queue, and show that $P\{L > n\} = \rho^{n+1}$.

In the remainder, we derive the KPIs of the above models.¹⁵

Opt 5.1.9. Derive the following expressions for the $M/M/c$ queue:

$$\rho = \frac{\lambda}{c\mu}, \quad (5.1.6a)$$

$$p(n) = \frac{1}{G} \frac{1}{\prod_{k=1}^n \min\{c, k\}} (c\rho)^n, \quad n \geq 1, \quad (5.1.6b)$$

$$G = \frac{1}{p(0)} = \sum_{n=0}^{c-1} \frac{(c\rho)^n}{n!} + \frac{(c\rho)^c}{(1-\rho)c!}, \quad (5.1.6c)$$

$$E[Q] = \sum_{n=c}^{\infty} (n-c)p(n) = \frac{(c\rho)^c}{c!G} \frac{\rho}{(1-\rho)^2} \quad (5.1.6d)$$

$$(5.1.6e)$$

Opt 5.1.10. Check that the performance measures of the $M/M/c$ queue reduce to those of the $M/M/1$ queue if $c = 1$.

Opt 5.1.11. Show that (5.1.5) holds.

Opt 5.1.12. Show that as $K \rightarrow \infty$, the performance measures of the $M/M/1/K$ converge to those of the $M/M/1$ queue.



¹⁵ These exercises are optional, i.e., you may skip them if you don't like algebra.



5.2 APPLICATIONS OF THE $M(n)/M(n)/1$ QUEUEING MODEL

In this section we apply the tools developed above to a number of interesting queueing systems. First, we discuss how to use the $M(n)/M(n)/1$ queue to organize a part of a production process. Second, we show how to plan the number of servers at a call center such that the average queue length remains small.

RILEY IS AN operations manager at a factory that produces custom metal parts for various industries. Riley's factory has a single heat treatment furnace to temper the metal parts. Recently, Riley noticed an increase in customer orders, leading to long delays and dissatisfied clients.

1. Should Riley sometimes block jobs to control the waiting time?
2. Should Riley buy another oven to increase the capacity?
3. If Riley bought a second oven, the utilization of both ovens may become quite low. However, as an oven is costly to heat up, it may be favorable to only switch on a second oven when queues are long.

To compare the effect of these different strategies, Riley hires us to model the operation of the oven using queueing theory. We consider the following metrics: average job waiting times, average number of jobs in the system, machine utilization, and job acceptance rate. For ease⁰, we assume that jobs arrive as a Poisson process with a rate λ jobs per hour, and production times are $\sim \text{Exp}(\mu)$ with $1/\mu = E[S]$. We then implement the level-crossing relations in Python for the appropriate variations of the $M(n)/M(n)/1$ queue.

THE KPIs FOR the queueing models below can be computed with the same formulas; the models differ only in their level-crossing equations. Thus, we can use an abstract base class, `MnMn`, to implement the common functionality. The code below needs some explanation. First, the implementation of the KPIs follows the discussion of Section 5.1.¹ Second, by putting `@abstractmethod` above a method, we indicate that derived classes have to provide the implementation of that method. Third, the computation of $p(n) = P\{L = n\}$ is perhaps slightly subtle. Start with $p(0) = 1$, and then compute, recursively, $p(n)$, for $n = 1, 2, \dots$ until $p(n) < \epsilon$, for some small ϵ . This idea is based on the assumption that, for some $\eta < 1$ and N , $\lambda(n)/\mu(n+1) < \eta$ whenever $n > N$, so that $p(n)$ decreases geometrically for $n > N$. Fourth, for the utilization, we assume that the service rate is non-decreasing in the number of jobs n in the system² and use (5.1.4) to compute the mean acceptance rate.

⁰ At a later stage we can make more complicated models.

¹ Think about the sequence in which we compute the KPIs.

² There is no practical reason to reduce service capacity when queue lengths become large.

```

1 from abc import abstractmethod
2 from functools import cache
3
4 eps = 1e-5 # Threshold for convergence.
5
6
7 def normalize(v):
```

```

8      "Normalize the array v such that its elements sum to 1."
9      G = sum(v[n] for n in range(len(v))) # normalization constant
10     return [v[n] / G for n in range(len(v))] # normalize
11
12
13     class MnMn:
14         def __init__(self):
15             self.compute_p()
16
17         @abstractmethod
18         def acceptance_rate(self, n):
19             pass
20
21         @abstractmethod
22         def service_rate(self, n):
23             pass
24
25         def compute_p(self):
26             # Recursively compute probabilities until convergence.
27             p = [1] # p[0] = 1
28             n = 0
29             while p[-1] > eps:
30                 p.append(p[n] * self.acceptance_rate(n) / self.service_rate(n + 1))
31                 n += 1
32             self.p = normalize(p)
33
34         @cache
35         def mean_acceptance_rate(self):
36             return sum(
37                 self.acceptance_rate(n) * self.p[n] for n in range(len(self.p))
38             )
39
40         def max_service_rate(self):
41             # Assumes service rate is non-decreasing in queue length.
42             return self.service_rate(len(self.p))
43
44         def ES(self):
45             return 1 / self.service_rate(1)
46
47         @cache
48         def EL(self):
49             return sum(self.p[n] * n for n in range(len(self.p)))
50
51         def EJ(self):
52             return self.EL() / self.mean_acceptance_rate()
53
54         def EW(self):
55             return self.EJ() - self.ES()
56
57         def EQ(self):
58             return self.mean_acceptance_rate() * self.EW()
59
60         def ELs(self):
61             return self.EL() - self.EQ()
62
63         def utilization(self):
64             return self.mean_acceptance_rate() / self.max_service_rate()
65
66         def loss(self):
67             return 1 - self.mean_acceptance_rate() / self.labda
68

```

```

69     def print_kpis(self):
70         print(f"EW: {self.EW():0.2f}")
71         print(f"EL: {self.EL():0.2f}")
72         print(f"acceptance rate: {self.mean_acceptance_rate():0.2f}")
73         print(f"utilization: {self.utilization():0.2f}")
74         print(f"loss: {self.loss():0.2f}")

```

IT IS NOW easy to construct various queueing models because we can derive much functionality from this `MnMn` class.

We start with the level-crossing equations for the $M/M/c$ queue.

```

1  class MMC(MnMn):
2      def __init__(self, labda, mu, c):
3          self.labda = labda
4          self.mu = mu
5          self.c = c
6          super().__init__()
7
8      def acceptance_rate(self, n):
9          return self.labda
10
11     def service_rate(self, n):
12         return self.mu * min(n, self.c)

```

The $M/M/1$ queue is the $M/M/c$ queue with $c = 1$.

```

1  class MM1(MMC):
2      def __init__(self, labda, mu):
3          super().__init__(labda, mu, c=1)

```

For the $M/M/c/K$ queue, we override the acceptance rate and the computation of p and implement a function to compute the fraction of lost jobs.

```

1  class MMCK(MMC):
2      def __init__(self, labda, mu, c, K):
3          self.K = K
4          super().__init__(labda, mu, c)
5
6      def acceptance_rate(self, n):
7          return self.labda * (n < self.K)
8
9      def compute_p(self):
10         p = [1] * (self.K + 1)
11         for n in range(1, self.K + 1):
12             p[n] = (
13                 p[n - 1] * self.acceptance_rate(n - 1) / self.service_rate(n)
14             )
15         self.p = normalize(p)

```

WE COMPUTE THE KPIs for the relevant queueing models.

We start with the base case.

```

1 labda = 1.9
2 mu = 2
3 mm1 = MM1(labda, mu)
4 mm1.print_kpis()

```

```

1 EW: 9.50
2 EL: 19.00
3 acceptance rate: 1.90
4 utilization: 0.95
5 loss: 0.00

```

Next, consider several variations of how to organize the process. Riley first considers reducing the inflow of jobs. What happens if we allow at most $K = 10$ jobs in the system?

```

1 mmck = MMCK(labda, mu, c=1, K=10)
2 mmck.print_kpis()

```

```

1 EW: 2.04
2 EL: 4.49
3 acceptance rate: 1.77
4 utilization: 0.88
5 loss: 0.07

```

This is interesting. The expected waiting time reduces by a factor 4.5, which is considerable, while the fraction of loss is $\approx 7\%$, which does not seem large.

As a check, we verify that the KPIs converge to those of the $M/M/1$ queue when K is large. The results confirm this.

```

1 mmck = MMCK(labda, mu, c=1, K=1000)
2 mmck.print_kpis()

```

```

1 EW: 9.50
2 EL: 19.00
3 acceptance rate: 1.90
4 utilization: 0.95
5 loss: 0.00

```

Riley's second idea is to buy a second oven. For this, we use the $M/M/2$ queue.

```

1 mmc = MMC(labda, mu, c=2)
2 mmc.print_kpis()

```

```

1 EW: 0.15
2 EL: 1.23
3 acceptance rate: 1.90
4 utilization: 0.47
5 loss: 0.00

```

Now, $E[W]$ is almost negligible, but oven utilization becomes low. As it is costly to keep both ovens at production temperature, this latter result is undesirable.

Next, we analyze a case in which the second oven switches on only when the system has m or more jobs.

```

1 class MMC_controlled(MMC):
2     def __init__(self, labda, mu, m):
3         self.m = m
4         super().__init__(labda, mu, c=1)
5
6     def service_rate(self, n):
7         return self.mu * ((n < self.m) + 2 * (n >= self.m))
8
9     def utilization_first(self):
10        return 1 - self.p[0]
11
12    def utilization_second(self):
13        return 1 - sum(self.p[n] for n in range(self.m))

```

We can compare the usual KPIs, but now we should also include the fraction of time the second oven is busy. For the $M/M/2$ queue, this fraction of time is $\sum_{n=2}^{\infty} p(n)$. In the controlled $M/M/2$, the second oven is used only when $n \geq 6$, so the fraction is $\sum_{n=6}^{\infty} p(n)$.

```

1 mmc_c = MMC_controlled(labda, mu, m=5)
2 mmc_c.print_kpis()
3 print(f"mmc_c.utilization_first()=:0.2f)")
4 print(f"mmc_c.utilization_second()=:0.2f)")

```

```

1 EW: 0.79
2 EL: 2.46
3 acceptance rate: 1.90
4 utilization: 0.47
5 loss: 0.00
6 mmc_c.utilization_first()=0.81
7 mmc_c.utilization_second()=0.14

```

Clearly, $E[W]$ increases a bit, while the second oven does not have to assist often.

To complete the analysis, we should make graphs of how the KPIs depend on the control parameters K or the switching level m . With the code above, this is not difficult anymore.³

LET US NEXT discuss an example of employee planning at a call center. Here we keep the models simple, but we include all necessary steps to solve a realistic planning problem.

We start with specifying the *arrival process*. It is reasonable to model it as a Poisson process, as there are many potential customers, each choosing with a small probability of calling at a certain moment in time. Thus, we only have to characterize the arrival rate. Estimating this for a call center is easy because all calls are registered.

³ As our educational goal (demonstrate how to build and use the models) is achieved; we leave it to Riley to make the graphs.

It is common to use a *demand profile* that shows the average number of customers arriving per hour. Then we model the arrivals as a Poisson process with an arrival rate that is constant during a certain hour, as specified by the demand profile.

It is also easy to find the *service distribution*; for ease, we model the service time distribution as exponential with mean 1.5 minutes.

For the *service objective* we prefer to keep the average number of jobs in the system such that $E[L] \leq 5$.⁴

Let us model the situation as an $M/M/1$ queue with one server that works as fast as c single servers combined. Solving for c in the inequality $E[L] \leq 5$ gives⁵

$$c \geq \frac{E[L] + 1}{E[L]} \lambda E[S] = \frac{6}{5} \lambda E[S] \approx 0.03 \lambda,$$

where, with the above estimate, $E[S] = 1.5/60$ hour. It is easy to apply this formula. For instance, we see in the demand profile Fig. 5.2.1 that $\lambda = 120$ customers per hour between 12 and 13, hence we need $c \approx 3.6 = 4$ employees for that period.

With this we can find a formula to convert the demand profile into a *load profile* to specify the minimal number of servers per hour needed to meet the service objective.

THE LAST STEP in the planning process is to *cover the load profile with service shifts*. This is typically not easy since shifts have to satisfy all kinds of restrictions with respect to breaks and durations. Moreover, shifts can also have different costs: evening shifts are typically more expensive per hour.

The usual way to solve such covering problems is by means of an integer programming problem. For instance, suppose only 4 *shift plans* are available as shown at the right.⁶

For the optimization, let x_i be the number of shifts of type i and c_i the cost of this type. Then the problem is to solve $\min \sum_i c_i x_i$, such that for all hours t the shifts cover the load, that is, $\sum_i x_i \mathbb{1}_{t \in s_i} \geq 0.03 \lambda_t$. (We write $t \in s_i$ if hour t is covered by shift type i .)

TF 5.2.1. Suppose $\frac{\lambda}{\mu} = 2$. Claim: 3 is the minimum number of servers required to make the system stable.

TF 5.2.2. Claim: It is always better to have a single fast server than multiple slow servers which together have the same rate.

TF 5.2.3. A repair/maintenance facility would like to determine how many employees should be working in its tool crib. The service time is exponential, with a mean of 4 minutes, and customers arrive as a Poisson process with a rate of 28 per hour. Claim: With one employee the system is not rate stable.

TF 5.2.4. Consider an $M/M/c/K$ queue that is not rate stable, but jobs sometimes balk. Claim: $\lim_{t \rightarrow \infty} \mathbb{P}(L(t) = K) = 1$.

TF 5.2.5. Claim: For the $M/M/c$ queue, adding a second server always halves the expected waiting time compared to the $M/M/1$ queue with the same arrival and service rates.

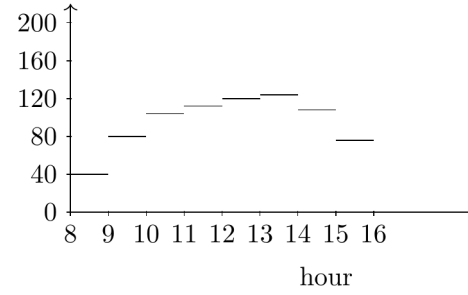


Fig. 5.2.1: A demand profile.

⁴ Other KPIs are also possible to use.

⁵ $E[L] = \rho / (1 - \rho) \implies \rho = E[L] / (E[L] + 1)$ and $\rho = \lambda E[S] / c$.

⁶ 1. + - - + + 2. + + + - + 3. + + - + + + 4. + + + - + +. Shift plans. A + indicates a working hour and - a break of an hour.

THE NEXT EXERCISES require some modeling skills. Hence, they may be quite hard even though the formulas are simple.

Ex 5.2.6. In the town hall of the municipality, people can, for example, renew their driver's license or passport. For ease, assume that there is just one employee and that the arrivals of customers are not planned. Why is it reasonable to model this queueing system as a $M/M/1$ queue?

Measurements show that $E[W] = 6$ minutes and $\lambda = 5$ per minute. What are the service rate and utilization? Calculate $E[Q]$, $E[L]$.

There should be a sufficient number of seats so that all waiting customers can sit down at least 90 percent of the time. How many seats should there minimally be? What would you advise the municipality?

Ex 5.2.7. The smallest barber shop in town works alone and has two chairs for waiting customers. Six jobs arrive per hour, and the barber needs 12 minutes for a haircut. We model this, for ease, as an $M/M/1/3$ queue. Estimate $E[Q]$ and the average number of customers served per hour. Then, estimate $E[Q]$ when the barber hires another location that allows for 3 extra chairs. Quantify the effect on the number of customers served per hour.

Ex 5.2.8. Occasionally there is a pizza stand at the corner of the Verlengde Visserstraat and the Westersingel in Groningen. As people (tend to) behave like sheep, the rate at which people join the queue depends on the actual queue length. When there is nobody waiting, people are somewhat hesitant to order: 'When there are no people waiting, most likely the pizzas are not good.'. However, when the queue is long, people prefer to go elsewhere. Measurements show that the number of arrivals per hour is well captured by the array `labda = np.array([10.0, 10.0, 20.0, 20.0, 5.0])`. The pizza oven can contain two pizzas and it takes 2 minutes to bake a pizza. We model the system as an $M/M/2$ queue.

Calculate the steady-state probabilities and from this the average arrival rate. Then, find $E[L]$, $E[Q]$, $E[J]$ and $E[W]$.

Ex 5.2.9. Fast food restaurants tend to fry patties before customers arrive. This way, they can serve customers faster, because when a customer arrives while there is still an inventory of prepared patties, the server only has to flip a patty in a bun, while otherwise the customer has to wait for the additional frying time of the patty. Assume the restaurant keeps at most 3 patties on stock. To keep the model simple, assume the restaurant has just one pan to fry a patty and the frying time $S \sim \text{Exp}(\mu)$ with $\mu = 15$ per hour. Customers arrive as a Poisson process with rate $\lambda = 12$ per hour. When there is a prepared patty upon arrival, the customer is served immediately, otherwise the customer has to wait for a patty.

Explain how to model this system as an $M/M/1$ system. Then compute the average number of patties on stock, the average number of people waiting, and the average waiting time of a patty before it ends up in a bun, and the average time a person has to wait for a hamburger.

You should remember this problem. It is of fundamental importance in inventory control and known as an *inventory system with backlogging*; it is (implicitly) used by nearly any company that keeps inventory.

Ex 5.2.10. [Continuation of [5.2.9]] What would be the impact of allowing four patties on stock?

Ex 5.2.11. [Continuation of [5.2.9]] Suppose that the restaurant does not keep patties in stock. What is the expected waiting time for the customer?

5.3 $M^B/M/1$ QUEUE AND REJECTION POLICIES

In this section, by using the renewal-reward theorem, we derive for the $M^B/M/1$ queue expressions for ρ , $E[W]$, and $E[L]$. To compute more difficult performance measures, such as the loss probability $P\{L > n\}$, we need expressions for the stationary distribution $\pi(n)$. We use level-crossing to derive a recursive scheme which, when combined with a policy to reject batches, can be used to compute these probabilities.

ASSUME THAT BATCHES arrive as a Poisson process with rate λ_B , and the number of jobs in a batch are iid $\sim B$. Let $S_{k,i}$ denote the service time of the i th job of batch k ; assume that the $S_{k,i}$ are iid $\sim S$ and independent of B . By (4.1.2) the utilization $\rho = \lambda_B E[B] E[S]$; as always, we require $\rho < 1$.

An arriving batch joins the end of the queue (if present). Once it arrives at the head of the queue, the batch moves in its entirety to the server⁰, and then the server processes the jobs one after another until the batch is finished. Writing Q^B for the number of batches in queue and L_s^B for the number of jobs (if any) at the server, the average time $E[W^B]$ a batch spends in queue is¹

$$E[W^B] = (E[L_s^B] + E[Q^B] E[B]) E[S]. \quad (5.3.1)$$

Substituting the equality² $E[Q^B] = \lambda_B E[W^B]$ and simplifying yields

$$E[W^B] = \frac{E[L_s^B]}{1 - \rho} E[S]. \quad (5.3.2)$$

We can relate the average time $E[W]$ a *job* spends in queue to $E[W^B]$ by observing that $E[L_s^B] + E[Q^B] E[B] = E[L]$,³ so from (5.3.1)

$$E[W^B] = E[L] E[S] \stackrel{1}{=} E[W],$$

where 1 follows from the memorylessness of the service time of the job in service (if any). Thus, to compute $E[W]$ it remains to find an expression for $E[L_s^B]$ in (5.3.2).

For this, we use the renewal reward theorem. Define⁴ $R(t) := \int_0^t L_s^B(s) ds$, where $L_s^B(s)$ is the number of jobs (of the batch in service) at the server at time s , and $Z_k = R(D_k) - R(D_{k-1})$ where D_k is the departure time of the k th batch. Suppose the k th batch has batch size B_k , then

$$Z_k | B_k = \int_{D_{k-1}}^{D_k} L_s^B(s) ds = B_k S_{k,1} + (B_k - 1) S_{k,2} + \cdots + S_{k,B_k},$$

because B_k jobs are at the server during the service of the first job, $B_k - 1$ during the service of the second, $\dots$ Using that the $S_{k,i} \sim S$,

$$E[Z_k | B_k] = B_k E[S] + (B_k - 1) E[S] + \cdots E[S] = \frac{B_k(B_k + 1)}{2} E[S].$$



⁰ This is the same batching model as in Section 6.2.

¹ First the jobs at the server have to be served, then the ones in queue.

² Little's law.

³ The total number of jobs in the system is the number at the server plus the number in queue. The latter equals the number of batches in queue times the average batch size.

⁴ Much like the proof of Little's law.

As $B_k \sim B$,

$$\mathbb{E}[Z_k] = \mathbb{E}[\mathbb{E}[Z_k|B_k]] = \mathbb{E}\left[\frac{B(B+1)}{2}\right] \mathbb{E}[S] = \frac{\mathbb{E}[B^2] + \mathbb{E}[B]}{2} \mathbb{E}[S].$$

We now have all elements to fill in $R = \alpha Z$. First, $R = \lim_{t \rightarrow \infty} R(t)/t = \mathbb{E}[L_s^B]$. Second, we sample at batch departure moments, so $\alpha = \delta_B$ which equals λ_B by rate stability. Third, $Z = \mathbb{E}[Z_k]$. Combining all this, and using that $\rho = \lambda_B \mathbb{E}[B] \mathbb{E}[S]$, we obtain

$$\mathbb{E}[L_s^B] = R = \lambda_B Z = \lambda_B \frac{\mathbb{E}[B^2]}{2} \mathbb{E}[S] + \frac{\rho}{2}. \quad (5.3.3)$$

We can brush this up by realizing that⁵

$$\frac{\mathbb{E}[B^2]}{(\mathbb{E}[B])^2} = \frac{\mathbb{E}[B^2] - (\mathbb{E}[B])^2 + (\mathbb{E}[B])^2}{(\mathbb{E}[B])^2} = \frac{\text{Var}[B] + (\mathbb{E}[B])^2}{(\mathbb{E}[B])^2} = C_B^2 + 1.$$

Substituting this into the above gives the final result

$$\mathbb{E}[W] = \frac{1 + C_B^2}{2} \frac{\rho}{1 - \rho} \mathbb{E}[B] \mathbb{E}[S] + \frac{1}{2} \frac{\rho}{1 - \rho} \mathbb{E}[S]. \quad (5.3.4)$$

This formula is an important result. It tells us that $\mathbb{E}[W]$ increases when the load ρ or the variability C_B^2 or $\mathbb{E}[B]$ or $\mathbb{E}[S]$ increases (even when the utilization ρ remains the same). As such, it gives us the most important clues about what to change if $\mathbb{E}[W]$ ⁶ has to be reduced.

RATHER THAN USING the time an entire batch spends at the server, we can concentrate on the expected time spent by a single job at the server. This will provide us with a simple expression for $\mathbb{P}\{L_s^B = i\}$, from which we also can derive (5.3.3).

In this case, let $R_i(t) = \int_0^t \mathbb{1}_{L_s^B(s)=i} ds$ be the total time there are i jobs at the server, so that $R_i(t)/t \rightarrow \mathbb{P}\{L_s^B = i\}$ as $t \rightarrow \infty$. By sampling $R(\cdot)$ at batch departure times $\{D_k\}$, see Fig. 5.3.1, we see that

$$Z_k = R_i(D_k) - R_i(D_{k-1}) = S_{k,i} \mathbb{1}_{B_k \geq i},$$

because only when $B_k \geq i$, we can see i jobs at the server for the k th batch, and then $S_{k,i}$ is the time this job occupies the server. With this,

$$Z = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n S_{k,i} \mathbb{1}_{B_k \geq i} = \mathbb{E}[S \mathbb{1}_{B \geq i}] = \mathbb{E}[S] G(i-1),$$

since $S_{k,i}$ and B_k are independent, and⁷ $G(i-1) = \mathbb{E}[\mathbb{1}_{B \geq i}] = \mathbb{P}\{B > i-1\}$ is the survivor function of B . With the renewal reward theorem and rate stability, $\delta_B = \lambda_B$, we conclude that

$$\mathbb{P}\{L_s^B = i\} = \lambda_B \mathbb{E}[S] G(i-1).$$

This expression provides us with another derivation of (5.3.4). Just note that $\mathbb{E}[L_s^B] = \sum_{i=0}^{\infty} i \mathbb{P}\{L_s^B = i\} = \lambda_B \mathbb{E}[S] \sum_{i=1}^{\infty} i G(i-1)$ and use [5.3.12] to simplify.

⁵ Recall that the scv of a rv B is $C_B^2 = \text{Var}[B] / (\mathbb{E}[B])^2$. We assume that $\mathbb{E}[B^2] < \infty$.

⁶ Hence, by Little's law, $\mathbb{E}[L]$.

⁷ Recall: $\mathbb{P}\{A\} := \mathbb{E}[\mathbb{1}_A]$.

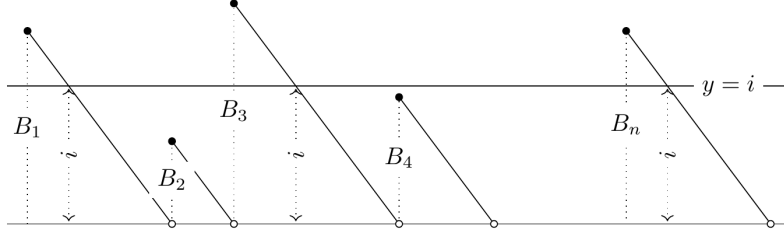


FIG. 5.3.1: A batch crosses the line $y = i$ iff it contains at least i jobs. Thus, during the service of a batch with i or more jobs, there is precisely one service of an i -th job.

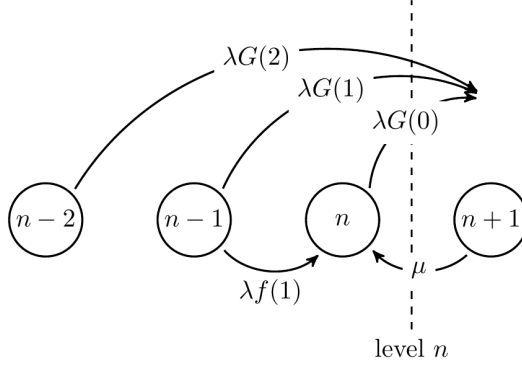


FIG. 5.3.2: When $L(t) = n$, the arrival of any batch crosses level n from below, but when $L(t) = n - 1$, only a batch with a size of at least two jobs can realize an up-crossing of level n , and so on.

THE SECOND TASK is to use level-crossing to find a recursive formula for the state probabilities $\{\pi(n)\}$.⁸ For this we need to generalize our earlier level-crossing equation $\lambda p(n) = \mu p(n - 1)$ because when a batch arrives, multiple levels can be up-crossed, see Fig. 5.3.2. We use the PASTA property to express level-crossing in terms of $\{\pi(n)\}$ instead of $\{p(n)\}$.

The down-crossing rate of level n is easy: since jobs are still served one-by-one, we have $\mu\pi(n + 1)$.

For the up-crossings of level n we should consider batch arrivals that see $m \leq n$ jobs in the system. Since batches arrive at a rate λ_B , PASTA implies that $\lambda_B\pi(m)$ is the arrival rate of batches that see m upon arrival.⁹ When an arriving batch sees m jobs in the system, and level n has to be up-crossed, the batch must contain more than $n - m$ jobs. As $G(n - m)$ is the probability that the batch size is larger than $n - m$ and as batch sizes are independent of arrival times, such batches arrive as a Poisson process with rate $\lambda_B\pi(m)G(n - m)$. Finally, to compute the rate at which level n is up-crossed, we merge all these thinned Poisson processes into a new Poisson process with rate $\lambda_B \sum_{m=0}^n \pi(m)G(n - m)$.

By level-crossing, the up-crossing and down-crossing rates must match, thus,

$$\lambda_B \sum_{m=0}^n \pi(m)G(n - m) = \mu\pi(n + 1). \quad (5.3.5)$$

It is an important check¹⁰ to retrieve (5.3.4) from the expression $E[L] = \sum_n n\pi(n)$ and Little's law.

⁸ Note: n means the number of jobs in the system, not the number of batches.

⁹ That is, the total arrival process is thinned with a fraction $\pi(m)$.

¹⁰ [5.3.13].

IT REMAINS TO find the normalization constant for π . However, this is not possible in general because (5.3.5) does not lead to a closed form expression for $\pi(n)$.¹¹ Numerically, though, we can start with $\pi(0) = 1$, apply the recursion, and truncate when the (unnormalized) probabilities become negligible. The RV class then normalizes the result. The function `mxm1` applies this idea.

¹¹ Compare (5.1.1).

```

_____ batchqueues.py _____
1  from random_variable import NumericRV as RV
2
3  labda_B, mu = 1, 3
4  B = RV({1: 1, 2: 1, 3: 1})
5  rho = labda_B * B.mean() / mu
6  rho_B = labda_B / mu
7
8
9  def mxm1(eps=1e-10):
10     "Compute stationary distribution of number of jobs in the system."
11     if rho >= 1:
12         print("The load is too high")
13         quit()
14     pi, n = [1], 0
15     batch_sizes = [int(m) - 1 for m in B.support()]
16     while n < batch_sizes[-1]:
17         res = sum(pi[m] * B.sf(n - m) for m in range(n + 1))
18         pi.append(res * rho_B)
19         n += 1
20     while pi[-1] > eps:
21         res = sum(pi[n - m] * B.sf(m) for m in batch_sizes)
22         pi.append(res * rho_B)
23         n += 1
24     return RV({i: pi[i] for i in range(len(pi))})

```

A different approach is to completely bypass the normalization problem by blocking the demand above some level K . There are three common policies for deciding which jobs in a batch to accept.

1. Complete acceptance: accept all batches that arrive when the system contains $K - 1$ or fewer jobs, and reject the entire batch when the system contains K or more jobs. For this case, we make the assumption that the batch size is bounded by the number K_B so that $L(t) \leq K + K_B$.¹²
2. Partial acceptance: accept whatever fits of a batch, i.e., up to K , and reject the rest.¹³
3. Complete rejection: if a batch does not fit entirely into the system, reject it completely.¹⁴

¹² [5.3.8].

¹³ [5.3.9].

¹⁴ [5.3.10].

In [5.3.9]–[5.3.10] we will derive recursions similar to (5.3.5) to compute $\{\pi(n)\}$ for these different blocking models.

TF 5.3.1. For the $M^B/M/1$ queue, let $E[Q]$ be the expected number of jobs in queue, $E[Q^B]$ the expected number of batches in the queue, $E[W]$ the expected waiting time of a job in queue, λ the rate at which jobs arrive, and λ_B the rate at which batches arrive. Claim: Little's law informs us that $E[Q^B] E[B] = \lambda E[W]$.

TF 5.3.2. For an $M^B/M/1$ queue we know that:

$$E[W] = \frac{1 + C_B^2}{2} \frac{\rho}{1 - \rho} E[B] E[S] + \frac{1}{2} \frac{\rho}{1 - \rho} E[S].$$

Suppose that $V[B] = 0$. Claim: This formula implies that if the batch size increases but $E[S]$ decreases such that ρ remains the same, the expected waiting time decreases.

TF 5.3.3. Suppose that the time to process a job has an exponential distribution. Claim: the inter-arrival times at which individual jobs arrive in a $M^B/M/1$ queue are exponentially distributed.

TF 5.3.4. Claim: This code correctly implements an $M^B/M/1/K$ queue with complete rejection.

```

1  def complete_rejection(K):
2      pi, n = {}, 0
3      pi[0] = 1
4      while n < K:
5          pi[n + 1] = sum(
6              pi[m] * (S.sf(n - m) - S.sf(K - m)) for m in range(n)
7          )
8          pi[n + 1] *= rho
9          n += 1
10     return RV(pi)

```

TF 5.3.5. Claim: In the $M^B/M/1$ queue, if the batch size B is deterministic, the waiting time formula reduces to the Pollaczek-Khinchine formula for the $M/M/1$ queue.

Ex 5.3.6. A company operates a machine that receives batches of various sizes. Management would like to know how a reduction in batch size variability would affect average queueing time. Suppose, for the sake of an example, that batch sizes are such that $P\{B = 1\} = P\{B = 2\} = P\{B = 3\} = 1/3$. The batches arrive at the rate $\lambda_B = 1/h$. The average service time for a job is 25 minutes. Calculate by how much $E[L]$ would decrease if $B \equiv 2$.

Suppose B is geometrically distributed¹⁵ with $E[B] = 1/p$.

Ex 5.3.7. Compute $E[L]$ for the $M^B/M/1$ queue when B is geometrically distributed with $E[B] = 1/p$. Check that if $E[B] = 1$ the $M/M/1$ queue results.

Ex 5.3.8. Derive a set of recursions analogous to (5.3.5) to compute $\pi(n)$ for the $M^B/M/1/K$ queue with complete acceptance.

Ex 5.3.9. Derive a set of recursions analogous to (5.3.5) to compute $\pi(n)$ for the $M^B/M/1/K$ queue with partial acceptance.

Ex 5.3.10. Derive a set of recursions analogous to (5.3.5) to compute $\pi(n)$ for the $M^B/M/1/K$ queue with complete rejection.

Opt 5.3.11. Show that when $B \leq K_B$, the probabilities $\pi(n)$ decrease geometrically fast.

Opt 5.3.12. Use [1.1.6] to show that $\sum_{i=1}^{\infty} iG(i-1) = (E[B^2] + E[B])/2$.

Opt 5.3.13. Use (5.3.5) in $E[L] = \sum_{n=0}^{\infty} n\pi(n)$ to derive (5.3.4).

¹⁵ Sometimes this distribution is called the First Success distribution. There is no consensus on the definition of the geometric distribution, see Wikipedia.



5.4 M/G/1 QUEUE: EXPECTED WAITING TIME

The Pollaczek-Khinchine (PK) formula for the average waiting time of an M/G/1 queue is a key result in Operations Research. We derive it using the renewal reward theorem, and follow the steps of Section 5.3.

WE START FROM the simple observation that before an arrival gets access to the server, it must first wait for the job in service⁰ to complete, then for the queue to clear. From PASTA it follows that $E[W] = E[S_r] + E[Q] E[S]$, where $E[S_r]$ is the (time-)average remaining service time of the job in service.¹ With Little's law $E[Q] = \lambda E[W]$ and $\rho = \lambda E[S]$,

$$E[W] = E[S_r] + \lambda E[W] E[S] \implies E[W] = \frac{E[S_r]}{1 - \rho}. \quad (5.4.1)$$

To proceed, we need an expression for $E[S_r]$.

Theorem 5.4.1. The expected remaining service time as observed by Poisson arrivals is

$$E[S_r] = \frac{\lambda}{2} E[S^2],$$

provided the second moment of S exists.

Proof. In Fig. 5.4.1, the service of job k starts at $\tilde{A}_k$, so the job departs at time $D_k = \tilde{A}_k + S_k$. Hence, at any time $s \in [\tilde{A}_k, D_k]$, the remaining service time of job k is $D_k - s$.

More generally, at some arbitrary time s , the number of departures is $D(s)$. Thus, $D(s) + 1$ is the index of the first job to depart after time s , and its departure time is $D_{D(s)+1}$. Consequently, the remaining service time at time s is $D_{D(s)+1} - s$, provided that this job is in service, i.e., $\tilde{A}_{D(s)+1} \leq s$. The total remaining service time as seen by the server up to time t is therefore

$$R(t) = \int_0^t (D_{D(s)+1} - s) \mathbb{1}_{\tilde{A}_{D(s)+1} \leq s} ds,$$

and $\lim_{t \rightarrow \infty} R(t)/t = E[S_r]$.

In Fig. 5.4.1, $R(D_k) - R(D_{k-1}) =: Z_k$ equals the area of the triangle. By inspecting $R(\cdot)$ at $\{D_k\}$, we see that $Z = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{k=1}^n S_k^2/2 = E[S^2]/2$. By the renewal-reward theorem, $R = \delta Z$, but $\delta = \lambda$ by rate-stability. The claim follows now from PASTA. $\square$

Noting that

$$\frac{E[S^2]}{(E[S])^2} = \frac{E[S^2] - (E[S])^2 + (E[S])^2}{(E[S])^2} = \frac{V[S] + (E[S])^2}{(E[S])^2} = C_s^2 + 1.$$

and substituting this in the expression for $E[S_r]$ we obtain from (5.4.1) the following result.

Theorem 5.4.2. The expected waiting time in the M/G/1 queue equals²

$$E[W] = \frac{1}{2} \frac{\lambda E[S^2]}{1 - \rho} = \frac{1 + C_s^2}{2} \frac{\rho}{1 - \rho} E[S]. \quad (5.4.2)$$

⁰ If any.

¹ The rv S_r is also known as the remaining lifetime and is widely used in maintenance theory and actuarial sciences.

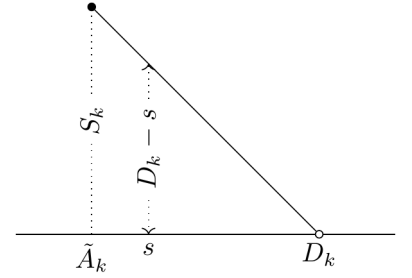


Fig. 5.4.1: Remaining service time.

² The Pollaczek-Khinchine (PK) formula.

THE POLLACZEK-KHINCHINE EQUATION can also be found as a limiting case of the waiting time of the $M^B/M/1$ queue by interpreting a job in the $M/G/1$ as a batch consisting of many tiny items in the $M^B/M/1$ queue.³ Suppose that in the $M^B/M/1$ queue the service times of the items are iid with mean $1/\nu$. If F is the cdf of the service times of the $M/G/1$ queue, then we take

$$P\{B = k\} = F(k/\nu) - F((k-1)/\nu)$$

as the probability that a batch has size k in an $M^B/M/1$ queue. Note that in the $M^B/M/1$ queue, when a batch has size k , the service time is the sum of k rvs that are $\text{Exp}(\nu)$; thus, the service time of a batch of size k is $\sim \text{Gamma}(k, \nu)$. When $\nu \gg 1$, the expected service time of a batch is

$$\begin{aligned} E[S_B] &= E[E[S_B|B]] = E\left[\frac{B}{\nu}\right] = \sum_{k=0}^{\infty} \frac{k}{\nu} P\{B = k\} \\ &= \sum_{k=0}^{\infty} \frac{k}{\nu} \left[F\left(\frac{k}{\nu}\right) - F\left(\frac{k-1}{\nu}\right) \right] \approx \int_0^{\infty} x dF(x) = E[S]. \end{aligned}$$

In words, when ν is large, the batch expected service time in an $M^B/M/1$ queue approximates the expected service time in an $M/G/1$ queue.

In fact, the proof that the expected waiting time of the $M^B/M/1$ converges to the Pollaczek-Khinchine equation when $\nu \rightarrow \infty$ is based on the fact that every cdf F , concentrated on $[0, \infty)$, can be approximated (weakly) by the sequence of cdfs⁴

$$F_{\nu}(x) = F(0) + \sum_{k=1}^{\infty} \left[F\left(\frac{k}{\nu}\right) - F\left(\frac{k-1}{\nu}\right) \right] \int_0^x \frac{\nu^k t^{k-1}}{(k-1)!} e^{-\nu t} dt, \quad \text{as } \nu \rightarrow \infty.$$

That is, the distribution of the service time of a batch whose items have $\text{Exp}(\nu)$ service times approaches the cdf F of the service time of the $M/G/1$ queue. Thus, when $\nu \rightarrow \infty$ in (5.3.4), the mean and scv of the batch service time converge to $E[S]$ and C_S^2 of the job, and the last term vanishes as it is proportional to $1/\nu$. This leads to (5.4.2).

TF 5.4.1. Consider the $M/G/1$ queue. Denote by $\tilde{A}_k$ the time job k starts service and by D_k its departure time, $k = 1, \dots, n$. Claim: the expression

$$\sum_{k=1}^n \int_0^{D_n} (D_k - s) \mathbb{1}_{\tilde{A}_k \leq s < D_k} ds$$

represents the total remaining service time up to $t = D_n$.

TF 5.4.2. Claim: For an $M/D/1$ queue, $E[W] = \frac{\rho}{1-\rho} \frac{E[S]}{2}$.

TF 5.4.3. Claim: The PK formula shows that $E[W]$ is linear in ρ .

TF 5.4.4. Claim: For the $M/G/1$ queue, the expected remaining service time $E[S_r]$ does not depend on the service time distribution, only on $E[S]$.

TF 5.4.5. Claim: The derivation of the PK formula uses both PASTA and Little's law.

³ If you are not interested in math, you may skip the next derivation.

⁴ See Asmussen, Section III.4, *Applied Probability and Queues*, 2003.

Ex 5.4.6. Consider a workstation with only one machine. We model the job arrival process as a Poisson process with rate $\lambda = 3$ per day. The average service time $E[S] = 2$ hours, $C_S^2 = 1/2$, and the store operates 8 hours per day. Show that $E[W] = 4.5$ h. What would you propose to reduce $E[W]$ to 2h?

Ex 5.4.7. Compare the expected waiting times in the $M/D/1$ and the $M/M/1$ queues.

Ex 5.4.8. Compute $E[J]$ for the $M/G/1$ queue with $S \sim U[0, \alpha]$.

Ex 5.4.9. Show that for the $M/G/1$ that $E[S_r] = \rho E[S_r | S_r > 0]$. As a consequence, for the $M/M/1$ queue, $E[S_r] \neq E[S]$. Why not?

5.5 M/G/1 QUEUE LENGTH DISTRIBUTION

In Section 5.3 we used level-crossing arguments to find a recursion for the stationary distribution $\pi(n)$ of the $M^B/M/1$ queue.⁰ Here we find a similar recursion to compute $\pi(n)$ for the $M/G/1$ queue. However, we cannot simply copy the ideas of Section 5.3 to the present situation, because in the $M^B/M/1$ queue the service times of the items are exponential, hence memoryless, while in the $M/G/1$ this is not the case.

IF WE WANT to characterize the state at all moments in time, we need to keep track of the number of jobs in the system *and* the remaining service time of the job in service (if any), because service times are not memoryless. But, by sampling at job departure times $\{D_k\}$,¹ we can restrict the state description to just the number in the system. For ease we write² $L_k = L(D_k)$ for the number of jobs in the system left behind at the departure epoch of the k th job.

We now find a useful recursion for $\{L_k\}$. Assume first that $L_{k-1} > 0$.³ If Y_k jobs arrive during the service S_k of job k , then $L_k = L_{k-1} - 1 + Y_k$. The situation is slightly more complicated when $L_{k-1} = 0$: we first have to wait for job k to arrive, and then, if Y_k jobs arrive while job k is in service, $L_k = Y_k$. Thus, in general,

$$L_k = [L_{k-1} - 1]^+ + Y_k.$$

With this recursion it is simple to find the level-crossing equations. Write $\delta_k(n) := P\{L_k = n\}$. The rvs $\{Y_k\}$ form a sequence of iid rvs $\sim Y$ with pmf $f(j) = P\{Y = j\}$ and survivor function G .

If L_k down-crosses level n , then $L_{k-1} = n + 1$; see Fig. 5.5.1 for an example at level $n = 3$. Hence, the probability of down-crossing level n at time D_k is $\delta_{k-1}(n + 1)f_0$.⁴ To up-cross level n , it is necessary that $L_{k-1} \leq n$ and $L_k > n$. When $L_{k-1} = 0$, then $L_k = Y_k$, so we need $Y_k > n$, giving probability $G(n)$. When $L_{k-1} = m \geq 1$, then $L_k = m - 1 + Y_k > n$ requires $Y_k > n + 1 - m$. The probability of up-crossing is then $\sum_{m=1}^n \delta_{k-1}(m)G(n + 1 - m) + \delta_{k-1}(0)G(n)$.⁵

It is possible to prove⁶ that $\delta(n) = \lim_{k \rightarrow \infty} \delta_k(n)$ exists for all n . Then, equating the up- and down-crossing probabilities, taking the limit $k \rightarrow \infty$

⁰ You can skip this if you are not interested.

¹ So that the remaining service time is guaranteed to be 0.

² i.e., not $L(D_k^-)$.

³ This implies that $A_k < D_{k-1}$.

⁴ Y_k and L_{k-1} are independent by assumption for the $M/G/1$ queue.

⁵ Beware off-by-one errors; check!

⁶ With some Markov chain theory.

at the LHS and the RHS, and noting that $\pi(n) = \delta(n)$, see (4.6.8), we arrive at a recursion for $\{\pi(n)\}$:

$$\pi(n+1)f(0) = \pi(0)G(n) + \sum_{m=1}^n \pi(m)G(n+1-m). \quad (5.5.1)$$

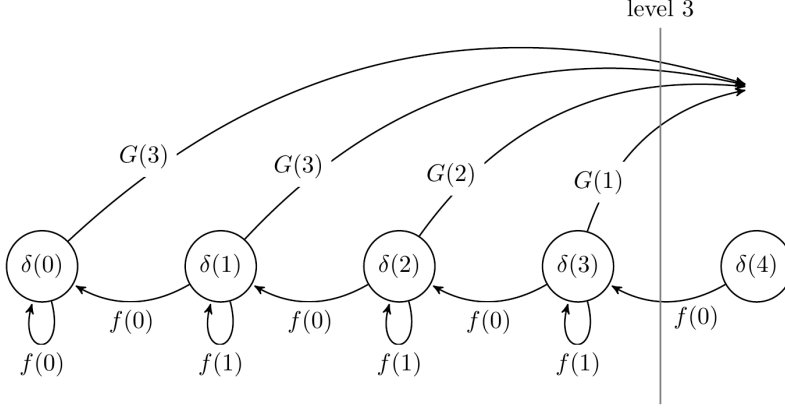


FIG. 5.5.1: Level crossing at departure moments.

FOR THE EVALUATION of the above recursion we can just follow the scheme of Section 5.3, but there is an important difference: here the $\{f(k)\}$ need to be computed.

We use a conditioning argument to find an expression for $P\{Y = j\}$. Since jobs arrive as a Poisson process, $Y|S = s \sim P(\lambda s)$, i.e.,

$$P\{Y = j | S = s\} = e^{-\lambda s} \frac{(\lambda s)^j}{j!}.$$

If $S \equiv s$, that is, a constant, then this is the distribution we are looking for. When F has a density f , then,

$$P\{Y = j\} = \int_0^\infty P\{Y = j | S = s\} f(s) ds = \int_0^\infty e^{-\lambda s} \frac{(\lambda s)^j}{j!} f(s) ds,$$

which can be solved by hand in simple cases, for instance when $S \sim \text{Exp}(\mu)$.⁷ When we cannot obtain a closed-form expression for the integral we need numerical methods. For this we can use the function `quad` of `scipy`.

⁷ [5.5.2].

```

1 import numpy as np
2 from scipy.integrate import quad
3
4 labda, mu = 2, 3
5 j = 5
6
7
8 def f(s):
9     return mu * np.exp(-mu * s)
10
11
12 def g(x):
13     return np.exp(-labda * x) * (labda * x) ** j / np.math.factorial(j) * f(x)
14
15
16 print(quad(g, 0, np.inf))

```

Ex 5.5.1. If $L(D_{k-1}) = 0$, why is $E[D_k - D_{k-1}] = E[X] + E[S]$?

Ex 5.5.2. If $S \sim \text{Exp}(\mu)$, show that

$$f(j) = P\{Y_k = j\} = \frac{\mu}{\lambda + \mu} \left(\frac{\lambda}{\lambda + \mu} \right)^j.$$

Ex 5.5.3. If $S \sim \text{Exp}(\mu)$, show that

$$G(j) = \sum_{k=j+1}^{\infty} f(k) = \left(\frac{\lambda}{\lambda + \mu} \right)^{j+1}.$$

Ex 5.5.4. Implement the recursion (5.5.1) in Python (or R) and test it for the $M/M/1$ queue.

The next exercise is a nice test of your algebra skills.⁸

Ex 5.5.5. Check that the queue length distribution $\{\pi(n)\}$ of the $M/M/1$ queue satisfies (5.5.1).

5.6 N -POLICIES FOR THE $M/G/1$ QUEUE

[2.2.6] motivates the study of the $M/G/1$ queue under an N -policy. We use the renewal-reward theorem to derive the time-average cost for this system by focusing on the average cost and duration of cycles that start and stop each time a job arrives at an empty system. As corollaries we obtain the EPQ formula of Section 4.3 and the PK formula of Section 5.4.

THE EXPECTED CYCLE duration is what we first need. For this, we consider the expected time $T(q)$ to clear the system when the server *starts working* on the first of q jobs present. We present three different ideas to obtain $T(q)$.

The first heuristic idea is this. We know that jobs arrive at a rate λ and are served at a rate $1/E[S] = \mu > \lambda$. Clearly, the net ‘drain rate’ of the queue is $\mu - \lambda$; hence we *guess* that⁰

$$T(q) = \frac{q}{\mu - \lambda} = \frac{E[S]}{1 - \lambda E[S]} q = \frac{E[S]}{1 - \rho} q. \quad (5.6.1)$$

Clearly, this reasoning completely ignores the stochasticity in arrival times and services: we cannot guarantee that the result is correct. For the second idea, consider a regular¹ $M/G/1$ queue. When a job arrives to an empty system, it takes a busy time² $E[U]$ to service this job and all the jobs that arrive during the service of this first job. In other words, it takes $E[U]$ time on average to move from 1 job in the system to 0 jobs, i.e., one job less. But then, when there are q jobs in the system, it must take $T(q) = q E[U]$ units of time to move from state q to state 0. By [4.3.7], $E[U] = E[S]/(1 - \rho)$, so we again obtain (5.6.1). The third idea is the most powerful. Write Y for the number of jobs that arrive during a service time. Then $T(q)$ satisfies the relation

$$T(q) = E[S] + E[T(q + Y - 1)], \quad q \geq 1, \quad (5.6.2)$$



⁸ This is a nice exercise to test your algebra skills.



⁰ By analogy: when you have a ‘queue’ of q km to cycle, and your speed is v km/h, then it takes q/v hours to complete the trip.

¹ That is, not controlled by an N -policy.

² Recall the explanation that led to (4.1.3).

because first the job in service must leave, and then, when $Y = k$, it takes $T(q + k - 1)$ to clear the system. To solve this equation, we substitute the guess $T(q) = aq + b$ and solve for a and b . It is clear³ that $T(0) = 0$, hence $b = 0$. Substituting aq into (5.6.2) gives $aq = E[S] + aq + aE[Y] - a$. Noting⁴ that $E[Y] = \lambda E[S]$, and solving for a we find (5.6.1). We note that the solution of (5.6.2) is unique once $T(0)$ is fixed.

For the expected cycle duration $C(N)$ as a function of N , observe that, right after the server switches off, we need N independent inter-arrival times to reach level N . As this takes N/λ in expectation, and then the system needs $T(N)$ to become empty again, $C(N) = N/\lambda + T(N)$.

FOR THE CYCLE COST, we proceed as follows. First, consider the expected cost $U(q)$ of waiting that accrues while one job is served and $q - 1$ jobs are in queue.⁵ When the service starts, the waiting cost for these q jobs is $hqE[S]$.⁶ Next, arrivals during the service of the job also contribute to the cost of waiting. The expected time these arrivals spend in the system equals $E\left[\int_0^S N(s) ds\right]$, where $N(s) \sim \text{Pois}(\lambda s)$. Conditional on $S = t$, $E\left[\int_0^S N(s) ds | S = t\right] = \int_0^t E[N(s)] ds = \int_0^t \lambda s ds = \lambda t^2/2$, hence $E\left[\int_0^S N(s) ds | S\right] = \lambda S^2/2$, hence $E\left[\int_0^S N(s) ds\right] = \lambda E[S^2/2]$. Combining both cost terms, the total cost of waiting during the service of the job is

$$U(q) = hqE[S] + \frac{1}{2}\lambda hE[S^2].$$

Let $V(q)$ be the expected queueing costs to clear the system just after a service starts and q jobs are in the system. By analogy with (5.6.2), $V(q)$ must be the solution of

$$V(q) = U(q) + E[V(q + Y - 1)], \quad q \geq 1. \quad (5.6.3)$$

As $U(q)$ has a term linear in q and a constant term, we substitute a quadratic form $V(q) = aq^2 + bq + c$ instead of a linear relation, assemble terms with the same power in q , and solve for a , b and c . Of course, $c = 0$ since $V(0) = 0$. For a and b we need some work to arrive at⁷

$$V(q) = \frac{h}{2} \frac{E[S]}{1-\rho} q^2 + h \frac{1+\rho C_s^2}{2} \frac{E[S]}{(1-\rho)^2} q.$$

For the cost during the build-up of the queue, we again use a recursive procedure. Write $W(q)$ for the accumulated queueing cost⁸ from the moment the server becomes idle up to the arrival time of the q th job (the job that sees $q - 1$ jobs in the system upon arrival). Then,⁹

$$W(q) = W(q-1) + h \frac{q-1}{\lambda} \implies W(q) = h \frac{q(q-1)}{2\lambda}.$$

BY COMBINING ALL the above cost factors and using the renewal reward theorem, we find for the long-run average cost¹⁰

$$\frac{W(N) + K + V(N)}{C(N)} = h \frac{1+C_s^2}{2} \frac{\rho^2}{1-\rho} + h\rho + h \frac{N-1}{2} + K \frac{\lambda(1-\rho)}{N}. \quad (5.6.4)$$

It is easy to retrieve the Pollaczek-Khinchine formula from this. The LHS in (5.6.4) is the average cost per unit time. Since in the $M/G/1$ queue,

³ If the system is empty, no time is needed to clear it.

⁴ Poisson arrivals $\implies$

$E[Y|S=s] = \lambda s \implies E[Y|S] = \lambda S$
 $\implies E[Y] = E[E[Y|S]] = \lambda E[S]$.

⁵ Thus, there are q jobs in the system.

⁶ That is, h times the area of the rectangle with base $E[S]$ and height q .

⁷ See [5.6.11]–[5.6.12].

⁸ Here W is not the waiting time in queue.

⁹ See [5.6.8].

¹⁰ [5.6.13].

each waiting customer pays $h = 1$ per unit time, the LHS corresponds to $E[L]$. For the RHS, in the $M/G/1$ queue, the setup cost $K = 0$, hence it is optimal to take $N = 1$. Consequently, just the first two terms of the RHS of (5.6.4) remain. The first term is $E[Q] = \lambda E[W]$, where $E[W]$ is given by the PK formula. The second is $\rho = E[L_s]$. But $E[L] = E[Q] + E[L_s]$, i.e., the LHS and RHS are equal.

A second interesting result is the EPQ formula for a production-inventory system with Poisson arrivals and no backlogging.¹¹ The optimal production quantity can be found by taking the derivative of (5.6.4) with respect to N , just as if it is a continuous variable. Setting the derivative to zero and solving for N results in, up to rounding,

$$N^* = \sqrt{\frac{2\lambda(1-\rho)K}{h}}.$$

TF 5.6.1. A machine can be turned on and off. If the queue length reaches N , the machine is turned on, and if the system becomes empty, the machine switches off. Let $I_k = 1$ if the machine is on in period k and $I_k = 0$ if it is off, let L_k be the number of jobs in the system at the end of period k . Claim: the next recursions model this queueing system.

$$I_{k+1} = \begin{cases} 1 & \text{if } L_k \geq N, \\ I_k & \text{if } 0 < L_k < N, \\ 0 & \text{if } L_k = 0, \end{cases}$$

$$I_{k+1} = \mathbb{1}_{L_k \geq N} + I_k \mathbb{1}_{0 < L_k < N},$$

$$d_k = \min\{L_{k-1}, c_k\},$$

$$L_k = L_{k-1} - (1 - I_k)d_k + a_k.$$

Assume that $I_0 = 0$ at time 0.

TF 5.6.2. We have an $M/G/1$ queue controlled by an N -policy, $\lambda = 1$ and $S_i \equiv 0.5$. Claim: the expected time to clear this queue after reaching N jobs is the same for a $D/D/1$ queue with the same arrival and service rates.

TF 5.6.3. Claim: For the $M/G/1$ queue under an N -policy, the expected time to clear the system starting from q jobs is $T(q) = qE[S]/(1-\rho)$.

TF 5.6.4. Claim: For the $M/G/1$ queue under an N -policy, the optimal N^* does not depend on the service time distribution, only on ρ .

TF 5.6.5. Claim: For the $M/G/1$ queue under an N -policy, setting $N = 1$ and $K = 0$ recovers the Pollaczek-Khinchine formula for $E[W]$.

Ex 5.6.6. Explain intuitively that the system is rate-stable for any N .

Ex 5.6.7. Why doesn't the utilization ρ depend on N ? 

Ex 5.6.8. Explain and solve the recursion for $W(q)$. 

Ex 5.6.9. Use the memoryless property for the $M/M/1$ queue to show that

$$T(q) = \frac{1}{\lambda + \mu} + \frac{\lambda}{\lambda + \mu} T(q+1) + \frac{\mu}{\lambda + \mu} T(q-1). \quad (5.6.5)$$

¹¹ This is more general than (4.3.1) because demand can arrive as a Poisson process. Although in its present form, backlogging is now not allowed, this is simple to include.

Ex 5.6.10. Show that for the $M/M/1$ queue, $V(q)$ satisfies a relation of the form:

$$V(q) = h \frac{q}{\lambda + \mu} + \frac{\lambda}{\lambda + \mu} V(q+1) + \frac{\mu}{\lambda + \mu} V(q-1). \quad (5.6.6)$$

Ex 5.6.11. Simplify $aq^2 + bq = aE[(q+Y-1)^2] + bE[q+Y-1] + U(q)$, where $U(q) = hqE[S] + \frac{1}{2}\lambda hE[S^2]$, by assembling powers in q to obtain:

$$a = \frac{h}{2} \frac{E[S]}{1 - E[Y]} = \frac{h}{2} \frac{E[S]}{1 - \rho},$$

$$b(1 - E[Y]) = a(E[Y^2] - 2E[Y] + 1) + \frac{1}{2}h\lambda E[S^2].$$



Opt 5.6.12. Derive the expression for $V(q)$ using the previous exercises.

Opt 5.6.13. Derive (5.6.4).

5.7 ANALYTIC RESULTS FOR THE BASE-STOCK POLICY

In this section, we establish a set of formulas to compute performance measures for the single-item base-stock inventory system of Section 2.5.⁰

WE INTRODUCE SOME notation: The period demands $\{D_i\}_{i=1}^{\infty}$ are assumed to be iid rvs¹; for notational ease we take $D_i = 0$ for $i \leq 0$. Write

$$D(i, j) = \sum_{k=i+1}^j D_k, \quad D[i, j] = D_i + D(i, j], \quad D[i, j] = D[i, j] - D_j,$$

and likewise for $Q(i, j]$ for the total outstanding reorders.

We assume that the lead time l is constant and a multiple of the period duration. Define $F_X(x) = P\{\sum_{i=1}^l D_i \leq x\}$; write $X \sim F_X$ for the lead-time demand, $G_X = 1 - F_X$ for the survivor function, and f_X for the pmf. Clearly, $E[X] = lE[D]$. Since the period demands are iid rvs, all rvs $\{D[k-l, k]\}_{k=l+1}^{\infty}$ have the same distribution F_X .

WE USE THE recursions of Section 2.5 to derive expressions for the long-run performance measures. The first step is to relate the outstanding orders to the demand.

Lemma 5.7.1. Under a base-stock policy with order-up-to level S , $Q(1, k) = D[1, k]$ for all periods $k \geq 1$.

Proof. Using the recursions for the inventory position under the base-stock policy repeatedly,

$$\begin{aligned} P_k &= P'_{k-1} + Q_k = P_{k-1} - D_{k-1} + Q_k \\ &= P_{k-1} - D[k-1, k] + Q(k-1, k] \\ &= P_1 - D[1, k] + Q(1, k]. \end{aligned}$$

The proof is finished by noting that $P_k = S$ for all $k \geq 1$. □

We can use this lemma to express I_k in terms of S and the lead-time demand.

⁰ This way, we do not have to simulate to see how performance depends on the order-up-to level S .

¹ Why did we not need to make this assumption earlier?

Lemma 5.7.2. Under a base-stock policy, if $I_1 = S$ and there are no outstanding orders of period $k \leq 1$, then for all periods $k \geq 1$,

$$I_k = S - D[k-l, k], \quad I'_k = I_k - D_k = S - D[k-l, k].$$

Proof. Using the recursion for the inventory level and the assumptions,

$$\begin{aligned} I_k &= I_{k-1} - D_{k-1} + Q_{k-l} \\ &= I_{k-1} - D[k-1, k] + Q(k-l-1, k-l] \\ &= I_1 - D[1, k] + Q(1-l, k-l] \\ &\stackrel{1}{=} S - D[1, k] + Q(1, k-l] \\ &\stackrel{2}{=} S - D[k-l, k]; \end{aligned}$$

1 uses the assumptions, 2 follows from the above lemma and $D[k-l, k] = D[1, k] - D[1, k-l]$. $\square$

This next theorem, which allows us to define the inventory level as an rv I , follows directly from the above lemmas.

Theorem 5.7.3. The rvs $\{I_k\}_{k=l+1}^{\infty}$ are identically² distributed as the rv

$$I = S - X.$$

² But not necessarily independent.

Proof. As observed earlier, each rv $D[k-l, k] \sim X$. As S is constant, the rvs $I_k = S - D[k-l, k] \sim S - X$ for all $k \geq l+1$. $\square$

NOW THAT WE have characterized the distribution of I in terms of the control parameter S and the lead-time demand X , we can find expressions for the performance measures. First, for the ready rate,

$$\alpha \stackrel{1}{=} P\{I' \geq 0\} \stackrel{2}{=} P\{I - D \geq 0\} \stackrel{3}{=} P\{S - X - D \geq 0\} = P\{X + D \leq S\}, \quad (5.7.1)$$

where 1 follows from the definition of α , 2 from $I' = I - D$, 3 from $I = S - X$. Second, for the fill rate, by taking expectations in (2.5.4), we obtain

$$\beta = \frac{E[\min\{D, I^+\}]}{E[D]}. \quad (5.7.2)$$

For the third performance measure, the cycle service level, I have not yet tried to find a simple expression in terms of expectations.

The mean inventory level is

$$E[I] = E[S - X] = S - E[X] = S - lE[D].$$

Next, noting that $I^+ - I^- = I$, the average on-hand inventory can be expressed in terms of the average backlog:

$$E[I^+] = E[I] + E[I^-] = S - lE[D] + E[I^-]. \quad (5.7.3)$$

From the definition of the backlog,

$$E[I^-] = E[[-I]^+] = E[(X - S)^+] = \sum_{k=0}^{\infty} (k - S)^+ f(k). \quad (5.7.4)$$

This summation runs to ∞ if the support of D is unbounded. For numerical purposes, we rewrite it as follows.

Lemma 5.7.4. The average backlog is equal to

$$\mathbb{E}[I^-] = l\mathbb{E}[D] - \sum_{j=0}^{S-1} G_X(j).$$

Proof. Since $\sum_{j=0}^{\infty} \mathbb{1}_{j < k} = k$, we find from (5.7.4)

$$\begin{aligned} \sum_{k=S}^{\infty} (k-S)f(k) &= \sum_{k=S}^{\infty} \sum_{j=0}^{\infty} \mathbb{1}_{j < k-S} f(k) = \sum_{j=0}^{\infty} \sum_{k=S}^{\infty} \mathbb{1}_{k > j+S} f(k) \\ &= \sum_{j=0}^{\infty} G_X(j+S) = \sum_{j=0}^{\infty} G_X(j) - \sum_{j=0}^{S-1} G_X(j). \end{aligned}$$

Since $\sum_{i=0}^{\infty} G_X(i) = l\mathbb{E}[D]$, the claim follows. $\square$

Recall from Th. 5.7.3 that I , hence I^+ and I^- , are functions of the reorder level S . Therefore, the average inventory cost $c(S) = h\mathbb{E}[I^+] + b\mathbb{E}[I^-]$ is also a function of S .

HERE WE ILLUSTRATE the Lighthouse Company example of Section 2.5. With the rv class from Section 1.4, our code stays nearly in one-to-one correspondence with the mathematical definitions above.

```

1  from functools import cache
2  import numpy as np
3
4  from random_variable import NumericRV as RV
5  from functions import Plus, Min
6
7  from lighthouse_case import D, K, N, leadtime, h, b

```

The base-stock policy follows directly once we note that it suffices to specify the rv X ; the other rvs then follow from X . Note that the performance measures depend on the control parameter S .

```

1  class Basestock:
2      """Implements KPIs for a base-stock inventory system."""
3
4      def __init__(self, D, leadtime, h, b):
5          self.D = D
6          self.leadtime = leadtime
7          self.h = h
8          self.b = b
9          self.X = sum(self.D for _ in range(leadtime)) if leadtime > 0 else RV({0: 1})
10
11     @cache
12     def IP(self, S):
13         return S
14
15     @cache
16     def IL(self, S):
17         return self.IP(S) - self.X
18
19     @cache
20     def Imin(self, S):
21         return self.IL(S).map(lambda x: Min(x))

```

```

22
23 @cache
24 def Iplus(self, S):
25     return self.IL(S).map(lambda x: Plus(x))
26
27 def cost(self, S):
28     return self.b * self.Imin(S).mean() + self.h * self.Iplus(S).mean()
29
30 def alpha(self, S):
31     return (self.IL(S) - self.D).sf(-1)
32
33 def beta(self, S):
34     m = RV.map2(self.D, self.Iplus(S), lambda x, y: min(x, y))
35     return m.mean() / self.D.mean()

```

Here are some tests to verify the implementation and the preceding algebra.

```

_____ basestock.py _____
1 S = 5
2
3 base = Basestock(D, leadtime, h, b)
4
5 theta = leadtime * D.mean()
6 print(np.isclose(base.IL(S).mean(), S - theta))
7 print(
8     np.isclose(
9         base.Imin(S).mean(), theta - sum(base.X.sf(j) for j in range(0, S))
10    )
11 )
12 print(np.isclose(base.alpha(S), (base.X + D).cdf(S)))

```

```

1 True
2 True
3 True

```

The following shows how to use the code. We see that, as a function of S , the service levels increase, but the cost first decreases and then increases.³

³ High service levels do not come for free.

```

_____ basestock.py _____
1 for S in range(5, 12):
2     print(f"{S=:} {base.alpha(S)=:.2f}, {base.beta(S)=:.2f}, {base.cost(S)=:.2f}")

```

```

1 S=5: base.alpha(S)=0.34, base.beta(S)=0.33, base.cost(S)=15.84
2 S=6: base.alpha(S)=0.47, base.beta(S)=0.47, base.cost(S)=9.42
3 S=7: base.alpha(S)=0.60, base.beta(S)=0.61, base.cost(S)=5.65
4 S=8: base.alpha(S)=0.72, base.beta(S)=0.74, base.cost(S)=4.08
5 S=9: base.alpha(S)=0.82, base.beta(S)=0.84, base.cost(S)=3.54
6 S=10: base.alpha(S)=0.89, base.beta(S)=0.91, base.cost(S)=3.63
7 S=11: base.alpha(S)=0.94, base.beta(S)=0.95, base.cost(S)=4.30

```

TF 5.7.1. In a base-stock model, the ready rate is given by $\alpha = P\{I - D \geq 0\}$. Claim: The next code implements this correctly.

```

1  class Basestock:
2      def __init__(self, D, L, h, b):
3          self.D = D
4          self.L = L
5          self.h = h
6          self.b = b
7          self.X = sum(self.D for _ in range(self.L))
8
9      def IL(self, S):
10         return S - self.X
11
12     def alpha(self, S):
13         return (self.IL(S) - self.D).sf(0)

```

TF 5.7.2. Claim: In a base-stock model with lead-time demand X , the backlog is given by $E[S - X]^+$.

TF 5.7.3. Claim: Increasing the order-up-to level S always decreases the average inventory cost $c(S) = hE[I^+] + bE[I^-]$.

TF 5.7.4. Claim: For the base-stock model, $E[I] = S - lE[D]$ can be negative.

TF 5.7.5. Claim: Under a base-stock policy, $I^+ - I^- = I$.

Ex 5.7.6. Suppose $l = 0$. What values do X , $E[I]$, $E[I^+]$ and $E[I^-]$ take?

Ex 5.7.7. Why does the definition of β follow from (2.5.4)?

The cost function $S \mapsto c(S)$ is convex and coercive.⁴

Ex 5.7.8. Prove that there exists an order-up-to level S^* that minimizes the average cost.

⁴ A function f is coercive if $\lim_{x \rightarrow \pm\infty} f(x) = \infty$.



5.8 ANALYTIC RESULTS FOR THE (Q, r) POLICY

In this section, we extend the analysis of a single-item inventory system under a base-stock policy to a (Q, r) inventory system.

THERE IS A convenient way to relate the (Q, r) system to a number of base-stock systems. To see this, imagine two animals, a squirrel that observes the (Q, r) system at the end of each demand period, and a bear that only wakes up when the inventory position right at the start of the interval equals some fixed i , where i must lie in the set $\{r+1, \dots, r+Q\}$; if not, the bear hibernates. Clearly, from the bear's point of view, the system behaves as a base-stock system with order-up-to level i , because whenever the bear wakes, the inventory position is i , and the position just before the bear falls asleep is $i - D$. More generally, we can associate a different bear with each position in the set $\{r+1, \dots, r+Q\}$, so exactly one bear is awake at any time. To combine the statistics observed by the bears and the statistics observed by the squirrel, we need, for each i , the long-run fraction of periods in which the inventory position P equals i . For instance, if the squirrel knows⁰

$$p(i) = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{k=1}^T \mathbb{1}_{P_k=i},$$

⁰ We assume that the limit over all sample paths $\{P_k\}$ exists almost surely, so that we can invoke the strong law of large numbers.

for all $i \in \{r+1, \dots, r+Q\}$, then it can retrieve the ready rate $\sum_{i=r+1}^{r+Q} p(i)\alpha(i)$ from the ready rate $\alpha(i)$ observed by the bear with order-up-to level i .¹

Theorem 5.8.1. Under the (Q, r) policy, $p(i) = P\{P = i\} = 1/Q$ for $i = r+1, \dots, r+Q$, in the long-run time limit. Consequently, I is distributed as $P - X$.

Proof. For simplicity take $Q = 3, r = 0$, so that the inventory position P cycles between the levels 1, 2 and 3.² We apply level-crossing arguments, as follows. Write $f_i = P\{D \in \{i, i+3, i+6, \dots\}\}$ for the probability that the period demand equals a multiple of 3 plus i , $i = 0, 1, 2$. Suppose that, at the start of period k , the position $P_k = 3$. Since we order in multiples of $Q = 3$, the probability $P\{P_{k+1} = 3 | P_k = 3\} = f_0$, i.e., equal to the probability that D_k is a multiple of 3. Likewise, $P\{P_{k+1} = 2 | P_k = 3\} = f_1$, and $P\{P_{k+1} = 1 | P_k = 3\} = f_2$. Next, since we again order in multiples of Q , $P\{P_{k+1} = 1 | P_k = 2\} = f_1$, and similarly for the other possibilities. Therefore, we can use level crossing to conclude that in the long run, $(f_1 + f_2)p(3) = f_1p(2) + f_2p(1)$. However, because of the (Q, r) ordering rule, we can cyclically change $p(1), p(2)$ and $p(3)$ in this equality. By symmetry, the only possible solution is $p(i) = 1/3$ for all i .

For the second claim, condition on the inventory position. When $P = i$, the system behaves as a base-stock system with order-up-to level i , so by Th. 5.7.3, the conditional rv $I | P = i \sim i - X$. Since P and X are independent (the position depends on past demands, while X is the lead-time demand over future periods), it follows that $I \sim P - X$. $\square$

WITH THE ABOVE theorem the ready rate for the (Q, r) model can be computed with the formula

$$\alpha = \frac{1}{Q} \sum_{i=r+1}^{r+Q} \alpha(i) = \frac{1}{Q} \sum_{i=r+1}^{r+Q} P\{X + D \leq i\}, \quad (5.8.1)$$

where $\alpha(i)$ is the ready rate of the base-stock model with reorder level i .

For the average inventory level, we write $I(i)$ for the inventory level of the base-stock model with order-up-to level i , and get

$$E[I] = \frac{1}{Q} \sum_{i=r+1}^{r+Q} E[I(i)] = \frac{1}{Q} \sum_{i=r+1}^{r+Q} (i - E[X]) = \frac{Q+1}{2} + r - E[X].$$

The expected number of backorders $E[I^-]$ and expected on-hand inventory $E[I^+]$ can be found by equivalent summations:

$$E[I^+] = \frac{1}{Q} \sum_{i=r+1}^{r+Q} E[I^+(i)], \quad E[I^-] = \frac{1}{Q} \sum_{i=r+1}^{r+Q} E[I^-(i)],$$

using (5.7.3) and (5.7.4) of the base-stock model.

Finally, with h and b the holding and backorder cost per item per period, the total average cost becomes

$$c(Q, r) = K \frac{E[D]}{Q} + hE[I^+] + bE[I^-].$$

With regard to the first term, note that $E[D]$ is the average demand per period, so $E[D]/Q$ is the order frequency.

¹ We label the ready rate of the base-stock model by the order-up-to level.

² The proof is the same for general Q and r , but needs a bit more notation.

It remains to find optimal values for r and Q . Fast algorithms exist to compute optimal r and Q , but we omit this. A simple workaround is to perform a complete grid search over a reasonable set of pairs of r and Q . Another simple heuristic is to use the EOQ³ formula to set $Q = \sqrt{2E[D]K/h}$ and then search for r such that the total average cost is minimal.⁴

With similar methods we can search for an r such that the α or β service level criteria are met.

TO CODE THE above formulas we use the RV class again so that our implementation can stay on a high level. Note in particular that the code relies on Th. 5.8.1.

³ See [5.8.7], and set $d = E[D]$.

⁴ For that value of Q !

```

_____ qr.py _____
1  from functools import cache
2
3  from random_variable import NumericRV as RV
4  from functions import Plus, Min
5
6  from lighthouse_case import D, K, N, leadtime, h, b
7
8
9  class Qr:
10     def __init__(self, D, leadtime, h, b, K=0):
11         self.D = D
12         self.leadtime = leadtime
13         self.h = h
14         self.b = b
15         self.K = K
16         self.X = sum(self.D for _ in range(leadtime)) if leadtime > 0 else RV({0: 1})
17
18     @cache
19     def IP(self, Q, r):
20         return RV({i: 1 / Q for i in range(r + 1, r + Q + 1)})
21
22     @cache
23     def IL(self, Q, r):
24         return self.IP(Q, r) - self.X
25
26     @cache
27     def Imin(self, Q, r):
28         return self.IL(Q, r).map(lambda x: Min(x))
29
30     @cache
31     def Iplus(self, Q, r):
32         return self.IL(Q, r).map(lambda x: Plus(x))
33
34     def order_freq(self, Q):
35         return self.D.mean() / Q
36
37     def cost(self, Q, r):
38         c = self.K * self.order_freq(Q)
39         c += self.b * self.Imin(Q, r).mean()
40         c += self.h * self.Iplus(Q, r).mean()
41         return c
42
43     def alpha(self, Q, r):
44         return (self.IL(Q, r) - self.D).sf(-1)
45
46     def beta(self, Q, r):
47         m = RV.map2(self.D, self.Iplus(Q, r), lambda x, y: min(x, y))

```

```
48         return m.mean() / self.D.mean()
```

Here is an example of how to use this class.

```

qr.py
1  qr = Qr(D, leadtime, h, b, K)
2  Q, r = 3, 10
3  print(f"{qr.IL(Q, r).mean()}")
4  print(f"{(Q + 1) / 2 + r - qr.X.mean()}")

1  qr.IL(Q, r).mean()=7.450000000000001
2  (Q + 1) / 2 + r - qr.X.mean()=7.45

```

TF 5.8.1. Claim: The (Q, r) inventory policy can be analyzed by comparing it with multiple base-stock systems.

TF 5.8.2. Claim: In a (Q, r) policy system, the inventory position P at the beginning of an interval must always be equal to the reorder level r .

TF 5.8.3. Consider the following Python code.

```

1  class Qr:
2      def __init__(self, D, leadtime, h, b):
3          self.D = D
4          self.leadtime = leadtime
5          self.h = h
6          self.b = b
7          self.X = sum(self.D for _ in range(self.leadtime))
8
9  qr_instance = Qr(D=2, leadtime=3, h=5, b=1)

```

Claim: The value of `qr_instance.X` will be 6.

TF 5.8.4. The fill rate for the (Q, r) inventory system is defined as

$$\beta = E[\min\{D, I^+\}] / E[D].$$

Claim: The following Python code implements this correctly:

```

1  def beta(self, Q, r):
2      m = RV.map2(self.D, self.Iplus(Q, r), lambda x, y: min(x, y))
3      return m.mean() / self.D.mean()

```

TF 5.8.5. Claim: The order frequency of the (Q, r) policy is equal to $Q/E[D]$.

TF 5.8.6. Claim: this Python code computes the average cost under a (Q, r) policy.

```

1  C = K * Q / D.mean()
2  C += b * Imin(Q, r).mean() + h * Iplus(Q, r).mean()

```

Ex 5.8.7. Perhaps the most important formula in practical inventory management is the Economic Order Quantity (EOQ) that determines the optimal order quantity in case demand is constant, i.e., $D \equiv d$, the lead time $l = 0$ and backlogging is not allowed. Use the (Q, r) model to show that it is optimal to take $Q^2 = 2dK/h$.



5.9 AN ALGORITHM FOR THE (s, S) AND (T, S) POLICIES

We develop an algorithm by which we can compute the long-run average cost for a single-item inventory system under periodic review and controlled by the (s, S) or the (T, S) policy. This algorithm finds its importance in its underlying ideas: hitting (or stopping) problems, the renewal reward theorem, and dynamic programming. We start with the (T, S) policy, then move on to the (s, S) policy, and then extend to a combination of the other two, the (T, s, S) policy, to be introduced presently.

RECALL THAT UNDER the (T, S) policy, we place a replenishment order every T periods such that the inventory position P equals S after the replenishment. When the inventory level I equals x , the inventory cost is given by

$$c(x) = bx^- + hx^+.$$

By conditioning on $P = x$, and using⁰ that $I = P - X$, it follows that

$$L(x) = E[c(I)|P = x] = E[c(x - X)] = \sum_{k=0}^{\infty} f_k c(x - k) \quad (5.9.1)$$

⁰ See Th. 5.7.3.

is the expected inventory cost per period when $P = x$.

Let us write $V(n, x)$ for the expected cost until reordering if we start at position $P = x$ and there are n , $0 \leq n \leq T$, periods remaining. By reasoning analogously as in Section 5.6, $V(n, x)$ satisfies the recursion

$$V(n, x) = \begin{cases} L(x) + E[V(n-1, x-D)], & n > 0, \\ K, & n = 0. \end{cases}$$

To see this, observe that when $P = x$ and $n > 0$, we first pay $L(x)$ for keeping inventory during one period, then a demand D arrives, and for the next period we have one less period to go until ordering. When $n = 0$, we pay the order cost K to replenish to S . Note that this recursion always terminates because $V(n, \cdot)$ is expressed in terms of $V(n-1, \cdot)$, so always one period less to go.

Since a (renewal) cycle starts at level S , the expected cycle cost is $V(T, S)$ and the expected cycle duration is T periods. The renewal reward theorem implies that the long-run average cost is given by $V(T, S)/T$.

To facilitate the development of the other models, we introduce some important definitions¹ that are used in the above recursion: $L(x)$ is called the *running cost*, K the *hitting cost*, T is the *hitting time*, and $\{n : n = 0\}$ is the *hitting set*.²

¹ Of dynamic programming.

² We use the adjectives *hitting* and *stopping* interchangeably.

WITH THIS TERMINOLOGY we can easily structure the analysis of the (s, S) policy. In view of (2.5.3), when the inventory position P enters (or hits) the stopping set $\{i : i \leq s\}$ we place an order and pay a stopping cost K . When $P > s$, we pay the running cost $L(x)$ and continue for another period. Hence, the expected cost until stopping satisfies the recursion

$$V(s, x) = \begin{cases} L(x) + E[V(s, x-D)], & x > s, \\ K, & x \leq s. \end{cases}$$

For the hitting time we find

$$T(s, x) = \begin{cases} 1 + E[T(s, x - D)], & x > s, \\ 0, & x \leq s, \end{cases}$$

because the running cost is 1 if we continue for another period, and when the process hits the stopping set, the time stops counting, so the stopping cost is 0. Note that this recursion is *structurally identical* to that of V .

To find an algorithm, we expand the recursion for V at $x > s$ as

$$V(s, x) = E[V(s, x - D)] = \sum_{k=0}^{\infty} f_k V(s, x - k) = f_0 V(s, x) + f_1 V(s, x - 1) + \cdots.$$

Clearly, $V(s, x)$ appears at the LHS and RHS, so $V(s, x)$ is expressed in terms of itself, that is, circular.³ The way out is to isolate $V(s, x)$ by rewriting this as

$$(1 - f_0)V(s, x) = f_1 V(s, x - 1) + f_2 V(s, x - 2) + \cdots. \quad (5.9.2)$$

Thus, assuming that⁴ $f_0 < 1$,

$$V(s, x) = (1 - f_0)^{-1} \sum_{k=1}^{\infty} f_k V(s, x - k). \quad (5.9.3)$$

We can now use one algorithm to compute V and T by adjusting the running and stopping cost accordingly.

Finally, when the expected cycle cost $V(s, S)$ and expected hitting time $T(s, S)$ have been computed, the long-run average cost is $V(s, S)/T(s, S)$.

BY COMBINING THE two policies above we obtain the (T, s, S) policy. The recursion for $V(n, s, x)$ specifies the behavior:

$$V(n, s, x) = \begin{cases} L(x) + E[V(n - 1, s, x - D)], & x > s, \\ K, & x \leq s \text{ or } n = 0. \end{cases} \quad (5.9.4)$$

The structure of the recursion for $T(n, s, x)$ is the same, but with running cost 1 and stopping cost 0.

To see how the (s, S) and (T, s, S) policy relate, realize that when T becomes very large, the set $set i : i \leq s$ will be hit with overwhelming probability long before the time-to-go has become 0. Therefore, the average cost under the (s, S) policy must be $V(s, S)/T(s, S) = \lim_{n \rightarrow \infty} V(n, s, S)/T(n, s, S)$. By similar reasoning, $V(T, S)/T = \lim_{s \rightarrow -\infty} V(T, s, S)/T(T, s, S)$.

IT IS INTERESTING to understand why we can obtain the closed-form expression $T(q) = q/(\mu - \lambda)$ in (5.6.1), while for the (s, S) and (T, s, S) policies we need an algorithm. The difference is that in the earlier situation⁵ $T(q) = 1 + E[T(q + Y - 1)]$ while in the case of inventory control, the inventory position can jump down by more than one unit when $P\{D \geq 2\} > 0$. Thus, the behavior of v and T just above s is more complicated. Moreover, in the queueing situation, the queue length can never be negative, while in the inventory situation the level I can take negative values. As a consequence, the cost function $L(x)$ is not linear⁶, hence the idea to substitute $T(q) = \alpha q + \beta$ and solve for α and β will not work in general.

³ This is different from the recursion for the (T, S) policy.

⁴ This is not restrictive, because $P\{D = 0\} = 1$ implies that there is no demand, in which case there is no need to keep inventory.

⁵ So, possibly big steps upward, but only single steps downward.

⁶ But still convex.

Next, we note that deriving a recursive algorithm for the cost under the (Q, r) policy seems not so attractive. To see why, observe that the stopping cost is given by $V(r, x) = V(r, x + Q)$, that is, it is no longer constant such as K , but dependent on other values of v . Therefore, the simplifying trick that led to (5.9.2) cannot be applied here.

WE NOW IMPLEMENT the algorithm for the (T, s, S) and (s, S) policies in Python. By comparing the recursions, we see that both policies have the running cost and stopping cost for the expected cost and time until the stopping set is hit. However, the two policies differ in their stopping sets and the details of the cost computations. We therefore put all common functionality in a base class and use a `hitting_value` method in the subclasses `TsSPolicy` and `sSPolicy` to cope with the differences.

For ease of understanding, we first discuss `TsSPolicy`; assume that the running cost and stopping cost are already provided by the super class. The `is_stop` function implements the stopping criterion.

```

TsS.py
1  from functools import cache
2
3  from inventory_base import InventoryModelPrimitives
4
5
6  class TsSPolicy(InventoryModelPrimitives):
7      def hitting_value(self, *, running_cost, stopping_cost):
8          is_stop = lambda n, s, x: x <= s or n == 0
9
10         @cache
11         def v(T, s, S):
12             if is_stop(T, s, S):
13                 return stopping_cost(S)
14             return running_cost(S) + self.D.E(lambda i: v(T - 1, s, S - i))
15
16         return v
17
18
19  class TSPolicy(TsSPolicy):
20      "Long run average cost for a TS inventory system."
21
22      def average_cost(self, T, S):
23          return super().average_cost(T, -float('inf'), S)

```

In this code, the method `hitting_value` returns a *function*⁷ v . This function, when applied to the arguments T, s, S , returns the number $v(T, s, S)$. You should solve [5.9.6] to understand how this idea works in detail.

⁷ Pay attention here: the returned object is a function, not a number.

Similarly, for the (s, S) policy we implement the recursive scheme of (5.9.3).

```

ss.py
1  from functools import cache
2
3  from inventory_base import InventoryModelPrimitives
4
5
6  class sSPolicy(InventoryModelPrimitives):
7      def hitting_value(self, *, running_cost, stopping_cost):

```

```

8         is_stop = lambda s, x: x <= s
9
10        @cache
11        def v(s, S):
12            if is_stop(s, S):
13                return stopping_cost(S)
14            res = running_cost(S)
15            res += self.D.E(lambda i: 0.0 if i == 0 else v(s, S - i))
16            return res / (1.0 - self.D.pmf(0))
17
18        return v

```

Here is the super class. Once again, solve [5.9.6] to understand why we put the arguments $(*args, **kwargs)$ at the end of the methods.⁸

⁸ Consult an LLM on the meaning of $*args$ and $**kwargs$.

```

_____ inventory_base.py _____
1  from abc import ABC, abstractmethod
2  from functools import cache
3
4  import numpy as np
5  from functions import Min, Plus
6  from random_variable import NumericRV as RV
7
8
9  class InventoryModelPrimitives:
10      "Common functionality for sS and TsS inventory control systems."
11
12      def __init__(self, D, leadtime, h, b, K):
13          if D.pmf(0) == 1:
14              raise ValueError("Invalid D: pmf(0) == 1")
15          self.D = D
16          self.leadtime = leadtime
17          self.h = h
18          self.b = b
19          self.K = K
20          self.X = sum(self.D for _ in range(leadtime)) if leadtime > 0 else RV({0: 1})
21
22      def c(self, x):
23          return self.b * Min(x) + self.h * Plus(x)
24
25      @cache
26      def L(self, x):
27          return self.X.E(lambda i: self.c(x - i))
28
29      @abstractmethod
30      def hitting_value(self, *, running_cost, stopping_cost):
31          raise NotImplementedError
32
33      @cache
34      def hitting_cost(self, *args, **kwargs):
35          return self.hitting_value(
36              running_cost=lambda x: self.L(x),
37              stopping_cost=lambda x: self.K,
38          )(*args, **kwargs)
39
40      @cache
41      def hitting_time(self, *args, **kwargs):
42          return self.hitting_value(
43              running_cost=lambda x: 1.0,
44              stopping_cost=lambda x: 0.0,
45          )(*args, **kwargs)

```

```

46
47     def average_cost(self, *args, **kwargs):
48         return self.hitting_cost(*args, **kwargs) / self.hitting_time(
49             *args, **kwargs
50     )

```

LET US EXAMINE a few numerical examples.

```

_____ sS_tss_experiments.py _____
1  from lighthouse_case import D, K, b, h, leadtime
2  from TsS import TSPolicy, TsSPolicy
3  from ss import sSPolicy
4
5
6  sS = sSPolicy(D, leadtime, h, b, K)
7
8  s, S = 16, 20
9  print("sS policy")
10 print(f"hitting time: {sS.hitting_time(s, S):.2f}")
11 print(f"hitting cost: {sS.hitting_cost(s, S):.2f}")
12 print(f"average cost: {sS.average_cost(s, S):.2f}")

```

```

1  sS policy
2  hitting time: 2.2865088
3  hitting cost: 72.04810303999999
4  average cost: 31.51009217196102

```

We can check that for T large, the (T, s, S) policy and the (s, S) have the same average cost.

```

_____ sS_tss_experiments.py _____
1  TsS = TsSPolicy(D, leadtime, h, b, K)
2
3  print("TsS policy")
4  print(f"average cost: {TsS.average_cost(T=1, s=s, S=S):.2f}")
5  print(f"average cost: {TsS.average_cost(T=2, s=s, S=S):.2f}")
6  print(f"average cost: {TsS.average_cost(T=10, s=s, S=S):.2f}")

```

```

1  TsS policy
2  average cost: 60.3
3  average cost: 38.59665071770335
4  average cost: 31.51019163236799

```

For the (T, S) policy, setting T to a large value is costly, as expected.

```

_____ sS_tss_experiments.py _____
1  TS = TSPolicy(D, leadtime, h, b, K)
2
3  print("TS policy")
4  print(f"average cost: {TS.average_cost(T=1, S=S):.2f}")
5  print(f"average cost: {TS.average_cost(T=2, S=S):.2f}")
6  print(f"average cost: {TS.average_cost(T=10, S=S):.2f}")
7  print(f"average cost: {TS.average_cost(T=20, S=S):.2f}")

```

```

1 TS policy
2 average cost: 60.3
3 average cost: 34.541666666666664
4 average cost: 35.8361856753891
5 average cost: 193.77674738939885

```

TF 5.9.1. Claim: For the (s, S) policy, the expected cost until stopping $V(s, x)$ satisfies the same recursion as the one for the (T, S) policy.

TF 5.9.2. Claim: The next formula is a correct implementation to compute the expected time to hit s under the (s, S) policy:

$$\begin{cases} (1 - f_0)^{-1}(1 + E[T(s, x - D)]), & x > s, \\ 0, & x \leq s, \end{cases}$$

TF 5.9.3. Claim: The next code can be used to compute the expected cost until stopping for the (s, S) and the (T, S) policy:

```

1 def hitting_cost(self, *args, **kwargs):
2     return self.hitting_value(
3         running_cost=lambda x: self.L(x),
4         stopping_cost=lambda x: self.K,
5     )(*args, **kwargs)

```

TF 5.9.4. Claim: This code correctly computes the expected cost until stopping for the (s, S) policy:

```

1 def hitting_value(self, *, running_cost, stopping_cost):
2     is_stop = lambda s, x: x <= s
3
4     @cache
5     def v(s, S):
6         if is_stop(s, S):
7             return stopping_cost(S)
8         res = running_cost(S)
9         res += self.D.E(lambda i: 0.0 if i == 0 else v(s, S - i))
10        return res
11
12    return v

```

TF 5.9.5. Claim: When $P = x$ and the leadtime $l > 0$, the inventory cost under the (T, s, S) policy is given by

$$c(x) = bx^- + hx^+.$$

Ex 5.9.6. Consider the following functions to illustrate how functions like `hitting_time` and `hitting_cost` can be understood.

```

1 def exp_apply(exponent):
2     def apply(base):
3         return base ** exponent
4     return apply
5
6 lion = exp_apply(2)
7 tiger = exp_apply(3)

```

Without running the code, what do you think will be printed by:

```

1 print(exp_apply(1)(9))
2 print(lion(5))
3 print(tiger(4))

```



5.10 HITTING PROBLEMS WITH APPLICATIONS IN SINGLE-ITEM INVENTORY CONTROL

In a single-item inventory under period review and controlled by an (s, S) policy, every time the inventory position becomes s or lower, a replenishment up to state S is placed at cost K .⁰ When the position is at $x > s$, the inventory cost is $L(x)$, see (5.9.1). Typically, we are interested in the long-run average cost when the position starts at S and stops when getting at or below s . By the renewal reward theorem, it suffices to compute the expected cost $\nu(s, S)$ and expected time $T(s, S)$ of one cycle that starts at S and stops at $x \leq s$. The long-run average cost is then given by $\nu(s, S)/T(s, S)$.

Both $\nu(s, S)$ and $T(s, S)$ can be found as the solution of a hitting problem. The stopping set is $x \leq s$. When in state $x > s$, the continuation (or running) cost for ν is, as said, $L(x)$, and 1 for T , because it costs 1 period to stay 1 period in state x . The stopping cost for ν is the ordering cost K , and it is 0 for T , because once stopped, time no longer progresses.

Below we discuss a general algorithm to numerically solve such hitting problems.

A BIT OF theory helps to set up this algorithm. We start with a discrete Markov chain $X = \{X_k, k \geq 0\}$ with a reasonably small state space, e.g., at most several thousand states.¹ The state space is split into two disjoint sets C and D . When $X_k \in C$, the process continues, while if $X_k \in D$, it stops. Define the stopping time $\tau = \inf\{k : X_k \in D\}$, and assume that the transition probabilities $P(x, y)$ of X are such that the chain gets absorbed in D in finite expected time, that is, $E_x[\tau] < \infty$ for all possible initial states x . We write $\tilde{L}(x) = L(x) \mathbb{1}_{x \in C} + K \mathbb{1}_{x \in D}$ for the cost associated with spending a period or stopping in state x . The interesting question is to ask: What is the expected cost until the process stops?²

Focus first on some state y , then, when starting in x , $E_x \left[\sum_{t=0}^{\tau} \mathbb{1}_{X_t=y} \tilde{L}(y) \right]$ is the expected cost for spending time in, or hitting, state y . With this, we write the total cost until stopping as

$$\nu(x) = \sum_y E_x \left[\sum_{t=0}^{\tau} \mathbb{1}_{X_t=y} \tilde{L}(y) \right].$$

By splitting the expectation and taking the summation over t out of the expectation we obtain a very useful expression for ν :

$$\nu(x) = \sum_y g(x, y) \tilde{L}(y),$$

where g , known as Green's kernel, is given by

$$g(x, y) = \sum_{t=0}^{\infty} E_x \left[\mathbb{1}_{X_t=y, t \leq \tau} \right].$$

⁰ If you don't like challenges, you can skip this section.

¹ Otherwise we should resort to other numerical methods.

² For suitable conditions on c , see for instance, Dynkin and Yushkevich 1968.

It represents the expected number of visits to y until absorption when the chain starts at x . In terms of probabilities, g can be written as³

$$g(x, y) = \sum_{t=0}^{\infty} P_x \{X_t = y, t \leq \tau\}. \quad (5.10.1)$$

Finally, if X can start at some state x with probability μ_x , the expected number of visits to y becomes

$$g(y) = \sum_x \mu_x g(x, y). \quad (5.10.2)$$

THE FOLLOWING SEEMINGLY simple, and certainly short, algorithm computes g as defined in (5.10.2).

```

1  from collections import defaultdict
2
3
4  def green(mu, P, is_stop, eps):
5      g = defaultdict(float)
6      while mu:
7          for x in list(mu.keys()):
8              g[x] += (p := mu[x])
9
10             mu[x] = 0
11             if is_stop(x):
12                 continue
13
14             for y, pxy in P(x):
15                 mu[y] += p * pxy
16             mu = defaultdict(float, {x: p for x, p in mu.items() if p > eps})
17         return g

```

Before explaining how this works, we discuss this function's arguments. The dictionary μ maps a state x to the probability μ_x that the chain starts in x . For instance, when the process starts (almost) surely at $X_0 = x$, then $\mu = \{x: 1\}$. Next, P is a function that maps a state x to a list of pairs $(y, p_{x,y})$ where y is a state to which X can move from x and $p_{x,y}$ is the probability that the transition from x to y happens. The function is_stop returns `True` when the chain should stop (is absorbed) at x , and `False` otherwise. Finally, eps is a small number that prevents keeping states that have negligible probability of being accessed.

So, intuitively speaking, how does this algorithm work? As long as not all probability mass has been absorbed⁴, a for loop cycles over all states x seen so far. The dictionary g counts the expected number of visits to some state x and is updated in every round of the loop. We need to fix the range of the for loop by calling `list(mu.keys())` because the keys of μ can change in the body of the loop. Next, $\mu[x]$ is the probability that x has been visited since the last pass of the for loop. In accordance with (5.10.1) we add this probability to the expected total number of visits to $g[x]$ and assign it to p for ease.

We set $\mu[x] = 0$ for two reasons. When the chain is absorbed (stopped) at x , we continue with the next state to be checked in the for loop. In this case, the total probability mass should decrease by the absorption probability p , which is indeed achieved by setting $\mu[x] = 0$. If the chain does

³ Use that $E[\mathbb{1}_A] = P\{A\}$ for an indicator event A .

⁴ That is, the dictionary μ is not yet empty.

not stop at x , the probability p should be distributed over all states y to which X can transition. If $y \neq x$, then $\text{mu}[y] += p * p_{xy}$, because y could have been accessed from other states too. When $y = x$, the probability of staying in x is pp_{xy} .⁵ Since we earlier set $\text{mu}[x] = 0$, we can also write $\text{mu}[x] += p * p_{xy}$ for state x . In other words, we can now use $+=$ for all states, instead of having to single out the state x with an `if` statement.

Once the `for` loop completes, we prune in the last line all states that have negligible probability of having been accessed.

There remains, however, a fundamental question: why is it allowed to change mu in the `for` loop itself, and does it really converge to the Green kernel we discussed above?

TO REALLY UNDERSTAND why the green function works, we need to discuss some further ideas of discrete Markov chains. Recall the interpretation of the Green kernel $g(x, y)$ as the expected number of times y will be visited up to absorption and when starting in $X_0 = x$. Clearly, using the Markov property, g should satisfy the recursive expression

$$\begin{aligned} g(x, y) &= \mathbb{E}_x [\mathbb{1}_{X_0=y}] + \mathbb{E}_x [g(X_1, y)] \\ &\stackrel{1}{=} \mathbb{1}_{x=y} + \sum_z \mathbb{E}_x [\mathbb{1}_{X_1=z}] g(z, y) \\ &\stackrel{2}{=} \mathbb{1}_{x=y} + \sum_z P(x, z) g(z, y); \end{aligned}$$

in 1 we use that $g(z, y)$ is a constant, hence can be taken out of the expectation, in 2 that $\mathbb{E}_x [\mathbb{1}_{X_1=z}] = P(x, z)$. Casting this in matrix format makes this structure clearer:

$$G = I + PG. \quad (5.10.3)$$

Then, by using this recursive structure, and noting that X will be absorbed in some set D in expected finite time and P is finite so that P^t converges geometrically fast to 0 elementwise as $t \rightarrow \infty$,

$$G = I + PG = I + P(I + PG) = I + P + P^2G = \sum_{t=0}^n P^t + P^{n+1}G \rightarrow \sum_{t=0}^{\infty} P^t.$$

And, if we define recursively $\mu^t = \mu^{t-1}P = \mu P^t$, so that $\mu_y^t = \sum_x \mu_x P^t(x, y)$,

$$g(y) = \sum_x \mu_x \sum_{t=0}^{\infty} P^t(x, y) = \sum_{t=0}^{\infty} \mu_y^t.$$

The advantage of this algorithm over solving $G = I + PG$ with matrix methods is as follows. The latter is a matrix equation for which we need to specify the size of G . However, the states that have a reasonable probability of being visited are not yet known. Thus, solving $G = I + PG$ may include many more or less superfluous states, which is numerically wasteful.

LET US DEVELOP a few intermediate algorithms that apply the above ideas to compute $g(y)$. Note that mu is the starting distribution of X_0 . Skipping the standard imports, the key part updates g and μ as $g^{k+1} = g^k + \mu^k$, and $\mu^{k+1} \leftarrow \mu^k P$. Thus, after k iterations, g equals $\sum_{t=0}^{k-1} \mu^t$ and mu equals μ^k . In the update of mu we prune on too small events. As μ^k becomes smaller and smaller as k increases, eventually, all elements of mu become less than `eps`, and the `while` loop terminates.

⁵ Recall that the expected number of consecutive periods to stay in state x is $\sim \text{Geom}(p_x)$.

```

1  g = defaultdict(float)
2  while mu:
3      for x in mu: # update g
4          g[x] += mu[x]
5
6      new_mu = defaultdict(float)
7      for x in mu: # update mu
8          if is_stop(x):
9              continue # It seems cryptic to continue when stopping ...
10         for y, pxy in P(x):
11             new_mu[y] += mu[x] * pxy
12
13     mu = {x: p for x, p in new_mu.items() if p > eps}

```

We can combine the updating of g and new_mu in one for loop because these dicts do not affect each other in the for loop. Observe that the assignment $p := mu[x]$ makes the code faster since we prevent intermediate lookups.

```

1  while mu:
2      new_mu = defaultdict(float)
3      for x in mu:
4          g[x] += (p := mu[x])
5
6          if is_stop(x):
7              continue
8          for y, pxy in P(x):
9              new_mu[y] += p * pxy
10
11     mu = {x: p for x, p in new_mu.items() if p > eps}

```

THE NEXT PHASE in the analysis of `green` is to see why it is allowed to update mu within the for loop, that is, why `green` gives the same value for $g(y)$ as the above algorithms. Let us repeat here the core part of `green`.

```

1  while mu:
2      for x in list(mu.keys()):
3          g[x] += (p := mu[x])
4
5          mu[x] = 0
6          if is_stop(x):
7              continue
8
9          for y, pxy in P(x):
10             mu[y] += p * pxy

```

The difference with the previous algorithms is this. In the while loop of the previous algorithms, we update $g^{k+1} = g^k + \mu^k$ first, and then we update $\mu^{k+1} = \mu^k P$. In other words, the entire vector μ^{k+1} is computed before adding it to g^k in the next pass of the while loop. However, in `green`, we update mu and g within the same iteration of the for loop. Note where we set $mu[x] = 0$: it should be placed before the stop condition, because when stopping, the chain is absorbed.

What actually happens exactly in `green`?

Theorem 5.10.1. Let P be a matrix such that $G(I - P) = I$ has a solution G , compare (5.10.3). If $g^0 = 0$ and $\mu^0 = \mu$, then $g^k \rightarrow \mu G$ when the update rules of green

$$p = \mu_x^k, \quad g^{k+1} = g^k + p\delta_x, \quad \mu^{k+1} = \mu^k - p\delta_x + p\delta_x P,$$

are applied sequentially for all x in the support of μ^k .

Proof. We first show that the quantity $H^k = g^k + \mu^k G$ remains the same for all $k \geq 0$ under the update rules. Using substitution of the update rules,

$$\begin{aligned} H^{k+1} &= g^{k+1} + \mu^{k+1} G &&= g^k + p\delta_x + (\mu^k - p\delta_x + p\delta_x P)G \\ &= g^k + p\delta_x + \mu^k G - p\delta_x(I - P)G \\ &= g^k + \mu^k G = H^k, \end{aligned}$$

because $(I - P)G = I$, cf. (5.10.3). Then, by recursion, $g^k + \mu^k G = g^0 + \mu^0 G = \mu G$ since $g^0 = 0$ and $\mu^0 = \mu$. Since each accessible state that is not absorbing is visited infinitely often and the chain as a whole is absorbing, $\mu^k \rightarrow 0$, hence $g^k \rightarrow \mu G$. $\square$

Concluding, the iterations in the `while` and `for` loop over `g` and `mu` in `green` lead to precisely the same Green kernel as in the other algorithms.

WE HAVE SOME final remarks.

- The advantage of `green` is that we do not have to make a guess of the states that will be visited by X with reasonable probability. When using matrix methods to compute G such that $G(I - P) = I$, we need to expand P to many states that are irrelevant (up to `eps`).
- In inventory control, the update over P is often a fixed list. In such cases, it is quite a bit faster not to pass P , but to write something like `for i, pi in D_items: mu[x - i] += p * pi`.
- The difference between the, say, regular approaches and `green` can be compared to a Jacobi versus Gauss-Seidel updating scheme. In my experiments with inventory control, the `green` algorithm converges in about half the time of the other schemes.

LET US DEMONSTRATE the `green` algorithm for the (s, S) inventory control algorithm. The parameters are provided in `lighthouse_case.py`.

```

1  """Lighthouse case: model parameters."""
2
3  from functools import cache
4
5  from functions import Min, Plus
6  from random_variable import NumericRV as RV
7
8  leadtime = 2 # lead time
9  h = 40 * 0.5 / 30 # daily holding cost
10 b = 100 * 0.2 # daily backlog cost
11 K = 50 # Order cost, used for sS and Qr
12 N = 100 # simulation duration

```

```

13
14 if N <= leadtime:
15     print(
16         f"The simulation duration {N=} is shorter than the lead time {leadtime=}."
17     )
18     quit()
19
20 # demand for one period
21 D = RV({0: 1 / 6, 1: 1 / 5, 2: 1 / 4, 3: 1 / 8, 4: 11 / 120, 5: 1 / 6})
22
23 # demand during leadtime
24 X = sum(D for _ in range(leadtime)) if leadtime > 0 else RV({0: 1})
25
26
27 def c(x):
28     return b * Min(x) + h * Plus(x)
29
30
31 @cache
32 def L(x):
33     return X.E(lambda i: c(x - i))

```

With these data, the rest follows from the above. Note that we replaced the call to `P` by the list `D` in the arguments of `green` as this runs somewhat faster.

```

1 from collections import defaultdict
2 from functools import cache
3
4 from pyticToc import TicToc
5 from random_variable import NumericRV as RV
6
7 from lighthouse_case import D, K, L
8
9
10 def green(mu, D, is_stop, eps):
11     g = defaultdict(float)
12     while mu:
13         for x in list(mu.keys()):
14             g[x] += (p := mu[x])
15             mu[x] = 0
16
17         if is_stop(x):
18             continue
19
20         for i, pi in D:
21             mu[x - i] += p * pi
22     mu = defaultdict(float, {x: p for x, p in mu.items() if p > eps})
23     return g
24
25
26 def sS(s, S, eps=1e-12):
27     mu = defaultdict(float)
28     mu[S] = 1.0
29     is_stop = lambda x: x <= s
30     D_items = list(D.items())
31     # P = lambda x: ((x - y, py) for y, py in D_items) # example P
32     visits = green(mu, D_items, is_stop, eps=eps)
33

```

```

34     t = sum(p for x, p in visits.items() if not is_stop(x))
35     cost = sum(p * L(x) for x, p in visits.items() if not is_stop(x))
36     cost += K * sum(p for x, p in visits.items() if is_stop(x))
37     return cost / t
38
39
40 S_star, s_star = 20, 16
41 print(f"sS(s_star, S_star)=:.4f")

```

```

1  sS(s_star, S_star)=31.5101

```

In a second example, we consider an inventory system that accepts returns. These returns can be seen as negative demand. In that case, the regular algorithm to compute the average cost under an (s, S) policy no longer works, but green has no problems with such cases.

```

1  from functools import cache
2  from lighthouse_case import c, leadtime, D
3
4
5  R = RV({0: 1 / 3, 1: 1 / 3, 2: 1 / 4, 3: 1 / 8})
6  D -= R
7  X = sum(D for _ in range(leadtime)) if leadtime > 0 else RV({0: 1})
8
9
10 @cache
11 def L(x):
12     return X.E(lambda i: c(x - i))
13
14
15 def sS(s, S, eps=1e-12):
16     mu = defaultdict(float)
17     mu[S] = 1.0
18     is_stop = lambda x: x <= s
19     D_items = list(D.items())
20     visits = green(mu, D_items, is_stop, eps=eps)
21
22     t = sum(p for x, p in visits.items() if not is_stop(x))
23     cost = sum(p * L(x) for x, p in visits.items() if not is_stop(x))
24     cost += K * sum(p for x, p in visits.items() if is_stop(x))
25     return cost / t
26
27
28 S_star, s_star = 20, 16
29 print(f"sS(s_star, S_star)=:.4f")

```

```

1  sS(s_star, S_star)=22.8449

```

We see that by accepting returns, the average cost decreases.

Opt 5.10.1. In the pruning step of green, why not prune like this

```

1  mu = defaultdict(float, {x: p for x, p in mu.items() if (p > eps or not stop(x))})

```



In Section 6.1 we introduce Sakasegawa's formula, which approximates the expected waiting time in a $G/G/c$ queue. With some license, one could say that the philosophy behind lean manufacturing and the Toyota production system rests on principles derived from this formula. We then illustrate how to use Sakasegawa's formula to estimate waiting times in three queueing settings where the service process is interrupted. In the first case, Section 6.2, the server produces jobs from different families with a change-over time required to switch from one family to another. Such setups reduce the server's available time, thereby increasing the load. To counter this, the server produces in batches of fixed size. In the second case, Section 6.3, the server occasionally requires small adjustments, for example to prevent production quality from degrading below a certain level. Such adjustments typically do not occur during a job's service, but can take place *between* any two jobs. Consequently, the number of jobs served between adjustments (or setups) varies, unlike the situation in which batch sizes are fixed. In the third example, Section 6.4, quality issues or breakdowns may occur *during* a job's service. These increase service-time variability, leading to longer expected queueing times.

Along the way, we use some interesting results from probability theory and the Poisson process.

6.1 THE $G/G/c$ QUEUE

In this section, we introduce Sakasegawa's formula, which gives an approximation for the expected waiting time for a $G/G/c$ queue. We discuss the insights it offers and show how to estimate the squared coefficient of variation of the inter-arrival time from data.

SAKASEGAWA'S FORMULA PROVIDES a reasonable approximation⁰ for $E[W]$ of the $G/G/c$ queue, and has the form

$$E[W] = \frac{C_a^2 + C_s^2}{2} \frac{\rho^{\sqrt{2(c+1)}-1}}{1-\rho} \frac{E[S]}{c}, \quad (6.1.1)$$

where the utilization¹ of the station is $\rho = \lambda E[S] / c \in [0, 1)$, and $C_a^2 = V[X] / (E[X])^2$ and $C_s^2 = V[S] / (E[S])^2$ are the scv of the inter-arrival time and service time, respectively.

Note that Sakasegawa's formula for the $G/G/c$ queue reduces to the PK formula for $E[W]$ for the $M/G/1$ queue; just fill in $c = 1$ and set $C_a^2 = 1$. Part of the intuition behind Sakasegawa's formula is based on the following idea. To generalize the formula for $E[W]$ from the $M/M/1$ queue to that of the $M/G/1$ queue, we replace $(1 + 1)/2$ by $(1 + C_s^2)/2$; the result remains exact. Now, to go from the $M/G/1$ queue to the $G/G/1$ queue, replace $(1 + C_s^2)/2$ by $(C_a^2 + C_s^2)/2$; this is no longer exact, but the approximation remains good

⁰ There is no known exact expression for the $G/G/c$ queue.

¹ Here is a subtle point. When there are multiple machines, the utilization of each machine is not necessarily equal to ρ . For instance, if there is a preference to choose the 'left-most' machine whenever it is free, then the utilization of this machine is larger than ρ . Only when the machines have the same speed and routing is such that each machine receives a fraction λ/c of jobs, all machines in one station have the same utilization.

for most practical purposes. For the details behind the power with c , we refer to Sakasegawa's paper.

IT IS CRUCIAL to memorize the insights this formula offers. First, we see that $E[W] \sim (1 - \rho)^{-1}$. Consequently, when ρ is large, the waiting time is (very) long. And, not only is $E[W]$ large, it is also extremely *sensitive* to the actual value of ρ . Clearly, such situations must be avoided and therefore, when trying to improve a queueing system, the first focus should be on reducing ρ .

Second, $\rho = \lambda E[S] / c$. Thus, when ρ must be made smaller, we have just three options.² We can reduce the arrival rate λ of jobs, for instance, by blocking demand or sending it elsewhere, such as to another machine.³ We can make $E[S]$ smaller by replacing a server with a faster one or by shifting some of the processing steps of a job to other servers, thereby making job sizes smaller. Finally, we can add servers, i.e., making c larger.⁴ If technically possible, adapting c to λ when $E[S]$ cannot be easily changed is a very effective mechanism for controlling waiting times.

Third, $E[W] \sim C_a^2$ and $E[W] \sim C_s^2$, which implies that when the job inter-arrival or service times are very variable, $E[W]$ is large. Thus, after ensuring that ρ is sufficiently small, it becomes important to concentrate on reducing C_a^2 and C_s^2 .⁵

Finally, $E[W] \sim E[S] / c$. This means that from the perspective of a job in queue, average job service times are c times as short as 'their own' service times.

CLEARLY, SAKASEGAWA'S EQUATION requires an estimate⁶ of C_a^2 and C_s^2 . Now it is not always easy in practice to determine the actual service-time distribution, since service times are often estimated by a planner rather than measured. Similarly, the actual arrival moments of jobs are often not registered, only the date, or perhaps the hour, that a customer arrived. Hence, it is often not possible to directly estimate C_a^2 and C_s^2 from the available information.

However, when the number of arrivals per period $\{a_n\}$ has been logged for some time, we can use $\{a_n\}$ to estimate C_a^2 as⁷

$$C_a^2 \approx \frac{\tilde{\sigma}^2}{\tilde{\lambda}},$$

where

$$\tilde{\lambda} = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n a_i, \quad \tilde{\sigma}^2 = \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n a_i^2 - \tilde{\lambda}^2.$$

We derive this result in steps.⁸

First, recall some results from earlier sections. Let the inter-arrival time X have mean $1/\lambda$ and variance σ^2 , so that

$$C_a^2 = \frac{V[X_i]}{(E[X_i])^2} = \frac{\sigma^2}{1/\lambda^2} = \lambda^2 \sigma^2.$$

Let A_k be the arrival time of the k th arrival and $A(t)$ the number of arrivals up to time t . Consider the following useful relation between $A(t)$ and A_k , see [1.2.7],

$$P\{A(t) < k\} = P\{A_k > t\}.$$

² And not more!

³ And if customers have a choice, they will take their own measures, simply by going elsewhere.

⁴ This is precisely what you see in supermarkets.

⁵ It works the other way too: systems with low variability can deal with higher load.

⁶ If you are not interested in statistics, you can skip this part.

⁷ This can of course also be used to estimate C_s^2 .

⁸ The arguments are based on a book by Cox on renewal theory. If you don't like to extend your knowledge of statistics, you can skip this derivation.

Since the inter-arrival times have finite mean and second moment by assumption, we use the central limit theorem to see that

$$\lim_{k \rightarrow \infty} \frac{A_k - k/\lambda}{\sigma \sqrt{k}} = N(0, 1),$$

where $N(0, 1)$ is a standard normal rv with distribution $\Phi(\cdot)$. Similarly, for an α yet to be determined,⁹

$$\frac{A(t) - \lambda t}{\alpha \sqrt{t}} \rightarrow N(0, 1).$$

Thus, $E[A(t)] = \lambda t$ and $V[A(t)] = \alpha^2 t$.

Using that $P\{N(0, 1) \leq y\} = P\{N(0, 1) > -y\}$, we get

$$\begin{aligned} \Phi(y) &\approx P\left\{\frac{A_k - k/\lambda}{\sigma \sqrt{k}} \leq y\right\} = P\left\{\frac{A_k - k/\lambda}{\sigma \sqrt{k}} > -y\right\} \\ &= P\left\{A_k > \frac{k}{\lambda} - y\sigma \sqrt{k}\right\}. \end{aligned}$$

We can use this relation between the distributions of $A(t)$ and A_k to see that $P\{A_k > t_k\} = P\{A(t_k) < k\}$ where we define for ease

$$t_k = \frac{k}{\lambda} - y\sigma \sqrt{k}.$$

With this we get,

$$\begin{aligned} \Phi(y) &\approx P\{A_k > t_k\} = P\{A(t_k) < k\} \\ &= P\left\{\frac{A(t_k) - \lambda t_k}{\alpha \sqrt{t_k}} < \frac{k - \lambda t_k}{\alpha \sqrt{t_k}}\right\}. \end{aligned}$$

Since $(A(t_k) - \lambda t_k)/\alpha \sqrt{t_k} \rightarrow N(0, 1)$ as $t_k \rightarrow \infty$, the above implies that

$$\frac{k - \lambda t_k}{\alpha \sqrt{t_k}} \rightarrow y,$$

as $t_k \rightarrow \infty$. Using the above definition of t_k , the LHS of this equation can be written as

$$\frac{k - \lambda t_k}{\alpha \sqrt{t_k}} = \frac{\lambda \sigma \sqrt{k}}{\alpha \sqrt{k/\lambda + \sigma \sqrt{k}}} y.$$

Since $t_k \rightarrow \infty$ is implied by (and implies) $k \rightarrow \infty$, we therefore want that α is such that

$$\frac{\lambda \sigma \sqrt{k}}{\alpha \sqrt{k/\lambda + \sigma \sqrt{k}}} y \rightarrow y,$$

as $k \rightarrow \infty$. This is precisely the case when

$$\alpha = \lambda^{3/2} \sigma.$$

Finally, for t large, or, by the same token, k large,

$$\frac{\sigma_k^2}{\lambda_k} = \frac{V[A(t)]}{E[A(t)]} \approx \frac{\alpha^2 t}{\lambda t} = \frac{\alpha^2}{\lambda} = \frac{\lambda^3 \sigma^2}{\lambda} = \lambda^2 \sigma^2 = C_a^2,$$

where the last equation follows from the above definition of C_a^2 .

⁹ This is a common trick: suppose that there exists a constant to take of the scaling, and then try an extra relation to 'ferret out' the scaling constant.

TF 6.1.1. Sakasegawa's approximation for the waiting time in a $G/G/c$ queue is

$$E[W] = \frac{C_a^2 + C_s^2}{2} \frac{\rho^{\sqrt{2(c+1)}-1}}{c(1-\rho)} E[S].$$

We claim that it is exact for the $M/G/1$ queue.

TF 6.1.2. Claim: For a $G/G/1$ queue, if $\rho = 0.95$ then a 5\

TF 6.1.3. Sakasegawa's approximation for the waiting time in a $G/G/c$ queue is

$$E[W] = \frac{C_a^2 + C_s^2}{2} \frac{\rho^{\sqrt{2(c+1)}-1}}{c(1-\rho)} E[S].$$

Claim: For a stable $D/D/1$ queue $C_a^2 = 0$ and $C_s^2 = 0$, therefore $E[W] = 0$.

Ex 6.1.4. Consider a single-server queue at which every minute a customer arrives, precisely at the first second and $S \equiv 50$ s. What are ρ , $E[L]$, C_a^2 , and C_s^2 ?

Ex 6.1.5. Consider the same single-server system as in [6.1.4], but now the customer service time is stochastic: with probability 1/2 a customer requires 1 minute and 20 seconds of service, and with probability 1/2 the customer requires only 20 seconds of service. What are ρ , C_a^2 , and C_s^2 ?

Ex 6.1.6. In a workplace, when a mechanic has used a specific high precision tool, the tool has to be inspected by a microscope for potential failures. The inter-arrival times of the tools at the microscope is $X \sim \text{Unif}([3,9])$ minutes, and the inspection time S is such that $E[S] = 5$ minutes and $V[S] = 4$ minutes squared. Use Sakasegawa's formula to estimate $E[W]$. Next, when tools arrive as a Poisson process, what would be $E[W]$? Discuss your findings.

Ex 6.1.7. A machine serves two types of jobs. The service time of jobs of type T , $T = 1, 2$, is exponentially distributed with parameter μ_T . The type T of a job is random and independent of anything else, and such that $P\{T = 1\} = p = 1 - q = 1 - P\{T = 2\}$. (An example is a desk serving adults and children, both of which require different average service times, and p is the probability that the customer in service is an adult.) Show that the expected service time and variance are given by

$$\begin{aligned} E[S] &= pE[S_1] + qE[S_2] \\ V[S] &= pV[S_1] + qV[S_2] + pq(E[S_1] - E[S_2])^2. \end{aligned}$$

Thus, even if $V[S_1] = V[S_2] = 0$, still $V[S] > 0$ if $E[S_1] \neq E[S_2]$. Mixing different job types increases variability; hence queueing times.

6.2 SERVER SETUPS

In some cases, machines must be set up before they can start producing jobs. Consider, for example, a machine that paints jobs in red and blue.⁰ When the machine requires a color change, it may be necessary to clean up the machine, which takes time. Another example is an oven that needs a

⁰ In realistic problems there can be tens of families. Often, the setup time depends on the sequence in which the families are produced, for example, a switch in color from white to black might take less cleaning time than from black to white. The problem then includes determining a production sequence that minimizes the sum of the setup times to produce the families

temperature change when different job types require different production temperatures. Service operations form another setting with setup times: When servers (personnel) have to move from one part of a building to another, the time spent moving cannot be spent serving customers.

In all such cases, the setups, or change-over times, consume a significant amount of time; in fact, setup times of an hour or more are not uncommon. To avoid overloading the server, it is necessary to produce in batches such that a server first processes a batch of jobs of one type¹, then performs a setup to serve a batch of another type, and so on.

Here we use Sakasegawa's formula to model the effect of setup times on the average sojourn time of jobs. In the main text, we provide a list of elements required to compute $E[J]$; in [6.2.6] we illustrate in detail how to carry out the computations.

ASSUME A SINGLE machine produces runs of red and blue jobs arriving at a rate λ_r and λ_b . The scv of the inter-arrival times of both job types is given by C_a^2 . When the server changes from one color to the other, it needs a setup time. Once the server is set up for the correct color, we assume for simplicity that for all jobs the *net service times* $\sim S_0$.² Finally, the setup times $R_k \sim R$ with mean $E[R]$ and variance $V[R]$, and are assumed to be independent of the service times of the job.

The sojourn time of a job in the *entire* system is built as follows. First, we assemble the jobs into batches of the same color. For simplicity, we take B constant and the same for both colors.³ Once B jobs of one color have arrived, the batch is complete, and the *batch enters* a queue; thus we consider a queue with batches, not single jobs. After some time, the batch reaches the head of the queue, and its service starts as soon as the machine becomes free. To serve a batch, the machine first performs a setup and then processes each job individually until the batch is finished. Once a job's service is completed, it leaves the server.

It remains to build a quantitative model of this queueing system.

WHEN A JOB of color i arrives, the expected time for its batch to be formed is given by⁴

$$\frac{B-1}{2\lambda_i}, \quad i \in \{r, b\}. \quad (6.2.1)$$

WHEN THE BATCH is complete, the batch joins the queue, so we next compute the average time batches spend in queue.⁵ For this we can use Sakasegawa's formula; we only have to find expressions for each of its components.

The total arrival rate of jobs is $\lambda = \lambda_b + \lambda_r$. Thus, $\lambda_B = \lambda/B$ is the arrival rate of *batches*.

The expected service time of a batch is

$$E[S_B] = E[R] + B E[S_0]. \quad (6.2.2)$$

With the batch arrival rate and the expected batch service time, the load becomes $\rho = \lambda_B E[S_B]$

¹ Producing one type of job is often called a 'run'.

² It is a simple exercise to generalize the notation to allow service times to depend on color.

³ In general, B should depend on the arrival rate of the job type when the arrival rates vary considerably between the families.

⁴ [6.2.7].

⁵ Note that we shift interpretation: batches (not individual jobs) form the queue.

It is essential that B is sufficiently large to ensure that $\rho < 1$. With (6.2.2) this leads to the condition that B must be larger than some minimal batch size,

$$B > B_m = \frac{\lambda E[R]}{1 - \lambda E[S_0]}.$$

Now that we have identified the arrival rate and the service times of the batches, it remains to find expressions for the scv of the batch inter-arrival times $C_{a,B}^2$ and the batch service times $C_{s,B}^2$. It is easy to derive that⁶

$$C_{a,B}^2 = \frac{C_a^2}{B}, \quad C_{s,B}^2 = \frac{V[S_B]}{(E[S_B])^2}, \quad (6.2.3)$$

where

$$V[S_B] = V[R] + B V[S_0].$$

Observe that now we have all components for Sakasegawa's formula.

It can be useful to convert the effects of the setup time into an *effective service time* of individual jobs. Since there are B jobs per batch, by dividing by B we see

$$E[S] = \frac{E[S_B]}{B} = E[S_0] + \frac{E[R]}{B}, \quad V[S] = \frac{V[S_B]}{B} = V[S_0] + \frac{V[R]}{B}. \quad (6.2.4)$$

Note in particular that the variance of the effective service time is larger than the variance of the net service time.

IT REMAINS TO find a rule to determine what happens to a job after it has been processed. If the job has to wait until all the jobs in the batch are served, the expected time it spends at the server is $E[R] + B E[S_0]$. However, if the job can leave right after being served, the expected time at the server is⁷

$$E[R] + \frac{B+1}{2} E[S_0]; \quad (6.2.5)$$

the setup takes $E[R]$, on average $\frac{B-1}{2}$ jobs are served before the job itself, and then the job requires $E[S_0]$ of service. Our model is complete!

WE CAN OBTAIN a number of important insights from the above model. Using the code from [6.2.6] we plot the sojourn time $E[J_r]$ of a red job for various values of B in the figure at the right.

First, as B increases, we see a sharp decrease of $E[J_r]$. The reason for this is that the load ρ decreases as a function of B . Since $E[W] \sim (1 - \rho)^{-1}$, it is essential to avoid critically loading the server.

When B becomes quite large, we see that the sojourn time increases linearly. This follows immediately from (6.2.1) and (6.2.5), because the time to (dis)assemble batches is linear in B .

Finally, observe that the graph is not symmetric around the minimum. It is much worse to take B too small than too large. More generally, when designing systems, we can use such graphs to understand how sensitive the system is to variation and measurement errors.

Here is the code used to make Fig. 6.2.1.⁸

```
1 import matplotlib.pyplot as plt
```

```
2
```

⁶ [6.2.8] [6.2.9].

⁷ [6.2.10].

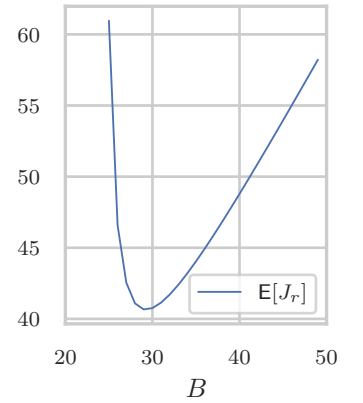


Fig. 6.2.1: The sojourn time of the red jobs as a function of the batch size B .

⁸ It follows directly from the above math.

```

3  from latex_figures import apply_figure_settings
4
5  apply_figure_settings(use=True)
6
7  labda_r = 0.5 # red jobs
8  labda_b = 2.5 # blue jobs
9  labda = labda_r + labda_b
10 Cae = 1.0 # assumption on inter arrival times
11 ES0 = 15.0 / 60 # hour
12 Cse = 1.0 # scv of service times
13 VS0 = Cse * ES0**2
14 ER = 2.0 # setup time
15 VR = 1.0 # Variance of setup times
16
17 x = list(range(25, 50))
18 y = []
19
20 for B in x:
21     ESe = ES0 + ER / B
22     rho = labda * ESe
23
24     # The time to form a red batch
25     EW_r = (B - 1) / (2 * labda_r)
26
27     # Now the time a batch spends in queue
28     CaB = Cae / B
29     VSe = B * VS0 + VR
30     ESb = B * ES0 + ER
31     CeB = VSe / ESb**2
32     EW = (CaB + CeB) / 2 * rho / (1 - rho) * ESb
33
34     # The time to unpack the batch, i.e., the time at the server.
35     ES = ER + (B - 1) / 2 * ES0 + ES0
36
37     total = EW_r + EW + ES
38     y.append(total)
39
40
41 def cm_to_inch(cm):
42     return cm / 2.54
43
44
45 plt.figure(figsize=(cm_to_inch(5), cm_to_inch(6)))
46 plt.xlim(20, 50)
47 plt.plot(x, y, label=r"$\mathsf{E}[J_r]$", lw=0.7)
48 plt.xlabel("$B$")
49 plt.legend(loc="lower right")
50 plt.tight_layout()
51 plt.savefig("/home/nicky/Dropbox/vakken/qts/stochastic_or/figures/setups.pdf")
52 plt.savefig("/home/nicky/Dropbox/vakken/qts/stochastic_or/figures/setups.png", dpi=300)

```

TF 6.2.1. Jobs arrive at a rate λ and are assembled in batches of size B . The average time a job waits until the batch is complete is $E[W] = \frac{B-1}{2\lambda}$.

TF 6.2.2. Claim: for a $G^B/G/1$ queue, the waiting time of a job can be reduced by making batches larger, as this means that there are fewer batches in the queue.

TF 6.2.3. Claim: In a system with setup times between batches, the load decreases when the batch size B increases.

TF 6.2.4. Claim: In a system with setup times between batches, batching reduces the variability of the arrival process; see (6.2.3).

TF 6.2.5. Claim: In a system with setup times between batches, increasing the batch size B always reduces the average sojourn time of a job.

Before dealing with technical derivations, let us first see how to apply the model.⁹

Ex 6.2.6. Jobs arrive at $\lambda = 3$ per hour at a machine with $C_a^2 = 1$; service times are exponential with an average of 15 minutes. Assume $\lambda_r = 0.5$ per hour, and hence $\lambda_b = 2.5$ per hour. Between two batches, the machine requires a clean-up of 2 hours, with a standard deviation of 1 hour. First find the smallest batch size that can be allowed, then compute the average time a red job spends in the system in the case $B = 30$ jobs.

Ex 6.2.7. Show that the average time a job has to wait to fill the batch (to which this job belongs) is given by (6.2.1).

Ex 6.2.8. Explain that the scv of the batch inter-arrival times is given by (6.2.3).

Ex 6.2.9. Show that $C_{s,B}^2$ takes the form as in (6.2.3).

Ex 6.2.10. Show that, when jobs can leave right after being served, the time at the server is given by (6.2.5)

⁹ Before dealing with technical derivations, let us first see how to apply the model.



6.3 SERVER ADJUSTMENTS

In Section 6.2 we study the effect of setup times between batches of jobs. There are, however, other reasons to interrupt the server while it is serving jobs in queue. For example, when a knife on a cutting machine becomes blunt, an operator has to stop the machine to replace the knife. Another example is a medical doctor who has to make many unexpected phone calls between seeing two patients. All such small (maintenance) tasks can be carried out in between jobs, but it is often hard to predict when they are required. Therefore, the number of jobs⁰ served between any two tasks¹ is no longer constant. But we know from Sakasegawa's formula that randomness in service times affects queueing times, and we also know from (6.2.4) that the batch size affects the service time. Hence, such unplanned tasks must make sojourn times longer.

We refer to this type of interruption as a server *adjustment*. It is important to realize that setups and adjustments are *non-preemptive outages*, i.e., they occur *between* jobs, not *during* job service times. Section 6.4 deals with the latter set of outages.

In this section, we develop a simple model to understand the impact of adjustments occurring randomly between jobs; we use the same notation as in Section 6.2 and follow the same line of reasoning. With this model, we can analyze a number of trade-offs, such as making fewer, but longer adjustments, or planning adjustments instead of just waiting until it becomes necessary at an unexpected moment.²

⁰ Compare this to a batch of jobs.

¹ And compare a task to a setup.

² [6.3.7].

LET US WRITE F_i for the Bernoulli rv that, when equal to 1, indicates that job i requires an adjustment, and, when 0, the adjustment is not necessary. We assume that $F_i \sim F$ such that $P\{F = 1\} = p = 1 - P\{F = 0\}$. Thus, with constant probability p an adjustment occurs between any two jobs. As a consequence, the number of jobs served between two consecutive adjustments is a geometric rv B with $E[B] = 1/p$. We further assume that the adjustment times $R_i \sim R$ with mean $E[R]$ and variance $V[R]$. Finally, we assume that the net service times $\{S_{0,i}\}$, $\{R_i\}$ and $\{F_i\}$ are independent.

Thus, whereas in Section 6.2 the batch sizes were constant between two setups, they are here geometrically distributed.

TO COMPUTE $E[W]$ with Sakasegawa's formula, we only have to find out how the adjustments affect the mean $E[S]$ and scv C_s^2 of the effective service time. As adjustments do not influence the job arrival process, λ and C_a^2 remain the same.

In the exercises below we show that the average effective service time is

$$E[S] = E[S_0] + pE[R] = E[S_0] + \frac{E[R]}{E[B]}, \quad (6.3.1)$$

and its variance is

$$V[S] = V[S_0] + pV[R] + p(1-p)(E[R])^2. \quad (6.3.2)$$

By dividing $V[S]$ by $(E[S])^2$ we obtain C_s^2 .

Assuming there is one server, we can now fill in Sakasegawa's formula!

BEFORE CONSIDERING SOME examples, let us compare the results of this model with those of Section 6.2. The expected effective service time is the same: an amount $E[R]/E[B]$ is added to the net service time $E[S_0]$. However, the impact on the variance is different. By comparing (6.2.4) to (6.3.2) we see that the latter has an extra (positive) term. This supports our intuition: Unexpected interruptions have a larger effect on variability than expected (planned) interruptions.

The first few exercises demonstrate how to apply the above; the rest are concerned with deriving (6.3.1) and (6.3.2).

TF 6.3.1. The number of jobs served between two consecutive adjustments follows a Poisson distribution.

TF 6.3.2. A repair shop is considering buying a new CNC machine. There are two options available, the Yamazaki and the Haas. The Haas is less reliable and breaks down every week on average. However, it can be easily fixed by running through the control checklist, which takes 60 minutes. The Yamazaki breaks down about once a month, but it is much more complicated to fix and takes 3.5 hours. The time till a breakdown is memoryless and the machines have the same service rate. Claim: The Yamazaki is better from a queueing perspective.

TF 6.3.3. Claim: In a system with random adjustments between jobs, the effective service time has the same variance as in a system with planned setups and the same average batch size.

TF 6.3.4. Claim: In a system with random adjustments between jobs, making fewer but longer adjustments (same total adjustment time on average) reduces the waiting time.

TF 6.3.5. Claim: In a system with random adjustments between jobs, if the probability of an adjustment is constant $p \in (0, 1)$ for each job, the number of jobs served between two consecutive adjustments follows a geometric distribution.

Ex 6.3.6. Jobs arrive as a Poisson process with a rate $\lambda = 9$ per working day. The machine works two 8 hour shifts a day. Work not processed on a day is carried over to the next day. The job service times are 1.5 hours, on average, with a standard deviation of 0.5 hours. Outages occur on average between 30 jobs. The average duration of an outage is 5 hours and has a standard deviation of 2 hours. Compute $E[J]$.

Ex 6.3.7. In the setting of [6.3.6] we can perhaps choose to make an adjustment every 20 jobs. For simplicity, assume that adjustments never occur at random. Also, assume that an adjustment takes less time, for instance 4.5 hours, and is constant. What is $E[J]$ now?

The easiest way to derive the expressions for $E[S]$ and $V[S]$ is to use Adam's and Eve's rule.³

Ex 6.3.8. Derive the expressions for $E[S]$ and $V[S]$.

6.4 SERVER FAILURES

In Sections 6.2 and 6.3 we assumed that servers are never interrupted while serving a job. However, this assumption is not always satisfied; for example, a machine may fail in the middle of processing a job. In this section, we develop a model⁰ to calculate the influence on the mean waiting time of such *preemptive outages*, again based on Sakasegawa's formula for the $G/G/1$ queue.

As in Section 6.3, we only have to derive expressions for the expectation and variance of the effective service time S . The other components of Sakasegawa's formula are not affected.

We assume that a failure can occur at any time during the service time of a job. The repair times $R_i \sim R$ have common mean $E[R]$ and finite second moment $E[R^2]$. Supposing that N interruptions occur during the net job service time S_0 , the effective service time becomes¹

$$S = S_0 + S_N = S_0 + \sum_{i=1}^N R_i. \quad (6.4.1)$$

In general, N is a random number, so we need to adjust for this in the computation of the mean and variance of S .

IT IS COMMON to assume that the time between two interruptions is memoryless with mean $1/\lambda_f$, where λ_f is the *failure rate*. Defining the MTTF² as $m_f = 1/\lambda_f$, then the *availability* is given by

$$A := \frac{m_f}{m_f + E[R]},$$



³

$$V[Y] = E[V[Y|X]] + V[E[Y|X]].$$

⁰ There is not much point in combining the models of Section 6.2–Section 6.4 into one large model. For example, when a machine can be set up while a part is being repaired, there may not be time wasted on setup. Everything depends on specific strategies to combine outages.

¹ In an insurance context, if we interpret $\{R_i\}$ as a set of claims, then $\sum_{i=1}^N R_i$ is the total claim size of N claims. Likewise, in an inventory system, this sum is the total demand of N customers.

² MTTF := mean time to fail.

from which follows easily that ³

$$A = (1 + \lambda_f E[R])^{-1}. \quad (6.4.2)$$

Next, because by assumption the time between two failures is $\sim \text{Exp}(\lambda_f)$ so that $N \sim \text{Pois}(\lambda_f S_0)$ if S_0 is given,

$$E[N] = E[E[N|S_0]] = E[\lambda_f S_0] = \lambda_f E[S_0]. \quad (6.4.3)$$

Therefore,

$$\begin{aligned} E[S] &= E[S_0] + E[N] E[R] = E[S_0] + \lambda_f E[S_0] E[R] \\ &= E[S_0] (1 + \lambda_f E[R]) = E[S_0] / A. \end{aligned}$$

With this expression for the expected effective service time, the load becomes

$$\rho = \lambda E[S] = \lambda \frac{E[S_0]}{A}.$$

Clearly, $A \in (0, 1]$, so the load increases due to failures.

WITH SOME MORE work, we obtain⁴ from (6.4.1),

$$V[S] = \frac{V[S_0]}{A^2} + \lambda_f E[R^2] E[S_0]. \quad (6.4.4)$$

If the repair times are exponentially distributed, it is possible to find a neat expression for the scv of the effective service time:⁵

$$C_s^2 = C_0^2 + 2A(1 - A) \frac{E[R]}{E[S_0]}, \quad (6.4.5)$$

where C_0^2 is the scv of S_0 .

TF 6.4.1. The normal service time of a job, without interruptions, is given by S_0 . The durations of interruptions are given by $R_i \sim R$. If N interruptions occur, the effective service time will then be

$$S = S_0 + \sum_{i=1}^N R_i.$$

Then all steps in the computation below are correct:

$$E \left[\sum_{i=1}^N R_i \right] = E \left[\sum_{n=0}^{\infty} \mathbb{1}_{N=n} \right] E \left[\sum_{i=1}^n R_i \right] = E[N] E[R]$$

TF 6.4.2. Assume that server failures are memoryless and occur at a rate of 6 per hour. Moreover, assume that the expected time to fix a failure is 18 minutes. An investment can be made which will result in a failure rate of 18 per hour and an expected repair time of 6 minutes per failure. Claim: This is a good investment.

TF 6.4.3. Claim: Server failures reduce the availability A but do not affect the effective service time $E[S]$.

TF 6.4.4. Claim: If the net service time S_0 is deterministic, then the effective service time $S = S_0 + \sum_{i=1}^N R_i$ is also deterministic.

TF 6.4.5. Claim: In the model of this section, the scv of the effective service time satisfies $C_s^2 \geq C_0^2$, where C_0^2 is the scv of the net service time.

Ex 6.4.6. Suppose that we have a machine with memoryless failure behavior, with a mean time-to-fail of $m_f = 3$ hours. Regular service times are deterministic with an average of $S_0 = 10$ minutes, jobs arrive as a Poisson process with a rate of $\lambda = 4$ per hour. Repair times are exponential with a mean duration of $E[R] = 30$ minutes. What is the average sojourn time?

Ex 6.4.7. Derive (6.4.2).

Ex 6.4.8. Derive (6.4.4) using Adam and Eve's rule.

Ex 6.4.9. Show that

$$C_s^2 = \frac{V[S]}{(E[S])^2} = C_0^2 + \frac{\lambda_f E[R^2] A^2}{E[S_0]},$$

Ex 6.4.10. Assuming that the repair times R are exponentially distributed, show (6.4.5).

Ex 6.4.11. How is the renewal reward theorem used in (6.4.2) to find an expression for the availability A ?

7.1 $G/G/c$ QUEUES IN TANDEM

OFTEN, QUEUES APPEAR in sequence. In such cases, customers are interested in the mean sojourn (and waiting times) of their jobs in the entire chain. For this reason, we need a formula for dealing with networks.

Consider two $G/G/1$ stations in tandem.⁰ Suppose we have the financial means to reduce the variability of the service times at one of the stations, but not at both. Clearly, we would like to improve the one that has the most impact on the total sojourn time in the line.

For the waiting time of the first machine, we can use Sakasegawa's formula, but to apply this to the second machine, we need $C_{a,2}^2$, i.e., the scv of the inter-arrival times at the second station. Now, noting that the output of the first machine forms the input of the second machine, it is clear that $C_{a,2}^2 = C_{d,1}^2$, where $C_{d,1}^2$ is the scv of the *inter-departure* times of the *first* station.

Consider the inter-departure times of a $G/G/1$ queue. Suppose that the utilization ρ is very high. Then the server will seldom be idle, so that most of the inter-departure times are equal to the service times of the first station. However, if the utilization is low, the server will be idle most of the time and the inter-departure times must be approximately equal to the inter-arrival times of the first station. By interpolating between these two extremes, we obtain the following approximation for the scv of the inter-departure times of a $G/G/c$ queue:

$$C_{d,1}^2 \approx 1 + (1 - \rho^2)(C_{a,1}^2 - 1) + \frac{\rho^2}{\sqrt{c}}(C_{s,1}^2 - 1).$$

It is simple to see that for the $G/G/1$ queue this reduces to

$$C_{d,1}^2 \approx (1 - \rho^2)C_{a,1}^2 + \rho^2 C_{s,1}^2. \quad (7.1.1)$$

Next, we can use this formula for each station in the tandem network.

Combining Sakasegawa's formula with this expression provides us with a very useful insight for a line of queues. If we reduce $C_{s,1}^2$, i.e., the scv of service times at the first station, $E[W_1]$ and $C_{d,1}^2$ become smaller. Since $C_{a,2}^2 = C_{d,1}^2$, $E[W_2]$ also becomes lower, but also $C_{d,2}^2$, etc. In other words, the entire chain benefits from an improvement in service variability at the first station.¹

TF 7.1.1. Suppose that in a tandem network of $G/G/c$ queues we can reduce C_s^2 of just one station by a factor 2. To improve the average waiting time throughout the chain, it is best to reduce $C_{s,1}^2$.

TF 7.1.2. A production system consists of 2 stations in tandem. The first station has one machine, the second has two identical machines. Machines never fail and service times are deterministic. Jobs arrive at a rate of 1 per

⁰ 'In tandem' means 'in line', i.e., one station after the other.

¹ In other words, try to improve from the start of the chain.

hour. The machine at the first station has a service time of 45 minutes per job, and a machine at the second station has a service time of 80 minutes. We claim that the second station is the bottleneck.

TF 7.1.3. Consider a network with n stations in tandem. At station i , the service times S_i for all machines at that station are the same and constant; station i contains N_i machines. The number of jobs required to keep all machines busy is $N = \sum_{i=1}^n N_i$, and the raw service time is $T_0 = \sum_{i=1}^n S_i$. Thus, if the number w of jobs allowed in the system is greater than N , the number of jobs waiting somewhere in the queue is $w - N$.

TF 7.1.4. We have two $M/M/1$ stations in tandem. Claim: The average queueing time for the network is given by

$$E[W] = \frac{\rho_1}{1 - \rho_1} + \frac{\rho_2}{1 - \rho_2}.$$

Ex 7.1.5. In a manufacturing setting, the Poisson process is not always a suitable model for the arrival process of jobs at a production station. Can you provide an example to show why this is the case? 

Ex 7.1.6. Consider two $G/G/1$ stations in tandem. Suppose $\lambda = 2$ per hour, $C_{a,1}^2 = 2$, $C_{s,1}^2 = C_{s,2}^2 = 0.5$, and $E[S_1] = 20$ minutes, and $E[S_2] = 25$ minutes. Compute $E[J] = E[J_1] + E[J_2]$.

7.2 DETERMINISTIC QUEUEING NETWORKS

Before we study queueing networks in stochastic settings, we should familiarize ourselves with the behavior of networks of queueing systems in the simplest setting possible. Arguably, this is a network in which all servers have constant service times, which is the topic of this section.

A QUEUEING NETWORK can be represented as a *graph*. *Stations* form the *nodes* in the graph, and *edges* between nodes represent the routing, i.e., the possibility that jobs can move from one station to another. A simple example is a network of n stations in tandem. Jobs arrive at station 1, and after service they move to station 2. Then they move to station 3, and so on, until they leave the network after being served at station n . A station can contain one or more servers. We assume that all servers have constant job processing times. However, service times need not be equal for all servers.

We need some more definitions. The *raw processing time* T_0 is the minimal amount of time needed to move from the start of the system to the end. More precisely, it is the sum of the long-run average processing times of each workstation in the routing. The *utilization* of station i is λ_i / μ_i , i.e., the rate λ_i at which work arrives divided by the processing rate of the server(s) at station i . The *bottleneck* station is the station with the highest utilization. We write r_b for the processing rate of the bottleneck station. The *critical work in progress (WIP)* W_0 is defined as $W_0 = r_b T_0$. This is the minimal amount of work required in the network to keep the bottleneck station fully loaded. To see how this relates to Little's law, observe that if jobs leave the network at rate r_b , they have to arrive at the network at rate r_b —we also assume that inter-arrival times are constant, so that no

explosions occur. Note that this is the largest arrival rate that the network can handle without queues growing to infinity. But then, with Little's law, the amount of work in the system is equal to the arrival rate times the time in the system.

In the next set of exercises we illustrate the above concepts by means of a network with n single-server stations in tandem, where all stations have the same service time S .

Ex 7.2.1. Suppose we would release just one job at a time in the above tandem system, that is, only when a job has visited all stations, the next job is released. What is the throughput of the network?

Ex 7.2.2. Argue that if we are prepared to release w jobs, the throughput of the network becomes

$$\mu(w) = \min\{w, n\} \frac{1}{T_0}.$$

Conclude that we need a minimal amount w in the network to guarantee that the network's throughput $\mu(w)$ exceeds the arrival rate λ , that is, there is a minimal w such that $\mu(w) \geq \lambda$.

Ex 7.2.3. Show that only when $w \geq n$,

$$\mu(w) = \frac{1}{S},$$

in which case the maximal capacity of the network is achieved.

Thus, for given w the largest arrival rate the network can handle is $\lambda = \mu(w)$, and the maximal rate at which it can process jobs is $r_b = 1/S$.

Ex 7.2.4. Show that when $\lambda = \mu(w)$, the waiting time in the network is

$$E[W] = \begin{cases} T_0 = nS, & \text{if } w \leq n, \\ \frac{w}{n} T_0 = wS, & \text{if } w \geq n. \end{cases}$$

Ex 7.2.5. What do you conclude from the above exercises?

In conclusion, to balance waiting times and throughput, we need to tune the amount of work in the system. The critical WIP W_0 is the minimal WIP we need to keep all bottleneck machines occupied without adding extra waiting time. Jobs can never pass through the network in less time than T_0 , and the maximal output rate of the network is $r_b = 1/S$. Thus,

$$T_0 = nS, \quad r_b = 1/S, \quad W_0 = r_b T_0 = n. \quad (7.2.1)$$

THE PROBLEMS BELOW are generalizations to networks with multi-server stations, servers with different processing rates, and multiple routings. They are meant to help you become acquainted with network behavior, and also with Little's law, which we use time and again here.⁰

Ex 7.2.6. A production system consists of 2 separate stations. Jobs never go from one machine to another. The processing times are $(t_1, t_2) = (1, 2)$ hours. The arrival rates are $(\lambda_1, \lambda_2) = (1/3, 1/3)$ per hour. What is the fastest machine? Which machine is the bottleneck?



⁰ Warning, these problems are quite challenging, despite their apparent simplicity; I had much less trouble posing the problems than solving them. In fact, to ensure that I got the answer right, I often tried to find at least two different ways to solve the same problem.

Ex 7.2.7. In the network of the previous exercise, what are T_0 and W_0 ?

Ex 7.2.8. A production network consists of 3 single-machine stations in tandem, i.e., in series. The processing times are (2, 3, 2) hours respectively, i.e., constant and such that $t_1 = 2$ hours, $t_2 = 3$ hours and $t_3 = 2$ hours. What are the critical WIP W_0 , bottleneck capacity r_b and raw processing time T_0 ?



Ex 7.2.9. A production network consists of 3 single-machine stations in tandem with processing times (2, 3, 2) hours. Suppose we modify this network by adding one extra machine to station 2, with processing time $t = 3$ hours. What are W_0 , r_b and T_0 ?

Ex 7.2.10. A production network consists of 3 single-machine stations in tandem with processing times (2, 3, 2) hours. Suppose we modify this network by adding one extra machine to station 2 with processing time $t = 4$ hours. What are W_0 , r_b and T_0 ?

Ex 7.2.11. Why does the critical WIP change in the different cases above?

Ex 7.2.12. A production network consists of 3 single-machine stations in tandem with processing times (2, 3, 2) hours. Suppose we modify this network by adding one extra machine to station 2 with processing time $t = 1$ hour. What are W_0 , r_b and T_0 ?

Ex 7.2.13. In the previous question, it might seem that one machine at the second station is superfluous. Which one, and why? Would you remove this superfluous machine in a real production network?

Ex 7.2.14. A production network consists of 3 single-machine stations in tandem with processing times (2, 3, 2) hours. Suppose we modify this network by adding one extra machine to station 2 with processing time $t = 7$ hours. What are W_0 , r_b and T_0 ?

Ex 7.2.15. A production network consists of 3 single-machine stations in tandem with processing times (2, 3, 2) hours. Suppose we modify this network such that 2/3 of the jobs *leave after the first* station, i.e., between station 1 and 2. What are W_0 , r_b and T_0 ?

Ex 7.2.16. A production network consists of 3 single-machine stations in tandem with processing times (2, 3, 2) hours. Suppose we modify this network such that 4/5 of the jobs *bypass the second* station, i.e., 1/5 of the jobs have the routing [1, 2, 3], but 4/5 have routing [1, 3]. What are W_0 , r_b and T_0 ?

Ex 7.2.17. Building upon the previous question, which stations would you include anyway in a CONWIP loop, and why?

Production situations can be pretty complicated. Jobs can enter the network at different places, not just at station 1 as we considered here. Jobs may also need rework at one or more stations. One consequence of these aspects is that the slowest machines or stations need not be the bottlenecks. If you like, you can try to find or develop a general algorithm to compute T_0 , W_0 and r_b for any network of reasonable size. One way to approach this problem might be to place (imaginary) huge amounts of work in front of each station and just see how the network drains, another is to try to find an analogy with an electrical network. All in all, it's an elegant problem.

7.3 OPEN SINGLE-CLASS PRODUCT-FORM NETWORKS

Up to now our analysis focused on single-station queueing systems. However, jobs, or patients, may need several 'processing' steps at different stations. In such cases, jobs move from one queueing station to another. In this section we analyze such queueing networks in simple settings. We start with two stations in tandem, and then extend to general networks. Throughout we assume that external jobs⁰ arrive as Poisson processes, and that service times at all stations are exponentially distributed. Under this condition we are able to obtain closed-form expressions for the stationary distribution of jobs at each station.¹

WE START BY stating the remarkable and important result that the *inter-departure times* of jobs of an $M/M/1$ queue are exponentially distributed with rate λ , just as the inter-arrival times.² This property makes the analysis of a *tandem network* of stations, i.e., stations placed in sequence, particularly easy.

When the first station is an $M/M/1$ queue, jobs arrive as a Poisson process, but its output process is also a Poisson process. Therefore the second station, i.e., the station immediately downstream of station 1, receives jobs as a Poisson process. If job service times at the second station are exponentially distributed, then this station is also an $M/M/1$ station. But then its departure process is a Poisson process in turn, and the third station receives jobs as a Poisson process.

With this insight, it follows from (5.1.2) that the expected total sojourn time in a tandem network of M stations is equal to

$$E[J] = \sum_{i=1}^M E[J_i] = \sum_{i=1}^M \frac{E[S_i]}{1 - \rho_i},$$

where $E[J_i]$ is the sojourn time at station i with $\rho_i = \lambda E[S_i]$.

IT IS NOT difficult to extend the above result to general networks of $M/M/1$ queues. For this, we first need to model such networks more formally. In particular, we assume that the probability that a job moves to station j after completing its service at station i is independent of anything else, and is given by the number $P_{ij} \in [0, 1]$.³ We assemble all these probabilities in a *routing matrix* P such that P_{ij} is the element of P on the i th row and j th column. Define $P_{i0} = 1 - \sum_{j>0} P_{ij}$ as the probability that a job leaves the network from station i . In Section 7.4 we discuss some properties of P such that the network is stable.

⁰ I.e., new jobs that have not visited any other station before.

¹ Recall that Section 7.1 only considers the expected sojourn times in tandem networks of $G/G/c$ queues, not the distribution of the number of jobs at each station in a network of $M/M/1$ queues.

² [7.3.2].

³ This is called Markov routing.

Consider station i , say, and assume that jobs arrive as a Poisson process with rate λ_i . Since service times are exponentially distributed, the departure process is also Poisson with rate λ_i . Then, after departure, jobs are sent with probability P_{ij} to station j . It follows from [1.2.13] that jobs sent to station j from station i form a thinned Poisson process with rate $\lambda_i P_{ij}$.

Now take the perspective of station j . This station receives a merged ‘stream’ of thinned Poisson processes from all other stations. But by the results of Section 1.2, this merged process is also a Poisson process with rate $\sum_{i=1}^M \lambda_i P_{ij}$. Assuming that external jobs arrive at station j as a Poisson process with rate γ_j , then by merging this process with the other Poisson stream, we obtain a Poisson process with rate $\gamma_j + \sum_{i=1}^M \lambda_i P_{ij}$.

By assumption, service times at station j are exponential, hence, the departure process of this station is Poisson. But then the thinned process resulting from routing its departing jobs to yet other stations is again Poisson, and so on.

It follows that this network behaves as a set of $M/M/1$ queues. Below we give a formal proof of this fact for two stations.

Note that we allow for external jobs arriving at any station. Therefore, this is an *open network*. This differs from so-called *closed networks*; in such networks jobs do not enter or leave the network.

IT IS EVIDENT that, when the network is stable, all jobs that enter a station must eventually leave that station. This insight leads us to the *traffic equations*, which state that for each station i the departure rate must equal the arrival rate, i.e.,

$$\lambda_i = \gamma_i + \sum_{j=1}^M \lambda_j P_{ji}, \quad i = 1, \dots, M. \quad (7.3.1)$$

For the *network as a whole*, the total external arrival rate is given by $|\gamma| = \sum_{i=1}^M \gamma_i$. Hence, this is also the departure rate of the network.⁴ In Section 7.4 we present a method to compute λ from γ and P .

⁴ Not for an individual station i , because $\delta_i = \lambda_i$.

LET US FOR the moment assume that we can solve the traffic equations, in other words, for a given $\gamma = (\gamma_1, \dots, \gamma_M)$ and routing matrix P we can find a set of numbers $\lambda = (\lambda_1, \dots, \lambda_M)$ that satisfies (7.3.1). Then, we can define $\rho_i = \lambda_i E[S_i]$ as the utilization of (the server of) station i . Clearly, we assume that $\rho_i < 1$ for all stations i .

Evidently, the average total number of jobs in this network of $M/M/1$ stations is equal to the sum of the average number of jobs at each station, hence, $E[L] = \sum_{i=1}^M E[L_i]$. With this we can also find an expression for the expected total sojourn time in the network $E[J]$. Applying Little’s law first to the network as a whole and then to each station individually, we see that

$$|\gamma| E[J] = E[L] = \sum_{i=1}^M E[L_i] = \sum_{i=1}^M \lambda_i E[J_i] = \sum_{i=1}^M \lambda_i \frac{E[S_i]}{1 - \rho_i}.$$

Dividing by $|\gamma|$ gives $E[J]$ in terms of the *visit ratios* $\lambda_i/|\gamma|$,

$$E[J] = \sum_{i=1}^M \frac{\lambda_i}{|\gamma|} \frac{E[S_i]}{1 - \rho_i}.$$

This is intuitive: the visit ratio $\lambda_i/|\gamma|$ tells how often station i is visited relative to the total number of arrivals. Thus, $E[J_i] \lambda_i/|\gamma|$ is the amount of time an ‘average’ job spends at station i before leaving the network.

ABOVE WE DERIVED expressions for the average waiting time in a network of $M/M/1$ queues, but it is possible to obtain a much stronger result. In fact, the stationary probability $p(n)$ that the system contains $n = (n_1, n_2, \dots, n_M)$ jobs at stations $1, \dots, M$ can be written as the *product* of the probabilities of the individual stations, specifically,

$$p(n) = P\{L_1 = n_1, \dots, L_M = n_M\} = \prod_{i=1}^M p(n_i),$$

where $p(n_i) = (1 - \rho_i) \rho_i^{n_i}$ is the stationary probability that station i contains n_i jobs, compare (5.1.1). In other words, the stationary probability $p(n_i)$ is *independent* of the state of the other stations. Hence, in stationarity, we can concentrate on each station individually. As a consequence, we can easily compute the excess probability $P\{L_i > n_i\}$ for each station i .

As a matter of fact, any stable open network of $M/M/c$ stations⁵ admits a *product-form solution*, a result known as *Jackson’s theorem*.

LET US PROVE this result for the case of two $M/M/1$ stations in tandem. Let $p(i, j) = P\{L_1 = i, L_2 = j\}$ be the probability that station 1 contains i jobs and station 2 contains j jobs. Specifically, we have to show that

$$p(i, j) = (1 - \rho_1)(1 - \rho_2) \rho_1^i \rho_2^j \quad (7.3.2)$$

satisfies the balance equations for all $i, j \geq 0$, see Section 4.6.

Here we focus on the case $i, j \geq 1$, see Fig. 7.3.1. As the normalization factor appears at both sides of the balance equations, we write $p(i, j) = \rho_1^i \rho_2^j$ for ease. Clearly, the balance equations hold for $i, j \geq 1$.

$$\begin{aligned} \text{rate out} &= (\lambda + \mu_1 + \mu_2) p(i, j) \\ &= \lambda \rho_1^i \rho_2^j + \mu_1 \rho_1^i \rho_2^j + \mu_2 \rho_1^i \rho_2^j \\ &= \mu_2 \rho_1^i \rho_2^{j+1} + \lambda \rho_1^{i-1} \rho_2^j + \mu_1 \rho_1^{i+1} \rho_2^{j-1} \\ &= \mu_2 p(i, j+1) + \lambda p(i-1, j) + \mu_1 p(i+1, j-1) \\ &= \text{rate in.} \end{aligned}$$

It remains to check the boundary⁶ to complete the claim.

Ex 7.3.1. For the $M/M/1$ queue, show that if $L(D_{k-1}) = 0$ and $S_k \sim \text{Exp}(\mu)$, the density of the rv $D_k - D_{k-1}$ is

$$f(t) = \frac{\lambda \mu}{\mu - \lambda} (e^{-\lambda t} - e^{-\mu t}).$$

Ex 7.3.2. For the $M/M/1$ queue, show that the inter-departure times $D_k - D_{k-1} \sim \text{Exp}(\lambda)$.

⁵ Note, $M/M/c$ queues, not just $M/M/1$ queues.

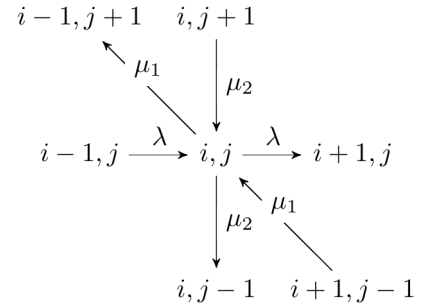


Fig. 7.3.1: Transitions in two stations in tandem.

⁶ [7.3.4]–[7.3.6].

Ex 7.3.3. We have a two-station single-server open network. Jobs enter the network at the first station with rate γ . A fraction α returns from station 1 to itself; the rest moves to station 2. At station 2 a fraction β_2 returns to station 2 again, a fraction β_1 goes to station 1.

Ex 7.3.4. Check that $p(0, 0)$ satisfies (7.3.2).

Ex 7.3.5. Check that $p(i, 0)$ satisfies (7.3.2) for $i > 0$.

Ex 7.3.6. Check that $p(0, j)$ satisfies (7.3.2) for $j > 0$.

7.4 ON $\lambda = \gamma + \lambda P$

In this final section, we concentrate on finding a general condition to ensure that $\lambda = \gamma + \lambda P$ has a solution.

TO START, WE define iteratively $\lambda^0 = 0$, $\lambda^n = \gamma + \lambda^{n-1}P$, for $n \geq 1$. Then, by substitution, we find

$$\lambda^n = \gamma + \lambda^{n-1}P = \gamma + (\gamma + \lambda^{n-2}P)P = \cdots = \gamma \sum_{m=0}^{n-1} P^m,$$

where P^0 is the identity matrix. Since P is a non-negative matrix, P^n is also non-negative for any n , hence, $\sum_{m=0}^n P^m$ is a monotonic increasing sequence (of matrices).⁰ Thus, for $\lambda = \gamma + \lambda P$ to have a (unique) solution, this sum must remain finite as $n \rightarrow \infty$.

Now, observe the similarity with $\sum_{i=0}^n x^i$ with $|x| < 1$. We conclude that this converges to $1/(1-x)$ for $n \rightarrow \infty$. Similarly, if there is an $\epsilon > 0$ such that for all i, j , $0 \leq P_{ij}^m \leq (1-\epsilon)^m$, then $0 \leq \sum_{m=0}^n P_{ij}^m \leq \sum_{m=0}^n (1-\epsilon)^m \leq 1/\epsilon$.

It remains to find a condition to ensure that such ϵ exists. For this, we discuss two ways.

FROM THE PREVIOUS section, we know that $P_{i0} = 1 - \sum_{j=1}^M P_{ij}$ is the probability that a job leaves the network from station i . Next, consider a station k with $P_{ki} > 0$. Then the probability that a job starts at k and leaves the network after a visit to i is $P_{ki}P_{i0} > 0$. Consequently, $P_{k0}^2 = \sum_{j=0}^M P_{kj}P_{j0} \geq P_{ki}P_{i0} > 0$.

More generally, we say that an $M \times M$ matrix P is *transient* when it is possible to leave the network from any station in at most M steps. In other words, for any station j there is a sequence of intermediate stations $j_1, j_2, \dots, j_{M-1}$ such that $P_{j0}^M \geq P_{jj_1}P_{j_1j_2} \cdots P_{j_{M-1}0} > 0$. Using this, it is not hard to prove that $P^n \leq (1-\epsilon)^n$ when P is transient.¹

FOR THE SECOND method, let us make the simplifying assumption that P is a diagonalizable matrix with M different eigenvalues.² In this case, there exists an invertible matrix V with the (left) eigenvectors of P as its rows and a diagonal matrix Λ with the eigenvalues of P on its diagonal such that $V P = \Lambda V$. Hence, pre-multiplying with V^{-1} , $P = V^{-1} \Lambda V$. But then $P^2 = V^{-1} \Lambda V \cdot V^{-1} \Lambda V = V^{-1} \Lambda^2 V$, and in general $P^n = V^{-1} \Lambda^n V$. Clearly, if each eigenvalue has modulus less than 1, then $\Lambda^n \rightarrow 0$ exponentially fast, hence $P^n \rightarrow 0$ exponentially fast.



⁰ Formally, non-decreasing sequence.

¹ [7.4.2].

² The argument below applies just as well to matrices reduced to Jordan normal form, but adds only notational clutter.

So, let us prove that all eigenvalues of a finite, transient routing matrix P have modulus less than 1. For this we use *Gershgorin's disk theorem*. Define the Gershgorin disk of the i th row of the matrix P as the disk in the complex plane:

$$B_i = \left\{ z \in \mathbb{C}; |z - p_{ii}| \leq \sum_{j \neq i} |p_{ij}| \right\}.$$

In words, this is the set of complex numbers that lie within a distance $\sum_{j \neq i} |p_{ij}|$ of the point p_{ii} . Next, assume for notational simplicity that for each row i of P we have that $\sum_j p_{ij} < 1$.³ Then this implies for all i that

$$1 > \sum_{j=1}^M p_{ij} = p_{ii} + \sum_{j \neq i} p_{ij}.$$

Since all elements of P are non-negative, so that $|p_{ij}| = p_{ij}$, it follows that

$$-1 < p_{ii} - \sum_{j \neq i} p_{ij} \leq p_{ii} + \sum_{j \neq i} p_{ij} < 1.$$

With this and using that p_{ii} is a real number (so that it lies on the real number axis), it follows that the disk B_i lies strictly within the complex unit circle $\{z \in \mathbb{C}; |z| \leq 1\}$. As this applies to any row i , the union of the disks $\cup_i B_i$ also lies strictly within the complex unit circle. Now we invoke Gershgorin's theorem, which states that all eigenvalues of the matrix P must lie in $\cup_i B_i$, to conclude that all eigenvalues of P lie strictly in the unit circle, and hence all eigenvalues have modulus smaller than 1.

Ex 7.4.1. What is the routing matrix P for a tandem network with M stations? Show that $P^M = 0$.

Ex 7.4.2. Show that when P is transient, $P^n \leq (1 - \epsilon)^n$ for some $\epsilon > 0$.

Ex 7.4.3. Why does the assumption of [7.4.2] not apply to an infinite transient matrix P ?

³ Otherwise apply the argument to P^M .



HINTS

h.1.1.6. For (1) write $\sum_{k=0}^{\infty} G(k) = \sum_{k=0}^{\infty} \sum_{m=k+1}^{\infty} P\{X = m\}$; reverse the summations. Then note that $\sum_{k=0}^{\infty} \mathbb{1}_{k < m} = m$. For (2):

$$\sum_{i=0}^{\infty} i G(i) = \sum_{n=0}^{\infty} P\{X = n\} \sum_{i=0}^{\infty} i \mathbb{1}_{n \geq i+1},$$

and reverse the summations.

h.1.2.6. Recall that $E[X] = M'_X(0)$, $E[X^2] = M''_X(0)$, and $V[X] = E[X^2] - (E[X])^2$.

h.1.2.7. Use that for independent rvs X, Y , $M_{X+Y}(t) = M_X(t)M_Y(t)$. Why is $M_{A_i}(t) = E[e^{tA_i}] = \prod_{k=1}^i E[e^{tX_k}]$?

h.1.2.8. LOTUS: $E[a^N] = \sum_{k=0}^{\infty} a^k P\{N = k\}$.

h.1.2.9. This result can be anticipated when you think about merging Poisson processes. Then, use $\{\min\{X, S\} > x\} = \{X > x\} \cap \{S > x\}$.

h.1.2.11. Think about splitting Poisson processes, and use that $P\{S > X\} = E[E[\mathbb{1}_{S > X} | X]]$.

h.1.2.13. Dropping the dependence of N on t for the moment for notational convenience, consider the rv

$$Y = \sum_{i=1}^N Z_i,$$

with $N \sim P(\lambda)$ and $Z_i \sim B(p)$ and $\{Z_i\}$ iid. Show that the moment-generating function of Y is equal to the moment-generating function of a Poisson rv with parameter λp .

h.2.1.3. Compute the number of jobs that depart from queue 1. Subtract the capacity used for these jobs from the total capacity to get the remaining capacity for queue 2.

h.2.1.6. If the mean service time is $ES = 2$ minutes, then the service rate per server equals $1/ES$ customers per minute. A reduction of 30 seconds corresponds to subtracting 0.5 minutes from the mean regular service time. Service capacity equals (number of servers) $\times$ (service rate per server).

h.2.1.7. The recursions can be found by introducing an extra variable p_k that specifies the production state of the nurse. If $p_k = 0$, the nurse serves the first queue, if $p_k = 1$ the nurse moves from queue 1 to queue 2, if $p_k = 2$ the nurse serves queue 2. Finally, if $p_k = 3$ the nurse moves from queue 2 to queue 1. Thus, the production state cycles from $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$.

h.2.1.8. We need three queues: L_{1a} for the new cases arriving at the clerks, L_2 for the judge, and L_{1b} for the cases sent by the judge for the second step of the clerks. Next, convert the arrivals to hours of work arriving, and interpret the queue lengths L as work in hours to be served.

h.2.1.9. Modify d_k in (2.1.1) to incorporate the modified service behavior. Then, substitute d_k in the expression for L_k .

h.2.2.6. Introduce a variable $I_k \in \{0, 1\}$ to keep track of the state of the server. Then, $I_{k+1} = \mathbb{1}_{L_k \geq N} + I_k \mathbb{1}_{0 < L_k < N}$ implements the N -policy.

h.2.3.9. When the machine makes n items, and each of these is faulty with probability p , then the number of faulty items is $\text{Bin}(n, p)$.

h.2.3.10. Make a queue for new and repaired items and use [2.1.4].

h.2.3.11. In addition to the regular conditions, we need to impose $c_{k+1}^1 \leq M - L_k^2$.

h.2.4.1. Use recursion. Then apply, in order, $\max\{\max\{a, b\}, c\} = \max\{a, b, c\}$; $L_0 = Z_0$; and $\max\{-a, -b\} = -\min\{a, b\}$.

h.2.4.2. Use the ideas of (1.2.7).

h.3.1.6. Use that $L(A_k-) > 0$ means that the system contains at least one job at the time of the k th arrival, and that $A_k- < D_{k-1}$ means that job k arrives (almost surely) before job $k-1$ departs.

h.3.4.7. To see why, explain that $\max\{k : A_{k-1} \leq t < A_k\} = 1 + A(t)$. Realize also that $\max\{k : A_{k-1} \leq t < A_k\} = \max\{k : A_{k-1} \leq t\}$ because the epochs A_k are ordered and separated in time (a.s.).

h.4.1.6. As a start, the function $\sin(t)$ does not have a limit as $t \rightarrow \infty$. However, the time-average $\sin(t)/t \rightarrow 0$. Now construct a function whose time-average does not converge; it must grow fast or oscillate increasingly wildly.

For the mathematically interested, we seek a function whose Cesàro limit does not exist.

h.4.2.6. Consider a queueing system with constant service times and constant inter-arrival times.

h.4.3.5. Observe that $R(t) = \int_0^t \mathbb{1}_{L(s) > 0} ds$ is the total time the server is busy during $[0, t]$, hence $R(t)/t$ is the fraction of time busy. Then take $T_k = D_k$ as the epochs at which to inspect $R(t)$, and realize that $R(D_k) = \sum_{i=1}^k S_i$.

h.4.3.6. For the direction of the inequalities (4.3.2), observe that t can lie halfway between a service interval, and $A(t) \geq D(t)$. Divide by t in (4.3.2). Then for the LHS, multiply by $A(t)/A(t)$ and take appropriate limits. Similar for the RHS, but multiply by $D(t)/D(t)$.

h.4.3.7. Take the process $R(\cdot)$ such that the system earns money when the server is busy, and sample at moments when busy times start.

h.4.4.6. Recall that $\lim_{t \rightarrow \infty} t^{-1} \int_0^t L_s(s) ds$ is the time-average number of servers busy up to t .

h.4.4.7. What happens if the population increases?

h.4.5.7. Solve [4.5.6] first. Use that $\lambda \geq \delta$ always holds. Thus, when $\lambda \neq \delta$, necessarily $\lambda > \delta$. What are the consequences of this inequality; How does the queue length behave as a function of time?

h.4.6.8. Apply PASTA and (4.6.8).

h.5.1.8. Use (5.1.1), show that $M_L(s) = (1 - \rho) \sum_n e^{sn} \rho^n$, then use the familiar expression of the sum over a geometric series. Similarly, $P\{L \geq n\} = \sum_{k \geq n} p(k)$.

h.5.1.10. Fill in $c = 1$. Realize that this is a check on the formulas.

h.5.1.12. Use that $\sum_{i=0}^n x^i = (1 - x^{n+1})/(1 - x)$.

Is it necessary for this expression to be true that $|x| < 1$? What should you require for $|x|$ when you want to take the limit $n \rightarrow \infty$?

h.5.2.6. $E[Q]$ follows right away from an application of Little's law. For the other quantities, we need to find $E[S]$. Use the expression for $E[W]$ for the queue $M/M/1$ to solve for ρ . Then, since λ is known, $E[S]$ follows.

h.5.2.9. Let T be the number of patties waiting and G the number of persons waiting. There cannot be a patty and a person waiting at the same time, hence $T \cdot G = 0$. Take L as the number of jobs in an $M/M/1$ queue. If $L = 0$, then there are 3 patties on stock, when $L = 1$ there is one patty less $T = 2, G = 0$; when $L = 2$, we have two patties less, so $T = 1, G = 0$, when $L = 3$, there are no patties on stock, but also no people waiting, hence $T = 0, G = 0$. When $L = 4$, we ran out of patties and one person must be waiting, hence $T = 0, G = 1$.

With this reasoning we see a pattern: at all moments in time, $3 = T + L - G$. Now map this system to an $M/M/1$ queue.

h.5.2.10. How is the *invariant* $3 = T + L - G$ affected?

h.5.3.7. $P\{B = k\} = q^{k-1}p$ with $q = 1 - p$. Look up the expectation and variance of a geometric rv.

h.5.3.11. Use [1.1.6], the recursion for $\pi(n)$, $\rho = \lambda_B / \mu \sum_{i=0}^{K_B-1} P\{B > i\}$, and realize that $E[B] \geq 1$ whatever the distribution of B .

h.5.3.12. Read the answer of [1.1.6].

h.5.3.13. Show first that

$$\mu E[L] = \mu \sum_{n=0}^{\infty} n\pi(n) = \lambda_B \frac{E[B^2]}{2} + \lambda_B E[B] E[L] + \lambda_B \frac{E[B]}{2}.$$

h.5.4.7. Use $V[S] = 0$ for the $M/D/1$ queue and $C_s^2 = 1$ for the $M/M/1$ queue.

h.5.4.9. Use the PASTA property. Note that when estimating $E[S_r]$ along a sample path, $S_r = 0$ for jobs that arrive at an empty system.

h.5.5.2. Use conditional probability or [1.2.10] and [1.2.11].

h.5.5.3. Use [5.5.2].

h.5.5.4. Use the results of [5.5.2].

h.5.5.5. Solve [5.5.2] and [5.5.3] first. Use the following shorthands: $\alpha = \lambda/(\lambda + \mu) \implies \mu/(\lambda + \mu) = 1 - \alpha \implies \alpha/(1 - \alpha) = \lambda/\mu = \rho$.

h.5.6.7. Use the argumentation that leads to (4.3.2).

h.5.6.8. Use that $\sum_{n=0}^N \alpha^n = \frac{1-\alpha^{N+1}}{1-\alpha}$.

h.5.6.11.

$$\begin{aligned} aq^2 &= aq^2, \\ bq &= 2aq\mathbb{E}[Y] - 2aq + bq + hq\mathbb{E}[S], \\ 0 &= a\mathbb{E}[Y^2] - 2a\mathbb{E}[Y] + a + b\mathbb{E}[Y] - b + \frac{1}{2}\lambda h\mathbb{E}[S^2]. \end{aligned}$$

h.5.7.8. Use the following properties for convex functions.

1. f convex, $\alpha \geq 0 \implies \alpha f$ convex.
2. f, g convex $\implies f + g$ convex.
3. f convex, $R(x) = -x \implies f \circ R$ convex.

h.5.8.7. No backlogging and $l = 0 \implies r = -1$. Use [5.7.6].

h.5.9.6. Suppose we rename lion and tiger as follows:

```

1 square = exp_apply(2)
2 cube = exp_apply(3)

```

h.5.10.1. How do we handle states at which `is_stop(x) = True` earlier in the algorithm?

h.6.1.6. What is λ ? What is C_a^2 in the $G/G/1$ setting; What is it in the $M/G/1$ setting?

h.6.1.7. Let S be the service (or service) time at the server, and S_T the service time of a type T job. Then,

$$S = \mathbb{1}_{T=1}S_1 + \mathbb{1}_{T=2}S_2.$$

h.6.2.7. An arbitrary job has to wait half the time it takes to form a batch.

h.6.3.6. Get the units right. Compute the load and then the rest.

h.6.3.7. Realize that we now deal with setups and batch processing.

h.6.4.6. Be sure to work in a consistent set of units, e.g., hours. It is easy to make mistakes.

h.6.4.7. Observe that $m_f = 1/\lambda_f$ and $m_r = \mathbb{E}[R]$.

h.6.4.8. Condition on S_0 and N .

h.6.4.9. Just realize that $E[S] = E[S_0] / A$, and use the above.

h.6.4.11. Define suitable moments $T_1 < T_2 < \dots$ to inspect the system, and try to find a suitable function $R(t)$ that captures what we want to measure.

h.7.1.5. Think about the inter-departure time of a machine that produces jobs every 10 minutes. What inter-arrival times will a downstream machine see?

h.7.2.4. Use Little's law.

h.7.2.5. If $w < n$, then what happens? If $w > n$, what are the $w - n$ jobs doing?

h.7.2.9. Is the new machine placed in parallel or in series?

h.7.2.10. Check your reasoning very carefully here. It is easy to do it wrong.

h.7.3.1. Do [5.5.1] first.

h.7.3.2. Conditioning on the server being idle or busy at a departure leads to $f_D(t) = f_{X+S}(t) P\{\text{server is idle}\} + f_S(t) P\{\text{server is busy}\}$. Next, use [7.3.1].

h.7.3.3. First, compute λ , then analyze what happens if $\alpha \rightarrow 1$ or $\beta_1 \rightarrow 0$.

h.7.3.4. Realize that an arrival is required to leave state $(0, 0)$, and a departure at the second queue is necessary to enter state $(0, 0)$.

h.7.4.1. After visiting M stations, any job must have left the tandem network.

h.7.4.2. Define $Q = P^M$. As P is transient, $Q_{ij} < 1$ for all i, j . Then show that $Q^n \rightarrow 0$.

SOLUTIONS

s.1.1.1. False. The value of a changes from 8 to 17 to 10 to 7.

s.1.1.2. True.

s.1.1.3. False. Since the key 1 is not yet in the dictionary a , the code will fail.

s.1.1.4. True. Since we now use a defaultdict, we can add numbers with $+=$ even when the key 1 is not yet present in the dictionary a . As the key 1 is not present, the defaultdict provides the default value 0, after which 3 is added.

s.1.1.5. True. A dictionary maps a *key* to a *value*, for example, a student ID (the key) to a student name (the value).

s.1.1.6. For (1): observe first that $\sum_{k=0}^{\infty} \mathbb{1}_{m>k} = m$, since $\mathbb{1}_{m>k} = 1$ if $k < m$ and $\mathbb{1}_{m>k} = 0$ if $k \geq m$. With this,

$$\begin{aligned} \sum_{k=0}^{\infty} G(k) &= \sum_{k=0}^{\infty} P\{X > k\} = \sum_{k=0}^{\infty} \sum_{m=k+1}^{\infty} P\{X = m\} \\ &= \sum_{k=0}^{\infty} \sum_{m=0}^{\infty} \mathbb{1}_{m>k} P\{X = m\} = \sum_{m=0}^{\infty} \sum_{k=0}^{\infty} \mathbb{1}_{m>k} P\{X = m\} \\ &= \sum_{m=0}^{\infty} m P\{X = m\} = E[X]. \end{aligned}$$

Since all terms are non-negative, interchanging the summations is valid.

For (2):

$$\begin{aligned} \sum_{i=0}^{\infty} i G(i) &= \sum_{i=0}^{\infty} i \sum_{n=i+1}^{\infty} P\{X = n\} = \sum_{n=0}^{\infty} P\{X = n\} \sum_{i=0}^{\infty} i \mathbb{1}_{n \geq i+1} \\ &= \sum_{n=0}^{\infty} P\{X = n\} \sum_{i=0}^{n-1} i = \sum_{n=0}^{\infty} P\{X = n\} \frac{(n-1)n}{2} \\ &= \sum_{n=0}^{\infty} \frac{n^2}{2} P\{X = n\} - \frac{E[X]}{2} = \frac{E[X^2]}{2} - \frac{E[X]}{2}. \end{aligned}$$

s.1.2.1. False. Since $N(0, s] + N(s, t] = N(0, t]$, the LHS just says that there was one arrival during $(0, t]$. The RHS says something different, in particular because $s < t$.

s.1.2.2. True.

s.1.2.3. False, it prints 8. To see why, observe that for $i = 2$, $A[i-1] = A[1] = 1$ and $X[2] = 2$. Thus, for $i = 2$ the RHS in the loop becomes 3. Now there is the $+=$ symbol, which implies that 3 is added to $A[2]$. As the latter is 1, $A[2]$ becomes $1 + 3 = 4$ after the first iteration of the for loop. In the second iteration, we, therefore, find that $4 + 3$ gets added to $A[3] = 1$, so $A[3]$ becomes 8.

s.1.2.4. True. Observe that

$$\mathbb{P}\{A_k \leq t\} - \mathbb{P}\{A_{k+1} \leq t\} = \frac{(\lambda t)^k}{k!} e^{-\lambda t}.$$

s.1.2.5. False, it prints 33, because the algorithm keeps adding 5 to 8 until the threshold 30 is exceeded. It does not print the intermediate values of `tot` because `print(tot)` is not part of the body of the loop.

s.1.2.6. For $t < \lambda$, and $f(t) = \lambda e^{-\lambda t} \mathbb{1}_{t \geq 0}$,

$$M_X(t) = \mathbb{E}[\exp(tX)] = \int_0^\infty e^{tx} f(x) dx = \int_0^\infty e^{tx} \lambda e^{-\lambda x} dx = \frac{\lambda}{\lambda - t}.$$

This last integral only converges when $\lambda > t$. Next, $M'_X(t) = \lambda/(\lambda - t)^2 \implies M'_X(0) = 1/\lambda$, $M''_X(t) = 2\lambda/(\lambda - t)^3 \implies \mathbb{E}[X^2] = 2\lambda^{-2}$. Thus, $\mathbb{V}[X] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 = \frac{2}{\lambda^2} - (\frac{1}{\lambda})^2 = \lambda^{-2}$.

s.1.2.7. Using the iid property of the $\{X_i\}$,

$$M_{A_i}(t) = \mathbb{E}[e^{tA_i}] = \mathbb{E}\left[\exp\left(t \sum_{k=1}^i X_k\right)\right] = \prod_{k=1}^i \mathbb{E}[e^{tX_k}] = \left(\frac{\lambda}{\lambda - t}\right)^i.$$

From a table of moment-generating functions, it follows immediately that $A_i \sim \text{Gamma}((i, \lambda))$, i.e., A_i is Gamma distributed.

s.1.2.8.

$$\sum_{k=0}^{\infty} \alpha^k \mathbb{P}\{N = k\} = \sum_{k=0}^{\infty} \alpha^k \frac{(\lambda)^k}{k!} e^{-\lambda} = e^{-\lambda} \sum_{k=0}^{\infty} \frac{(\alpha\lambda)^k}{k!} = \exp(\lambda(\alpha - 1)).$$

Taking $\alpha = e^s$ in this equation and differentiating gives

$$M'_{N(t)}(s) = \lambda t e^s \exp(\lambda t(e^s - 1)).$$

Hence $\mathbb{E}[N(t)] = M'_{N(t)}(0) = \lambda t$. Next, $M''_{N(t)}(s) = (\lambda t e^s + (\lambda t e^s)^2) \exp(\lambda t(e^s - 1))$, hence $\mathbb{E}[(N(t))^2] = M''_{N(t)}(0) = \lambda t + (\lambda t)^2$, and thus, $\mathbb{V}[N(t)] = \mathbb{E}[(N(t))^2] - (\mathbb{E}[N(t)])^2 = \lambda t + (\lambda t)^2 - (\lambda t)^2 = \lambda t$.

s.1.2.9. $\mathbb{P}\{Z > x\} = \mathbb{P}\{\min\{X, S\} > x\} = \mathbb{P}\{X > x, S > x\} = \mathbb{P}\{X > x\} \mathbb{P}\{S > x\} = e^{-\lambda x} e^{-\mu x}$, since X and S are independent.

s.1.2.10. From (1.2.5) and (1.2.7) and

$$\begin{aligned} \mathbb{P}\{N_\lambda(t) = 1 \mid N_\lambda(t) + N_\mu(t) = 1\} &= \frac{\mathbb{P}\{N_\lambda(t) = 1\} \mathbb{P}\{N_\mu(t) = 0\}}{\mathbb{P}\{N_{\lambda+\mu}(t) = 1\}} \\ &= \frac{\lambda t \exp(-\lambda t) \exp(-\mu t)}{((\lambda + \mu)t \exp(-(\lambda + \mu)t))} = \frac{\lambda t \exp(-(\lambda + \mu)t)}{((\lambda + \mu)t \exp(-(\lambda + \mu)t))} = \frac{\lambda}{\lambda + \mu}. \end{aligned}$$

s.1.2.11. With conditioning, $\mathbb{P}\{S > X \mid X = t\} = \mathbb{P}\{S > t\} = e^{-\mu t}$. With the fundamental bridge and conditional expectation, $\mathbb{E}[\mathbb{1}_{S>X} \mid X] = e^{-\mu X}$, hence $\mathbb{E}[\mathbb{1}_{S>X}] = \mathbb{E}[e^{-\mu X}]$. But this last formula is equal to $M_X(-\mu)$, so taking $t = -\mu$ in the MGF of X we obtain $\lambda/(\lambda + \mu)$.

s.1.2.12.

$$\begin{aligned} M_{N_\lambda(t)+N_\mu(t)}(s) &= M_{N_\lambda(t)}(s) \cdot M_{N_\mu(t)}(s) = \exp(\lambda t(e^s - 1)) \cdot \exp(\mu t(e^s - 1)) \\ &= \exp((\lambda + \mu)t(e^s - 1)). \end{aligned}$$

s.1.2.13.

$$\mathbb{E}[e^{sZ}] = e^0 \mathbb{P}\{Z = 0\} + e^s \mathbb{P}\{Z = 1\} = (1 - p) + e^s p,$$

from which

$$\mathbb{E}[e^{s \sum_{i=1}^n Z_i}] = (\mathbb{E}[e^{sZ}])^n = (1 + p(e^s - 1))^n,$$

where we use that the Z_i are iid. Thus, (see the hint) $\mathbb{E}[e^{sY}|N = n] = \mathbb{E}[e^{s \sum_{i=1}^n Z_i}] = \alpha^n$ where $\alpha = 1 + p(e^s - 1)$. With Adam's law and [1.2.8], we get

$$\mathbb{E}[e^{sY}] = \mathbb{E}[\alpha^N] = e^{\lambda(\alpha - 1)} = \exp(\lambda p(e^s - 1)).$$

This is the MGF of a Poisson rv with rate λp .

s.1.3.1. False. The i th row of A contains the first $T - 1$ number of arrival times for patient i because $A[0] = 0$. Hence, when $T = 8$, the column $A[:, T]$ corresponds to the 7th arrival time of the patients.

s.1.3.2. True.

s.1.3.3. True.

s.1.3.4. False. The function is applied to the string '8', not the number (integer) 8.

s.1.3.5. True. The solution follows directly from standard probability and Section 1.2. Take $U \sim \text{Unif}((0, 1))$, and $f(x) = -\frac{1}{\lambda} \log x$. Then,

$$\mathbb{P}\{f(U) \leq a\} \stackrel{1}{=} \mathbb{P}\{U \geq f^{-1}(a)\} \stackrel{2}{=} 1 - f^{-1}(a) \stackrel{3}{=} 1 - e^{-\lambda a},$$

where 1 follows from the fact that f is strictly decreasing, 2 from the general property of the uniform distribution $\mathbb{P}\{U \geq b\} = 1 - b$, and 3 because $f^{-1}(y) = e^{-\lambda y}$. Thus, if we define $X := f(U)$, then $X = -\frac{1}{\lambda} \log U \sim \text{Exp}(\lambda)$. In other words, the rv $X = -\frac{1}{\lambda} \log U$ is exponentially distributed.

Next, define

$$N(t) = \inf\{j : A_j \leq t < A_{j+1}\},$$

in words, $N(t)$ is the index of the last arrival before or equal to t . Moreover, when $\{U_i\}$ is a sequence of iid uniform rvs on $(0, 1)$ then the $X_i = -\frac{1}{\lambda} \log U_i$ are iid $\sim \text{Exp}(\lambda)$. Since $A_k = A_{k-1} + X_k$, the event

$$\begin{aligned} \{N(t) = k\} &= \{A_k \leq t < A_{k+1}\} \\ &= \left\{ \sum_{i=1}^k X_i \leq t < \sum_{i=1}^{k+1} X_i \right\} \\ &= \left\{ -\frac{1}{\lambda} \sum_{i=1}^k \log U_i \leq t < -\frac{1}{\lambda} \sum_{i=1}^{k+1} \log U_i \right\}. \end{aligned}$$

Now taking $t = 1$, we arrive at the equality:

$$\{N(1) = k\} = \left\{ -\frac{1}{\lambda} \sum_i^k \log U_i \leq 1 < -\frac{1}{\lambda} \sum_i^{k+1} \log U_i \right\}.$$

Since $-\frac{1}{\lambda} \log U \sim \text{Exp}(\lambda)$, the sum of k such rvs is $\text{Gamma}(k, \lambda)$, and, by (1.2.4) and (1.2.5),

$$P\{N(1) = k\} = P\left\{ -\frac{1}{\lambda} \sum_i^k \log U_i \leq 1 < -\frac{1}{\lambda} \sum_i^{k+1} \log U_i \right\} = \frac{\lambda^k}{k!} e^{-\lambda}.$$

The next algorithm implements the above.

```

1 def Pois(labda):
2     R = 0
3     T = - np.log(uniform()) / labda
4     while T < 1:
5         R += 1
6         T += -np.log(uniform()) / labda
7     return R

```

Here we divide every rv `uniform()` by λ , but this is numerically wasteful, which we can avoid by multiplying the threshold by λ .

Note that each uniform rv in the while loop must be a new drawing (i.e., realization of a rv). If not, we keep on adding the same number, which is wrong.

s.1.4.1. False, it prints $40/5 = 8$.

s.1.4.2. False. The line `c[i, j] += p` is incorrect.

Here is a possible variation of this question. Claim: if we apply this function to two independent rvs X and Y and take $f(i, j) = i/j$, we get a RV that represents $Z = X/Y$.

```

1 def map2(X, Y, f):
2     "Make the rv f(X, Y) for the independent rvs X and Y."
3     c = defaultdict(float)
4     for i in X.support():
5         for j in Y.support():
6             p = X.pmf(i) * Y.pmf(j)
7             c[f(i, j)] += p
8     return RV(c)

```

Yet another variation is to write `p = X.pmf(i) / Y.pmf(j)`. This code is incorrect.

s.1.4.3. False when F has jumps.

s.1.4.4. False. Check the index of the second summation.

s.1.4.5. True. The denominators are limited to 1000, so the program looks for the fraction, with a denominator with at most 1000, that is closest to a and b .

s.1.4.6. Type on the Google prompt: 'realpython defaultdict', click on the link that appears. Then read and think hard. Recall that coding involves intellectual effort.

s.2.1.1. True.

s.2.1.2. True.

s.2.1.3. True.

s.2.1.4. True. Queue 2 minimally needs $c_k^2 = \min\{L_{k-1}^2, r^2\}$, and with this,

$$d_k^1 = \min\{L_{k-1}^1, c_k - c_k^2\}, \quad d_k^2 = \min\{L_{k-1}^2, c_k - d_k^1\}.$$

s.2.1.5. False. The code only sets c_k when $L_{k-1} \geq 48$ or $L_{k-1} \leq 24$. When $24 < L_{k-1} < 48$, c_k is not modified, so it retains whatever value it had before.

s.2.1.6.

$$\begin{aligned} d_k^f &= \min\{L_{k-1}^f, \beta(E[S] - 0.5)c_k / E[S]\}, \\ L_k^f &= L_{k-1}^f + \alpha a_k - d_k^f, \\ d_k^r &= \min\{L_{k-1}^r, (1 - \beta)c_k\}, \\ L_k^r &= L_{k-1}^r + (1 - \alpha)a_k - d_k^r, \end{aligned}$$

s.2.1.7. Recall that code like `(3 <= 4)` implements the indicator function. Focus on the logic of for loop, and on how the state of the nurse is encoded. The rest is just code to make a nice figure. BTW, making nice visual reports is an important skill (but not so much for this course).

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from latex_figures import apply_figure_settings
4  from enum import Enum, auto
5
6  apply_figure_settings(use=True)
7
8
9  class State(Enum):
10     A_T0_B = auto()
11     B_T0_A = auto()
12     AT_A = auto()
13     AT_B = auto()
14
15
16  rng = np.random.default_rng(3)
17  thres_A, thres_B = 40, 10
18
19  num_weeks = 100
20  a1 = rng.poisson(3, size=num_weeks)
21  a2 = rng.poisson(4, size=num_weeks)
22  c1 = rng.poisson(7, size=num_weeks)
23  c2 = rng.poisson(8, size=num_weeks)
24
25  LA = np.zeros(num_weeks)
26  LB = np.zeros(num_weeks)
27  LA[0], LB[0] = 30, 50

```

```

28
29 p = np.zeros(num_weeks, dtype='object') # production state
30 p[0] = State.AT_A
31
32 for k in range(1, num_weeks):
33     if p[k - 1] == State.AT_A and LA[k - 1] <= thres_A:
34         p[k] = State.A_T0_B
35     elif p[k - 1] == State.A_T0_B:
36         p[k] = State.AT_B
37     elif p[k - 1] == State.AT_B and LB[k - 1] <= thres_B:
38         p[k] = State.B_T0_A
39     elif p[k - 1] == State.B_T0_A:
40         p[k] = State.AT_A
41     else: # don't change
42         p[k] = p[k - 1]
43
44     d1 = min(LA[k - 1] + a1[k], c1[k] * (p[k] == State.AT_A))
45     d2 = min(LB[k - 1] + a2[k], c2[k] * (p[k] == State.AT_B))
46     LA[k] = LA[k - 1] + a1[k] - d1
47     LB[k] = LB[k - 1] + a2[k] - d2
48
49
50 # Convert cm → inches for Matplotlib
51 cm_to_inch = 1 / 2.54
52 fig_width = 6 * cm_to_inch
53 fig_height = 5.5 * cm_to_inch
54
55 xx = range(num_weeks)
56 plt.figure(figsize=(fig_width, fig_height))
57 plt.step(xx, LA, ls=":", lw=0.7, where="pre", label="LA")
58 plt.step(xx, LB, ls=":", lw=0.7, where="pre", label="LB")
59 plt.xlabel('Time')
60 plt.legend()
61 plt.tight_layout()
62 plt.savefig("/home/nicky/Dropbox/vakken/qts/stochastic_or/figures/nurse.png", dpi=300)

```

s.2.1.8. Here is the code.

```

1 import numpy as np
2 from numpy.random import default_rng
3
4 rng = default_rng(3)
5 num_weeks = 1000
6 ala = rng.poisson(10, num_weeks) # new arriving cases
7 a2 = np.zeros(len(ala))          # arrivals at judge
8 alb = np.zeros(len(ala))          # arriving for a second step at the clerks
9 d1a = np.zeros(len(ala))          # departures from first round at clerks
10 d2 = np.zeros(len(ala))           # departures from judge
11 d1b = np.zeros(len(ala))          # departures after second round at clerks
12 L1a = np.zeros(len(ala))          # amount of work in hours at the first stage at clerks
13 L2 = np.zeros(len(ala))           # amount of work in hours at judge
14 L1b = np.zeros(len(ala))
15 c1 = 80 * np.ones(len(ala))        # weekly hours available of clerks
16 c2 = 34 * np.ones(len(ala))        # hours available of judges
17
18 ES1a = 5                           # work in hours per case at first stage

```

```

19 ES1b = 2                                # work in hours per case at second stage
20 ES2 = 3                                # work of judge per case
21
22
23 for k in range(num_weeks):
24     d1a[k] = min(L1a[k - 1], c1[k])
25     L1a[k] = L1a[k - 1] + a1a[k] * ES1a - d1a[k]
26     a2[k] = d1a[k] // ES1a
27     d2[k] = min(L2[k - 1], c2[k])
28     L2[k] = L2[k - 1] + a2[k] * ES2 - d2[k]
29     a1b[k] = d2[k] // ES2
30     d1b[k] = min(L1b[k - 1], c1[k] - d1a[k])
31     L1b[k] = L1b[k - 1] + a1a[k] * ES1b - d1b[k]
32
33 print(f"{L1b.mean():.2f}")
34 print(f"{L2.mean():.2f}")
35 print(f"{L1a.mean():.2f}")

```

s.2.1.9.

$$\begin{aligned}
 d_k &= \min\{L_{k-1} + a_k, c_k\}, \\
 L_k &= L_{k-1} + a_k - d_k = L_{k-1} + a_k - \min\{L_{k-1} + a_k, c_k\} \\
 &\stackrel{1}{=} -\min\{c_k - L_{k-1} - a_k, 0\} \stackrel{2}{=} \max\{L_{k-1} + a_k - c_k, 0\},
 \end{aligned}$$

where 1 uses that $x - \min\{x, y\} = -\min\{0, y - x\}$ and 2 that $-\min\{x, 0\} = \max\{-x, 0\}$.

s.2.2.1. False. Replace the code at line 2 by: $c2 = c[k] - \max(r1, d1)$. The problem with the code of the question is that the reserved capacity for the first queue is not used in the computation of the capacity available for the second queue.

s.2.2.2. False. The utilization $\rho = \lambda E[S]$ depends only on the arrival rate and the service time, not on N . The N -policy affects the lengths of busy and idle periods, but not the long-run fraction of time the server is busy.

s.2.2.3. False. The N -policy increases waiting times because arriving jobs have to wait while the server is switched off. The purpose of the policy is to reduce switch-on costs, not to reduce waiting time.

s.2.2.4. True. When $L_{k-1} = 0$, the first term is 0 (since $0 < N$) and the second term is $I_{k-1} \cdot 0 = 0$ (since $0 < L_{k-1}$ is false). Hence $I_k = 0$.

s.2.2.5. True. The server switches on when $I_k = 1$ and $I_{k-1} = 0$, so $I_k > I_{k-1}$.

s.2.2.6. Understanding the state variable I_k is the hardest part.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from latex_figures import apply_figure_settings
4
5 apply_figure_settings(use=True)
6

```

```

7  rng = np.random.default_rng(3)
8
9  num_weeks = 100
10 a = rng.poisson(1, size=num_weeks)
11 c = rng.poisson(5, size=num_weeks)
12 N = 20
13
14 L = np.zeros(num_weeks)
15 I = np.zeros(num_weeks) # on or not
16
17 for k in range(1, num_weeks):
18     I[k] = (L[k - 1] >= N) + I[k - 1] * (0 < L[k - 1] < N)
19     c[k] *= I[k]
20     d = min(L[k - 1] + a[k], c[k])
21     L[k] = L[k - 1] + a[k] - d
22
23
24 cm_to_inch = 1 / 2.54
25 fig_width = 6 * cm_to_inch
26 fig_height = 5.5 * cm_to_inch
27 xx = range(num_weeks)
28 plt.figure(figsize=(fig_width, fig_height))
29 plt.step(xx, L, ":", lw=0.7, where="pre", label="L")
30 plt.step(xx, 30 * I, "-", ms=0.5, lw=0.4, where="pre", label="I")
31 plt.xlabel('Time')
32 plt.legend()
33 plt.tight_layout()
34 plt.savefig("/home/nicky/Dropbox/vakken/qts/stochastic_or/figures/N-policy.png", dpi=300)

```

Note that when arrivals cannot be served in the period in which they arrive, it might be that the system is never empty. For instance, if $a_k \sim \text{Unif}((4, 10))$, then $L_k \geq a_k \geq 4$ for all k .

s.2.2.7. The last line of the code is the most interesting. Ask an LLM to explain it to you if you find it hard.

```

1  beta = 0.5
2  K = 50
3  h = 1
4
5
6  cost = beta * I.sum()
7  cost += h * L.sum()
8  cost += K * sum((I[1:] > I[:-1]))

```

s.2.3.1. False. The condition allows $\lambda = \mu$, in which case stochastic queues are not stable.

s.2.3.2. False. We can also block jobs, or reduce the variability of the arrival or service process.

s.2.3.3. False. Since, on average, two jobs arrive per period and one leaves, the queue length increases on average by one job per period. After 1000 periods, the system must contain about 1000 jobs. As the Poisson distribution has a small relative variability (if $X \sim \text{Pois}(\lambda)$, then $\sqrt{X}/(E[X])^2 = 1/\lambda$), the probability that $L_{1000} < 100$ is exceedingly small.

s.2.3.4. True.

s.2.3.5. False. Variability in payment time also affects the queue length.

s.2.3.6. False.

s.2.3.7. True.

s.2.3.8. All jobs that arrive such that the queue becomes larger than K must be dropped.

```

1  import numpy as np
2
3  rng = np.random.default_rng(3)
4  a = rng.integers(3, 8, size=100)
5  c = 2 * np.ones_like(a)
6  L = np.zeros_like(a)
7  l = np.zeros_like(a) # store the lost jobs
8
9  K = 8 # loss level
10
11 for k in range(1, len(a)):
12     d = min(L[k - 1], c[k])
13     Lp = L[k - 1] + a[k] - d # without loss
14     L[k] = min(Lp, K) # chop off at K
15     l[k] = Lp - L[k] # lost
16
17 print(f"sum(l) / sum(a)=:.2f") # fraction lost.
```

s.2.3.9. Here is the code.

```

1  import numpy as np
2
3  rng = np.random.default_rng(3)
4  a = rng.integers(3, 8, size=100)
5  c = 7 * np.ones_like(a)
6  L = np.zeros_like(a)
7  d = np.zeros_like(a)
8  p = 0.1
9
10 for k in range(1, len(a)):
11     produced = min(L[k - 1], c[k])
12     faulty = rng.binomial(produced, p)
13     d[k] = produced - faulty
14     L[k] = L[k - 1] + a[k] - d[k]
15
16 print(f"L.mean()=:.2f")
```

Observe that faulty items remain in the system.

An interesting challenge: Can you use these recursions to *prove* that the long-run average service capacity $n^{-1} \sum_{k=1}^n c_k$ must be larger than $\gamma/(1-p)$, where $\gamma = \lim_{n \rightarrow \infty} n^{-1} \sum_{k=1}^n a_k$ is the arrival rate of new jobs?

s.2.3.10. In code:

```

1  a = [0, 4, 8, 2, 1]
2  c = [3] * len(a)
3  L_R = [0] * len(a) # repair jobs
4  d_R = [0] * len(a) # departing repair jobs
5  L_N = [0] * len(a) # new jobs
6  d_N = [0] * len(a) # departing new jobs
7  p = 0.2
8
9  L_R[0] = 2
10 L_N[0] = 8
11
12 for k in range(1, len(a)):
13     l = L_R[k - 1]
14     if l % 2 == 1:
15         l -= 1
16     d_R[k] = min(l, 2 * c[k])
17     c_N = c[k] - d_R[k] / 2 # capacity left for new jobs
18     d_N[k] = min(L_N[k - 1], c_N)
19     a_R = int(p * d_N[k] + 0.5) # rounding
20     a_N = a[k]
21     L_R[k] = L_R[k - 1] + a_R - d_R[k]
22     L_N[k] = L_N[k - 1] + a_N - d_N[k]
23
24 print(L_R)
25 print(L_N)

```

We need a trick to get around the division by 2 in line 17 (because if it is odd we might end up with 0.5). For this, we should check whether it is odd or not. If so, it must be at least 1, and then we can subtract 1 to make l even. This l can be used to compute the number of departures.

The above code is easily converted to mathematics. Lines 13 to 15 might be a bit problematic, but that is also easy. When writing line 16 like this,

$$d_{R,k} = 2 \min\{\lfloor L_{R,k-1}/2 \rfloor, c_k\}, \quad (7.4.1)$$

we obtain the effect of lines 13 to 16 in one step.

As a general observation: there are many ways to organize the repairs. Do we serve them with priority or not, do we bin them and make new items instead, do we do the repairs on another, separate, station? Such differences should be reflected in a simulation model.

s.2.3.11. Using [2.2.1] with $d_k^1 = \min\{L_{k-1}^1, c_k^1, M - L_{k-1}^2\}$ is nearly correct. But what if $L_{k-1}^2 > M$ for the first few periods? Therefore, instead take $d_k^1 = \min\{L_{k-1}^1, c_k^1, [M - L_{k-1}^2]^+\}$.

```

1  a1 = [0, 2, 3, 8, 0, 9]
2  c1 = [3] * len(a1)
3  c2 = [2] * len(a1)
4  L1 = [0] * len(a1)
5  L2 = [0] * len(a1)
6
7  M2 = 10
8
9  if L2[0] > M2:
10     print("The starting value of L2 is too large")
11     exit(0)

```

```

12
13 for k in range(1, len(a1)):
14     d1 = min(L1[k - 1], c1[k], M2 - L2[k - 1])
15     L1[k] = L1[k - 1] + a1[k] - d1
16     d2 = min(L2[k - 1], c2[k])
17     L2[k] = L2[k - 1] + d1 - d2

```

s.2.3.12. The idea is as follows. The dictionary counts the number of jobs that see 0, 1, and so on, in the system.

Here is the code. As an aside, it was hard to make it this simple.

```

1 from collections import defaultdict
2
3 L_count = defaultdict(int)
4
5 a = [0, 2, 5, 1, 2]
6 c = [0, 1, 1, 1, 1]
7
8 d = [0] * len(a)
9 L = [0] * len(a)
10
11 for k in range(1, len(a)):
12     d[k] = min(L[k - 1], c[k])
13     L[k] = L[k - 1] + a[k] - d[k]
14     for i in range(a[k]):
15         L_count[L[k - 1] - d[k] + i] += 1
16
17
18 # normalize
19 tot = sum(L_count.values())
20 L_dist = {k: v / tot for k, v in L_count.items()}
21
22 print(L_count)
23 print(L_dist)

```

s.2.4.1.

$$\begin{aligned}
 L_k - Z_k &= \max\{\max\{L_{k-2} - Z_{k-2}, -Z_{k-1}\}, -Z_k\} \\
 &= \max\{L_{k-2} - Z_{k-2}, -Z_{k-1}, -Z_k\} \\
 &= \max\{L_0 - Z_0, -Z_1, \dots, -Z_k\} \\
 &= \max\{0, -Z_1, \dots, -Z_k\} \\
 &= -\min\{0, Z_1, \dots, Z_k\}.
 \end{aligned}$$

s.2.4.2.

$$\begin{aligned}
 P_m\{Z_k = n\} &= P\{m + N_{\lambda,k} - N_{\mu,k} = n\} = P\{N_{\lambda,k} = n - m + N_{\mu,k}\} \\
 &= \sum_{j=0}^{\infty} P\{N_{\lambda,k} = n - m + j, N_{\mu,k} = j\} = \sum_{j=0}^{\infty} e^{-\lambda k} \frac{(\lambda k)^{n-m+j}}{(n-m+j)!} e^{-\mu k} \frac{(\mu k)^j}{j!} \\
 &= e^{-(\lambda+\mu)k} (\lambda k)^{n-m} \sum_{j=0}^{\infty} \frac{(\lambda \mu k^2)^j}{j!(n-m+j)!}.
 \end{aligned}$$

s.2.5.1. False. It is the inventory position.

s.2.5.2. True.

s.2.5.3. True.

s.2.5.4. True. Variation. Consider the (T, S) model. Then $Q_k = \min\{Q, S - P'_{k-1}\}$ only if k is a multiple of T .

s.2.5.5. False. An order is placed only when the inventory position drops to or below the reorder level s . If the position is above s , no order is placed.

s.2.5.6. In a queueing system, an item is not produced before the demand for it arrives. This is a cost-effective way to produce when items are very expensive, so holding cost is high, or customer specific. In addition, services (which are not products) cannot be put on stock. For instance, keeping an inventory of haircuts seems impossible.

Note that in a queueing system, the server has a finite capacity. In the inventory models above, the production capacity is assumed infinite.

A base-stock model with $s = -1$ is appropriate. When a customer arrives, the server switches on, and keeps on producing until there are no further jobs in queue. Note also that a job in queue corresponds to a customer in backlog.

s.2.5.7. We would prefer all service levels to be zero. If $h \gg b$ for some inventory system, the cost structure expresses that we dislike stock on hand. In other words, $h \gg b$ expresses that we want to produce according to MTO, that is, as a queueing system with zero on-hand inventory.

s.2.5.8. Note the differences between these schemes. In the (s, S) policy, the order level is fixed, but in the case of stochastic demand, the specific period in which an order is triggered is not fixed. The order quantity (which is not the same as the order-up-to level) may vary from period to period. In the (Q, r) policy, the quantity of orders is always a multiple of Q , but the specific period in which a replenishment order is triggered varies in general. The (T, S) policy fixes the period but leaves the order quantity free. The (s, S) policy can achieve the lowest average cost, because it is the least constrained of the policies.

s.2.5.9. Since only the on-hand inventory I_k is available to meet demand, the rules for a loss system become

$$\begin{array}{ll}
 D'_k = \min\{I_k, D_k\} & \text{accepted demand} \\
 L_k = D_k - D'_k & \text{lost demand} \\
 P'_k = P_k - D'_k, & \text{subtract the accepted demand} \\
 I'_k = I_k - D'_k, & \text{subtract the accepted demand.}
 \end{array}$$

Then use the standard rules to compute Q_{k+1} , and update P_{k+1} and I_{k+1} accordingly. The (s, S) and (Q, r) policy do not control demand, so these rules for $D'_k, \dots$ apply then too.

For the cost, it is evident that there is no backlog. However, we should include a term for the fraction of lost demand, for instance, proportional to $n^{-1} \sum_{k=1}^n L_k$.

s.2.5.10. The variable $O_k = 1 - \prod_{i=1}^N \mathbb{1}_{P_{i,k-1} > s_i}$ equals 1 if an order is placed at the end of period $k-1$, otherwise it is 0. Then take $Q_{i,k} = (S_i - P'_{i,k-1})O_k \mathbb{1}_{P'_{i,k-1} \leq c_i}$, and $P_{i,k} = P'_{i,k-1} + Q_{i,k}$. For the cost, $\sum_{k=1}^n (KO_k + K' \sum_{i=1}^N \mathbb{1}_{Q_{k,i} > 0})$.

Suppose we have set $S_i - s_i = 10$ for all items. Then c_i can take 10 values, hence the total number of can-order levels is 10^N . A supermarket has $N \gg 100$ item types (SKUs).

s.2.6.1. The (Q, r) policy with $Q = 10$ and $r = s = 20$. However, this version is correct; the loop in the problem statement is wrong.

```

1  for t in range(1, len(D)):
2      Pprime = P[t - 1] - D[t - 1]
3      Q[t] = Q_size * (Pprime <= s)
4      P[t] = P[t - 1] + Q[t] - D[t]

```

s.3.1.1. True. A simple variation: Claim: with our notation, the following mapping is correct:

$$A(t) : \mathbb{R} \rightarrow \mathbb{N}, \quad \text{time (real number) to number of jobs (integer).}$$

s.3.1.2. False. Suppose $A_3 = 10$ and $A_4 = 20$. Take $t = 15$. Then $\min\{k : A_k \geq 15\} = 4$, since $A_3 < t = 15 < A_4$. However, $\max\{k : A_k \leq t\} = 3$. In fact, at time $t = 15$, 3 jobs arrived, not 4. Here is a simple variation: Claim: the number of arrivals $A(t)$ during $[0, t]$ can be defined as $\min\{k : A_k > t\}$?

s.3.1.3. False.

s.3.1.4. False. The first part is false, the second is true. $A(t)$ is the number of arrivals during $[0, t]$. Suppose that $A(t) = n$. This n th job arrived at time A_n . Thus, $A_{A(t)}$ is the arrival time of the last job that arrived before or at time t . In a similar vein, A_n is the arrival time of the n th job. Thus, the number of arrivals up to time A_n , i.e., $A(A_n)$, must be n .

s.3.1.5. False. $J(A_{A(t)})$ is nonsense. The correct expression is $[J_{A(t)} - (t - A_{A(t)})]^+ = [D_{A(t)} - t]^+$ because $A_{A(t)} + J_{A(t)} = D_{A(t)}$.

s.3.1.6. The claim is evident: for, if the system contains a job when job k arrives, it is not empty. But if it is not empty, then at least the last job that arrived before job k , i.e., job $k-1$, must still be in the system. That is, $D_{k-1} \geq A_k$. A more formal proof proceeds along the following lines. Using that $A(A_k-) = k-1 = D(D_{k-1})$,

$$\begin{aligned}
 L(A_k-) > 0 &\iff A(A_k-) > D(A_k-) \iff A(A_k-) \geq D(A_k) \\
 &\iff k-1 \geq D(A_k) \iff D(D_{k-1}) \geq D(A_k) \iff D_{k-1} \geq A_k,
 \end{aligned}$$

where the last relation follows from the fact that $D(t)$ is a counting process, hence monotone non-decreasing.

s.3.1.7. Job k arrives at time $t = A_k$. Its sojourn time is J_k . If no further jobs arrive, the system will be empty at time $D_k = A_k + J_k$. Therefore, the work in the system at that moment is $J_k = D_k - t = D_k - A_k$.

s.3.1.8. $s_k = \operatorname{argmin}_{i \in \{1, \dots, c\}} \{w_k(i) + S_k / \mu(i)\}$.

s.3.2.1. False. All three statements are true, so it is not the case that exactly one is false. The first two statements are evident. For the third, K is the capacity of the system, not the queue capacity. Hence, the queue cannot contain more than $K - c$ jobs.

s.3.2.2. False. Service times are iid and exponential.

s.3.2.3. False. The service for business customers follows a priority queue. Therefore, one of the $c + 1$ servers, as perceived by the economy customers, works differently.

s.3.2.4. True. In Kendall's notation, M stands for memoryless (Poisson/exponential), G for general, and 1 for a single server.

s.3.2.5. False. The c denotes the number of servers. The K denotes the system capacity, i.e., the maximum number of jobs allowed in the system.

s.3.3.1. False. The correct quantity is $D[1:] . \text{mean}()$.

s.3.3.2. True.

s.3.3.3. False, it neglects the dependence between A_k and D_{k-1} .

s.3.3.4. False. It picks the server with the smallest waiting time. When the service rates are different, the sojourn time is also affected by the speed of the server.

s.3.3.5. False. This recursion is only valid for the $G/G/1$ queue with a single server. For $c > 1$ servers, we must track the workload at each server and assign the job to the server that becomes free earliest.

s.3.4.1. True.

s.3.4.2. False. It prints the entry with 'Zeno', because 1.3 is the smallest first element among the tuples.

```
1 (1.3, 'Zeno')
```

s.3.4.3. False. A min-heap pops the element with the *smallest* key first. This is why we use heap queues in discrete-event simulation: the event with the earliest time is processed first.

s.3.4.4. True. When $L(T) = 0$, $\int_0^T L(s) ds = \sum_{k=1}^{A(T)} J_k$, so the time-average follows from the sojourn times and the simulation horizon T .

s.3.4.5. False. Events are processed in order of their scheduled *time*, not their creation order. This is precisely why a heap queue is used: it efficiently retrieves the event with the earliest time.

s.3.4.6. First, compute the sum of the sojourn times of all jobs that departed before t ; in a formula $\sum_{k=1}^n J_k \mathbb{1}_{D_k \leq t}$. Then add to this the times of the jobs arrived before t but not yet departed: $\sum_{k=1}^n (t - A_k) \mathbb{1}_{A_k \leq t < D_k}$. Divide the total by t .

s.3.4.7.

$$\max\{k : A_{k-1} \leq t < A_k\} = \max\{A_{k-1} \leq t\} = \sum_{k=1}^{\infty} \mathbb{1}_{A_{k-1} \leq t} = \sum_{k=0}^{\infty} \mathbb{1}_{A_k \leq t} = 1 + A(t).$$

s.3.5.1. False. Compare with the correct code.

s.3.5.2. True.

An interesting variation. Claim: the job records the number of free servers as seen just prior to arrival. This claim is false because the code first pops a server from the pool of free servers, and then sets the `job.free_servers` attribute.

s.3.5.3. True.

s.3.5.4. False. In discrete-event simulation, time jumps from one event to the next. The time increments are variable and determined by the event schedule.

s.3.5.5. True. The base class implements the general event loop and common logic, while subclasses override methods like `handle_arrival` to model specific queueing policies.

s.3.5.6. Read the pages on RealPython.

s.3.5.7. Give this string to an LLM: ‘What is the advantage of a python deque over a heapq. Discuss the algorithmic efficiency, too.’

s.3.5.8. In this case, there are servers idling while there are still customers in queue. If such events occur, the system is not work-conserving.

s.3.6.1. False. Compare with the example code.

s.3.6.2. False. It waits until the system contains `thres_N` plus one jobs.

s.3.6.3. False. Compare with the provided code in the main text. There is no check on the availability of the business servers.

s.3.6.4. False. A common policy is to assign the arriving job to the server that becomes free earliest, i.e., the one with the smallest workload. This minimizes the waiting time, not the remaining service time.

s.3.6.5. True, provided the system is stable. Higher priority jobs are served before lower priority jobs, so their expected waiting time and hence sojourn time is shorter.

s.4.1.1. False. In any reasonable queueing system this limit is ∞ .

s.4.1.2. False.

s.4.1.3. False. δ differs from μ .

s.4.1.4. True. By blocking or rejecting jobs, the effective arrival rate can be reduced below the service rate, making the system rate-stable.

s.4.1.5. False. The load can exceed 1; it is the utilization that is at most 1. When $\rho > 1$ the system is not rate-stable.

s.4.1.6. If $A(t) = 3t^2$, then clearly $A(t)/t = 3t$. This does not converge to a limit.

Another example, let the arrival rate $\lambda(t)$ be given as follows:

$$\lambda(t) = \begin{cases} 1 & \text{if } 2^{2k} \leq t < 2^{2k+1} \\ 0 & \text{if } 2^{2k+1} \leq t < 2^{2(k+1)}, \end{cases}$$

for $k = 0, 1, 2, \dots$. Let $A(t) = \lambda(t)t$. Then $A(t)/t$ fails to converge. Of course, these examples are quite pathological, and are not representative of 'real life cases'. (Though this is somewhat vague: What is actually a real-life case?)

s.4.2.1. False.

$$\frac{1}{t} \sum_{k=1}^{A(t)} \mathbb{1}_{W_k \leq x} = \frac{A(t)}{t} \frac{1}{A(t)} \sum_{k=1}^{A(t)} \mathbb{1}_{W_k \leq x} \rightarrow \lambda \mathbb{P}\{W \leq x\}.$$

Moreover, the final w in the claim should be x .

s.4.2.2. True.

s.4.2.3. False. These are equal only under special conditions, such as when arrivals form a Poisson process (PASTA). As a counterexample, with constant inter-arrival time $X = 10$ and service time $S = 9$, arrivals always see an empty system, but the time-average number in the system is $9/10$.

s.4.2.4. True. This is the time-average definition of $\mathbb{P}\{L \geq n\}$.

s.4.2.5. True. This is a sample-average over arrivals, so it reflects the customer perspective.

s.4.2.6. Take $L(0) = 0$, $X_k = 10$ and $S_k = 10 - \epsilon$ for some tiny $\epsilon > 0$. Then $L(t) = 1$ almost always. Indeed, $\lim_{t \rightarrow \infty} t^{-1} \int_0^t L(s) ds = 1 - \epsilon/10$. However, $L(A_k -) = 0$ for all k .

s.4.2.7.

$$\mathbb{E}[L] = \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t L(s) ds \neq \lim_{t \rightarrow \infty} \frac{L(t)}{t}.$$

If $L(t) = 1$ for all t , $\mathbb{E}[L] = 1$, yet $L(t)/t \rightarrow 0$.

s.4.3.1. True. The cost increase per failure is c . Thus, this c takes the role of Z in the renewal reward theorem. The time between the failures is $\alpha = 1/\mathbb{E}[X]$, where X is the inter-arrival time between two failures. Therefore, $R = c\alpha = c/\mathbb{E}[X]$.

s.4.3.2. True. See [4.3.7].

s.4.3.3. False. In an (s, S) policy, s denotes the reorder point and S the order-up-to level.

s.4.3.4. False. The quantity α is the rate at which the epochs $\{T_k\}$ occur, which can be any renewal process, not necessarily the arrival process.

s.4.3.5. As $k/T_k \rightarrow \delta$, it follows from the renewal reward theorem that $R(t)/t \rightarrow \delta \mathbb{E}[S]$. By rate stability, $\delta = \lambda$, so that $\rho = \lambda \mathbb{E}[S] = \delta \mathbb{E}[S] = R$, from which the claim follows.

s.4.3.6. For the LHS, as $A(t) \rightarrow \infty$ as $t \rightarrow \infty$,

$$\lim_{t \rightarrow \infty} \frac{1}{t} \sum_{k=1}^{A(t)} S_k = \lim_{t \rightarrow \infty} \frac{A(t)}{t} \frac{1}{A(t)} \sum_{k=1}^{A(t)} S_k = \lim_{t \rightarrow \infty} \frac{A(t)}{t} \cdot \lim_{t \rightarrow \infty} \frac{1}{A(t)} \sum_{k=1}^{A(t)} S_k = \lambda \mathbb{E}[S].$$

Apply similar limits to the RHS. The limit of the middle term gives the fraction of time the server is busy.

s.4.3.7. Take $R(t) = \int_0^t \mathbb{1}_{L_s(s)=1} ds$, so that $R(t)/t \rightarrow \rho$, i.e., the fraction of time there is someone on the server. (We could also take $R(t) = \int_0^t \mathbb{1}_{L(s)>0} ds$, because $L_s(t) = 1 \iff L(t) > 0$.) Writing U_k for the k th busy time and I_k for the k th idle time, we sample $R(\cdot)$ at moments $T_k = \sum_{j=1}^k (I_j + U_j)$, that is, we check $R(\cdot)$ at the end of each busy period. Then, by the definition of $R(\cdot)$, its increase between two sample moments is $Z_k := R(T_k) - R(T_{k-1}) = U_k$. The rate at which we sample the system is $\alpha \approx N(t)/t \approx k/T_k \rightarrow 1/(\mathbb{E}[I] + \mathbb{E}[U])$ as $k \rightarrow \infty$ (by the strong law). And thus, with the renewal reward theorem $\rho = R = \alpha Z = \mathbb{E}[U]/(\mathbb{E}[I] + \mathbb{E}[U])$.

s.4.3.8. Suppose $0 \leq s < S$. The average holding cost for the rectangle with base T and height s is hTs , i.e., h times the area of the rectangle. The cost of the triangular area with base T and height $S - s$ is $hT(S - s)/2$. Thus, the total cost of a cycle is $K + hT(S + s)/2$. Dividing by T gives the time-average cost.

The other case follows by the same reasoning.

s.4.4.1. False.

s.4.4.2. False. First, W_k should be J_k . Second, even after replacing W_k by J_k , it is only valid at times T when the system is empty.

s.4.4.3. True, [4.4.6].

s.4.4.4. False.

s.4.4.5. False. Reducing λ also affects ρ , which in turn affects $\mathbb{E}[J]$. While $\mathbb{E}[L]$ typically decreases, the relationship is not simply proportional because $\mathbb{E}[J]$ depends on λ through ρ .

s.4.4.6. Since the queue is stable, the jobs leave at rate λ , thus each job that enters the box leaves it. The average time a job spends in the box is $\mathbb{E}[S]$. By Little's law, the average number in the box is $\mathbb{E}[L_s] = \lambda \mathbb{E}[S]$.

s.4.4.7. Let us make a very simple model to clarify the problem. We start with an initial population size of A . Then assume that each individual dies after exactly 80 years (and does not live a day longer). Moreover, assume that the population increases by 1% per year. After 79 years, the population size is $A \sum_{k=0}^{79} (1.01)^k$. If we assume (as we do when we apply Little's law) that each year contains the same amount of people, then the number of people per year equals

$$\frac{A}{80} \sum_{k=0}^{79} (1.01)^k = \frac{A}{80} \frac{(1.01)^{80} - 1}{1.01 - 1} = 1.5A.$$

However, after 80 years, only A people die, not $1.5A$. Interestingly, $4000/1.5 = 2700$, which is much closer to 2900 per week.

In conclusion, Little's law must be applied with caution; during periods when the population increases or decreases on average, it does not hold.

s.4.5.1. False. In the $G/M/1$ queue, jobs do not arrive as a Poisson process.

s.4.5.2. True.

s.4.5.3. True.

s.4.5.4. False. First we should use $L_S(A_k-)$ instead of $L_S(A_k)$. Second, by PASTA α must be the load, which is ρ for the $M/G/1$ queue; $1 - \rho$ is the fraction of time the server is idle.

s.4.5.5. False. PASTA requires that arrivals form a Poisson process. For a general $G/G/1$ queue, the arrival process need not be Poisson, so sample and time averages can differ.

s.4.5.6. If $\lambda > \delta$, then $L(t) \rightarrow \infty$. But then there must be a last time s , say, that $L(s) = n + 1$, and $L(t) > n + 1$ for all $t > s$. Hence, after time s there will never be a job that will see the system with n jobs. Thus $A(n, t) = A(n, s)$ for all $t > s$ (recall that $A(n, t)$ can only change at time t , for example when a job arriving at t sees n in the system. But since $L(t) > n + 1$ for $t > s$, this will never happen after s .) As $A(n, t)$ remains finite, $\lim_{t \rightarrow \infty} A(n, t)/t = 0$.

s.4.5.7. The assumptions lead us to conclude that $\lambda > \delta$. As a consequence, the queue length must increase in the long run (jobs come in faster than they leave). Hence, $A(n, t)/t \rightarrow 0$ for all n , and $D(n, t)/t \rightarrow 0$. Consequently, $\pi(n) = \delta(n) = 0$, which is the only sensible reconciliation with (4.6.7).

s.4.5.8. By the memoryless property of the exponentially distributed service times of the $M/M/1$ queue, the remaining service time of a job in service, if any, is $\text{Exp}(\mu)$. Therefore, at an arrival moment, all jobs in the system (whether in service or not) have the same expected duration. Hence, the expected time to spend in queue equals the expected number of jobs in the system times the expected service time of each job, i.e., $E[W] = E[L] E[S]$. Note that we use PASTA to see that the expected number of jobs in the system at an arrival is $E[L]$. For the $M/G/1$ queue, the remaining service time of a job in service (if any) does not have the same distribution as the jobs in the queue. Hence, for the $M/G/1$ queue, the *expected time in queue is not* $E[L] E[S]$.

s.4.6.1. True.

s.4.6.2. True.

s.4.6.3. False. The figure sketches the $M/M^2/1/3$ queue.

s.4.6.4. False. Since service times are constant in the $M/D/1$ queue and thus not memoryless, $\mu \neq \mu(n + 1)$ for the $M/D/1$ queue.

s.4.6.5. True. When jobs arrive and depart as single units, the number of up-crossings and down-crossings of any level n can differ by at most one, leading to this equation in the long run.

s.4.6.6. 1. $A_k = 2(k - 1)$, $k = 1, 2, \dots$, as jobs arrive at $t = 0, 2, 4, \dots$. Thus, $A(t) = 1 + \lfloor t/2 \rfloor$ for $t \geq 0$.

(BTW, $A(t)/t \rightarrow 1/2$ as $t \rightarrow \infty$.) We also know that $L(s) = 1$ if $s \in [2k, 2k + 1)$ and $L(s) = 0$ for $s \in [2k - 1, 2k)$ for $k = 0, 1, 2, \dots$. Thus, $L(A_k -) = L(2k -) = 0$. Therefore, $A(0, t) = A(t)$ for $t \geq 0$, and $A(n, t) = 0$ for $n \geq 1$.

1. All arrivals see an empty system. Hence, $A(0, t)/A(t) \approx (t/2)/(t/2) = 1$, and $A(n, t) = 0$ for $n > 0$. Thus, $\pi(0) = \lim_{t \rightarrow \infty} A(0, t)/A(t) = 1$ and $\pi(n) = 0$ for $n > 0$. Recall from the other exercises that $p(0) = 1/2$. Hence, statistics as obtained via time-averages are not necessarily the same as statistics obtained at arrival moments (or any other point process).
2. $\lambda = \lim_{t \rightarrow \infty} A(t)/t = 1/2$. $\lambda(0) = 1$, $p(0) = 1/2$, and $\pi(0) = 1$. Hence,

$$\lambda\pi(0) = \lambda(0)p(0) \implies \frac{1}{2} \times 1 = 1 \times \frac{1}{2}.$$

For $n > 0$, the argument is straightforward: everything is 0.

1. Observe that the system never contains more than 1 job. Hence, $Y(n, t) = 0$ for all $n \geq 2$. Then we see that $Y(1, t) = \int_0^t \mathbb{1}_{L(s)=1} ds$. Now observe that for our queueing system $L(s) = 1$ for $s \in [0, 1)$, $L(s) = 0$ for $s \in [1, 2)$, $L(s) = 1$ for $s \in [2, 3)$, and so on. Thus, when $t < 1$, $Y(1, t) = \int_0^t \mathbb{1}_{L(s)=1} ds = \int_0^t 1 ds = t$. When $t \in [1, 2)$,

$$L(t) = 0 \implies \mathbb{1}_{L(t)=0} \implies Y(1, t) \text{ does not change.}$$

Continuing to $[2, 3)$ and so on gives

$$Y(1, t) = \begin{cases} t & t \in [0, 1), \\ 1 & t \in [1, 2), \\ 1 + (t - 2) & t \in [2, 3), \\ 2 + (t - 4) & t \in [4, 5), \end{cases}$$

and so on. With this, we see that the general formula must be

$$Y(1, t) = \begin{cases} \lfloor t/2 \rfloor + t - \lfloor t \rfloor, & \text{if } \lfloor t \rfloor \text{ is even,} \\ 1 + \lfloor t/2 \rfloor, & \text{if } \lfloor t \rfloor \text{ is odd.} \end{cases}$$

Since $Y(n, t) = 0$ for all $n \geq 2$, $L(s) = 1$ or $L(s) = 0$ for all s , therefore,

$$Y(0, t) = t - Y(1, t).$$

5.

$$\begin{aligned} \lambda(0) &\approx \frac{A(0, t)}{Y(0, t)} \approx \frac{t/2}{t/2} = 1, \\ \lambda(1) &\approx \frac{A(1, t)}{Y(1, t)} \approx \frac{0}{t/2} = 0, \\ p(0) &\approx \frac{Y(0, t)}{t} \approx \frac{t/2}{t} = \frac{1}{2}, \\ p(1) &\approx \frac{Y(1, t)}{t} \approx \frac{t/2}{t} = \frac{1}{2}. \end{aligned}$$

For the rest $\lambda(n) = 0$, and $p(n) = 0$, for $n \geq 2$.

$$6. D(0, t) = \sum_{k=1}^{\infty} \mathbb{1}_{D_k \leq t, L(D_k)=0}.$$

From the graph of $\{L(s)\}$ we see that all jobs leave an empty system behind. Thus, $D(0, t) \approx t/2$, and $D(n, t) = 0$ for $n \geq 1$. With this, $D(0, t)/Y(1, t) \sim (t/2)/(t/2) = 1$, and so,

$$\mu(1) = \lim_{t \rightarrow \infty} \frac{D(0, t)}{Y(1, t)} = 1,$$

and $\mu(n) = 0$ for $n \geq 2$.

7. $\lambda(0)p(0) = 1 \cdot 1/2 = 1/2$, $\lambda(n)p(n) = 0$ for $n > 1$, as $\lambda(n) = 0$ for $n > 0$.

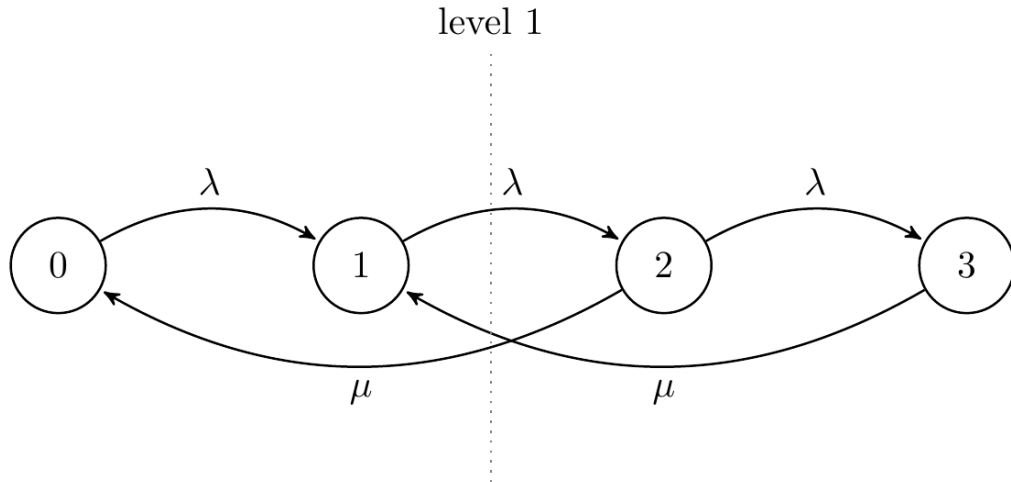
Next, $\mu(1) = 1$, hence $\mu(1)p(1) = 1 \cdot 1/2 = 1/2$. Moreover, $\mu(n) = 0$ for $n \geq 2$. Clearly, for all n we have $\lambda(n)p(n) = \mu(n+1)p(n+1)$.

s.4.6.7. Noting that $L(s) = \sum_{n=0}^{\infty} n \mathbb{1}_{L(s)=n}$, we see that

$$\frac{1}{t} \int_0^t L(s) ds = \frac{1}{t} \int_0^t \left(\sum_{n=0}^{\infty} n \mathbb{1}_{L(s)=n} \right) ds = \sum_{n=0}^{\infty} \frac{n}{t} \int_0^t \mathbb{1}_{L(s)=n} ds = \sum_{n=0}^{\infty} n \frac{Y(n, t)}{t}$$

Taking the limit $t \rightarrow \infty$ and applying (4.5.1) (and reversing the limit and the summation, which we assume is okay here) we obtain that $E[L] = \sum_n n p(n)$.

s.4.6.8. Use the references of the hint to see that $\rho = \sum_{i=1}^{\infty} p(i) = \sum_{i=1}^{\infty} \pi(i) = \sum_{i=1}^{\infty} \delta(i)$.



s.4.6.9.

With level-crossing:

$$\lambda p(0) = \mu p(2), \quad \text{the level between 0 and 1,}$$

$$\lambda p(1) = \mu p(2) + \mu p(3), \quad \text{see level 1,}$$

$$\lambda p(2) = \mu p(3), \quad \text{the level between 2 and 3.}$$

Solving this in terms of $p(0)$ gives $p(2) = \rho p(0)$, $p(3) = \rho p(2) = \rho^2 p(0)$, and

$$\lambda p(1) = \mu(p(2) + p(3)) = \mu(\rho + \rho^2)p(0) = (\lambda + \lambda^2/\mu)p(0),$$

hence $p(1) = p(0)(\mu + \lambda)/\mu$. For the final answer, use the normalization constraint $p(0) + p(1) + p(2) + p(3) = 1$. (As this is simple, we skip it.)

Note that the figure tells what type of transitions (that is, jumps from one state to another) are allowed, and the rate (that is, the inverse of the expected time of the exponential time the system remains in a state). It does not imply that, once a state is entered, the state will be left immediately.

s.4.6.10.

$$\lim_{t \rightarrow \infty} \frac{I(n, t)}{t} = \lim_{t \rightarrow \infty} \frac{A(n-1, t)}{t} + \lim_{t \rightarrow \infty} \frac{D(n, t)}{t} = \lambda(n-1)p(n-1) + \mu(n+1)p(n+1)$$

and

$$\lim_{t \rightarrow \infty} \frac{O(n, t)}{t} = \lim_{t \rightarrow \infty} \frac{A(n, t)}{t} + \lim_{t \rightarrow \infty} \frac{D(n-1, t)}{t} = \lambda(n)p(n) + \mu(n)p(n).$$

s.5.1.1. False. In this queueing model, the arrival rate $\lambda(n)$ can depend on the number of jobs in the queue. If this is the case, the inter-arrival times can still be independent but not identically distributed.

s.5.1.2. True. See [5.1.8].

s.5.1.3. False. For the $M/M/c/c$ queue, $\lambda^+(n) = \lambda \mathbb{1}_{n < c}$, so $\lambda^+(n) = 0$ when $n \geq c$.

s.5.1.4. True.

s.5.1.5. False. This formula holds only for $n \leq c$. For $n > c$, all c servers are busy and the formula changes to $p(n) = p(0) \frac{\lambda^n}{c! c^{n-c} \mu^n}$.

s.5.1.6. The fraction of time the system contains n jobs is $\pi(n)$, by PASTA. When the system contains $n > 0$ jobs, the number in the queue is one less, namely, $n-1$.

$$\mathbb{E}[Q] = \sum_{n=1}^{\infty} (n-1)\pi(n) = \sum_{n=1}^{\infty} n\pi(n) - \sum_{n=1}^{\infty} \pi(n) = \mathbb{E}[L] - (1 - \pi(0)) = \mathbb{E}[L] - \rho.$$

s.5.1.7. Take $\lambda(n) = (N-n)\lambda$ and $\mu(n) = n\mu$. Then

$$\begin{aligned} p(n+1) &= \frac{\lambda(n)}{\mu(n+1)} p(n) = \frac{(N-n)\lambda}{(n+1)\mu} p(n) = \frac{(N-n)(N-(n-1))}{(n+1)n} \frac{\lambda^2}{\mu^2} p(n-1) \\ &= \frac{N!}{(N-(n+1))!} \frac{1}{(n+1)!} \rho^{n+1} p(0) = \binom{N}{n+1} \rho^{n+1} p(0). \end{aligned}$$

Therefore, $G = \sum_{k=0}^N \rho^k \binom{N}{k}$. Now, using that $(a+b)^m = \sum_{k=0}^m \binom{m}{k} a^k b^{m-k}$, we see that $(1+\rho)^N = \sum_{k=0}^N \binom{N}{k} \rho^k 1^{N-k} = G$. So,

$$p(n+1) = \binom{N}{n+1} \frac{\rho^{n+1}}{(1+\rho)^N}.$$

When $\rho = 0$, $p(n) = 0$ for all $n > 1$. When $\rho > 1$, then $(1+\rho)^{-N} \rightarrow 0$ as $N \rightarrow \infty$, but $\binom{N}{n+1} \rightarrow 1/(n+1)!$. Hence, $p(n) \rightarrow 0$ if $N \rightarrow \infty$ for all n .

s.5.1.8.

$$M_L(s) = \mathbb{E}[e^{sL}] = \sum_{n=0}^{\infty} e^{sn} p(n) = (1-\rho) \sum_{n=0}^{\infty} e^{sn} \rho^n = \frac{1-\rho}{1-e^s \rho},$$

where we assume that s is such that $e^s \rho < 1$. Then,

$$M'_L(s) = (1-\rho) \frac{1}{(1-e^s \rho)^2} e^s \rho \implies \mathbb{E}[L] = M'_L(0) = \frac{\rho}{1-\rho},$$

$$\mathbb{E}[L^2] = M''(0) = \frac{2\rho^2}{(1-\rho)^2} + \frac{\rho}{1-\rho} = \rho \frac{1+\rho}{(1-\rho)^2},$$

$$\mathbb{V}[L] = \mathbb{E}[L^2] - (\mathbb{E}[L])^2 = \frac{\rho(1+\rho)}{(1-\rho)^2} - \frac{\rho^2}{(1-\rho)^2} = \frac{\rho}{(1-\rho)^2},$$

$$\mathbb{P}\{L \geq n\} = \sum_{k=n}^{\infty} p(k) = \sum_{k=n}^{\infty} p(0) \rho^k = (1-\rho) \sum_{k=n}^{\infty} \rho^k = (1-\rho) \rho^n \sum_{k=0}^{\infty} \rho^k = (1-\rho) \rho^n \frac{1}{1-\rho} = \rho^n.$$

s.5.1.9. For $n \geq 1$,

$$\begin{aligned}
 p(n) &= \frac{\lambda(n-1)}{\mu(n)} p(n-1) = \frac{\lambda}{\min\{c, n\}\mu} p(n-1) = \frac{1}{\min\{c, n\}} (c\rho) p(n-1) \\
 &= \frac{1}{\min\{c, n\} \min\{c, n-1\}} (c\rho)^2 p(n-2) \\
 &= \frac{1}{\prod_{k=1}^n \min\{c, k\}} (c\rho)^n p(0).
 \end{aligned}$$

To obtain the normalization constant $G = 1/p(0)$,

$$\begin{aligned}
 1 &= \sum_{n=0}^{\infty} p(n) = p(0) + \sum_{n=1}^{c-1} p(n) + \sum_{n=c}^{\infty} p(n) \\
 &= p(0) + p(0) \sum_{n=1}^{c-1} \frac{(c\rho)^n}{n!} + p(0) \sum_{n=c}^{\infty} \frac{c^c}{c!} \rho^n \\
 &= p(0) \sum_{n=0}^{c-1} \frac{(c\rho)^n}{n!} + p(0) \sum_{n=c}^{\infty} \frac{(c\rho)^c}{c!} \rho^{n-c} \\
 &= p(0) \sum_{n=0}^{c-1} \frac{(c\rho)^n}{n!} + p(0) \frac{(c\rho)^c}{c!} \sum_{n=0}^{\infty} \rho^n \\
 &= p(0) \sum_{n=0}^{c-1} \frac{(c\rho)^n}{n!} + p(0) \frac{(c\rho)^c}{c!(1-\rho)}.
 \end{aligned}$$

For $n \geq c$, observe that $p(n) = (c\rho)^n / (c^{n-c} c!) = c^c \rho^n / c!$, so that

$$\begin{aligned}
 E[Q] &= \sum_{n=c}^{\infty} (n-c) p(n) = \sum_{n=c}^{\infty} (n-c) \frac{c^c}{c!} \rho^n p(0) \\
 &= \frac{c^c \rho^c}{G c!} \sum_{n=c}^{\infty} (n-c) \rho^{n-c} = \frac{c^c \rho^c}{G c!} \sum_{n=0}^{\infty} n \rho^n \\
 &= \frac{c^c \rho^c}{G c!} \frac{\rho}{(1-\rho)^2}.
 \end{aligned}$$

The derivation of the expected number of jobs in service becomes easier if we pre-multiply the normalization constant G :

$$\begin{aligned}
 G E[L_s] &= G \left(\sum_{n=0}^c n p(n) + \sum_{n=c+1}^{\infty} c p(n) \right) \\
 &= \sum_{n=1}^c n \frac{(c\rho)^n}{n!} + \sum_{n=c+1}^{\infty} c \frac{c^c \rho^n}{c!} = \sum_{n=1}^c \frac{(c\rho)^n}{(n-1)!} + \frac{c^{c+1}}{c!} \sum_{n=c+1}^{\infty} \rho^n \\
 &= \sum_{n=0}^{c-1} \frac{(c\rho)^{n+1}}{n!} + \frac{(c\rho)^{c+1}}{c!} \sum_{n=0}^{\infty} \rho^n = c\rho \left(\sum_{n=0}^{c-1} \frac{(c\rho)^n}{n!} + \frac{(c\rho)^c}{c!(1-\rho)} \right).
 \end{aligned}$$

Observe that the RHS is precisely equal to $\rho c G$, so that $E[L_s] = c\rho$.

s.5.1.10. Take $c = 1$

$$\begin{aligned}
 p(0) &= \frac{1}{G} \frac{(c\rho)^0}{0!} = \frac{1}{G}, \\
 p(n) &= \frac{1}{G} \frac{c^c \rho^n}{c!} = \frac{1}{G} \frac{1^n \rho^n}{1!} = \frac{\rho^n}{G}, \quad n = 1, 2, \dots \\
 G &= \sum_{n=0}^{c-1} \frac{(c\rho)^n}{n!} + \frac{(c\rho)^c}{(1-\rho)c!} = \sum_{n=0}^0 \frac{\rho^0}{0!} + \frac{\rho}{(1-\rho)} = 1 + \frac{\rho}{1-\rho} = \frac{1}{1-\rho}, \\
 E[L] &= \frac{(c\rho)^c}{c!G} \frac{\rho}{(1-\rho)^2} = \frac{\rho}{1/(1-\rho)} \frac{\rho}{(1-\rho)^2} = \frac{\rho^2}{1-\rho}, \\
 E[L] &= \sum_{n=0}^c np(n) + \sum_{n=c+1}^{\infty} cp(n) = p(1) + 1 \sum_{n=2}^{\infty} p(n) = 1 - p(0) = \rho.
 \end{aligned}$$

Everything agrees with the formulas we derived earlier for the $M/M/1$ queue.

s.5.1.11. With the other results for the $M/M/1/K$ queue,

$$\begin{aligned}
 1 - p(0) &= 1 - \frac{1 - \rho}{1 - \rho^{K+1}} = \frac{1 - \rho^{K+1} - 1 + \rho}{1 - \rho^{K+1}} = \rho \frac{1 - \rho^K}{1 - \rho^{K+1}}, \\
 \rho(1 - p(K)) &= \rho \frac{1 - \rho^{K+1} - \rho^K + \rho^{K+1}}{1 - \rho^{K+1}} = \rho \frac{1 - \rho^K}{1 - \rho^{K+1}}.
 \end{aligned}$$

s.5.1.12. To take the limit $K \rightarrow \infty$ —not the limit $n \rightarrow \infty$ —, write

$$G = \frac{1 - \rho^{K+1}}{1 - \rho} = \frac{1}{1 - \rho} - \frac{\rho^{K+1}}{1 - \rho}.$$

Since $\rho^{K+1} \rightarrow 0$ as $K \rightarrow \infty$ (recall, $\rho < 1$), we get

$$G \rightarrow \frac{1}{1 - \rho},$$

as $K \rightarrow \infty$. Hence, $p(n) = \rho^n / G \rightarrow \rho^n (1 - \rho)$, and the latter are the steady-state probabilities of the $M/M/1$ queue. Finally, if the steady-state probabilities coincide, the performance measures derived from $p(n)$ coincide as well.

s.5.2.1. True.

s.5.2.2. False. This depends on the objective. The service time at a slow server is longer. However, when multiple servers are present, if one server breaks down, service can still continue with the other servers, while when a single fast server fails, everything stops.

s.5.2.3. True.

s.5.2.4. False. Just after jobs balk, the number of jobs in the system is less than K .

s.5.2.5. False. Adding a second server reduces the waiting time, but not by exactly half. The reduction depends nonlinearly on ρ and is typically much more than a factor 2.

s.5.2.6. As people arrive without making appointments, it is reasonable to say that the inter-arrival times are memoryless. As for the services, the employee deals with many different questions, some are short, some are long. So, we also model service times as exponential.

With Little's law:

```

1 labda = 5. # per minute
2 EW = 6.
3 EQ = labda * EW
4 EQ

```

We can use the expression for $E[W]$ to solve for ρ , but we can also do a simple search.

```

1 from scipy.optimize import bisect
2
3 EW = 6.
4
5 def find_W(rho):
6     # return W -1 for given rho
7     ES = rho / labda
8     return rho / (1 - rho) * ES - EW
9
10
11 rho = bisect(find_W, 0, 0.999)
12 rho
13 ES = rho / labda
14 ES
15 J = EW + ES
16 J
17 L = labda * J
18 L

```

Next, find n such that $\sum_{j=1}^n p_j > 0.9$. We start counting at 1 because while the system contains one job, this customer is in service, hence stands at the desk of the employee.

```

1 n, p = 0, 1 - rho
2 total = p
3 while total <= 0.9:
4     p *= rho
5     total += p
6     n += 1
7
8 total
9 n - 1

```

Note that the while loop breaks when n is one too large.

The average queue is huge: hire an extra employee.

s.5.2.7. This is an $M/M/1/3$ queue: there is room for 1 customer in service and two in queue.

We import `numpy` and convert the lists to arrays to fix the output precision to 3, otherwise we get long floats in the output.

First the case with $b = 2$.

```

1 import numpy as np
2 np.set_printoptions(precision=3)
3
4 labda, mu, c = 6.0, 5.0, 1

```

```

5 rho = labda / mu
6 K = c + 2
7
8
9 p = np.array([rho ** n for n in range(K + 1)]) # range(n) is up to n
10 G = sum(p)
11 p /= G # normalize
12 p
13 L = sum(n * p[n] for n in range(len(p)))
14 L
15 Q = sum(max(n - c, 0) * p[n] for n in range(len(p)))
16 Q
17 lost = labda * p[-1] # the last element of p
18 labda - lost # accepted, hence served

```

Now with three extra chairs.

```

1 K += 3
2 K
3 p = np.array([rho ** n for n in range(K + 1)]) # range(n) is up to n
4 G = sum(p)
5 p /= G # normalize
6 lost = labda * p[-1] # the last element of P
7 labda - lost # accepted, hence served

```

We see that since the server is overloaded, the acceptance is not much affected by increasing the number of chairs. We need an extra server.

s.5.2.8. The computations.

```

1 import numpy as np
2 np.set_printoptions(precision=3)
3
4 labda = np.array([10.0, 10.0, 20.0, 20.0, 5.0])
5 mu = np.array([0., 30., 60., 60., 60.])
6 c = 2
7
8 p = np.ones_like(mu)
9 for i in range(len(labda)):
10     p[i+1] = labda[i] * p[i] / mu[i+1]
11
12 p /= p.sum()
13 p
14 labdaBar = sum(labda[n] * p[n] for n in range(len(labda)))
15 labdaBar
16 L = sum(n * p[n] for n in range(len(p)))
17 L
18 Q = sum(max(n - c, 0) * p[n] for n in range(len(p)))
19 Q
20 J = L / labdaBar
21 J
22 W = Q / labdaBar
23 W

```

s.5.2.9. The rv L behaves like the number of jobs in an $M/M/1$ queue. If we have L , then with the two properties $TG = 0$ and $3 = T + L - G$, we can retrieve (uniquely at all moments in time) the number of patties T on stock and the number of people waiting G . In inventory terms, G represents the number of *backlogged* patties.

With this mapping, the expected number of patties on stock is $E[T] = \sum_{l=0}^3 (3-l)p(l)$.

```

1  labda = 12.0 # per hour
2  mu = 15.0 # per hour
3  rho = labda / mu
4  max_patties = 3
5
6  ET, p = 0, 1 - rho
7  for l in range(max_patties):
8      ET += (max_patties - l) * p
9      p *= rho
10
11  ET

```

For the expected number of people waiting, use that $3 = T + L - G$ always. Taking expectations, we see that $3 = E[T] + E[L] - E[G]$, where $E[G]$ is the expected number of people.

```

1  EL = rho/(1-rho)
2  EG = ET + EL - max_patties
3  EG

```

Computing the waiting times is tricky. For the patties, the rate at which ‘jobs’ arrive, is the arrival rate of people. For the people, the rate at which ‘jobs’ arrive is the arrival rate of patties.

```

1  ET/labda # Waiting for patties
2  EG/mu # waiting time for people

```

What would be the impact of allowing 4 patties maximally?

s.5.2.10. The *invariant* $3 = T + L - G$ becomes $4 = T + L - G$.

Estimating the impact of allowing maximally four patties is simple with the code we already have.

```

1  max_patties = 4
2
3  ET, p = 0, 1 - rho
4  for l in range(max_patties):
5      ET += (max_patties - l) * p
6      p *= rho
7
8  ET
9  EG = ET + EL - max_patties
10  EG

```

s.5.2.11. The *invariant* $3 = T + L - G$ becomes $0 = T + L - G$. This implies that $T = 0$ and $L = G$. Hence $E[G] = E[L]$.

s.5.3.1. True. We have for the batches that $\lambda_B E[W^B] = E[Q^B]$. We also find that $\lambda_B = \lambda / E[B]$, and $E[W^B] = E[W]$.

s.5.3.2. True. $V[B] = 0 \implies C_B^2 = 0 \implies$ the first term on the RHS does not change when ρ remains constant. The second term becomes smaller.

s.5.3.3. False. Jobs come in batches; hence, multiple jobs can arrive at the same time, implying that the inter-arrival distribution is no longer exponential.

s.5.3.4. False. The range over m should include n ; now it runs up to (but not including) n .

s.5.3.5. False. When B is deterministic, $C_B^2 = 0$, but the waiting time formula still contains the factor $E[B]$ from the batch structure. Only when $B \equiv 1$ does the formula reduce to the $M/M/1$ result.

s.5.3.6. Start with the simple case. $B \equiv 2 \implies V[B] = 0 \implies C_s^2 = 0$, $\rho = \lambda_B E[B] E[S] = 1 \cdot 2 \cdot 25/60 = 5/6$. Hence,

$$E[L] = \frac{1}{2} \frac{5/6}{1/6} 2 + \frac{1}{2} \frac{5/6}{1/6} = 5 + \frac{5}{2}.$$

Now we have the other case. $E[B^2] = (1 + 4 + 9)/3 = 14/3$. Hence, $V[B] = 14/3 - 4 = 2/3$. Hence, $C_B^2 = \frac{1}{6}$. And thus,

$$E[L] = \frac{1 + 1/6}{2} \frac{5/6}{1/6} 2 + \frac{1}{2} \frac{5/6}{1/6} = \frac{7}{6} 5 + \frac{5}{2}.$$

The ratio between $E[L]$ is 10/9: the average waiting time can be reduced by about 10\%

s.5.3.7. For a geometric rv,

$$E[B] = \frac{1}{p}, \quad V[B] = \frac{1}{p^2} - \frac{1}{p}, \quad C_B^2 = \frac{V[B]}{(E[B])^2} = p^2 \left(\frac{1}{p^2} - \frac{1}{p} \right) = 1 - p,$$

$$(1 + C_B^2)/2 = 1 - p/2,$$

With this, fill in the formula for $E[W]$ to get

$$E[L] = \left(1 - \frac{p}{2}\right) \frac{\rho}{1 - \rho} \frac{1}{p} + \frac{1}{2} \frac{\rho}{1 - \rho} = \frac{\rho}{1 - \rho} \frac{1}{p}.$$

For the $M/M/1$ queue, $E[B] = 1 \implies p = 1 \implies E[L] = \rho / (1 - \rho)$.

If you cannot find the expectation and variance of a geometric rv, then here is the derivation.

$$\begin{aligned} M_B(s) &= E[e^{sB}] = \sum_{k=0}^{\infty} e^{sk} P\{B = k\} \\ &= \sum_{k=0}^{\infty} e^{sk} p q^{k-1} = \frac{p}{q} \sum_{k=0}^{\infty} (q e^s)^k = \frac{p}{q} \frac{1}{1 - q e^s}, \\ E[B] &= M'_B(0) = \frac{p}{q} \frac{q}{(1 - q e^s)^2} \Big|_{s=0} = \frac{p}{(1 - q)^2} = \frac{1}{p}, \\ E[B^2] &= M''_B(0) = \frac{2}{p^2} - \frac{1}{p}, \\ V[B] &= E[B^2] - (E[B])^2 = \frac{2}{p^2} - \frac{1}{p} - \frac{1}{p^2} = \frac{1}{p^2} - \frac{1}{p}, \end{aligned}$$

s.5.3.8. The complete acceptance policy is quite simple. As any batch will be accepted when $n \leq K$, the queue length is not bounded. Only when the number of jobs in the system is larger than K , do we reject jobs.

$$\mu\pi(n+1) = \begin{cases} \lambda_B \sum_{m=0}^n \pi(m)G(n-m), & \text{for } n < K, \\ \lambda_B \sum_{m=0}^K \pi(m)G(n-m), & \text{for } n \geq K. \end{cases}$$

Use the code of [5.3.9] to get a working example.

```

batchqueues.py
1 def complete_acceptance(K):
2     pi, n = {}, 0
3     pi[0] = 1
4     while B.sf(n - K) > 0:
5         pi[n + 1] = sum(pi[m] * B.sf(n - m) for m in range(min(n, K) + 1))
6         pi[n + 1] *= rho_B
7         n += 1
8     return RV(pi)

```

s.5.3.9. For the partial acceptance case, any job is accepted, but the system accepts only what fits. As level $n \in 0, 1, \dots, K-1$ is still up-crossed by any batch of size at least $n-m$ when the system is in state m , the formula for the up-crossing rate is identical to the case without this acceptance policy. Hence, $\mu\pi(n+1) = \lambda_B \sum_{m=0}^n \pi(m)G(n-m)$, for $n = 0, 1, \dots, K-1$.

This is the code.

```

batchqueues.py
1 def partial_acceptance(K):
2     pi, n = {}, 0
3     pi[0] = 1
4     while n < K:
5         pi[n + 1] = sum(pi[m] * B.sf(n - m) for m in range(n + 1))
6         pi[n + 1] *= rho_B
7         n += 1
8     return RV(pi)

```

s.5.3.10. Suppose a batch of size k arrives when the system contains m jobs. When $m+k \leq K$, the batch can be accepted since the entire batch will fit into the queue; otherwise, it will be rejected. Further, level $n, 0 \leq n < K$, can only be crossed when $m+k > n$. Thus, for $n = 0, \dots, K-1$,

$$\begin{aligned} \mu\pi(n+1) &= \lambda_B \sum_{m=0}^n \pi(m) P\{n-m < B \leq K-m\} \\ &= \lambda_B \sum_{m=0}^n \pi(m) [G(n-m) - G(K-m)], \end{aligned}$$

Let us check this for some simple cases. First, when $K \rightarrow \infty$, then $G(K-m) \rightarrow 0$, so we get our earlier result. Second, take $n = K$. Then $G(n-m) - G(K-m) = 0$ for all m , so that the RHS is 0, as it should. Third, take $n = 0$ and $K = 1$, then $\mu\pi(1) = \lambda_B \pi(0)$. This also makes sense.

Use the code of [5.3.9] to get a working example.

```

batchqueues.py
1 def complete_rejection(K):
2     pi, n = {}, 0

```

```

3     pi[0] = 1
4     while n < K:
5         pi[n + 1] = sum(
6             pi[m] * (B.sf(n - m) - B.sf(K - m)) for m in range(n + 1)
7         )
8         pi[n + 1] *= rho_B
9         n += 1
10    return RV(pi)

```

s.5.3.11. The recursion for $\pi(n+1)$ states that

$$\pi(n+1) = \lambda_B / \mu \sum_{i=0}^{K_B-1} \pi(n-i) P\{B > i\}.$$

Start with taking $\pi(0) = 1$, so we consider the unnormalized probabilities. Then, by the recursion, $\pi(1) = \lambda_B / \mu \cdot \pi(0) \leq \rho$, where we use the hint. Since $\pi(1) < \pi(0)$, $\pi(2) \leq \lambda_B / \mu \sum_{i=0}^1 P\{B > i\} \leq \rho$. By similar reasoning, $\pi(n) \leq \rho$ for all $n = 1, \dots, K_B - 1$. This in turn implies that

$$\pi(K_B) = \lambda_B / \mu \sum_{i=0}^{K_B-1} \pi(n-i) P\{B > i\} \leq \lambda_B / \mu \cdot \rho \sum_{i=0}^{K_B-1} P\{B > i\} = \rho^2.$$

Continuing like this we see that $\pi(n) \leq \rho^2$ for all $n = K_B, \dots, 2K_B - 1$. Clearly, this in turn implies that $\pi(n) \leq \rho^3$ for all $n = 2K_B, \dots, 3K_B - 1$, and so on.

s.5.3.12.

$$\sum_{i=1}^{\infty} iG(i-1) = \sum_{i=0}^{\infty} (i+1)G(i) = \sum_{i=0}^{\infty} iG(i) + \sum_{i=0}^{\infty} G(i) = (E[B^2] - E[B])/2 + E[B].$$

s.5.3.13. We use that $\mu\pi(n) = \lambda_B \sum_{i=0}^{n-1} \pi(i)G(n-1-i)$ and the results of the exercises of Section 5.3 to see that

$$\begin{aligned}
 \mu E[L] &= \sum_{n=0}^{\infty} n\mu\pi(n), \quad \text{now substitute for } \mu\pi(n) \text{ the recursion (5.3.5),} \\
 &= \lambda_B \sum_{n=0}^{\infty} n \sum_{i=0}^{n-1} \pi(i)G(n-1-i) = \lambda_B \sum_{n=0}^{\infty} n \sum_{i=0}^{\infty} \mathbb{1}_{i < n} \pi(i)G(n-1-i) \\
 &= \lambda_B \sum_{i=0}^{\infty} \pi(i) \sum_{n=0}^{\infty} \mathbb{1}_{i < n} nG(n-1-i) = \lambda_B \sum_{i=0}^{\infty} \pi(i) \sum_{n=i+1}^{\infty} nG(n-1-i) \\
 &= \lambda_B \sum_{i=0}^{\infty} \pi(i) \sum_{n=0}^{\infty} (n+i+1)G(n) \\
 &= \lambda_B \sum_{i=0}^{\infty} \pi(i) \left[\sum_{n=0}^{\infty} nG(n) + (i+1) \sum_{n=0}^{\infty} G(n) \right] \\
 &= \lambda_B \sum_{i=0}^{\infty} \pi(i) \sum_{n=0}^{\infty} nG(n) + \lambda_B E[B] \sum_{i=0}^{\infty} \pi(i)(i+1) \\
 &= \lambda_B \sum_{i=0}^{\infty} \pi(i) \frac{E[B^2] - E[B]}{2} + \lambda_B E[B] (E[L] + 1) \\
 &= \lambda_B \frac{E[B^2] - E[B]}{2} + \lambda_B E[B] E[L] + \lambda_B E[B] \\
 &= \lambda_B \frac{E[B^2]}{2} + \lambda_B E[B] E[L] + \lambda_B \frac{E[B]}{2}.
 \end{aligned}$$

Dividing both sides by μ and using that $\lambda_B E[B] / \mu = \rho$, we obtain

$$E[L] = \lambda_B \frac{E[B^2]}{2} E[S] + \rho E[L] + \frac{\rho}{2}.$$

By bringing $\rho E[L]$ to the LHS, the RHS becomes equal to (5.3.3).

s.5.4.1. True.

s.5.4.2. True.

s.5.4.3. False. The factor $\rho/(1-\rho)$ makes $E[W]$ a nonlinear, convex function of ρ that diverges as $\rho \uparrow 1$.

s.5.4.4. False. $E[S_r] = \lambda E[S^2]/2$, which depends on the second moment of the service time, not just the mean.

s.5.4.5. True. PASTA is used to conclude that $E[W] = E[S_r] + E[Q] E[S]$, and Little's law to replace $E[Q]$ by $\lambda E[W]$.

s.5.4.6. The code computes the numbers.

```

1  lambda = 3.0 / 8
2  ES = 2
3  rho = lambda * ES
4  ca2 = 1
5  cs2 = 1 / 2
6  EW = (ca2 + cs2) / 2 * rho / (1 - rho) * ES
7  print(f"{EW=}")
8
9  cs2 = 0
10 EW = (ca2 + cs2) / 2 * rho / (1 - rho) * ES
11 print(f"{EW=}")
12
13 ES = 1
14 rho = lambda * ES
15 cs2 = 1 / 3
16 EW = (ca2 + cs2) / 2 * rho / (1 - rho) * ES
17 print(f"{EW=}")

```

```

1  EW=4.5
2  EW=3.0
3  EW=0.4

```

If we were able to reduce all service variability, i.e., $C_s^2 = 0$, then still $E[W] = 3$ h. Hence, capacity must be increased by adding extra machines or $E[S]$ must be reduced by other means.

Alternatively, job arrivals can be planned such that $C_a^2 = 0$. However, typically this is not feasible.

s.5.4.7. $V[S] = 0 \implies C_s^2 = 0 \implies E[W_{M/D/1}] = E[W_{M/M/1}]/2$.

s.5.4.8.

$$\begin{aligned}
E[S] &= \alpha/2, & E[S^2] &= \frac{1}{\alpha} \int_0^\alpha x^2 dx = \alpha^2/3, \\
V[S] &= \alpha^2/3 - \alpha^2/4 = \alpha^2/12, & C_s^2 &= (\alpha^2/12)/(\alpha^2/4) = 1/3, \\
\rho &= \lambda\alpha/2, \\
E[W] &= \frac{1+C_s^2}{2} \frac{\lambda\alpha/2}{1-\lambda\alpha/2} \frac{\alpha}{2}, & E[J] &= E[W] + \frac{\alpha}{2}.
\end{aligned}$$

s.5.4.9. The probability of finding the server busy upon arrival is ρ , and only arrivals that find the server busy observe $S_r > 0$.

$$E[S_r] = \rho E[S_r | S_r > 0] + (1 - \rho) E[S_r | S_r = 0] = \rho E[S_r | S_r > 0].$$

For the $M/M/1$, service times are memoryless, so $E[S_r | S_r > 0] = E[S]$, which implies $E[S_r] = \rho E[S]$ for the $M/M/1$ queue.

s.5.5.1. After job $k-1$ left, job k first has to arrive. Hence, $E[D_k - D_{k-1}] = E[X_k + S_k] = 1/\lambda + E[S]$, where we use that X_k is memoryless.

s.5.5.2. Use conditional probability to see that

$$\begin{aligned}
P\{Y_n = j\} &= \int_0^\infty e^{-\lambda x} \frac{(\lambda x)^j}{j!} dF(x) = \int_0^\infty e^{-\lambda x} \frac{(\lambda x)^j}{j!} \mu e^{-\mu x} dx = \frac{\mu}{j!} \lambda^j \int_0^\infty e^{-(\lambda+\mu)x} x^j dx \\
&= \frac{\mu}{j!} \left(\frac{\lambda}{\lambda+\mu} \right)^j \int_0^\infty e^{-(\lambda+\mu)x} ((\lambda+\mu)x)^j dx = \frac{\mu}{j!} \left(\frac{\lambda}{\lambda+\mu} \right)^j \frac{j!}{\lambda+\mu}.
\end{aligned}$$

Method 2. Consider the Poisson process with rate $\lambda + \mu$, and thin with probability $\mu/(\lambda + \mu)$. Then the probability that j ‘failures’ occur before a ‘success’ is precisely $P\{Y = j\}$.

s.5.5.3. Take $\alpha = \lambda/(\lambda + \mu)$ so that $f(j) = (1 - \alpha)\alpha^j$.

$$G(j) = \sum_{k=j+1}^\infty f(k) = (1 - \alpha) \sum_{k=j+1}^\infty \alpha^k = (1 - \alpha)\alpha^{j+1} \sum_{k=0}^\infty \alpha^k = \alpha^{j+1}.$$

s.5.5.4. This is the most direct, but not the fastest, implementation.

```

1  import numpy as np
2
3  num = 50
4
5
6  def compute_mgl_pi(pmf):
7      cdf = pmf.cumsum()
8      sf = 1 - cdf
9      pi = np.zeros(num)
10     pi[0] = 1
11     for n in range(len(pi) - 1):
12         res = pi[0] * sf[n]
13         res += sum(pi[m] * sf[n + 1 - m] for m in range(1, n + 1))
14         pi[n + 1] = res / pmf[0]
15     pi /= pi.sum()

```

```

16     return pi
17
18
19     labda = 1
20     mu = 2
21     rho = labda / mu
22     pmf = np.ones(num)
23     for i in range(1, len(pmf)):
24         pmf[i] *= pmf[i - 1] * labda / (labda + mu)
25     pmf /= pmf.sum()
26
27
28     pi = compute_mgl_pi(pmf)
29
30     # simple checks
31     print(f"{1 - rho=}, {pi[0]=}")
32     print(f"{(1 - rho) * rho**3=}, {pi[3]=}")

```

```

1 1 - rho=0.5, pi[0]=np.float64(0.4999999999999981)
2 (1 - rho) * rho**3=0.0625, pi[3]=np.float64(0.06249999999999966)

```

s.5.5.5. Observe that $f(j) = (1 - \alpha)\alpha^j$, and $G(j) = \alpha^{j+1}$. As the normalization factor cancels on both sides, we drop the normalization and just write $\pi(n) = \rho^n$ to simplify the algebra.

For $n = 0$: $f(0)\pi(1) = \pi(0)G(0) \iff (1 - \alpha)\rho = 1\alpha$, and this checks with the hint. For $n \geq 1$:

$$\begin{aligned}
 (1 - \alpha)\rho^{n+1} &= \pi(0)G(n) + \sum_{m=1}^n \pi(m)G(n+1-m) = \alpha^{n+1} + \sum_{m=1}^n \rho^m \alpha^{n-m+2} \\
 &= \alpha^{n+1} + \alpha^{n+2} \sum_{m=1}^n (\rho/\alpha)^m = \alpha^{n+1} + \alpha^{n+1} \rho \sum_{m=0}^{n-1} (\rho/\alpha)^m \\
 &= \alpha^{n+1} + \alpha^{n+1} \rho \frac{1 - (\rho/\alpha)^n}{1 - \rho/\alpha} \\
 &= \alpha^{n+1} - \alpha^{n+1}(1 - (\rho/\alpha)^n), \quad \text{as } 1 - \rho/\alpha = -\rho, \\
 &= \alpha^{n+1}(\rho/\alpha)^n = \alpha\rho^n.
 \end{aligned}$$

Since $\rho = \alpha/(1 - \alpha)$ we see that the LHS and RHS are the same.

s.5.6.1. False. It should be this: $d_k = I_k \min\{L_{k-1}, c_k\}$, $L_k = L_{k-1} - d_k + a_k$.

s.5.6.2. True.

s.5.6.3. True. This follows from the fact that the net drain rate is $\mu - \lambda$, and can be verified by substituting $T(q) = aq$ into the recursion (5.6.2).

s.5.6.4. False. The optimal N^* depends on λ , ρ , K , and h , but $\rho = \lambda E[S]$ itself depends on the mean service time. However, N^* does not depend on higher moments of the service time such as C_s^2 .

s.5.6.5. True. With $N = 1$ the server never switches off, and with $K = 0$ there is no setup cost, so the N -policy reduces to the standard $M/G/1$ queue.

s.5.6.6. When we switch on the server, the queue drains at a rate $\mu - \lambda > 0$, with $\mu = 1/\mathbb{E}[S]$. Consequently, no matter how large N , $T(N) < \infty$. Whenever the system is empty, the stochastic process restarts. As such cycles start over and over again, the queue length cannot ‘escape to infinity’.

s.5.6.7. The total number $A(t)$ of jobs that arrive during $[0, t]$ does not depend on N . Thus, in (4.3.2), $\sum_{k=1}^{A(t)} S_k$ does not depend on N . Now use rate-stability.

s.5.6.8. The cost up to the q th job is the cost $W(q-1)$ up to the arrival of job $q-1$ plus the cost while there are $q-1$ jobs in the system. The time between the arrival of job $q-1$ and q is $1/\lambda$. When there are $q-1$ jobs in the system, $h(q-1)/\lambda$ is the expected cost between the arrivals of job $q-1$ and job q . And thus, $W(q) = h\lambda^{-1} \sum_{i=1}^q (i-1)$.

s.5.6.9. Consider an arbitrary moment in time at which $q > 0$ and the server is busy. Now, either of two events happens first: a new job enters the system, or the job in service leaves. The probability that an arrival occurs first is $\alpha = \lambda/(\lambda + \mu)$,⁴ the probability of a departure to occur first is $\beta = 1 - \alpha = \mu/(\lambda + \mu)$. In addition, the expected time for an arrival or a departure, whichever is first, is $1/(\lambda + \mu)$.⁵ In other words, the system stays in state q for an expected time $1/(\lambda + \mu)$ until an arrival or departure occurs. Then, it moves to state $q+1$ or $q-1$, and from there it takes $T(q+1)$ or $T(q-1)$ until the system is empty. This reasoning depends crucially on the memoryless property.

⁴ See [1.2.11].

⁵ See [1.2.9].

s.5.6.10. Note that the queueing cost is hq per unit time when there are q jobs in the system; it costs $hq/(\lambda + \mu)$ until an arrival or departure occurs. For the rest, we follow the reasoning by which we derive the recursion for $T(q)$ for the $M/M/1$ queue.

s.5.6.11. In the hint, the first equation is superfluous. In the second, bq cancels on both sides, by which we find a . The third now follows.

s.5.6.12. For b , using the expressions for $\mathbb{E}[Y]$ and $\mathbb{E}[Y^2]$,

$$\begin{aligned}
 b(1 - \mathbb{E}[Y]) &= a(\mathbb{E}[Y^2] - 2\mathbb{E}[Y] + 1) + \frac{1}{2}h\lambda\mathbb{E}[S^2] \\
 &= \frac{h\mathbb{E}[S]}{2(1 - \mathbb{E}[Y])}(\mathbb{E}[Y^2] - 2\mathbb{E}[Y] + 1) + \frac{1}{2}h\lambda\mathbb{E}[S^2] \\
 &= \frac{h\mathbb{E}[S]}{2(1 - \lambda\mathbb{E}[S])} \left(\lambda^2\mathbb{E}[S^2] + \lambda\mathbb{E}[S] - 2\lambda\mathbb{E}[S] + 1 + \lambda\mathbb{E}[S^2] \frac{1 - \lambda\mathbb{E}[S]}{\mathbb{E}[S]} \right) \\
 &= \frac{h\mathbb{E}[S]}{2(1 - \lambda\mathbb{E}[S])} \left(\lambda^2\mathbb{E}[S^2] - \lambda\mathbb{E}[S] + 1 + \frac{\lambda\mathbb{E}[S^2]}{\mathbb{E}[S]} - \lambda^2\mathbb{E}[S^2] \right) \\
 &= \frac{h\mathbb{E}[S]}{2(1 - \lambda\mathbb{E}[S])} \left(1 + \frac{\lambda\mathbb{E}[S^2]}{\mathbb{E}[S]} - \lambda\mathbb{E}[S] \right) \\
 &= \frac{h\mathbb{E}[S]}{2(1 - \lambda\mathbb{E}[S])} \left(1 + \frac{\lambda\mathbb{E}[S^2]}{\mathbb{E}[S]} - \lambda \frac{(\mathbb{E}[S])^2}{\mathbb{E}[S]} \right) \\
 &= \frac{h\mathbb{E}[S]}{2(1 - \lambda\mathbb{E}[S])} \left(1 + \lambda \frac{\mathbb{V}[S]}{\mathbb{E}[S]} \right) \\
 &= \frac{h\mathbb{E}[S]}{2(1 - \lambda\mathbb{E}[S])} \left(1 + \lambda \frac{\mathbb{V}[S]}{(\mathbb{E}[S])^2} \mathbb{E}[S] \right) \\
 &= \frac{h\mathbb{E}[S]}{2(1 - \lambda\mathbb{E}[S])} (1 + \rho C_s^2).
 \end{aligned}$$

Divide now both sides by $1 - E[Y]$.

s.5.6.13. Note first that $C(N) = N(1/\lambda + E[S]/(1-\rho)) = N/(\lambda(1-\rho))$. Then,

$$\begin{aligned}
 \frac{V(N) + K + W(N)}{C(N)} &= (aN^2 + bN + K + hN(N-1)/2\lambda) \frac{\lambda(1-\rho)}{N} \\
 &= \frac{h}{2}\rho N + \frac{h}{2} \frac{\rho}{1-\rho} (1 + \rho C_s^2) + \frac{h}{2}(N-1)(1-\rho) + K \frac{\lambda(1-\rho)}{N} \\
 &= \frac{h}{2} \frac{\rho}{1-\rho} (1 + \rho C_s^2) + \frac{h}{2}(N-1 + \rho) + K \frac{\lambda(1-\rho)}{N} \\
 &= \frac{h}{2} \frac{\rho}{1-\rho} (\rho + \rho C_s^2 + 1 - \rho) + \frac{h}{2}(N-1 + \rho) + K \frac{\lambda(1-\rho)}{N} \\
 &= \frac{h}{2} \frac{\rho^2}{1-\rho} (1 + C_s^2) + \frac{h}{2}\rho + \frac{h}{2}(N-1 + \rho) + K \frac{\lambda(1-\rho)}{N} \\
 &= \frac{h}{2} \frac{\rho^2}{1-\rho} (1 + C_s^2) + h\rho + h \frac{N-1}{2} + K \frac{\lambda(1-\rho)}{N}.
 \end{aligned}$$

s.5.7.1. False. It should be `(self.IL(S) - self.D).sf(-1)`, so -1 instead of 0 .

s.5.7.2. False. The backlog is $E[(X - S)^+]$, not $E[(S - X)^+]$. The expression $[S - X]^+$ is the on-hand inventory, see (5.7.4).

s.5.7.3. False. Increasing S reduces backlog cost $bE[I^-]$ but increases holding cost $hE[I^+]$. Since $c(S)$ is convex, there is an optimal S^* beyond which the cost increases.

s.5.7.4. True. If S is small relative to the lead-time demand, then $E[I] < 0$, meaning that on average there is a backlog.

s.5.7.5. True. By definition, $I^+ = \max\{I, 0\}$ and $I^- = \max\{-I, 0\}$, so $I^+ - I^- = I$.

s.5.7.6. If $l = 0$, then $X = 0$, hence $f_0 = P\{X = 0\} = 1$. Consequently, $E[I] = S$, and $E[I^-] = 0$ from (5.7.4). Then from (5.7.3) it follows that $E[I^+] = S$.

s.5.7.7. D_k and I_k are independent, but D_k and I'_k not.

s.5.7.8. Fix i . The function $S \rightarrow [S - i]^+$ is convex. Since a weighted sum of convex functions is still convex, $\sum_{i=0}^{\infty} f(i)[S - i]^+$ is also convex. But $\sum_{i=0}^{\infty} f(i)[S - i]^+ = E[(S - X)^+] = E[I^+]$, and therefore $hE[I^+]$ is a convex function of S . Similarly, $S \rightarrow [i - S]^+$ is convex, and therefore $E[I^-]$ is convex. Consequently, $c(S)$, being the sum of two convex functions, is convex. As $E[I^+] \rightarrow \infty$ when $S \rightarrow \infty$, and $E[I^-] \rightarrow \infty$ when $S \rightarrow -\infty$, $c(\cdot)$ attains a minimum.

s.5.8.1. True.

s.5.8.2. False.

s.5.8.3. True. The sum operation in the constructor computes $2 + 2 + 2 = 6$ based on $D = 2$ and $leadtime = 3$.

s.5.8.4. True.

s.5.8.5. False.

s.5.8.6. False. The order cost should be $K * D.\text{mean}() / Q$, not $K * Q / D.\text{mean}()$, since the order frequency is $E[D]/Q$.

s.5.8.7. When $l = 0$, we have $E[X] = 0$ so that for the base-stock model, $E[I^+(i)] = i$. Hence, for the (Q, r) model, $E[I^+] = Q(Q-1)/2Q = (Q-1)/2$. Since demand arrives at a constant rate d , the average cost becomes $Kd/Q + hQ/2 - h/2$. Differentiating with respect to Q and setting the derivative to zero yields the result.

s.5.9.1. False. For the (s, S) policy the recursion is circular in x and cannot be solved by backward induction in time, and therefore needs the extra factor $1 - f_0$; this is explicitly contrasted with the (T, S) policy.

s.5.9.2. False. It is the time to hit the set $\{i : i \leq s\}$, not the time to hit s .

s.5.9.3. True.

s.5.9.4. False. See the comment to see which line is wrong.

```

1 def hitting_value(self, *, running_cost, stopping_cost):
2     is_stop = lambda s, x: x <= s
3
4     @cache
5     def v(s, S):
6         if is_stop(s, S):
7             return stopping_cost(S)
8         res = running_cost(S)
9         res += self.D.E(lambda i: 0.0 if i == 0 else v(s, S - i))
10        return res # this line is wrong
11    return v

```

s.5.9.5. False. It should be $L(x)$.

s.5.9.6. Observe that the function `exp_apply` returns an inner function `apply`. This inner function **captures** the variable `exponent` from its surrounding scope. As a result, the returned function remembers the value of `exponent`. For example, in `square = exp_apply(2)`, the function `exp_apply(2)` does not perform any computation yet. Instead, it returns a function that gets assigned to `square` for which the exponent is fixed to 2. Calling the returned function on a number completes the computation:

```

1 print(exp_apply(1)(9)) # 9
2 print(square(5))      # 25 = 5**2
3 print(cube(4))        # 64 = 4**3

```

In general terms: the outer function `exp_apply` specifies the general structure of a computation: it describes the pattern “take a number and raise it to a fixed exponent,” without yet committing to a particular exponent. By calling the outer function and assigning its output to a new name, we specialize this general procedure. The returned function has the same structure, but with the free parameter `exponent` bound to a concrete value. In this way, we obtain a new, more specific function from a general computational template.

Look how elegant the `sqrt` function can be defined.

```
1 sqrt = exp_apply(0.5)
```

A compact version of `exp_apply` would be like this.

```
1 def exp_apply(exponent):
2     return lambda base: base**exponent
```

Hopefully you find such ideas interesting: the concepts illustrated in this exercise are called *functional programming*, *currying*, and *closures*.

s.5.10.1. When a state x is in the stopping set, it has been visited once. We need to count this visit because there can be a cost associated with stopping at this x .

s.6.1.1. True.

s.6.1.2. True. Suppose for ease that the scvs do not change when we reduce the load ρ by 5%, and that we achieve this load by reducing λ by 5% so that $E[S]$ remains the same as well. Then the only factor that changes in Sakasegawa's formula is $\rho/(1-\rho)$. Now, when $\rho = 0.95$, this factor is $0.95/(1-0.95) \approx 20$. When ρ becomes 0.9, then $\rho/(1-\rho) = 0.9/(1-0.9) \approx 10$. Since 10 is half of 20, the reduction in waiting time is roughly a factor 2.

s.6.1.3. True. If the queue is not stable, $E[W] = \infty$.

s.6.1.4. $\rho = \lambda E[S] = 1/60 \cdot 50 = 5/6$. Since job arrivals do not overlap any job service, the number of jobs in the system is 1 for 50 seconds, then the server is idle for 10 seconds, and so on. Thus, $E[L] = 1 \cdot 5/6 = 5/6$. There is no variance in the inter-arrival times, and also not in the service times, thus $C_a^2 = C_s^2 = 0$. Also $E[W] = 0$ since $E[Q] = 0$.

s.6.1.5. Again, $E[S]$ is 50 seconds, so that $\rho = 5/6$. Also, $C_a^2 = 0$. For C_s^2 , we have to do some work.

$$\begin{aligned} E[S] &= \frac{20}{2} + \frac{80}{2} = 50 \\ E[S^2] &= \frac{400}{2} + \frac{6400}{2} = 3400 \\ V[S] &= E[S^2] - (E[S])^2 = 3400 - 2500 = 900 \\ C_s^2 &= \frac{V[S]}{(E[S])^2} = \frac{900}{2500} = \frac{9}{25}. \end{aligned}$$

s.6.1.6. First, the $G/G/1$ case. Observe that in this case, the inter-arrival time $X \sim U[3,9]$, that is, never smaller than 3 minutes, and never longer than 9 minutes.

```
1 a = 3.0
2 b = 9.0
3 EX = (b + a) / 2.0 # expected inter-arrival time
4 labda = 1.0 / EX # per minute
5 VA = (b - a) * (b - a) / 12.0
6 CA2 = VA / (EX * EX)
```

```

7 print(f"EX=, {labda=:.4f}, {CA2=:.4f}")
8
9 ES = 5.0
10 VS = 4
11 CS2 = VS / (ES * ES)
12 rho = labda * ES
13 print(f"CS2=:.4f}, {rho=:.4f}")
14
15 W = (CA2 + CS2) / 2.0 * rho / (1.0 - rho) * ES
16 print(f"W=:.4f}")

```

```

1 EX=6.0, labda=0.1667, CA2=0.0833
2 CS2=0.1600, rho=0.8333
3 W=3.0417

```

Now the $M/G/1$ case. In that case $C_a^2 = 1$.

```

1 W = (1.0 + CS2) / 2.0 * rho / (1.0 - rho) * ES
2 print(f"W=:.4f}")

```

```

1 W=14.5000

```

The arrival process with uniform inter-arrival times is much more regular than a Poisson process. In the first case, tool arrivals are spaced in time at least by 3 minutes.

s.6.1.7.

$$\begin{aligned}
 E[S] &= E[\mathbb{1}_{T=1}S_1] + E[\mathbb{1}_{T=2}S_2] \\
 &= E[\mathbb{1}_{T=1}]E[S_1] + E[\mathbb{1}_{T=2}]E[S_2], \quad \text{by the independence of } T, \\
 &= P\{T=1\}E[S_1] + P\{T=2\}E[S_2] \\
 &= pE[S_1] + qE[S_2].
 \end{aligned}$$

For the variance, we need some algebra. Since,

$$\mathbb{1}_{T=1}\mathbb{1}_{T=2} = 0 \text{ and } \mathbb{1}_{T=1}^2 = \mathbb{1}_{T=1},$$

we get

$$\begin{aligned}
 V[S] &= E[S^2] - (E[S])^2 \\
 &= E[(\mathbb{1}_{T=1}S_1 + \mathbb{1}_{T=2}S_2)^2] - (E[S])^2 \\
 &= E[\mathbb{1}_{T=1}S_1^2 + \mathbb{1}_{T=2}S_2^2] - (E[S])^2 \\
 &= pE[S_1^2] + qE[S_2^2] - (E[S])^2 \\
 &= pV[S_1] + p(E[S_1])^2 + qV[S_2] + q(E[S_2])^2 - (E[S])^2 \\
 &= pV[S_1] + p(E[S_1])^2 + qV[S_2] + q(E[S_2])^2 - p^2(E[S_1])^2 - q^2(E[S_2])^2 - 2pqE[S_1]E[S_2] \\
 &= pV[S_1] + qV[S_2] + pq(E[S_1])^2 + pq(E[S_2])^2 - 2pqE[S_1]E[S_2], \quad \text{as } p = 1 - q \\
 &= pV[S_1] + qV[S_2] + pq(E[S_1] - E[S_2])^2.
 \end{aligned}$$

s.6.2.1. True, [6.2.7]

s.6.2.2. False.

s.6.2.3. True. A larger batch means that the setup cost is amortized over more jobs, so the effective service time per job decreases, and with it the load.

s.6.2.4. True. Aggregating B inter-arrival times into one batch inter-arrival time reduces the scv by a factor B .

s.6.2.5. False. Larger batches reduce the time in queue (lower load, lower variability), but increase the time a job waits for its batch to be formed, which is $(B - 1)/(2\lambda_i)$. There is a trade-off.

s.6.2.6. First, check the load.

```

1  labda = 3  # per hour
2  ES0 = 15.0 / 60  # hour
3  ER = 2.0
4  Bmin = labda * ER / (1 - labda * ES0)
5  print(f"{ES0=}, {Bmin=}")

```

```

1  ES0=0.25, Bmin=24.0

```

```

1  B = 30
2  ES = ES0 + ER / B
3  rho = labda * ES
4  print(f"{rho=}")

```

```

1  rho=0.95

```

The time to form a red batch is

```

1  labda_r = 0.5
2  EW_r = (B - 1) / (2 * labda_r)
3  print(f"{EW_r=}")  # in hours

```

```

1  EW_r=29.0

```

And the time to form a blue batch is

```

1  labda_b = labda - labda_r
2  EW_b = (B - 1) / (2 * labda_b)
3  print(f"{EW_b=}")  # in hours

```

```
1 EW_b=5.8
```

The time a batch spends in queue.

```
1 CaE = 1.0
2 CaB = CaE / B
3 Ce = 1.0 # scv of service times
4 VS0 = Ce * ES0 * ES0
5 VR = 1.0 * 1.0 # Var setups is sigma squared
6 VS = B * VS0 + VR
7 ESb = B * ES0 + ER
8 CeB = VS / (ESb * ESb)
9 EW = (CaB + CeB) / 2 * rho / (1 - rho) * ESb
10 print(f"{CaB=:.4f}, {CeB=:.4f}, {EW=:.4f}")
```

```
1 CaB=0.0333, CeB=0.0319, EW=5.8833
```

The time to unpack the batch, i.e., the time at the server.

```
1 Eunpack = ER + (B - 1) / 2 * ES0 + ES0
2 print(f"{Eunpack=}")
```

```
1 Eunpack=5.875
```

The overall time red jobs spend in the system.

```
1 total = EW_r + EW + Eunpack
2 print(f"{total=:.4f}")
```

```
1 total=40.7583
```

s.6.2.7. Suppose that a batch is just finished. The first job of a new batch needs to wait, on average, $B - 1$ inter-arrival times until the batch is complete, the second $B - 2$ inter-arrival times, and so on. The last job does not have to wait at all. Thus, the total time to form a batch is $(B - 1)/\lambda_r$. An arbitrary job can be anywhere in the batch, hence its expected time is half the total time.

s.6.2.8. The variance of the inter-arrival time of batches is B times the variance in job inter-arrival times. The inter-arrival times of batches is also B times the inter-arrival times of jobs. Thus,

$$C_{a,B}^2 = \frac{BV[X]}{(BE[X])^2} = \frac{V[X]}{(E[X])^2} \frac{1}{B} = \frac{C_a^2}{B}.$$

s.6.2.9. The variance of a batch is $V[R + \sum_{i=1}^B S_{0,i}] = V[R] + B V[S_0]$, since the normal service times $S_{0,i}, i = 1, \dots, B$, of the jobs are independent, and also independent of the setup time R of the batch.

s.6.2.10. First, wait until the setup is finished, then wait (on average) for half of the batch (minus the job itself) to be served, and then the job has to be served itself, that is, $E[R] + \frac{B-1}{2} E[S_0] + E[S_0]$.

s.6.3.1. False.

s.6.3.2. False. Having fewer but larger breakdowns increases the variance more. This is bad from a queueing perspective.

s.6.3.3. False. Random adjustments add an extra term $p(1-p)(E[R])^2$ to the variance compared to the planned setup case; see (6.3.2) versus (6.2.4). Unplanned interruptions increase variability.

s.6.3.4. False. Fewer but longer adjustments increase the variance of the effective service time, which increases the waiting time through Sakasegawa's formula.

s.6.3.5. True. Each job independently triggers an adjustment with probability p , so the number of jobs between adjustments is geometric with parameter p .

s.6.3.6. First we determine the load.

```

1  EB = 30
2  p = 1 / EB
3  ES0 = 1.5
4  lambda = 9.0 / (2 * 8) # arrival rate per hour
5  ER = 5.0
6  ES = ES0 + p * ER
7  rho = lambda * ES
8  print(f"ES=:.4f", {rho=:.4f})

```

```

1  ES=1.6667, rho=0.9375

```

So, at least the system is stable.

```

1  VS0 = 0.5 * 0.5
2  VR = 2.0 * 2.0
3  VS = VS0 + p * VR + p * (1 - p) * ER * ER
4  Ce2 = VS / (ES * ES)
5  print(f"VS=:.4f", {Ce2=:.4f})

```

```

1  VS=1.1889, Ce2=0.4280

```

And now we can fill in the waiting time formula.

```

1 Ca2 = 1 # Poisson arrivals
2 EW = (Ca2 + Ce2) / 2 * rho / (1 - rho) * ES
3 EJ = EW + ES
4 print(f"EW=:.4f, {EJ=:.4f}")

```

```

1 EW=17.8500, EJ=19.5167

```

s.6.3.7. We can use the model of Section 6.2.

```

1 B = 20
2 ER = 4.5
3 VR = 0
4 ES = ES0 + ER / B
5 rho = lambda * ES
6 VS = VS0 + VR / B
7 Ce2 = VS / (ES * ES)
8 EW = (Ca2 + Ce2) / 2 * rho / (1 - rho) * ES
9 EJ = EW + ES
10 print(f"ES=:.4f, {rho=:.4f}, {Ce2=:.4f}")
11 print(f"EW=:.4f, {EJ=:.4f}")

```

```

1 ES=1.7250, rho=0.9703, Ce2=0.0840
2 EW=30.5586, EJ=32.2836

```

Comparing this to the results of [6.3.6], we see that the load becomes somewhat higher. Since ρ becomes close to one, regular adjustments are not a good idea.

s.6.3.8. Applying Adam's and Eve's rule is straightforward, but the details require attention. Using independence where necessary (and allowed by our assumptions),

$$\begin{aligned}
 E[S|F] &= E[S_0 + RF|F] = E[S_0] + FE[R], \\
 E[S] &= E[E[S|F]] = E[S_0] + E[F]E[R] = E[S_0] + pE[R], \\
 V[E[S|F]] &= V[E[S_0] + FE[R]] = (E[R])^2 V[F] = (E[R])^2 p(1-p), \\
 V[S|F] &= V[S_0 + RF|F] = V[S_0] + FV[R], \\
 E[V[S|F]] &= V[S_0] + pV[R], \\
 V[S] &= E[V[S|F]] + V[E[S|F]] = V[S_0] + pV[R] + p(1-p)(E[R])^2.
 \end{aligned}$$

s.6.4.1. False. The two expectations in the middle cannot be split like this.

s.6.4.2. True. The service time is less variable.

s.6.4.3. False. Since $E[S] = E[S_0] / A$, a lower availability increases the effective service time, and consequently the load $\rho = \lambda E[S]$ increases too.

s.6.4.4. False. Even with deterministic S_0 , the number of failures N and the repair times R_i are random, so S is random.

s.6.4.5. True. From (6.4.5), $C_s^2 = C_0^2 + 2A(1-A)E[R]/E[S_0]$, and the second term is non-negative.

s.6.4.6. Let us first check that $\rho < 1$.

```

1 labda = 4.0
2 ES0 = 10.0 / 60 # in hours
3 labda_f = 1.0 / 3
4 ER = 30.0 / 60 # in hours
5 A = 1.0 / (1 + labda_f * ER)
6 ES = ES0 / A
7 rho = labda * ES
8 print(f"A=:.4f, {ES=:.4f}, {rho=:.4f}")

```

```

1 A=0.8571, ES=0.1944, rho=0.7778

```

```

1 Ca2 = 1.0
2 C02 = 0.0 # deterministic service times
3 Ce2 = C02 + 2 * A * (1 - A) * ER / ES0
4 EW = (Ca2 + Ce2) / 2 * rho / (1 - rho) * ES
5 EJ = EW + ES
6 print(f"{Ca2=}, {EW=:.4f}, {EJ=:.4f}")

```

```

1 Ca2=1.0, EW=0.5903, EJ=0.7847

```

s.6.4.7. The time to fail is the time between two interruptions. By assumption, the failure times are $\text{Exp}(\lambda_f)$, hence $m_f = 1/\lambda_f$. The expected duration of an interruption is $E[R]$. With this

$$A = \frac{m_f}{m_f + E[R]} = \frac{1/\lambda_f}{1/\lambda_f + E[R]} = (1 + \lambda_f E[R])^{-1}.$$

s.6.4.8. When doing the calculations by hand, I worked from bottom to top. However, for the solutions, the current sequence is perhaps easier to understand.

$$\begin{aligned}
 E[S|S_0, N] &= E\left[S_0 + \sum_{i=1}^N R_i | S_0, N\right] = S_0 + NE[R], \text{ since } \{R_i\} \text{ are iid,} \\
 E[S|S_0] &= E[E[S|S_0, N] | S_0] = E[S_0 + NE[R] | S_0] = S_0 + \lambda_f S_0 E[R] = S_0 / A \\
 E[S] &= E[E[S|S_0]] = E[S_0 / A] = E[S_0] / A. \\
 V[S|S_0, N] &= V\left[S_0 + \sum_{i=1}^N R_i | S_0, N\right] = NV[R], \text{ as } V[S_0 | S_0] = 0, \\
 V[S|S_0] &= E[V[S|S_0, N] | S_0] + V[E[S|S_0, N] | S_0] = \lambda_f S_0 V[R] + V[S_0 + NE[R] | S_0] \\
 &= \lambda_f S_0 V[R] + (E[R])^2 V[N | S_0] = \lambda_f S_0 V[R] + (E[R])^2 \lambda_f S_0 = \lambda_f E[R^2] S_0 \\
 V[S] &= E[V[S|S_0]] + V[E[S|S_0]] = \lambda_f E[R^2] E[S_0] + V[S_0] / A^2.
 \end{aligned}$$

s.6.4.9.

$$\begin{aligned}
 C_s^2 &= \frac{V[S]}{(E[S])^2} = \frac{V(S)A^2}{(E[S_0])^2} = \frac{E[S_0^2] + \lambda_f E[R^2] E[S_0] A^2 - (E[S_0])^2}{(E[S_0])^2} \\
 &= \frac{E[S_0^2] - (E[S_0])^2}{(E[S_0])^2} + \frac{\lambda_f E[R^2] E[S_0] A^2}{(E[S_0])^2} = C_0^2 + \frac{\lambda_f E[R^2] A^2}{E[S_0]}.
 \end{aligned}$$

s.6.4.10. When repair times are exponentially distributed with mean $E[R]$, we have $E[R^2] = 2(E[R])^2$. Since $A = 1/(1 + \lambda_f E[R])$,

$$\begin{aligned}\lambda_f E[R^2] A^2 &= 2\lambda_f (E[R])^2 A^2 = 2 \frac{\lambda_f E[R]}{1 + \lambda_f E[R]} A E[R] \\ &= 2 \left(1 - \frac{1}{1 + \lambda_f E[R]} \right) A E[R] = 2(1 - A) A E[R].\end{aligned}$$

s.6.4.11. Take T_k as the end of the k th repair, and $R(t) = \int_0^t u(s) ds$, where $u(s) = 1$ if the server is up (operational) and 0 if the server is down (broken). Thus, $R(t)$ is the total amount of time the system has been up during $[0, t]$, and $R(t)/t$ becomes the fraction of uptime (which is the availability) in the limit. For Z_k , this is $Z_k = R(T_k) - R(T_{k-1})$, which is the uptime in $[T_{k-1}, T_k]$. Therefore, in the limit, Z is the average uptime between two failures, hence $Z \sim m_f$. Finally, the time between two inspections consists of an uptime and a breakdown. Hence, the rate at which such inspection epochs occur is $1/(m_f + E[R])$.

s.7.1.1. True.

s.7.1.2. False. The second station has a utilization of $80/(2 * 60) = 8/12 = 2/3$, while the first has a utilization of $45/60 = 3/4$, which is higher.

s.7.1.3. False. In general, the number of jobs in queue is much higher than $w - N$. Consider the example $S_1 = 10$ and $N_1 = 10$, and $S_2 = 1$, $N_2 = 20$, and $n = 2$. Clearly, 19 machines at station 2 are always empty.

s.7.1.4. False. This is incorrect: the units on the LHS and RHS do not match.

s.7.1.5. When service times at a station are nearly constant and the jobs of this station are sent to a second station for further processing, the inter-arrival times at the second station must be roughly equal. But then the inter-arrival times are not well approximated by the exponential distribution, consequently, the arrival process is not well described by a Poisson process.

s.7.1.6. First station 1.

```

1  lambda = 2.0
2  S1 = 20.0 / 60
3  rho1 = lambda * S1
4  ca1 = 2.0
5  cs1 = 0.5
6  EW1 = (ca1 + cs1) / 2 * rho1 / (1 - rho1) * S1
7  EJ1 = EW1 + S1
8  print(f"EJ1=:.4f")

```

```

1  EJ1=1.1667

```

Now station 2. We first need to compute C_{d1}^2 .

```

1 cd1 = (1 - rho1 ** 2) * ca1 + rho1 ** 2 * cs1
2 print(f"{cd1=:.4f}")
3 labda = 2
4 S2 = 25.0 / 60
5 rho2 = labda * S2
6 ca2 = cd1 # here we use our formula
7 cs2 = 0.5
8 EW2 = (ca2 + cs2) / 2 * rho2 / (1 - rho2) * S2
9 EJ2 = EW2 + S2
10 print(f"{EJ2=:.4f}")
11 print(f"{EJ1 + EJ2=:.4f}")

```

```

1 cd1=1.3333
2 EJ2=2.3264
3 EJ1 + EJ2=3.4931

```

s.7.2.1. In this case, the throughput μ of the network is $1/T_0$: only one server at a time is serving a job, the rest of the servers are forced to be idle as we only allow one job at a time in the network.

s.7.2.2. If there is one job in the system, $\mu(1) = 1/T_0$. If there are two, both jobs can be simultaneously worked on, each at a different station, hence $\mu(2) = 2/T_0$. And so on, until all stations are filled, at the same time, by a job, in which case $\mu(n) = n/T_0$. When there are yet more jobs in the network, only n can be simultaneously in service, the rest must wait at some station.

s.7.2.3.

$$\mu(w) = \frac{n}{T_0} = \frac{n}{nS} = \frac{1}{S}.$$

s.7.2.4.

$$E[W] = \frac{E[L]}{\lambda} = \frac{w}{\mu(w)} = \frac{w}{\min\{w, n\}} T_0 = \begin{cases} T_0 = nS, & \text{if } w \leq n, \\ \frac{w}{n} T_0 = wS, & \text{if } w \geq n. \end{cases}$$

s.7.2.5. When $w > n$, there must be $w - n$ jobs in queue and the other n jobs are in service. So, in conclusion, in deterministic tandem networks adding more than n jobs does not increase the network's throughput, but only increases the waiting times. On the other hand, setting $w < n$ minimizes the waiting times, but also negatively affects the throughput of the network.

s.7.2.6. The processing rates are $\mu_1 = 1/1$ and $\mu_2 = 1/2$ per hour, respectively. The utilizations $\rho_i = \lambda_i/\mu_i$ are $1/3$ and $2/3$, respectively. Thus, machine 2 is the bottleneck: it has the highest utilization.

s.7.2.7. Since these machines are not related by a routing (jobs going from one machine to another), computing T_0 and so on does not make much sense for the network. We can just consider each machine on its own. As we need 2 jobs to keep each machine busy, it follows that $W_0 = 2$.

We can use Little's law to find T_0 . The rate at which jobs can be served is $\mu = 1 + 1/2 = 3/2$. Thus,

$$T_0 = W_0/\mu = 2/(3/2) = 4/3.$$

Clearly, the average time a job spends in this system is $4/3$ time units if jobs arrive at the rate they are served. (Recall, everything is deterministic.)

Can we see this in another way? Yes, because 2 out of 3 jobs have service time of 1, and 1 out of 3 has a service time of 2,

$$\frac{2}{3}1 + \frac{1}{3}2 = \frac{4}{3}.$$

s.7.2.8. The raw processing time is just the sum of the processing times at each station. All stations have the same arrival rate, hence the second machine, being the slowest, is the bottleneck. Hence,

$$\begin{aligned} T_0 &= 2 + 3 + 2 = 7 \\ r_b &= \mu_2 = \frac{1}{3} \\ W_0 &= r_b T_0 = \frac{7}{3}. \end{aligned}$$

s.7.2.9. Realize that machines at a station are in parallel, not in series. Thus, when we add capacity to station 2, we don't add an extra processing step to station 2, we increase capacity.

Now,

$$r_2 = \frac{1}{3} + \frac{1}{3} = \frac{2}{3},$$

so that station 2 is no longer the bottleneck. Station 1 and station 3 are the bottlenecks.

$$\begin{aligned} r_b &= \frac{1}{2} \\ T_0 &= 2 + 3 + 2 = 7 \\ W_0 &= r_b T_0 = \frac{7}{2}. \end{aligned}$$

Often, students think that the processing rate is the inverse of the processing time, i.e., $t = r^{-1}$. This is not the case for multi-server stations. Thus, realize that the raw processing time at station 2 is still 3 hours, not $3/2$.

s.7.2.10. Now,

$$r_2 = \frac{1}{3} + \frac{1}{4} = \frac{7}{12} > \frac{1}{2},$$

hence, stations 1 and 3 are bottlenecks.

However, t_2 cannot be 3 hours anymore: some jobs will spend 4 hours at station 2 rather than 3 hours. In fact, the computation of the raw processing time at station 2, t_2 say, is a bit tricky. Some students reason that both machines serve half of the jobs, so that $t_2 = (3 + 4)/2$ hours. This, however, is not quite realistic. This way, you would load the slow machine all of the time, and the faster machine would be idle quite a bit of the time.

Rather than spreading the jobs evenly over the two machines, we can also assume that each machine takes in work in proportion to its processing rate. Then we reason as follows. Consider 12 hours. In those 12 hours, the slow machine processes 3 jobs with duration 4 and the fast machine processes 4 jobs with duration 3. Therefore:

$$t_2 = \frac{3}{7}4 + \frac{4}{7}3 = \frac{24}{7}.$$

Another way to get this is like this. We can ‘stuff’ station 2 with jobs. The most jobs that fit in simultaneously is 2, one job per machine. Then, with Little’s law,

$$t_2 = \frac{W}{r_2} = \frac{2}{7/12} = \frac{24}{7},$$

i.e., the same as the other answer.

Now, finally,

$$\begin{aligned} r_b &= \frac{1}{2} \\ T_0 &= 2 + \frac{24}{7} + 2 = 4 + 3 + \frac{3}{7} = 7\frac{3}{7} \\ W_0 &= r_b T_0. \end{aligned}$$

Finally, the most realistic assumption is that the fast machine is always working, and the slow machine covers the rest. In that case, suppose that the fast machine processes jobs at rate r_1 with processing time $s_1 = 3$ and the slow at rate r_2 with processing time $s_2 = 4$. Then, since jobs arrive with rate r_a , the fast machine processes a fraction of r_1/r_a of the jobs, and the slow machine processes the leftovers, i.e., a fraction of $(r_a - r_1)/r_a$ of the jobs. The raw processing time then becomes

$$\begin{aligned} t_2 &= \frac{r_1}{r_a} s_1 + \frac{r_a - r_1}{r_a} s_2 \\ &= \frac{1/3}{1/2} 3 + \frac{1/2 - 1/3}{1/2} 4 \\ &= 2 + \frac{1/6}{1/2} 4 \\ &= 2 + \frac{4}{3} = 3\frac{1}{3}, \end{aligned}$$

and then,

$$T_0 = 2 + t_2 + 2 = 7\frac{1}{3},$$

and this is a tiny bit less than the previous way of organizing things.

So, why are the above answers different? One reason is that the objective is not explicitly stated. It might be that, to minimize time in a multi-server station, it is optimal to let a fast machine idle and wait until the next job comes in. If we were to assign this next job to a slow machine, the time in the system may be longer. In general, it might be that schedules that minimize T_0 (on average) are not work-conserving.

s.7.2.11. Because T_0 increases and/or the bottleneck rate increases. In both cases, we need more W_0 to fill the network and achieve that the throughput can be equal to r_b in the best case.

s.7.2.12. Now the slow machine at station 2 becomes superfluous. The fast machine at station 2 can cope with all jobs that arrive from station 1.

$$\begin{aligned} r_b &= \frac{1}{2} \\ T_0 &= 2 + 1 + 2 = 5 \\ W_0 &= r_b T_0. \end{aligned}$$

Thus, why would you assign part of the jobs to the slow machine at station 2? There is no queue, the fast machine can cope with all the jobs. Moreover, assigning any job to the slow machine just adds to the cycle time.

Taking $t_2 = (1 + 3)/2 = 2$ is plainly silly.

s.7.2.13. See above. I would not remove the slow machine. In real life there is variability: the fast machine might fail for instance. Spare capacity is useful. However, if it is very costly to keep the slow machine operational, I would scrap it.

s.7.2.14.

$$r_2 = \frac{1}{3} + \frac{1}{7} = \frac{10}{21} < \frac{1}{2},$$

hence station 2 is still the bottleneck

$$t_2 = \frac{3}{10}7 + \frac{7}{10}3 = \frac{42}{10} = \frac{21}{5} \text{ hour}$$

$$T_0 = 2 + \frac{21}{5} + 2$$

$$r_b = \frac{10}{21}$$

$$W_0 = r_b T_0.$$

Note that we add capacity, so that we can process more jobs per unit time, but the raw processing time increases!

s.7.2.15. Now there are two loops. One-third of the jobs pass all three stations. Station 1 can process these jobs at rate

$$\frac{1}{3}r_1 = \frac{1}{3} \frac{1}{2} = \frac{1}{6}$$

per hour. Since this is smaller than r_2 and r_3 , station 1 is the bottleneck rate for this loop. Thus, W_0 in this loop is $r_b T_0 = 1/6 \cdot 7 = 7/6$.

The other loop only contains station 1. The bottleneck rate in this loop is

$$r_1 \frac{2}{3} = \frac{1}{2} \frac{2}{3} = \frac{1}{3}.$$

Thus, $W_0 = r_b T_0 = 1/3 \cdot 2 = 2/3$ for this loop.

Hence, the total WIP is $7/6 + 2/3 = 11/6$ and

$$T_0 = \frac{1}{3}7 + \frac{2}{3}2 = \frac{11}{3},$$

since $1/3$ take the long loop with $T = 7$ and $2/3$ the short loop with $T = 2$. Thus, T_0 is the weighted average raw processing time.

Another way to analyze this situation is like this. Station 1 is the bottleneck, because its utilization is 1, while

$$\rho_2 = \lambda_2 t_2 = \frac{1}{2} \frac{1}{3} 3 = \frac{1}{2}$$

$$\rho_3 = \lambda_3 t_3 = \frac{1}{2} \frac{1}{3} 2 = \frac{1}{3}.$$

Since station 1 is the bottleneck in this network, $r_b = 1/2$. Since $T_0 = 11/3$, which we obtain as the weighted average computation above, we must have that

$$W_0 = r_b T_0 = \frac{1}{2} \frac{11}{3} = \frac{11}{6},$$

which agrees with our earlier result.

s.7.2.16. Now stations 1 and 3 are the bottlenecks with $r_b = 1/2$. For T_0 we have

$$T_0 = \frac{1}{5}7 + \frac{4}{5}4 = \frac{23}{5}.$$

Thus,

$$W_0 = r_b T_0 = \frac{1}{2} \frac{23}{5} = \frac{23}{10}.$$

s.7.2.17. Now stations 1 and 3 are the bottlenecks. Thus, both stations need to be part of the conwip loop. For that matter, it is just as easy to also include station 2, and just include all work on the floor in the WIP measurements.

s.7.3.1. Since $X \sim \text{Exp}(\lambda)$ and $S \sim \text{Exp}(\mu)$, and X and S are independent, their joint density is $f_{X,S}(x, y) = \lambda\mu e^{-\lambda x - \mu y}$. With this,

$$\begin{aligned} P\{X + S \leq t\} &= \lambda\mu \int_0^\infty \int_0^\infty e^{-\lambda x - \mu y} \mathbb{1}_{x+y \leq t} dx dy \\ &= \lambda\mu \int_0^t \int_0^{t-x} e^{-\lambda x - \mu y} dy dx \\ &= \lambda\mu \int_0^t e^{-\lambda x} \int_0^{t-x} e^{-\mu y} dy dx \\ &= \lambda \int_0^t e^{-\lambda x} (1 - e^{-\mu(t-x)}) dx \\ &= \lambda \int_0^t e^{-\lambda x} dx - \lambda e^{-\mu t} \int_0^t e^{(\mu-\lambda)x} dx \\ &= 1 - e^{-\lambda t} - \frac{\lambda}{\mu - \lambda} e^{-\mu t} (e^{(\mu-\lambda)t} - 1) \\ &= 1 - e^{-\lambda t} - \frac{\lambda}{\mu - \lambda} e^{-\lambda t} + \frac{\lambda}{\mu - \lambda} e^{-\mu t} \\ &= 1 - \frac{\mu}{\mu - \lambda} e^{-\lambda t} + \frac{\lambda}{\mu - \lambda} e^{-\mu t}. \end{aligned}$$

The density $f_{X+S}(t)$ is the derivative of this expression with respect to t , hence,

$$\begin{aligned} f_{X+S}(t) &= \frac{\lambda\mu}{\mu - \lambda} e^{-\lambda t} - \frac{\mu\lambda}{\mu - \lambda} e^{-\mu t} \\ &= \frac{\lambda\mu}{\lambda - \mu} (e^{-\mu t} - e^{-\lambda t}). \end{aligned}$$

Conditioning is much faster:

$$\begin{aligned}
 f_{X+S}(t) &= P\{X+S \in dt\} = \int_0^t P\{S+x \in dt\} P\{X \in dx\} \\
 &= \int_0^t f_S(t-x) f_X(x) dx = \int_0^t \mu e^{-\mu(t-x)} \lambda e^{-\lambda x} dx \\
 &= \lambda \mu e^{-\mu t} \int_0^t e^{x(\mu-\lambda)} dx = \frac{\lambda \mu}{\mu-\lambda} e^{-\mu t} (e^{(\mu-\lambda)t} - 1).
 \end{aligned}$$

s.7.3.2.

$$\begin{aligned}
 f_D(t) &= (1-\rho)f_{X+S}(t) + \rho \mu e^{-\mu t} = (1-\rho) \frac{\mu \lambda}{\lambda-\mu} (e^{-\mu t} - e^{-\lambda t}) + \rho \mu e^{-\mu t} \\
 &= \left(1 - \frac{\lambda}{\mu}\right) \frac{\mu \lambda}{\lambda-\mu} (e^{-\mu t} - e^{-\lambda t}) + \rho \mu e^{-\mu t} \\
 &= \frac{\mu-\lambda}{\mu} \frac{\mu \lambda}{\lambda-\mu} (e^{-\mu t} - e^{-\lambda t}) + \frac{\lambda}{\mu} \mu e^{-\mu t} = -\lambda (e^{-\mu t} - e^{-\lambda t}) + \lambda e^{-\mu t}
 \end{aligned}$$

s.7.3.3.

$$P = \begin{pmatrix} \alpha & 1-\alpha \\ \beta_1 & \beta_2 \end{pmatrix}, \quad (\lambda_1, \lambda_2) = (\gamma, 0) + (\lambda_1, \lambda_2)P.$$

Solving first for λ_2 leads to $\lambda_2 = (1-\alpha)\lambda_1 + \beta_2\lambda_2$, so that

$$\lambda_2 = \frac{1-\alpha}{1-\beta_2} \lambda_1.$$

Next, using this and that $\lambda_1 = \alpha\lambda_1 + \beta_1\lambda_2 + \gamma$ gives

$$\begin{aligned}
 \gamma &= \lambda_1(1-\alpha) - \beta_1\lambda_2 = \lambda_1 \left(1 - \alpha - \beta_1 \frac{1-\alpha}{1-\beta_2}\right) \\
 &= \lambda_1(1-\alpha) \left(1 - \frac{\beta_1}{1-\beta_2}\right) = \lambda_1(1-\alpha) \frac{1-\beta_1-\beta_2}{1-\beta_2}.
 \end{aligned}$$

Hence,

$$\lambda_1 = \frac{\gamma}{1-\alpha} \frac{1-\beta_2}{1-\beta_1-\beta_2}, \quad \lambda_2 = \frac{1-\alpha}{1-\beta_2} \lambda_1 = \frac{\gamma}{1-\beta_1-\beta_2}.$$

We want of course that $\lambda_1 < \mu_1$ and $\lambda_2 < \mu_2$. With the above expressions this leads to conditions on α , β_1 and β_2 . Note that we have three parameters, and two equations; there is no single condition that guarantees stability. If $\alpha \uparrow 1$, the arrival rate at node 1 explodes. If $\beta_1 = 0$ no jobs are sent from node 2 to node 1.

s.7.3.4. The rate into state $(0,0)$ is $\mu_2 p(0,1) = \mu_2 \rho_2$. The rate out of state $(0,0)$ is $\lambda p(0,0) = \lambda$. Since $\rho_2 = \lambda/\mu_2$, these two rates are the same.

s.7.3.5.

$$\begin{aligned}
 \text{rate out} &= (\lambda + \mu_1)p(i,0) = \mu_1 \rho_1^i + \lambda \rho_1^i = \lambda \rho_1^{i-1} + \mu_2 \rho_1^i \rho_2 \\
 &= \lambda p(i-1,0) + \mu_2 p(i,1) = \text{rate in}.
 \end{aligned}$$

s.7.3.6. We show that the rate out is the rate in.

$$\begin{aligned}\text{rate out} &= (\lambda + \mu_2)p(0, j) = \mu_2\rho_2^j + \lambda\rho_2^j = \mu_1\rho_1\rho_2^{j-1} + \mu_2\rho_2^{j+1} \\ &= \mu_1p(1, j-1) + \mu_2p(0, j+1) = \text{rate in}.\end{aligned}$$

s.7.4.1. $P_{i,i+1} = 1$, and $P_{ij} = 0$ for $j \neq i+1$. As P is an upper-triangular matrix of dimension $M \times M$, $P^M = 0$.

s.7.4.2. P transient $\implies Q_{i0} = P_{i0}^M > 0 \implies \sum_{j=1}^M Q_{ij} < 1$. Since M is a finite number, there exists an $\epsilon > 0$ such that $\max\{\sum_{j=1}^M Q_{ij}\} < 1 - \epsilon$. Writing $\mathbf{1}$ for the vector $(1, 1, \dots, 1)$, this means that $Q\mathbf{1} < (1 - \epsilon)\mathbf{1}$. But then $Q^2\{\mathbf{1}\} = Q(Q\{\mathbf{1}\}) < (1 - \epsilon)Q\{\mathbf{1}\} < (1 - \epsilon)^2\{\mathbf{1}\}$. As $Q \geq 0$ element-wise, we see that $0 \leq Q^n < (1 - \epsilon)^n$. And then $P^n = Q^{n/M} < (1 - \epsilon)^{n/M}$.

s.7.4.3. The above argument is not necessarily valid for matrices P that are infinite, since $\inf\{P_{ij}^M\}$ need not be strictly positive for all i, j .

NOTATION

a_k	= Number of arrivals in k th period
$A(t)$	= Number of arrivals in $[0, t]$
A_k	= Arrival time of k th job
$\tilde{A}_k$	= Start of service of k th job
B	= General batch size
B_k	= Batch size at k th arrival time
c_k	= Service/production capacity in k th period
c	= Number of servers
C	= General batch size during a service time
C_a^2	= Squared coefficient of variation of inter-arrival times
C_s^2	= Squared coefficient of variation of service times
$D(t)$	= Number of departures in $[0, t]$
D_k	= Departure time of k th job
d_k	= Number of departures in k th period
δ	= Departure rate
F	= Distribution of service time of a job
I	= Idle time of single server
J_k	= Sojourn time of k th job
$L(t)$	= Number of customers/jobs in system at time t
L_k	= Number in system at end of k th period
$L_s(t)$	= Number of customers/jobs in service at time t
λ	= Arrival rate
μ	= Service rate
$N(t)$	= Number of events of a Poisson process in $[0, t]$
$N(s, t)$	= Number of arrivals in $(s, t]$
$p(n)$	= Long-run time-average that the system contains n jobs
$\pi(n)$	= Stationary probability that an arrival sees n jobs in system
$Q(t)$	= Number of customers/jobs in queue at time t
Q_k	= Queue length as seen by k th job, or at end of k th period
ρ	= Utilization of a single server
S	= Generic service time
S_k	= Service time of k th job
U	= Busy time of single server
W	= Generic waiting time (in queue)
W_k	= Waiting time in queue of k th job
X	= Generic inter-arrival time between two consecutive jobs
X_k	= Inter-arrival time between job $k - 1$ and job k